

Anexo

Vicente Muñoz, Raúl

Tabla de contenidos

Carga de Datos	1
Manipulación de los datos	2
Función de preprocesado	4
Países con trayectorias de contagios similares	5
Partitional Clustering	6
<i>k</i> -Shape Clustering	32
TADPole Clustering	37
Fuzzy Clustering	42
Hierarchical clustering	52
Países con trayectorias de muertes similares	60
Partitional Clustering	60
<i>k</i> -Shape Clustering	91
TADPole Clustering	96
Fuzzy Clustering	98
Hierarchical clustering	105
Países con trayectorias de contagios y muertes similares	113
Partitional Clustering	113
Fuzzy Clustering	135
Hierarchical clustering	147
Distancia DTW	147
Países con trayectoria de hospitalizados similar	160
Partitional Clustering	160
<i>k</i> -Shape Clustering	187

Carga de Datos

Procedemos a cargar los datos del covid

```
url =
  ↪ "https://catalog.ourworldindata.org/garden/covid/latest/compact/compact.csv"
download.file(url, destfile = "compact.csv", mode = "wb")
df = read.csv("compact.csv", encoding = "UTF-8")
```

Manipulación de los datos

Como vamos a estudiar las tendencias por país, veamos cuáles son los países disponibles en los datos.

```
unique_countries = unique(df$country)
sort(unique_countries)[1:10]
```

```
[1] "Afghanistan"      "Africa"           "Albania"
[4] "Algeria"          "American Samoa"   "Andorra"
[7] "Angola"           "Anguilla"         "Antigua and Barbuda"
[10] "Argentina"
```

En los datos se observan continentes, “países” raros y repeticiones (se incluye Inglaterra y Reino Unido). Por tanto vamos a quedarnos con los países que nos interesan en el estudio.

```
library(dplyr)
clean_countries = c(
  "Afghanistan", "Albania", "Algeria", "Andorra", "Angola", "Antigua and
  ↪ Barbuda",
  "Argentina", "Armenia", "Australia", "Austria", "Azerbaijan", "Bahamas",
  "Bahrain", "Bangladesh", "Barbados", "Belarus", "Belgium", "Belize",
  ↪ "Benin",
  "Bhutan", "Bolivia", "Bosnia and Herzegovina", "Botswana", "Brazil",
  ↪ "Brunei",
  "Bulgaria", "Burkina Faso", "Burundi", "Cambodia", "Cameroon", "Canada",
  "Cape Verde", "Central African Republic", "Chad", "Chile", "China",
  ↪ "Colombia",
  "Comoros", "Congo", "Costa Rica", "Cote d'Ivoire", "Croatia", "Cuba",
  ↪ "Cyprus",
  "Czechia", "Democratic Republic of Congo", "Denmark", "Djibouti",
  ↪ "Dominica",
  "Dominican Republic", "East Timor", "Ecuador", "Egypt", "El Salvador",
  "Equatorial Guinea", "Eritrea", "Estonia", "Eswatini", "Ethiopia", "Fiji",
  "Finland", "France", "Gabon", "Gambia", "Georgia", "Germany", "Ghana",
  ↪ "Greece",
```

```

"Grenada", "Guatemala", "Guinea", "Guinea-Bissau", "Guyana", "Haiti",
  ↪ "Honduras",
"Hungary", "Iceland", "India", "Indonesia", "Iran", "Iraq", "Ireland",
  ↪ "Israel",
"Italy", "Jamaica", "Japan", "Jordan", "Kazakhstan", "Kenya", "Kiribati",
  ↪ "Kuwait",
"Kyrgyzstan", "Laos", "Latvia", "Lebanon", "Lesotho", "Liberia", "Libya",
"Liechtenstein", "Lithuania", "Luxembourg", "Madagascar", "Malawi",
  ↪ "Malaysia",
"Maldives", "Mali", "Malta", "Marshall Islands", "Mauritania", "Mauritius",
"Mexico", "Micronesia (country)", "Moldova", "Monaco", "Mongolia",
  ↪ "Montenegro",
"Morocco", "Mozambique", "Myanmar", "Namibia", "Nauru", "Nepal",
  ↪ "Netherlands",
"New Zealand", "Nicaragua", "Niger", "Nigeria", "North Korea", "North
  ↪ Macedonia",
"Norway", "Oman", "Pakistan", "Palau", "Palestine", "Panama", "Papua New
  ↪ Guinea",
"Paraguay", "Peru", "Philippines", "Poland", "Portugal", "Puerto Rico",
  ↪ "Qatar", "Romania",
"Russia", "Rwanda", "Saint Kitts and Nevis", "Saint Lucia",
"Saint Vincent and the Grenadines", "Samoa", "San Marino",
"Sao Tome and Principe", "Saudi Arabia", "Senegal", "Serbia", "Seychelles",
"Sierra Leone", "Singapore", "Slovakia", "Slovenia", "Solomon Islands",
"Somalia", "South Africa", "South Korea", "South Sudan", "Spain", "Sri
  ↪ Lanka",
"Sudan", "Suriname", "Sweden", "Switzerland", "Syria", "Taiwan",
  ↪ "Tajikistan",
"Tanzania", "Thailand", "Togo", "Tonga", "Trinidad and Tobago", "Tunisia",
"Turkey", "Turkmenistan", "Tuvalu", "Uganda", "Ukraine",
"United Arab Emirates", "United Kingdom", "United States", "Uruguay",
"Uzbekistan", "Vanuatu", "Vatican", "Venezuela", "Vietnam", "Yemen",
  ↪ "Zambia"
)

df= df %>% filter(country %in% clean_countries)

```

Una vez que tenemos datos de los países que nos interesan expliquemos qué vamos a analizar.

En primer lugar, veremos países con tendencias similares en cuanto a nuevos casos por millón de habitantes. Por ello realizaremos clustering de series temporales usando el paquete `dtwclust`, que permite incluir distancias ideales para series temporales. Detalles de esta se pueden ver en la memoria.

Se ha elegido esa variable pues es comparable entre países al tener en cuenta el tamaño de la población. Tras esto estudiaremos otras relaciones entre países, como aquellos con series de muertes por millón similares. Se incluirá una sección final en que se realizará cluster incluyendo las variables a la vez.

Dicho esto preparemos los datos para los contagios por millón.

Destacar que nos quedaremos con fechas hasta el 31 de diciembre de 2023, pues se ha visto que a partir de entonces el número de casos registrados es muy poco significativo. Además estudiaremos patrones por meses, ya que tomar datos diarios nos haría incluir demasiado ruido y aumentaría demasiado el tiempo de cómputo. Creemos una función para preprocesar las series como queremos.

Función de preprocesado

```
library(rlang)
library(lubridate)

preparar_series_temporales = function(df, columna, fecha_limite =
  ↪ as.Date("2023-12-31")) {
  df = df %>% mutate( date = as.Date(date), year_month = floor_date(date,
  ↪ unit = "month") ) %>% filter(year_month <= fecha_limite)

  # Convertir nombre de columna a símbolo
  columna_sym <- sym(columna)

  # Agregación mensual por país de aquellos países que hayan registrado
  ↪ contagios en al menos 3 meses de los 48 considerados
  df_monthly = df %>% group_by(country, year_month) %>% summarise(valor =
  ↪ sum(!!columna_sym, na.rm = TRUE), .groups = "drop") %>% group_by(country)
  ↪ %>% filter(sum(valor > 0, na.rm = TRUE) > 3) %>% ungroup()

  # Crear lista de series temporales por país
  series_list = df_monthly %>% group_by(country) %>% arrange(year_month) %>%
  ↪ summarise(serie = list(valor), .groups = "drop") %>% pull(serie)

  nombres = df_monthly %>% distinct(country) %>% pull(country)

  names(series_list) = nombres
  return(series_list) }
```

Países con trayectorias de contagios similares

Realicemos un cluster de series temporales para la variable `new_cases_per_million`.

```
serie_new_cases = preparar_series_temporales(df,"new_cases_per_million")
```

Veamos si son diferentes las longitudes de cada serie.

```
unique(lengths(serie_new_cases))
```

```
[1] 48
```

Por tanto tenemos series de idéntica longitud.

Realicemos un escalado para que las técnicas de cluster funcionen mejor.

```
# Función para aplicar Z-score
z_score = function(x) {
  if (sd(x) == 0) return(rep(0, length(x))) # evitar división por 0
  (x - mean(x)) / sd(x)
}

# Aplicar a la serie
serie_new_cases_z = lapply(serie_new_cases, z_score)
```

Construimos una distancia personalizada a raíz de DTW.

```
library(dtwclust)
ndtw = function(x, y, ...) {
  dtw(x, y, ...,
      step.pattern = asymmetric,
      distance.only = TRUE)$normalizedDistance
}
# Register the distance with proxy
proxy::pr_DB$set_entry(FUN = ndtw, names = c("nDTW"),
                       loop = TRUE, type = "metric", distance = TRUE,
                       description = "Normalized, asymmetric DTW")
```

Se usa `asymmetric` por si un país tuvo una ola de contagios más temprana que otra. Buscamos patrones generales entre los países, es decir, países con una sola ola, países con una gran ola y rebrotes, etc.

En este caso la normalización no es necesaria pues todas las longitudes son iguales, sin embargo no es mala práctica dejarla.

Ahora pasemos a aplicar los diferentes tipos de cluster que se han estudiado en la memoria.

Partitional Clustering

Probaremos con diferentes configuraciones de distancia y de centroides.

Distancia nDTW

Prototipo DBA

En vez de la distancia DTW usaremos la versión asimétrica y normalizada que construimos. Como se describe en la memoria, el prototipo ideal cuando usamos DTW es DBA (DTW Bary-center Averaging). Para determinar el k que usaremos nos basaremos en el índice CH. Como nuestra matriz de distancias es asimétrica no podemos fiarnos de índices como el Silhouette o el índice de Dunn. Además al usar DBA, el índice de Davies Bouldin también puede inducirnos error, por ello se ha elegido el índice CH para ayudarnos a determinar el k .

```
# Definimos el rango de k a evaluar
k_values = 2:6
ch_scores = numeric(length(k_values))

for (i in seq_along(k_values)) {
  k = k_values[i]
  cat("Evaluando k =", k, "...\\n")
  set.seed(42)
  cl = tsclust(serie_new_cases_z, type = "partitional", k = k, distance =
↪  "ndtw", centroid = "dba", seed = 12345)

  resumen_metricas = cvi(cl,type="internal")
  ch = resumen_metricas["CH"]

  # Promedio de los indice CH
  ch_scores[i] = ch
}
```

Evaluando k = 2 ...

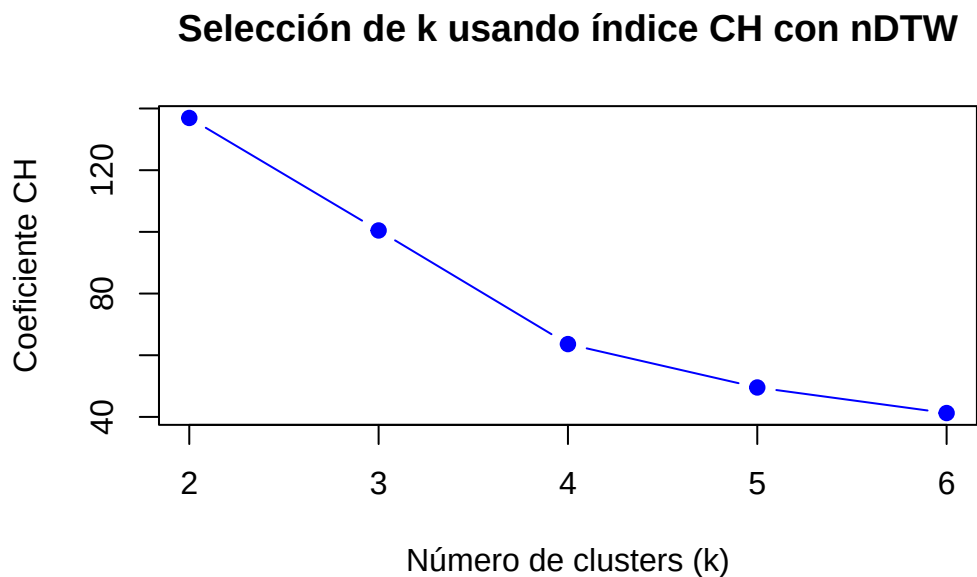
Evaluando k = 3 ...

Evaluando $k = 4$...

Evaluando $k = 5$...

Evaluando $k = 6$...

```
# Graficar los resultados
par(mfrow = c(1, 1))
plot(k_values, ch_scores, type = "b", pch = 19, col = "blue",
     xlab = "Número de clusters (k)", ylab = "Coeficiente CH",
     main = "Selección de k usando índice CH con nDTW")
```



Vemos que el máximo se alcanza para $k = 2$, veremos que aunque es verdad que separando en 2 cluster obtenemos grupos bien diferenciados, se puede hacer una división más fina con $k = 3$, de modo que no saquemos patrones tan generales.

Caso $k = 2$

Vamos a construir el cluster correspondiente. Recordemos que tomaremos la distancia ndtw definida y centroides por DBA.

```
set.seed(42)
cl_ndtw_dba_2 = tsclust(serie_new_cases_z, type = "partitional", k = 2,
  ↪ distance = "ndtw", centroid = "dba", seed = 12345)
cl_ndtw_dba_2
```

```
partitional clustering with 2 clusters
Using ndtw distance
Using dba centroids
```

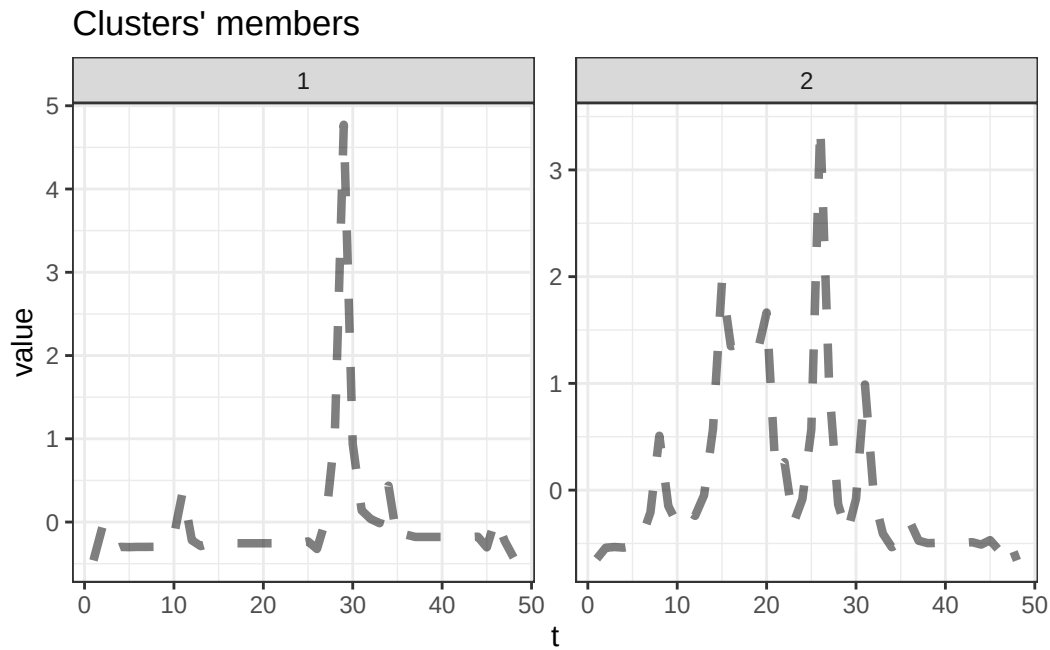
Time required for analysis:

user	system	elapsed
4.25	0.33	3.68

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	52	0.1683963
2	140	0.2145771

```
plot(cl_ndtw_dba_2,type="centroids")
```



En los centroides vemos claramente el patrón genérico que se detecta.

Un primer grupo está formado por países que han tenido un único pico importante de contagios que sobresale del resto, mientras que el segundo grupo está formado por países que pese a tener un gran pico importante, presentan numerosos repuntes. Construyamos funciones que nos permitirán ver esto que estamos diciendo ahora.


```

visualizar_series_centroides = function(series, cl, k, ylab =
  ↪ "Valor",tamano_ventana=c(1,k)) {
  par(mfrow = tamano_ventana, mar = c(6, 6, 2, 1), mgp = c(4, 1.5, 0))

  fechas = seq(as.Date("2020-01-01"), as.Date("2023-12-01"), by = "month")
  meses_anos = format(fechas, "%b-%Y")

  for (cluster_id in 1:k) {
    series_cluster = series[cl@cluster == cluster_id]
    series_cluster = lapply(series_cluster, as.numeric)

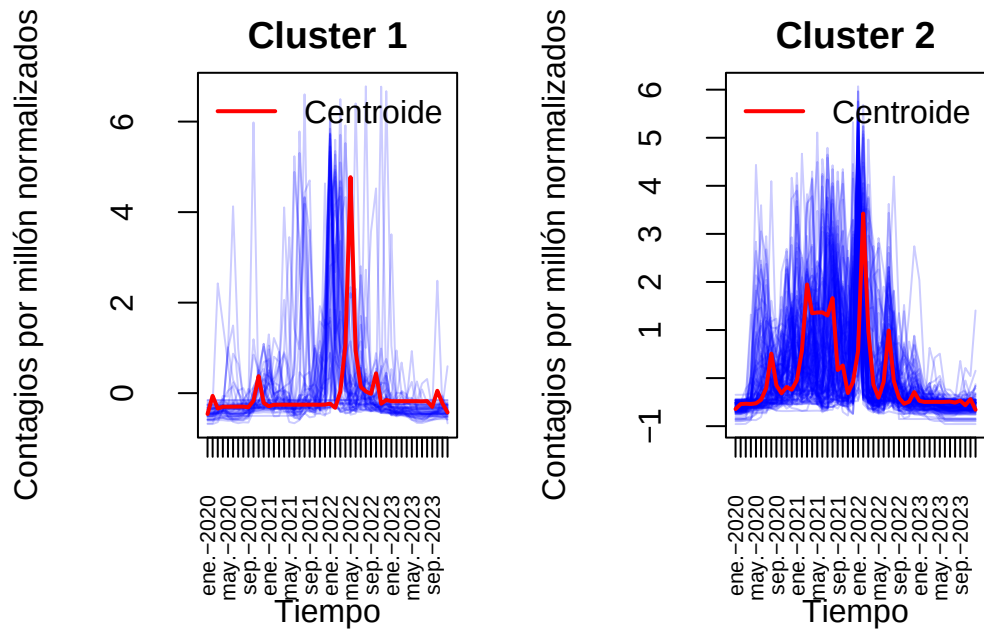
    matplot(do.call(cbind, series_cluster), type = "l",
      lty = 1, col = rgb(0, 0, 1, 0.2),
      main = paste("Cluster", cluster_id),
      ylab = ylab,
      xlab = "Tiempo",
      xaxt = "n")

    axis(1, at = 1:length(fechas), labels = meses_anos,
      las = 2, cex.axis = 0.7)

    lines(cl@centroids[[cluster_id]], col = "red", lwd = 2)
    legend("topright", legend = "Centroide", col = "red", lwd = 2, bty = "n")
  }
}

visualizar_series_centroides(serie_new_cases_z,cl_ndtw_dba_2,2,ylab =
  ↪ "Contagios por millón normalizados")

```



Pese a que parecen dos grupos bien definidos, se observa gran variabilidad en el segundo grupo. Esto es normal, pues realmente estamos hallando patrones bastante genéricos. Este análisis nos permite decir países que tuvieron una ola muy pronunciada sin repuntes demasiado altos o al menos comparado con ese gran repunte y países que tuvieron varios repuntes significativos además de esa gran ola. Visualicemos varios ejemplos

```
library(ggplot2)
library(tibble)
library(purrr)
library(scales)

visualizar_ejemplos = function(series, cl, n_cluster, n_ejemplos,
  ↪ tamaño_ventana,
                                semilla = NULL, ylab = "Valor", n_breaks_y =
  ↪ 4) {

  if (!is.null(semilla)) set.seed(semilla)

  ind_cluster <- sample(which(cl@cluster == n_cluster), n_ejemplos)
  nombres <- names(series)
  if (is.null(nombres)) nombres <- paste("Serie", seq_along(series))

  fechas <- seq(as.Date("2020-01-01"), as.Date("2023-12-01"), by = "month")
```

```

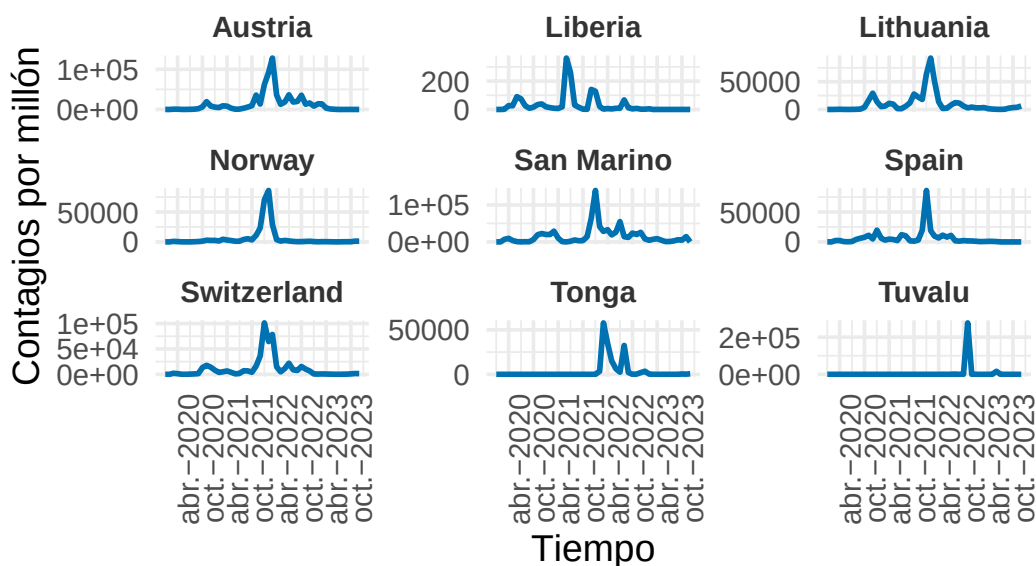
datos <- map_df(ind_cluster, function(i) {
  tibble(
    Tiempo = fechas[seq_along(series[[i]])],
    Valor = series[[i]],
    Serie = nombres[i]
  )
})

ggplot(datos, aes(x = Tiempo, y = Valor)) +
  geom_line(color = "#0072B2", size = 1) +
  facet_wrap(~ Serie, nrow = tamano_ventana[1],
             ncol = tamano_ventana[2], scales = "free_y") +
  labs(title = paste("Ejemplos del Cluster", n_cluster),
       x = "Tiempo", y = ylab) +
  scale_x_date(date_breaks = "6 months", date_labels = "%b-%Y") +
  scale_y_continuous(
    breaks = scales::breaks_pretty(n = n_breaks_y) # máximo n_breaks_y
    ↪ etiquetas
  ) +
  theme_minimal(base_size = 14) +
  theme(
    strip.text = element_text(face = "bold", color = "#333333"),
    plot.title = element_text(size = 18, face = "bold", hjust = 0.5),
    axis.text.x = element_text(angle = 90, hjust = 1)
  )
}

visualizar_ejemplos(serie_new_cases, cl_ndtw_dba_2, 1, 9, c(3, 3), 2, ylab = "Contagios
↪ por millón", 2)

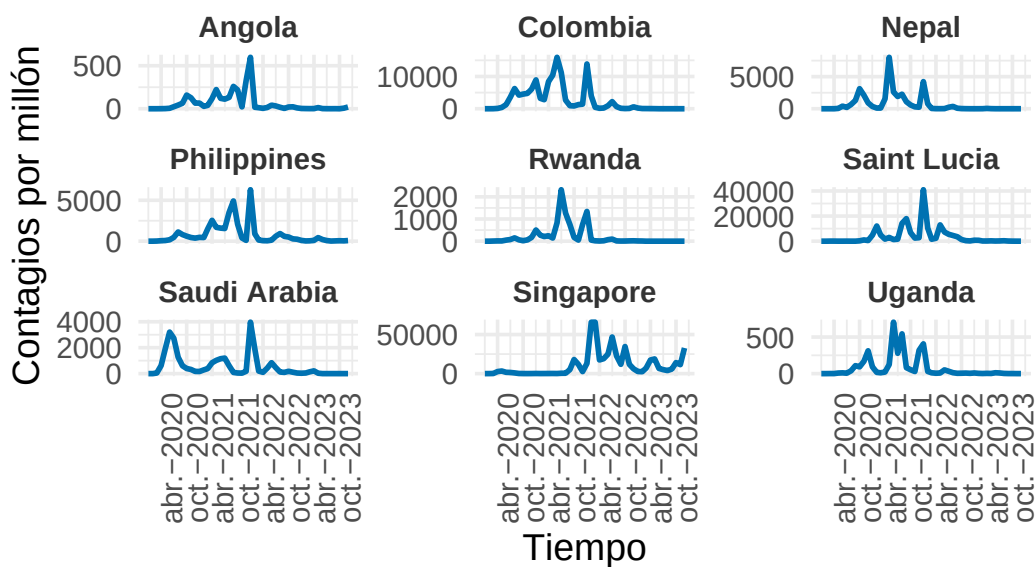
```

Ejemplos del Cluster 1



```
visualizar_ejemplos(serie_new_cases,cl_ndtw_dba_2,2,9,c(3,3),2,ylab="Contagios
↵ por millón",2)
```

Ejemplos del Cluster 2



En esta agrupación observamos una diferencia clara entre los dos clusters. En el Cluster 1, la mayoría de los países presentan un pico de contagios muy marcado y concentrado en el tiempo, lo que sugiere un brote intenso pero relativamente aislado. Aunque hay alguna excepción (como el tercer ejemplo), la mayoría de las series tienen una forma muy similar.

En cambio, en el Cluster 2 predominan los países con varios repuntes a lo largo del tiempo. Aunque también pueden haber tenido un pico pronunciado, este no domina completamente la serie, permitiendo identificar fases posteriores de reactivación o nuevos brotes.

En resumen, esta partición permite distinguir entre países que experimentaron una ola principal claramente definida y aquellos que sufrieron múltiples oleadas o una evolución epidémica más prolongada.

Aunque con dos clusters hemos logrado distinguir claramente dos patrones de comportamiento epidémico, se observa que dentro del segundo grupo existe cierta heterogeneidad, lo que sugiere que podría beneficiarse de una segmentación más detallada.

Por este motivo, vamos a explorar una partición en tres clusters, correspondiente al segundo valor de k según la métrica evaluada. El objetivo es comprobar si esta opción permite generar grupos más homogéneos internamente y separados entre sí, lo que podría aportar una mejor interpretación epidemiológica de los patrones temporales. Además en este caso exploraremos otras distancias y experimentaremos con otros centroides para dar variedad al estudio.

Caso $k = 3$

```
set.seed(42)
cl_ndtw_dba_3 = tsclust(serie_new_cases_z, type = "partitional", k = 3,
  ↪ distance = "ndtw", centroid = "dba", seed = 12345)
cl_ndtw_dba_3
```

```
partitional clustering with 3 clusters
Using ndtw distance
Using dba centroids
```

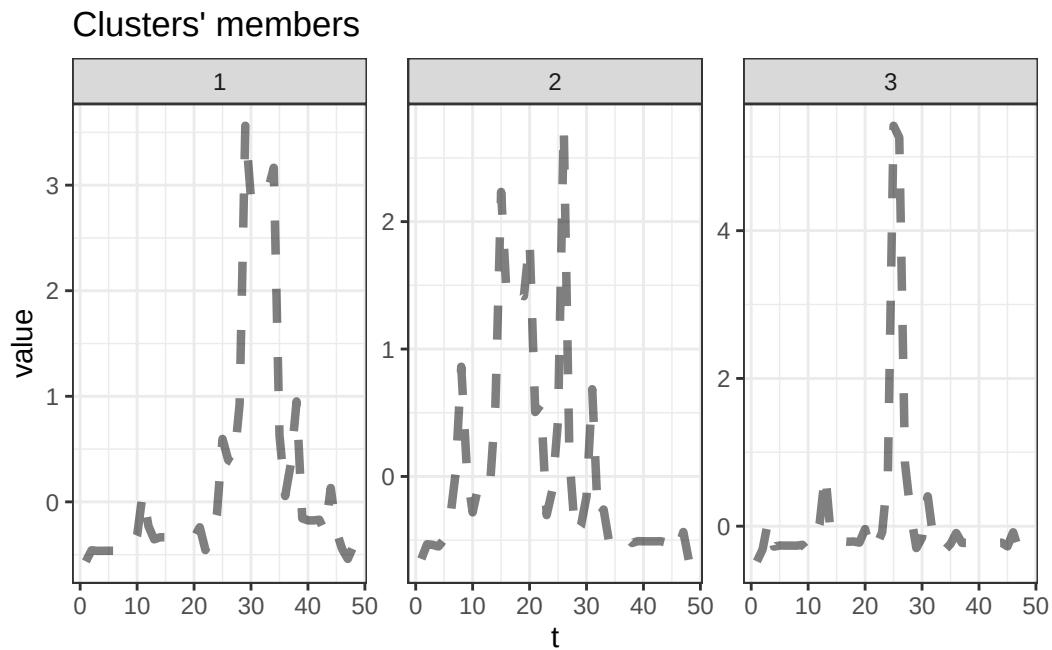
Time required for analysis:

	user	system	elapsed
	5.41	0.47	5.00

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	19	0.1659522
2	118	0.2195602
3	55	0.1668458

```
plot(cl_ndtw_dba_3,type="centroids")
```

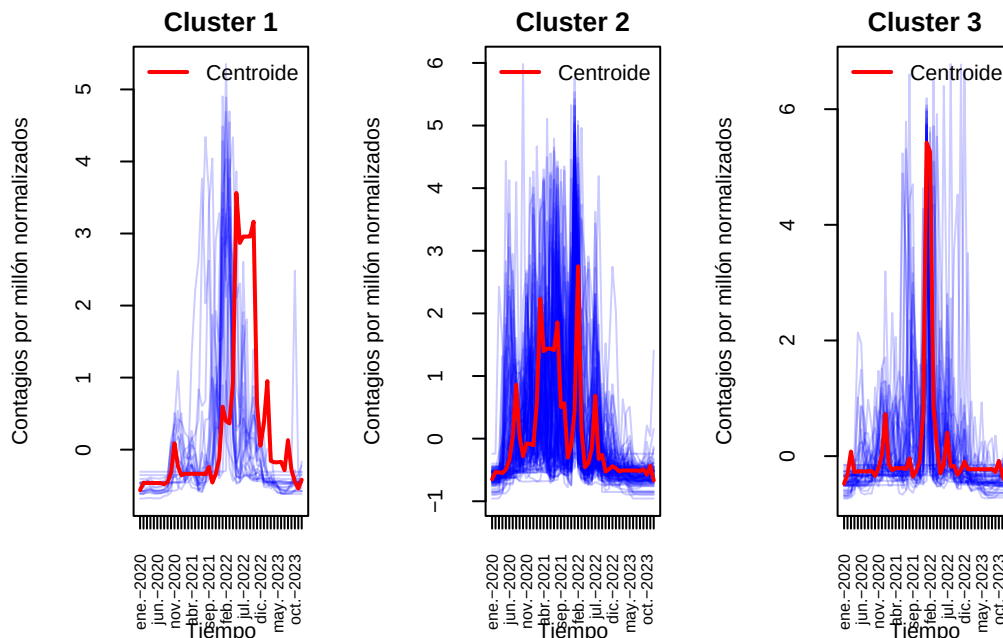


La segmentación en tres clusters parece haber permitido una mayor diferenciación entre los grupos, como se aprecia en el tamaño relativo de cada uno.

Al analizar los centroides de cada grupo, observamos que ha emergido un nuevo patrón característico: un grupo de países que, además de haber experimentado un pico epidémico muy marcado, presentan posteriormente un repunte significativo.

Este nuevo cluster complementa a los ya identificados en la partición anterior, ofreciendo una clasificación más rica y matizada. A continuación, visualizamos las series temporales correspondientes a cada grupo junto con sus centroides para examinar mejor estos patrones.

```
visualizar_series_centroides(serie_new_cases_z,cl_ndtw_dba_3,3,ylab =  
  ↪ "Contagios por millón normalizados")
```



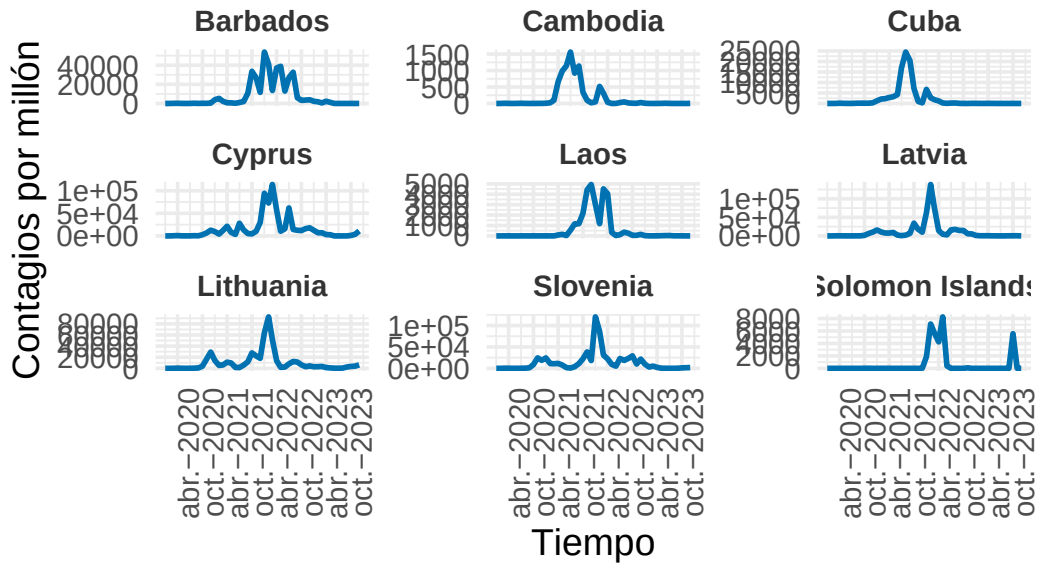
Debemos tener en cuenta que la distancia empleada en el clustering busca capturar similitudes en la forma general de las series temporales, independientemente de su alineación exacta en el tiempo. Esto implica que dos series pueden considerarse similares aunque presenten sus picos o patrones en momentos distintos.

Esta característica puede explicar por qué, a simple vista, el primer centroide no parece representar fielmente a su grupo. Es probable que, al visualizar ejemplos individuales, observemos que las series presentan formas similares al centroide, pero con desplazamientos temporales. Es decir, los brotes o subidas de casos se producen antes o después, pero con una evolución comparable.

Por otro lado, el primer grupo revela un nuevo patrón característico como dijimos anteriormente: países que han experimentado un brote importante, sostenido en el tiempo, antes de descender. Esta persistencia del pico lo diferencia claramente de los otros clusters y justifica su separación como grupo independiente.

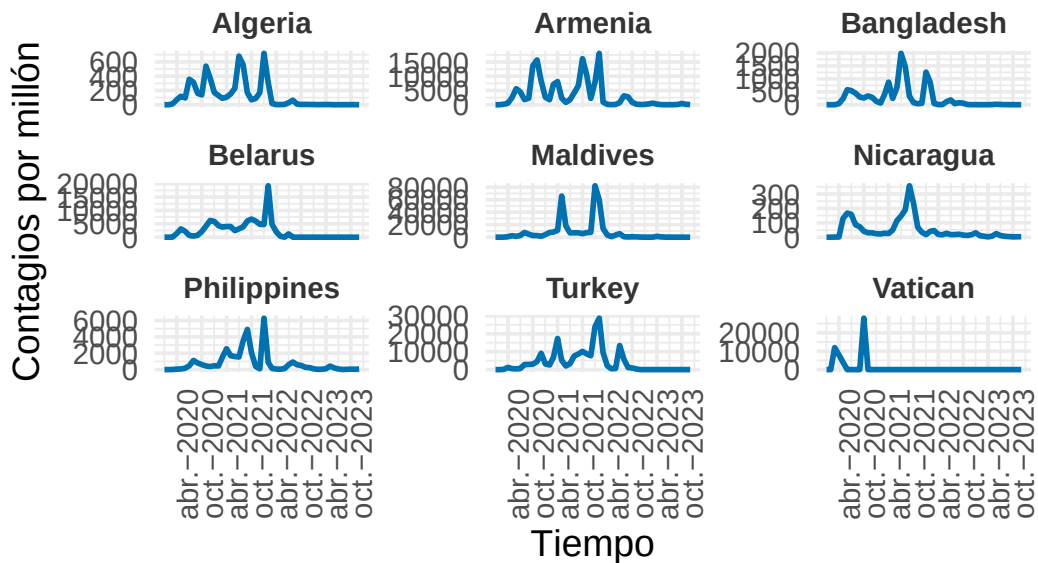
```
visualizar_ejemplos(serie_new_cases,cl_ndtw_dba_3,1,9,c(3,3),42,ylab =
  ↪ "Contagios por millón")
```

Ejemplos del Cluster 1



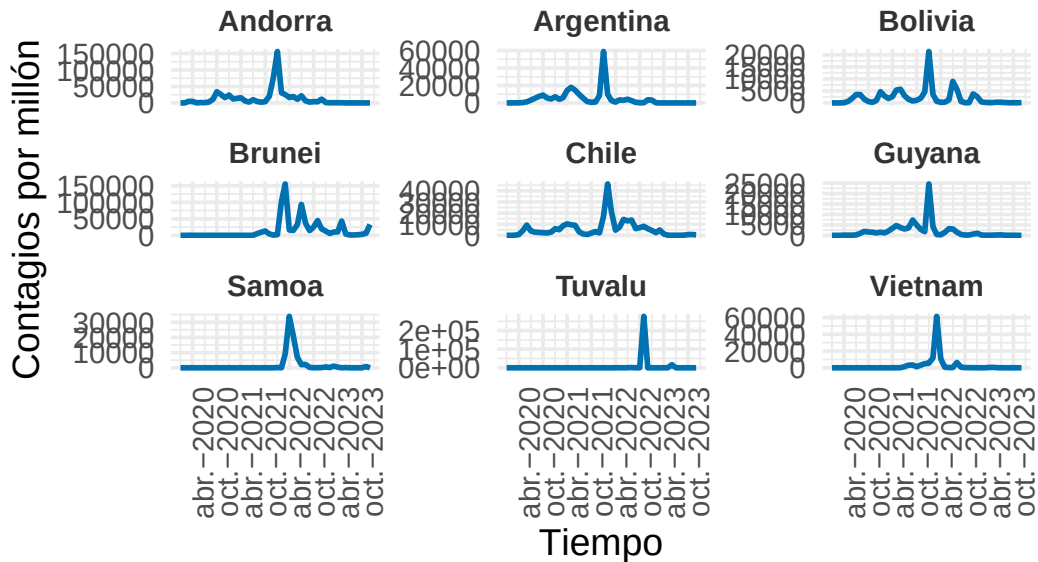
```
visualizar_ejemplos(serie_new_cases,cl_ndtw_dba_3,2,9,c(3,3),42,ylab =  
↪ "Contagios por millón")
```

Ejemplos del Cluster 2




```
visualizar_ejemplos(serie_new_cases, cl_ndtw_dba_3, 3, 9, c(3, 3), 42, ylab =
  ↪ "Contagios por millón")
```

Ejemplos del Cluster 3



Observamos exactamente lo anticipado: en los ejemplos pertenecientes al cluster 1, la mayoría de países presentan un descenso más progresivo de los contagios tras un primer pico pronunciado, e incluso muestran un rebrote posterior. Este comportamiento contrasta con el del cluster 3, donde predominan casos con un único gran pico de contagios, seguido de una estabilización sin rebotes significativos (con la posible excepción de Brunei).

En esta nueva segmentación, el cluster 2 resulta más homogéneo que el cluster con más observaciones que construimos antes. Agrupa principalmente países con un pico muy elevado y múltiples brotes en general. La única excepción notable es el Vaticano, cuya serie presenta un comportamiento atípico, aunque esperable dada su condición de país muy pequeño. Por tanto, su inclusión como outlier no compromete la coherencia del grupo.

En conjunto, esta nueva partición ha permitido formar grupos más homogéneos internamente, manteniendo al mismo tiempo diferencias claras entre clusters. Antes de explorar otras configuraciones de clustering para $k = 3$, analizaremos brevemente el caso de $k=4$, el cual presentaba un valor notablemente inferior del índice de Calinski-Harabasz. Esto sugiere que podría no ser una segmentación tan adecuada, por lo que quizás resulte razonable considerar $k = 3$ en futuros cluster.

Caso $k = 4$

```
set.seed(42)
cl_ndtw_dba_4 = tsclust(serie_new_cases_z, type = "partitional", k =4,
  ↪ distance = "ndtw", centroid = "dba",seed = 12345)
cl_ndtw_dba_4
```

partitional clustering with 4 clusters

Using ndtw distance

Using dba centroids

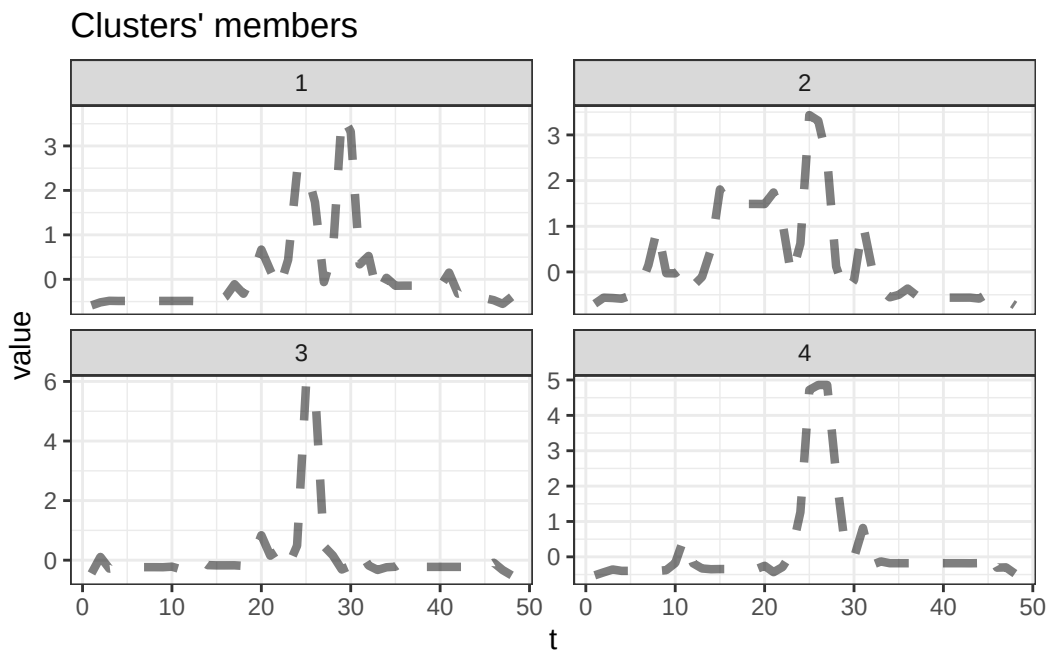
Time required for analysis:

user	system	elapsed
11.51	0.59	11.25

Cluster sizes with average intra-cluster distance:

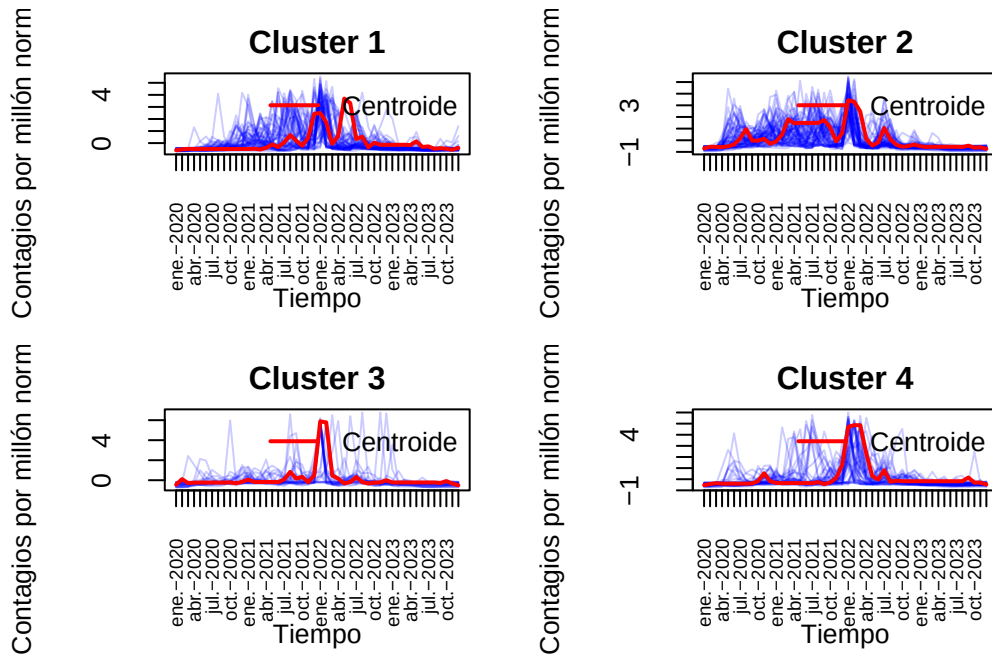
	size	av_dist
1	55	0.1862013
2	67	0.2298470
3	26	0.1635835
4	44	0.1708976

```
plot(cl_ndtw_dba_4,type="centroids")
```



Veamos si el tomar un grupo más nos permite obtener clusters mas homogéneos internamente manteniendo diferencias entre sí.

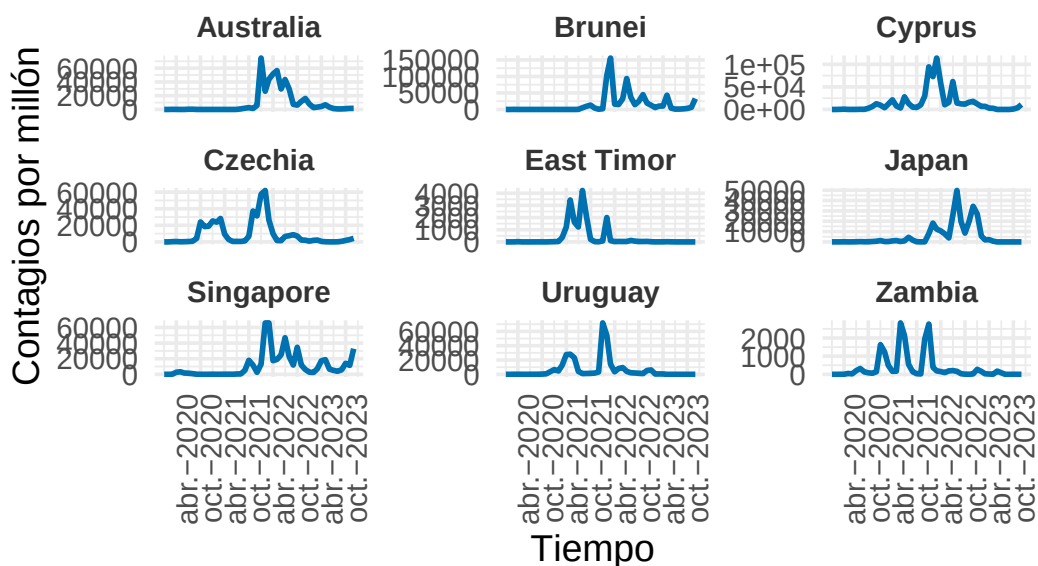
```
visualizar_series_centroides(serie_new_cases_z,cl_ndtw_dba_4,4,ylab =
  ↪ "Contagios por millón normalizados",tamano_ventana = c(2,2))
```



En los centroides se observa una similitud entre el cluster 3 y 4. Tomemos algunos ejemplos y veamos si hay homogeneidad.

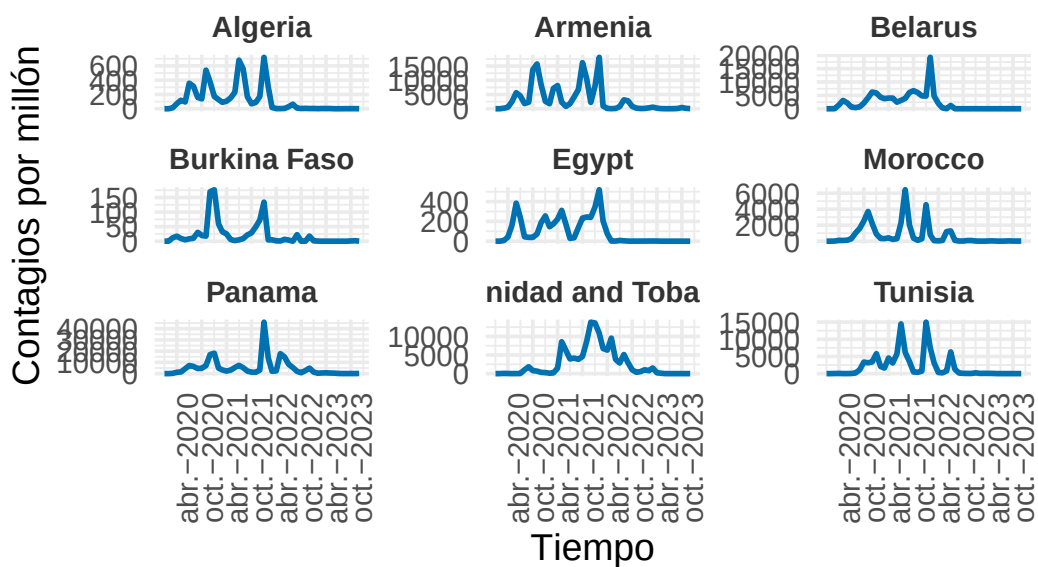
```
visualizar_ejemplos(serie_new_cases,cl_ndtw_dba_4,1,9,c(3,3),42,ylab =
  ↪ "Contagios por millón")
```

Ejemplos del Cluster 1



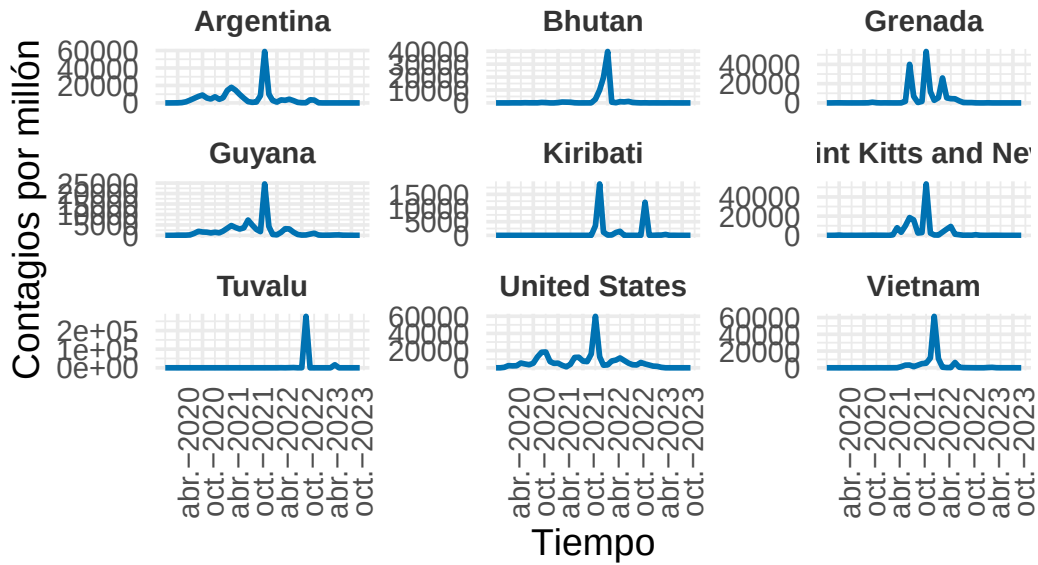
```
visualizar_ejemplos(serie_new_cases,cl_ndtw_dba_4,2,9,c(3,3),42,ylab =
↪ "Contagios por millón")
```

Ejemplos del Cluster 2



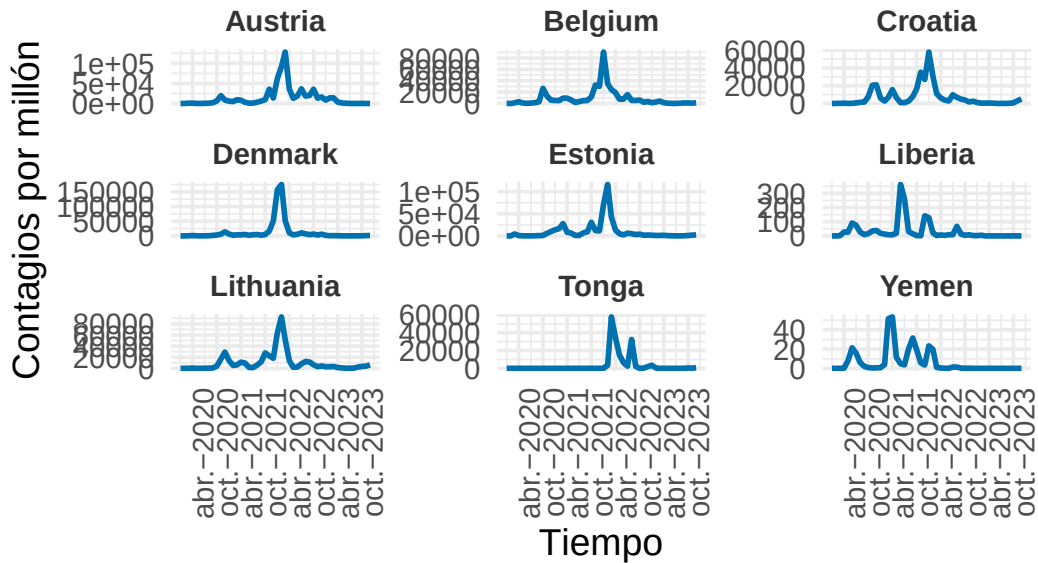
```
visualizar_ejemplos(serie_new_cases,cl_ndtw_dba_4,3,9,c(3,3),42,ylab =
  ↪ "Contagios por millón")
```

Ejemplos del Cluster 3



```
visualizar_ejemplos(serie_new_cases,cl_ndtw_dba_4,4,9,c(3,3),42,ylab =
  ↪ "Contagios por millón")
```

Ejemplos del Cluster 4



Vemos que como hemos dicho anteriormente, no se aprecie una diferencia clara entre el cluster 3 y el cluster 4, ni si quiera en forma. Esto respalda que el coeficiente CH asociado a 4 cluster sea bastante más bajo que con $k = 3$. Ahora pasaremos a ver los resultados con otras combinaciones estudiadas en la memoria para el caso $k = 3$ que es el que puede resultar no trivial (comparando con $k = 2$).

Prototipo PAM

```
set.seed(42)
cl_ndtw_pam_3 = tsclust(serie_new_cases_z, type = "partitional", k = 3,
  ↪ distance = "ndtw", centroid = "pam", seed = 12345)
cl_ndtw_pam_3
```

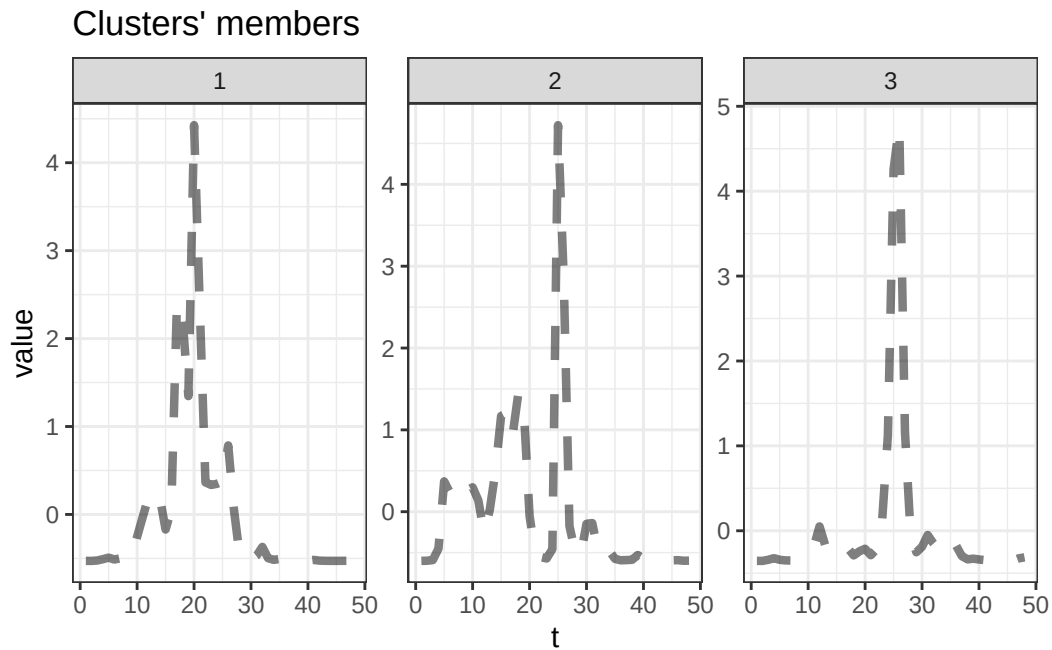
```
partitional clustering with 3 clusters
Using ndtw distance
Using pam centroids
```

```
Time required for analysis:
  user  system elapsed
37.67   0.46   39.87
```

```
Cluster sizes with average intra-cluster distance:
```

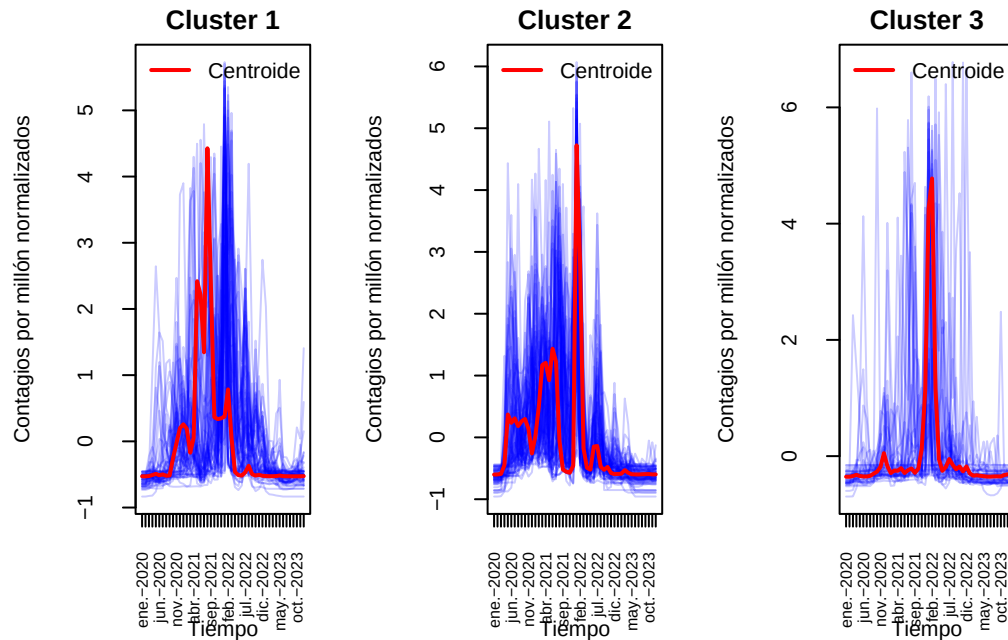
	size	av_dist
1	64	0.2089751
2	85	0.2464933
3	43	0.1658914

```
plot(cl_ndtw_pam_3,type="centroids")
```



En este caso observamos una diferencia considerable al comparar el uso de DBA como método de cálculo de centroides frente a simplemente seleccionar una serie representativa del grupo. DBA resulta mucho más adecuado cuando se utiliza una distancia basada en DTW, ya que permite generar un prototipo que refleja mejor el comportamiento promedio del grupo. En contraste, elegir una única serie como centroide puede resultar poco representativo, especialmente cuando existe alta variabilidad entre las series del grupo.

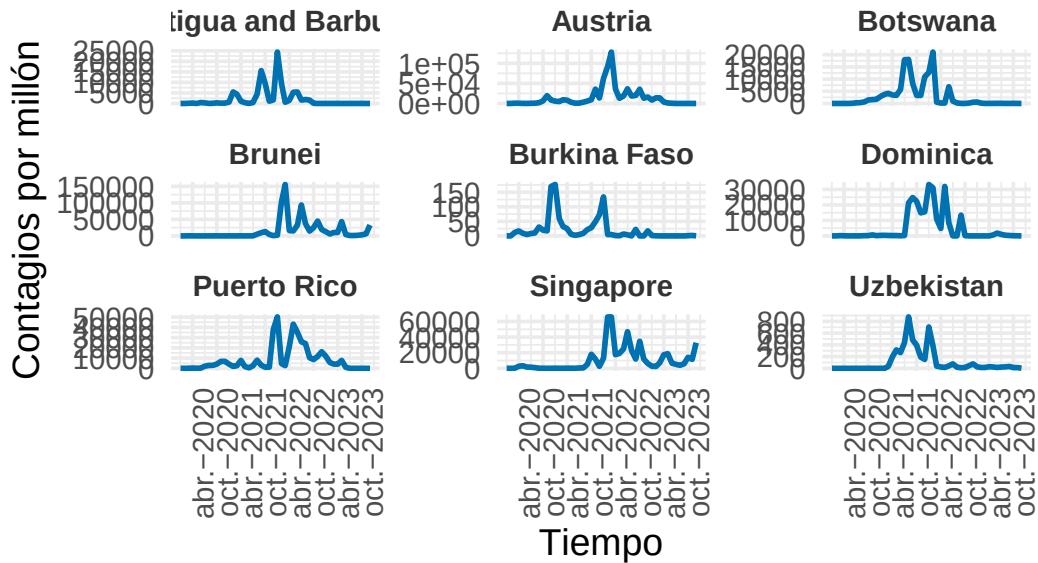
```
visualizar_series_centroides(serie_new_cases_z,cl_ndtw_pam_3,3,ylab=
  ↪ "Contagios por millón normalizados")
```



El grupo 3 al igual que cuando usamos DBA parece estar bien representado, aunque surgen dudas sobre los dos primeros. Veamos algunos ejemplos.

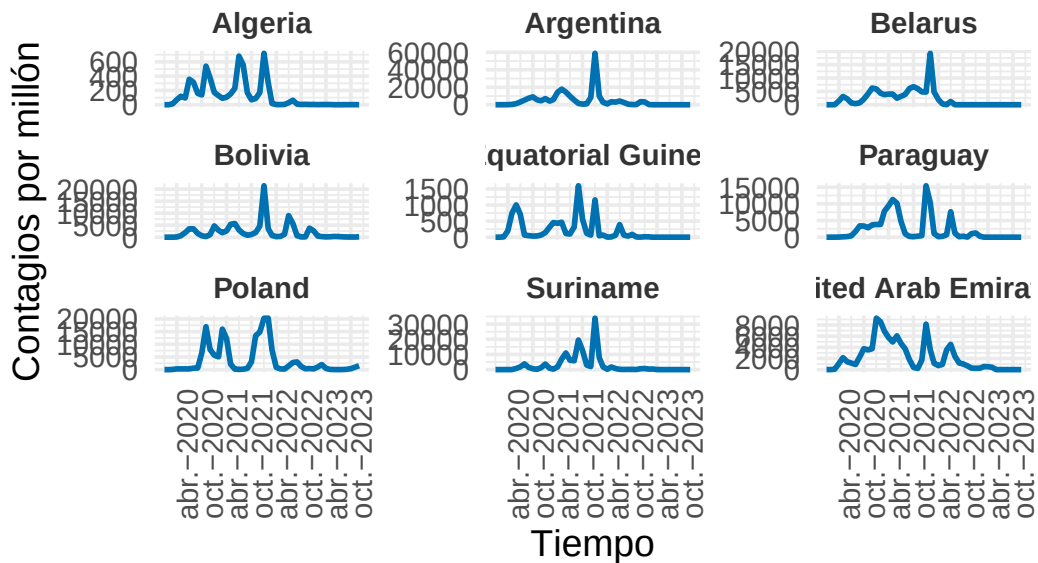
```
visualizar_ejemplos(serie_new_cases,cl_ndtw_pam_3,1,9,c(3,3),42,ylab=
  ↪ "Contagios por millón")
```


Ejemplos del Cluster 1



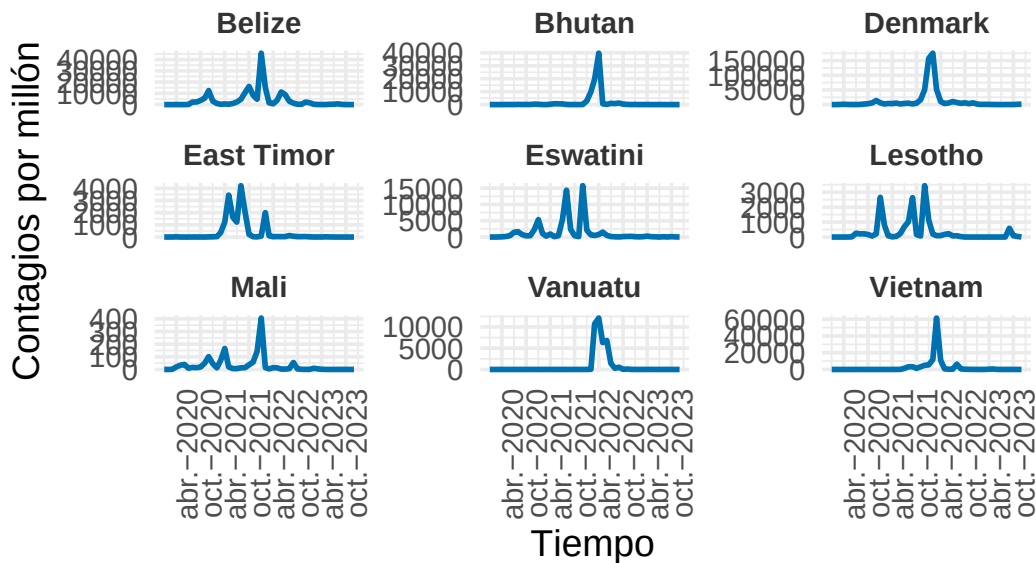
```
visualizar_ejemplos(serie_new_cases,cl_ndtw_pam_3,2,9,c(3,3),42,ylab=
  ↪ "Contagios por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_cases,cl_ndtw_pam_3,3,9,c(3,3),42,ylab=
↪ "Contagios por millón")
```

Ejemplos del Cluster 3



Parece que hemos obtenido clusters menos homogéneos que con el método DBA. Esto tiene sentido pues la elección de un buen prototipo es muy importante en los algoritmo particional por cómo se construyen estos. Veamos si las métricas reflejan esto que decimos y observamos. Aún así se observa que la segmentación tiene sentido y de cierta manera es similar a la que obtuvimos con DBA, aunque menos refinada.

```
set.seed(42)
cvi(cl_ndtw_dba_3,type="internal")
```

	Sil	SF	CH	DB	DBstar	D
	0.04234107	0.46096160	101.12711703	1.35249337	1.40840462	0.13057901
COP	0.62831404					

```
cvi(cl_ndtw_pam_3,type="internal")
```

	Sil	SF	CH	DB	DBstar	D
	0.09623965	0.43879222	95.31191715	1.70332776	1.77139047	0.16326364
COP	0.60424572					

Observamos que, en la métrica principal de evaluación, el índice de Calinski-Harabasz (CH), la agrupación obtenida mediante DBA presenta un rendimiento superior respecto a la agrupación alternativa. También se aprecia una mejora en la métrica SF, lo cual refuerza las observaciones realizadas anteriormente.

Por otro lado, vamos a comentar brevemente algunos índices cuya fiabilidad puede verse afectada por la asimetría de la función de distancias que consideramos. Aun así, métricas como DB y DB*, que penalizan la dispersión interna de los clústeres, muestran valores significativamente más favorables para el agrupamiento con DBA. En cuanto al índice Silhouette, aunque su valor es levemente superior en el modelo alternativo, esta diferencia es pequeña y su interpretación debe tomarse con cautela debido a la asimetría mencionada.

En conjunto, estos resultados sugieren que DBA produce una estructura de clústeres más coherente bajo la distancia nDTW. A continuación, exploraremos el comportamiento del modelo utilizando un tipo distinto de distancia y analizaremos los nuevos resultados obtenidos.

Distancia Soft-DTW

Usaremos ahora una versión diferenciable de DTW, la conocida SDTW.

En cuanto al prototipo, usaremos los centroides que mejor se adaptan a la distancia considerada.

```
set.seed(42)
cl_sdtw_3 = tsclust(serie_new_cases_z, type = "partitional", k = 3, distance =
  ↪ "sdtw", centroid = "sdtw_cent", seed = 12345)
cl_sdtw_3
```

```
partitional clustering with 3 clusters
Using sdtw distance
Using sdtw_cent centroids
```

Time required for analysis:

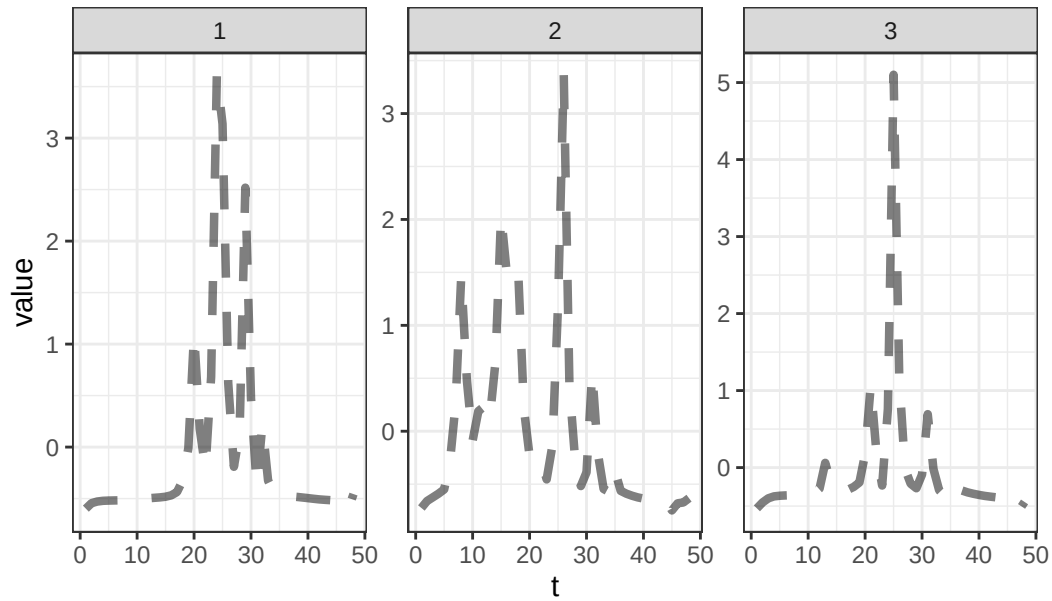
	user	system	elapsed
	72.09	0.23	5.10

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	78	6.403398
2	39	6.422278
3	75	5.650636

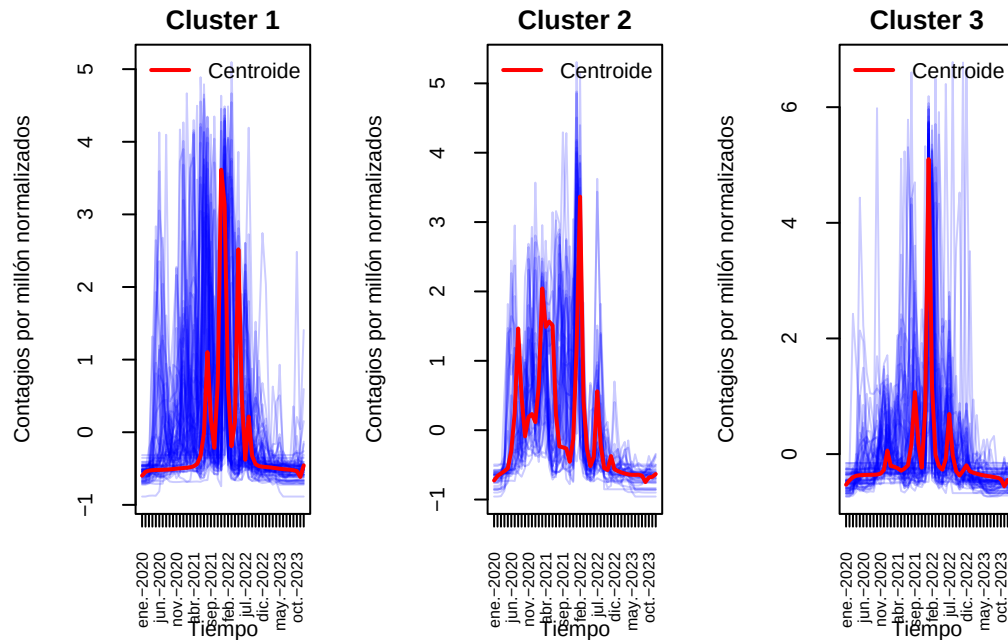
```
plot(cl_sdtw_3,type="centroids")
```

Clusters' members



Visualicemos los centroides junto a las series.

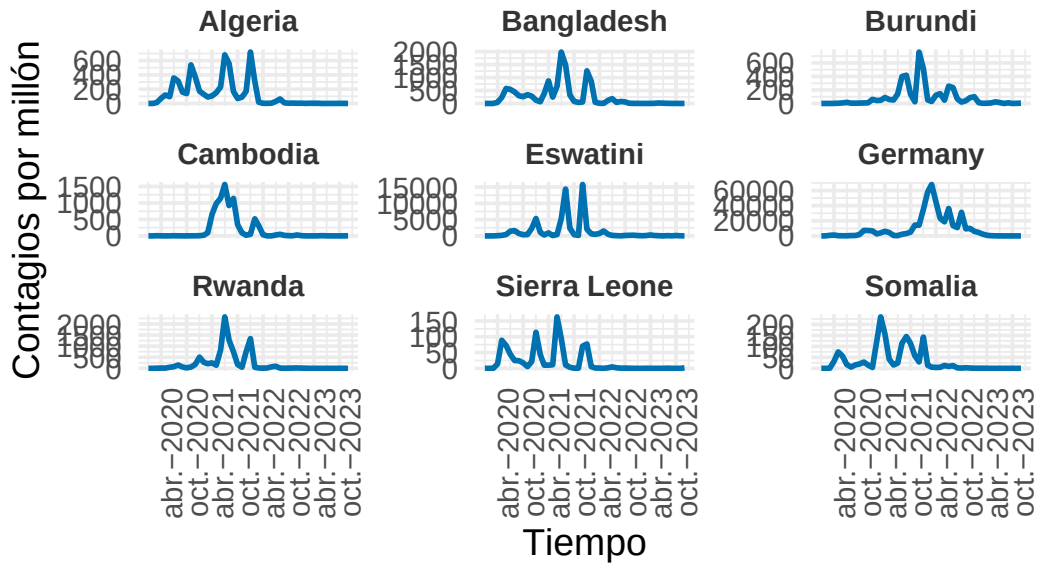
```
visualizar_series_centroides(serie_new_cases_z,cl_sdtw_3,3,ylab="Contagios  
↪ por millón normalizados")
```



Observamos que en este caso los centroides parecen tener sentido y de hecho recuerdan bastante a los de la mejor agrupación de 3 hasta ahora. Obtenemos un primer grupo con una gran ola y algún repunte significativo tras la misma. En el segundo grupo vemos varios picos significativos y en el tercer grupo un gran pico simplemente. Apoyémonos en algunos ejemplos.

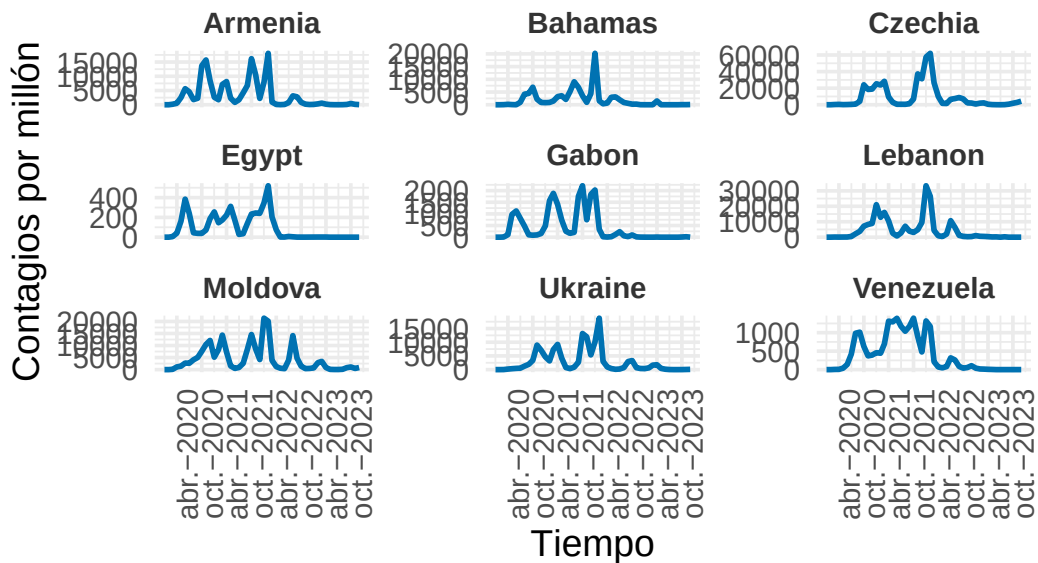
```
visualizar_ejemplos(serie_new_cases,cl_sdtw_3,1,9,c(3,3),42,ylab="Contagios  
↪ por millón")
```

Ejemplos del Cluster 1



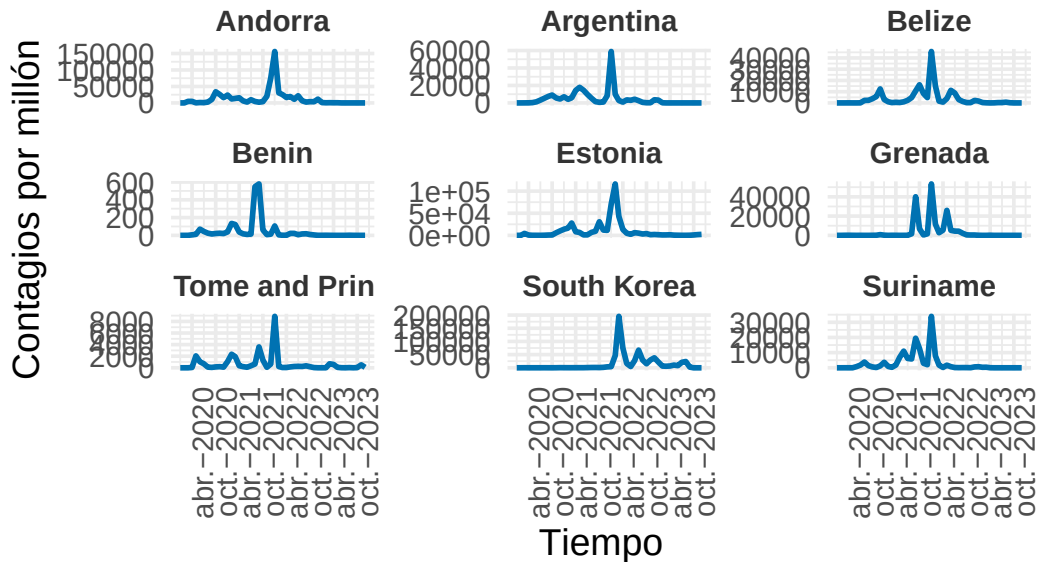
```
visualizar_ejemplos(serie_new_cases,cl_sdtw_3,2,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_cases,cl_sdtw_3,3,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 3



En los ejemplos se observa que las diferencias entre el grupo 1 y el grupo 2 no es tan clara como pudiera parecer en los centroides. Al evaluar las métricas veremos si esa similitud visual hace que obtengamos coeficientes peores.

```
set.seed(42)
cvi(cl_ndtw_dba_3,tipe="internal")
```

Si1	SF	CH	DB	DBstar	D
0.04234107	0.46096160	101.12711703	1.35249337	1.40840462	0.13057901
COP					
0.62831404					

```
cvi(cl_sdtw_3,type="internal")
```

Si1	SF	CH	DB	DBstar	D
1.672823e-01	2.591865e-08	4.674657e+01	1.469092e+00	1.499846e+00	4.394610e-02
COP					
2.501866e-01					

Debido a que Soft-DTW no garantiza una distancia nula entre una serie y sí misma, algunas métricas, como Silhouette, Dunn y COP, pueden estar levemente distorsionadas, por lo que su interpretación debe hacerse con cautela en este contexto.

Aunque Soft-DTW presenta un mejor índice Silhouette, el resto de las métricas internas de validación muestran un rendimiento claramente inferior en comparación con nDTW utilizando DBA. En particular, destaca una diferencia significativa en el índice de Calinski-Harabasz (CH), considerado una de las métricas más representativas en la evaluación de agrupamientos en estas circunstancias, lo cual refuerza la superioridad del enfoque basado en DBA.

Además, resulta especialmente preocupante el valor extremadamente bajo del índice SF en el caso de Soft-DTW, lo que sugiere una separación muy débil entre los clústeres generados (como se puede visualizar en los ejemplos).

Cabe señalar que algunas métricas, como Silhouette, Dunn y COP, pueden verse afectadas por la asimetría de la matriz de distancias, lo cual limita su fiabilidad en este contexto.

En conjunto, estos resultados respaldan que la agrupación obtenida con nDTW y DBA genera una estructura de clústeres más coherente y mejor definida.

Probemos ahora con un cluster similar al particional, *k*-Shape Clustering.

***k*-Shape Clustering**

Este método se obtiene en dtwclust al usar la distancia Shape-Based (SBD) y el centroeide shape extraction.

```
set.seed(42)
cl_sbd_3 = tsclust(serie_new_cases_z, type = "partitional", k = 3, distance =
  ↪ "sbd", centroid = "shape", seed = 12345)
cl_sbd_3
```

```
partitional clustering with 3 clusters
Using sbd distance
Using shape centroids
Using zscore preprocessing
```

```
Time required for analysis:
  user  system elapsed
0.42    0.03    0.39
```

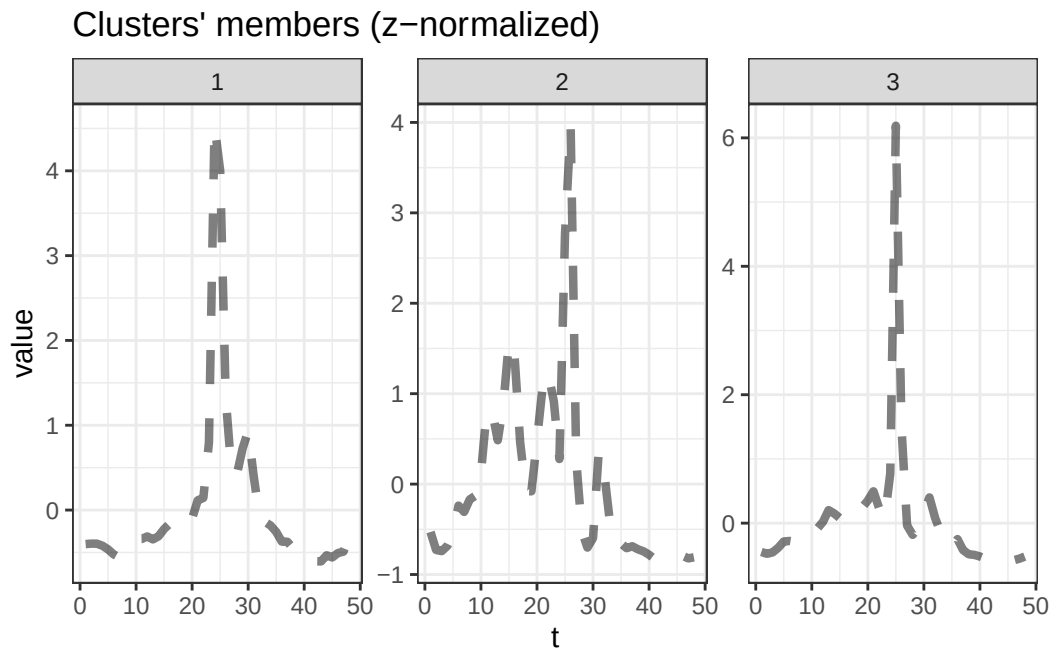
```
Cluster sizes with average intra-cluster distance:
```

```
size  av_dist
```



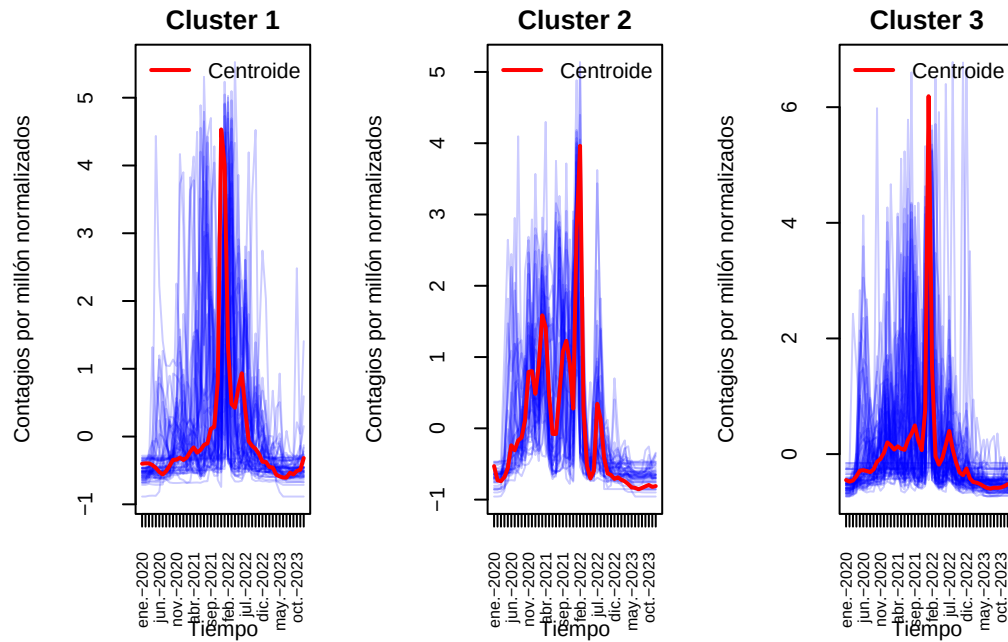
```
1 63 0.1729161
2 43 0.2124054
3 86 0.1847209
```

```
plot(cl_sbd_3,type="centroids")
```



Cabe destacar la velocidad tan alta a la que se ha realizado la agrupación. Además los centroides vuelven a estar relacionados con nDTW usando DBA. Veamos las representaciones necesarias.

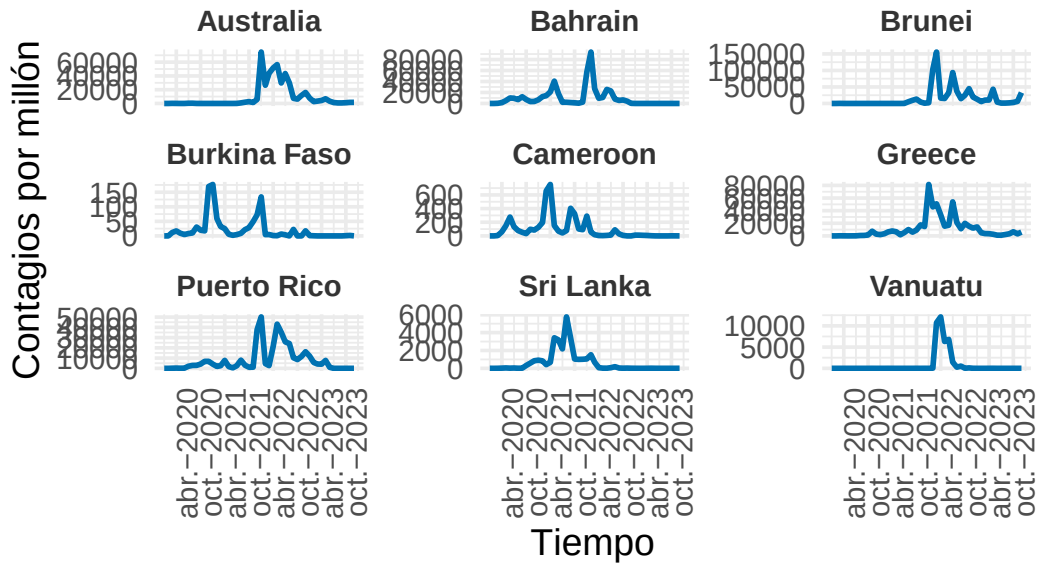
```
visualizar_series_centroides(serie_new_cases_z,cl_sbd_3,3,ylab="Contagios por  
↪ millón normalizados")
```



Corroboramos la información anterior, veamos algunos ejemplos.

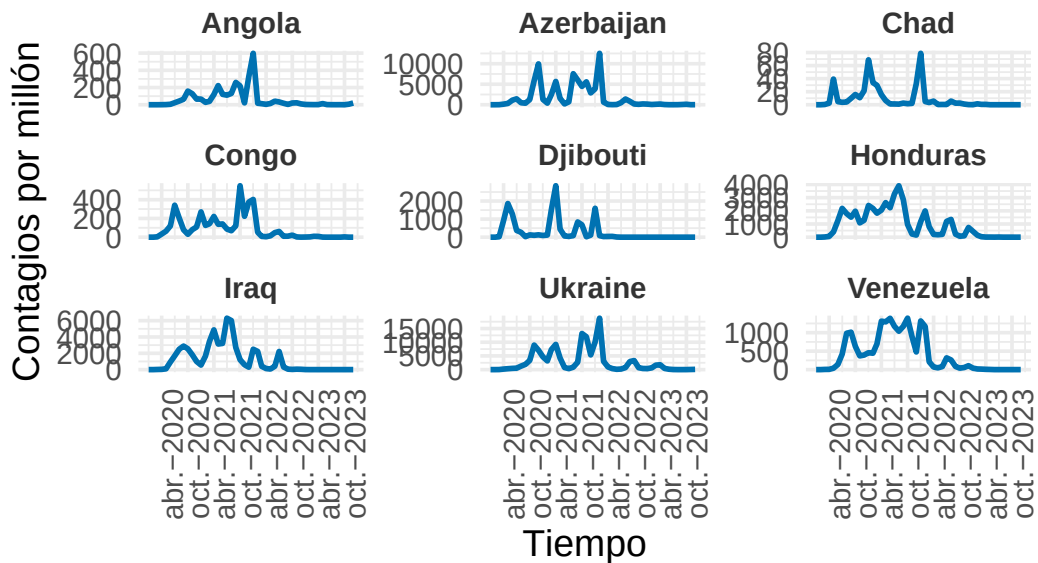
```
visualizar_ejemplos(serie_new_cases,cl_sbd_3,1,9,c(3,3),42,ylab="Contagios
↳ por millón")
```

Ejemplos del Cluster 1



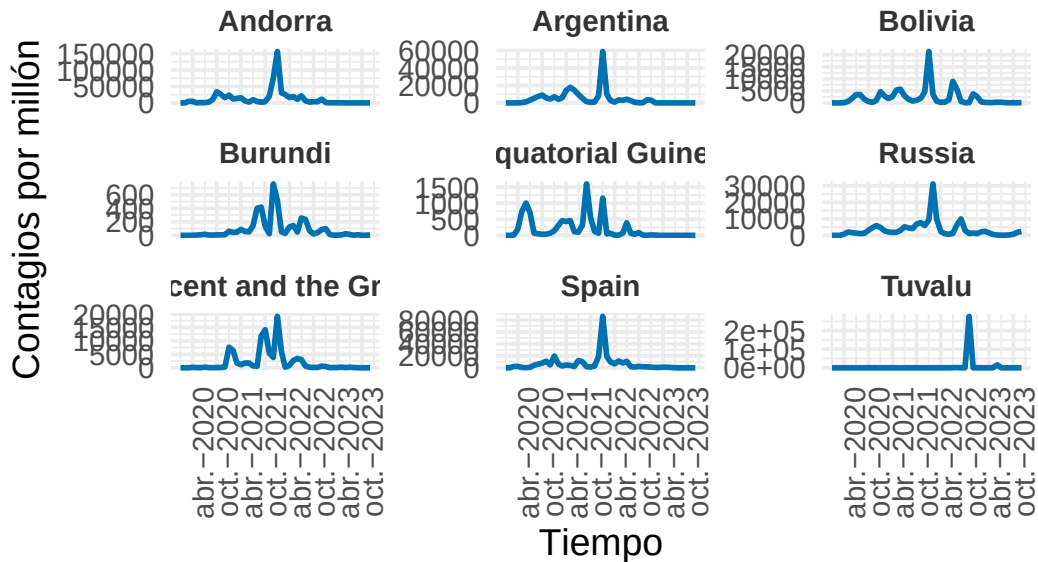
```
visualizar_ejemplos(serie_new_cases,cl_sbd_3,2,9,c(3,3),42,ylab="Contagios
↳ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_cases,cl_sbd_3,3,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 3



En los ejemplos se observa que pese a que en el grupo 2 parece haber más homogeneidad, no se observan patrones claros en los grupos 1 y 3 más halla de que tienen un gran pico. Veremos si en las métricas se refleja dicha similitud.

```
set.seed(42)
cvi(cl_ndtw_dba_3,type="internal")
```

Si1	SF	CH	DB	DBstar	D
0.04234107	0.46096160	101.12711703	1.35249337	1.40840462	0.13057901
COP					
0.62831404					

```
cvi(cl_sbd_3,type="internal")
```

Si1	SF	CH	DB	DBstar	D
0.12244222	0.44188563	46.68591316	2.06324740	2.20372386	0.02101961
COP					
0.36785978					

Efectivamente se observan mejores resultados en casi todos los coeficientes, destacando el más representativo en este caso, el índice de Calinski-Harabasz. En esta métrica observamos que el CH obtenido con esta partición es aproximadamente la mitad que en el cluster con distancia nDTW y centroides hallados por DBA.

Pasemos ahora a otro tipo de Clustering similar al particional, TADPole Clustering.

TADPole Clustering

Detalles de cómo funciona este algoritmo se pueden consultar en la memoria. Debemos dar un valor de cutoff distance (dc) y un valor de ventana. El valor de dc se refiere a cuán cercano consideramos vecino entre series. Usaremos el percentil 20 de todas las distancias. TADPole usa internamente la distancia DTW.

```
# Distancias DTW sobre nuestros datos normalizados
dist_matrix_dtw = proxy::dist(serie_new_cases_z, method = "dtw_basic")
# Cálculo del percentil para dc
dc = quantile(dist_matrix_dtw, probs = 0.2)
```

En cuanto al valor ventana que tomaremos, este será en torno a un 20% de la longitud de nuestras series.

```
window = floor(length(serie_new_cases_z[[1]])*20/100)
window
```

```
[1] 9
```

Ahora estamos en condiciones de utilizar el método TADPole.

```
set.seed(42)
cl_tadpole_3 = tsclust(serie_new_cases_z, k = 3, type = "t", trace = TRUE,
control = tadpole_control(dc = dc, window.size = window), seed = 12345)
```

```
Entering TADPole...
```

```
Computing lower and upper bound matrices
Pruning during local density calculation
Pruning during nearest-neighbor distance calculation (phase 1)
Pruning during nearest-neighbor distance calculation (phase 2)
Pruning percentage = 20.9%
Performing cluster assignment
```

TADPole completed for $k = 3$ & $dc = 18.4$

Elapsed time is 0.14 seconds.

```
cl_tadpole_3
```

tadpole clustering with 3 clusters
Using LB+DTW2 distance
Using PAM (TADPole) centroids

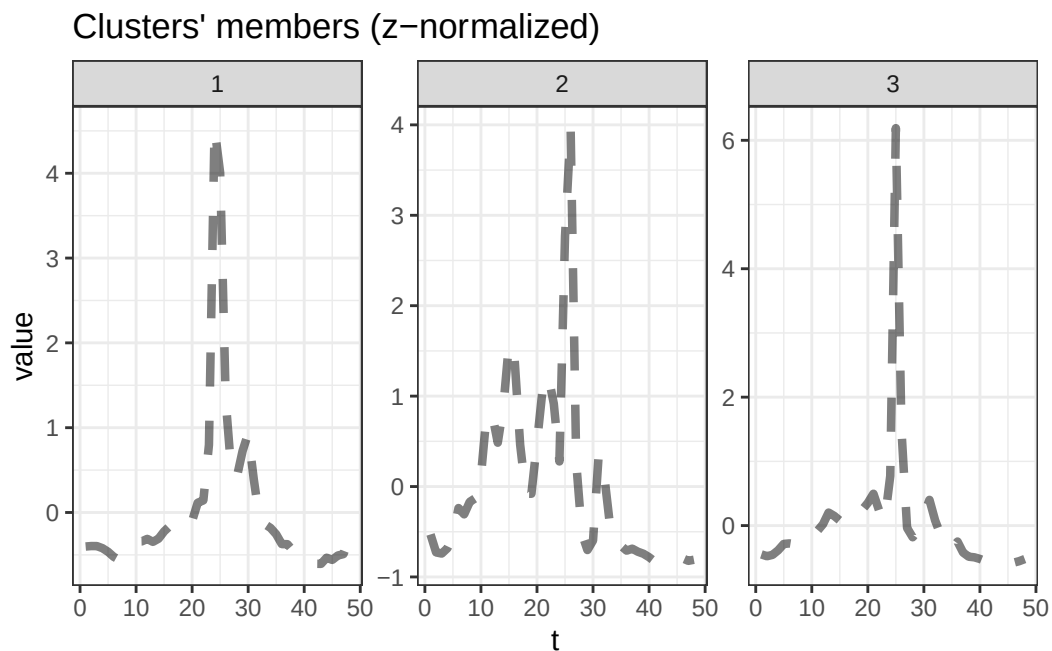
Time required for analysis:

	user	system	elapsed
	0.28	0.02	0.14

Cluster sizes with average intra-cluster distance:

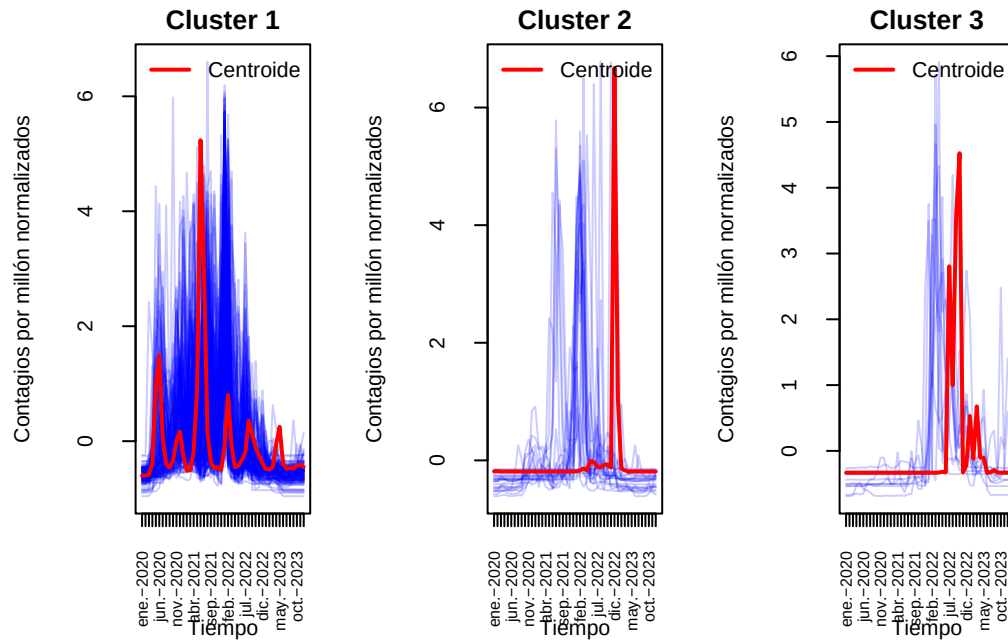
	size	av_dist
1	158	4.159463
2	23	6.072767
3	11	3.754293

```
plot(cl_sbd_3,type="centroids")
```



Nuevamente parece que hemos obtenido centroides con forma similar al mejor cluster hasta el momento. Representemos todas las series también.

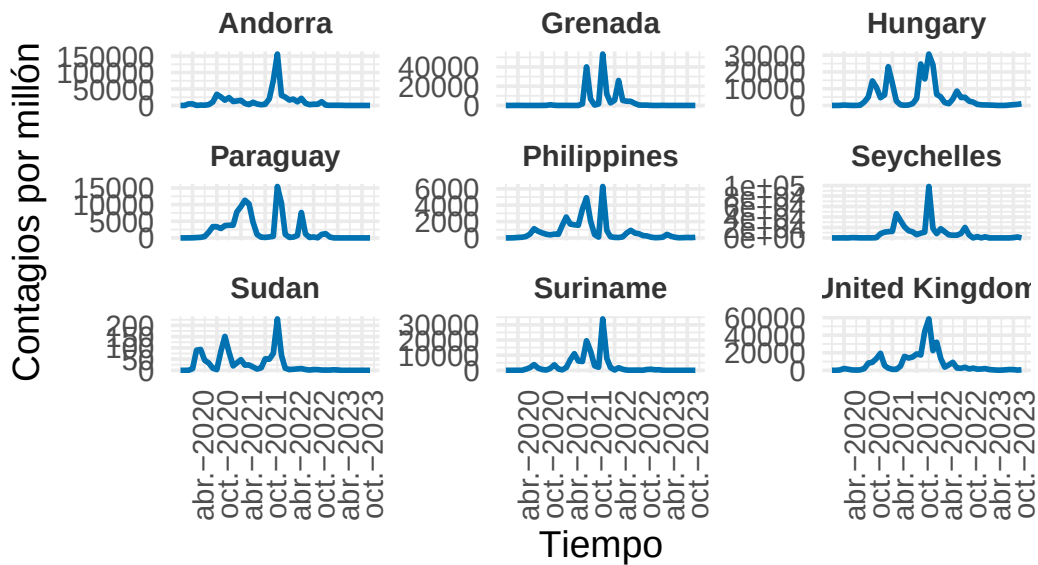
```
visualizar_series_centroides(serie_new_cases_z,cl_tadpole_3,3,ylab="Contagios  
↪ por millón normalizados")
```



Posiblemente tengamos una gran variabilidad interna en el primer grupo al tener la mayor parte de la población. Veamos algunos ejemplos.

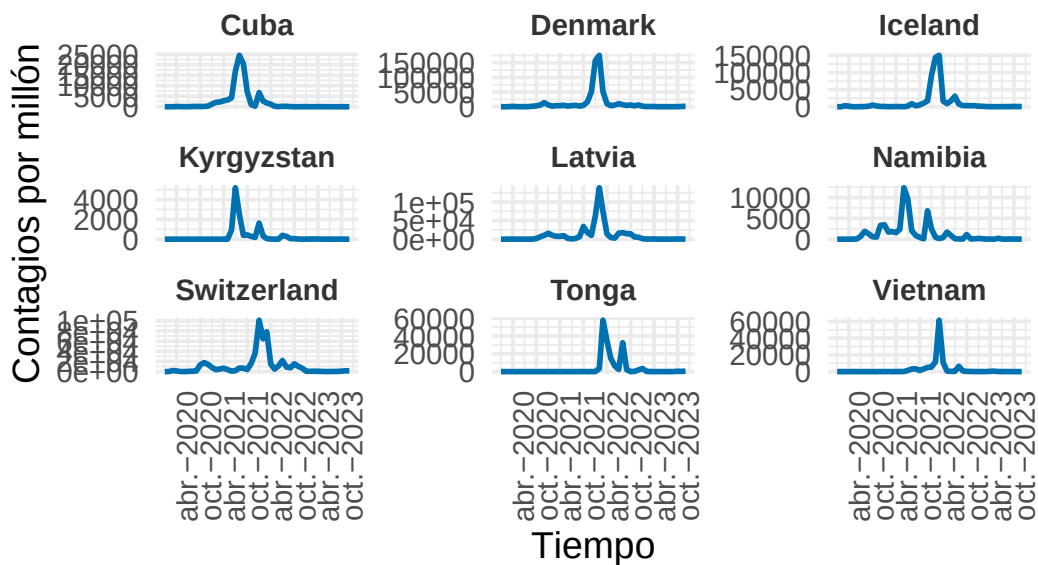
```
visualizar_ejemplos(serie_new_cases,cl_tadpole_3,1,9,c(3,3),42,ylab="Contagios  
↪ por millón")
```

Ejemplos del Cluster 1



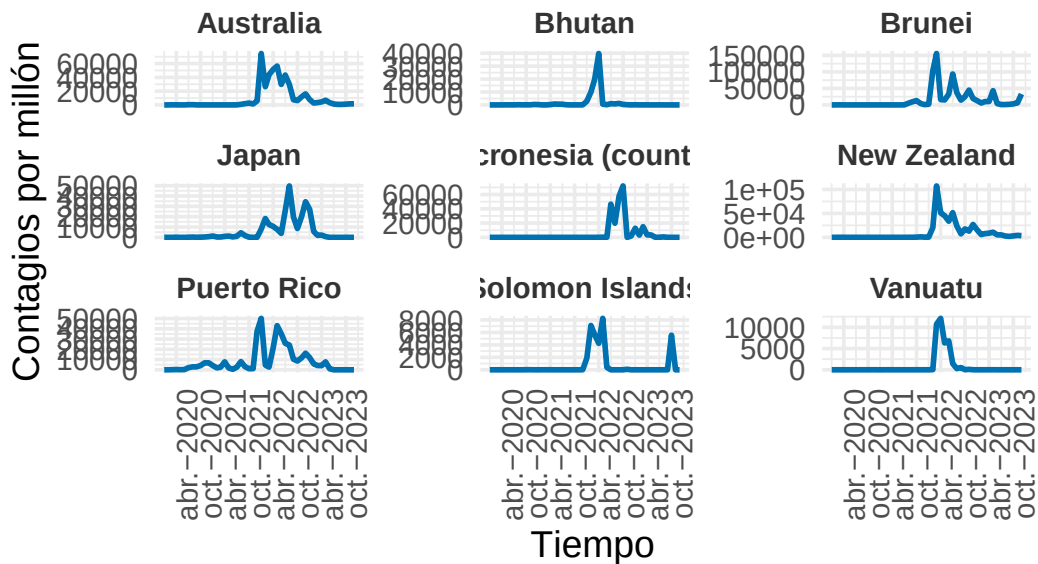
```
visualizar_ejemplos(serie_new_cases,cl_tadpole_3,2,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 2




```
visualizar_ejemplos(serie_new_cases,cl_tadpole_3,3,9,c(3,3),42,ylab="Contagios  
→ por millón")
```

Ejemplos del Cluster 3



Se observa una variabilidad interna clara en el primer grupo como era de esperar. En cuanto al segundo y tercer grupo, parecen muy similares al obtenido usando nDTW con DBA. Veamos en las métricas si se refleja esa variabilidad interna que se observa en los datos del cluster 1.

```
set.seed(42)  
cvi(cl_ndtw_dba_3,type="internal")
```

Sil	SF	CH	DB	DBstar	D
0.04234107	0.46096235	101.13033768	1.35249337	1.40840462	0.13057901
COP					
0.62831404					

```
cvi(cl_tadpole_3,type="internal")
```

Sil	SF	CH	DB	DBstar	D
2.842881e-01	1.453311e-06	3.538404e+01	3.214234e+00	3.300986e+00	3.634037e-03
COP					
8.532753e-01					

Observando la métrica fiable, el índice CH y SF, vemos que es significativamente mejor en el cluster usando nDTW y DBA, es decir seguimos observando y comprobando con estos índices que el mejor clúster hasta el momento es el que obtuvimos con nDTW y DBA. Pasemos ahora con un enfoque distinto, Fuzzy Clustering.

Fuzzy Clustering

Distancia nDTW

Usaremos ahora el método Fuzzy para hacer clustering. Detalles de dicho método pueden consultarse en la memoria.

```
set.seed(42)
cl_fuzzy_ndtw_3 = tsclust(serie_new_cases_z, type = "fuzzy", k = 3, distance =
  ↪ "ndtw", centroid = "fcddd", seed = 12345)
cl_fuzzy_ndtw_3
```

fuzzy clustering with 3 clusters

Using ndtw distance

Using fcddd centroids

Time required for analysis:

user	system	elapsed
36.06	0.56	38.01

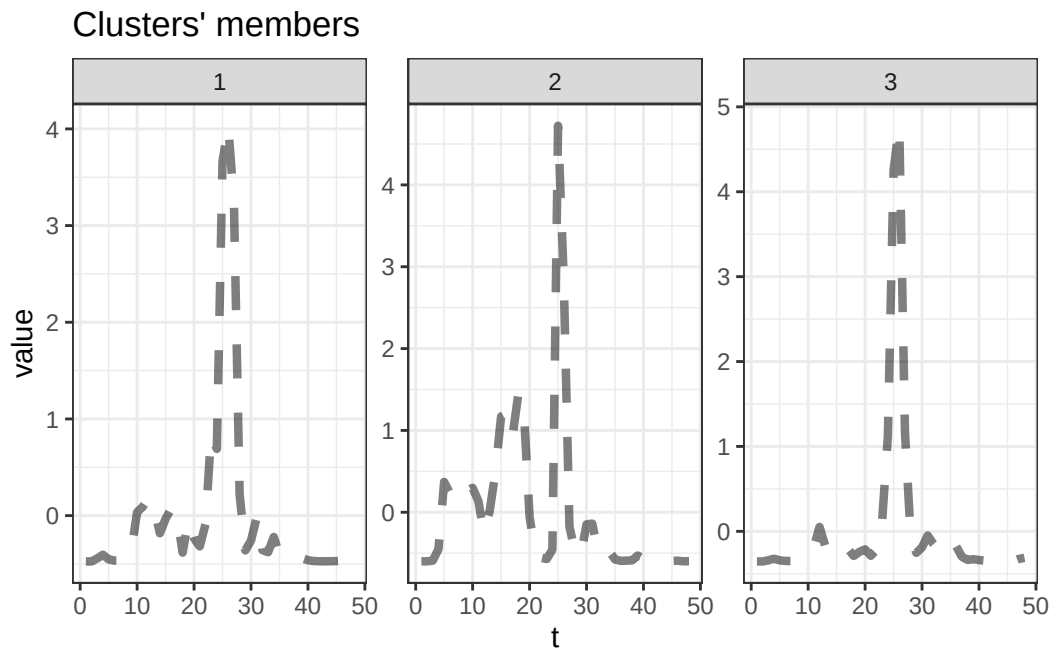
Head of fuzzy memberships:

	cluster_1	cluster_2	cluster_3
Afghanistan	0.4621098	0.2454305	0.2924596
Albania	0.3465260	0.4418097	0.2116643
Algeria	0.3078373	0.5035715	0.1885913
Andorra	0.5403627	0.1548687	0.3047687
Angola	0.3036125	0.4999648	0.1964227
Antigua and Barbuda	0.3405284	0.3549312	0.3045403

```
table(cl_fuzzy_ndtw_3@cluster)
```

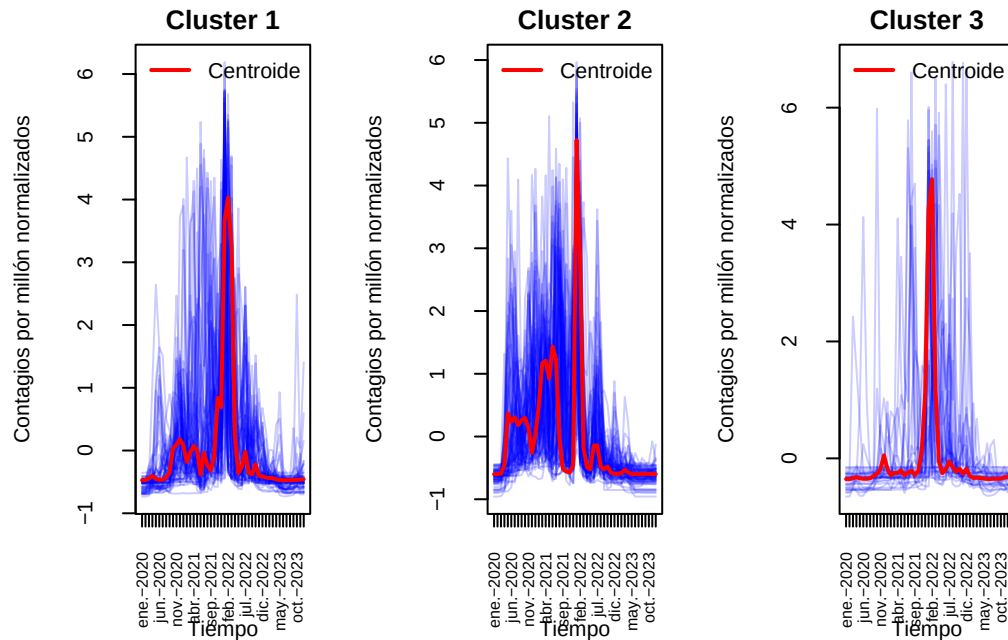
```
1 2 3
71 86 35
```

```
plot(cl_fuzzy_ndtw_3,type="centroids")
```



En este caso como era de esperar por el tipo de algoritmo que es fuzzy obtenemos una probabilidad de pertenencia a cada cluster. En los centroides vemos cierta similitud entre el primer y tercer cluster. Veamos los gráficos con las series.

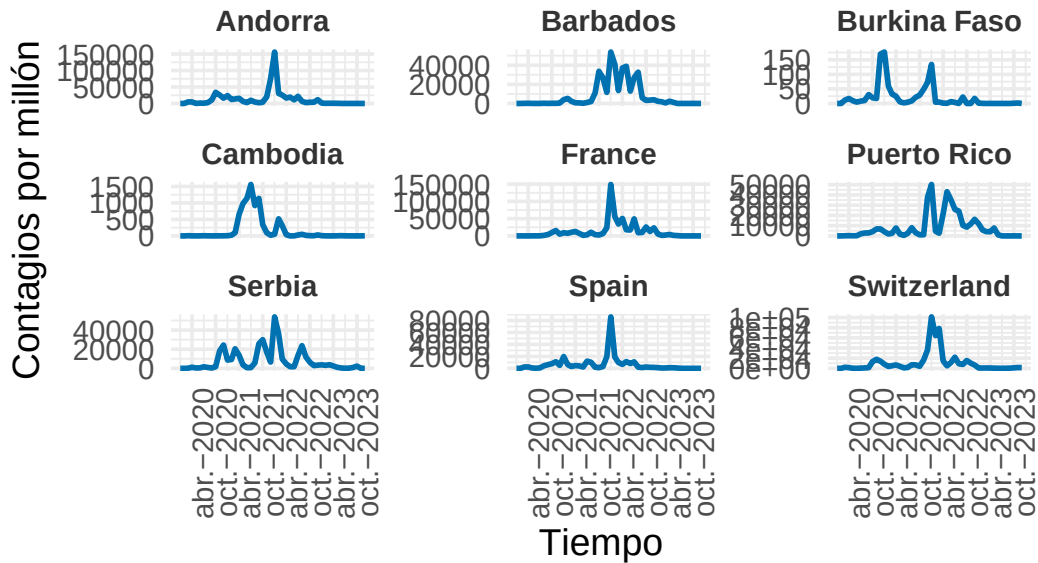
```
visualizar_series_centroides(serie_new_cases_z,cl_fuzzy_ndtw_3,3,ylab="Contagios  
↪ por millón normalizados")
```



Los gráficos nos muestran que parece que tenemos un agrupamiento bien separado. Un primer grupo que además del gran pico presenta pequeños rebrotes antes y/o después del mismo. Un segundo grupo con varios brotes y un tercer grupo con únicamente un gran pico de contagios. Veamos algunos ejemplos.

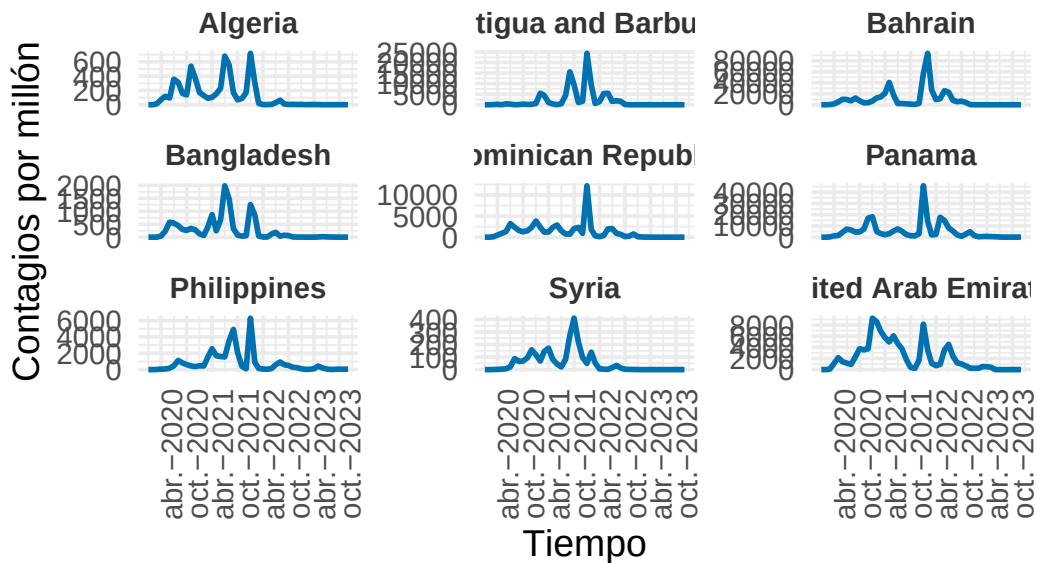
```
visualizar_ejemplos(serie_new_cases,cl_fuzzy_ndtw_3,1,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 1



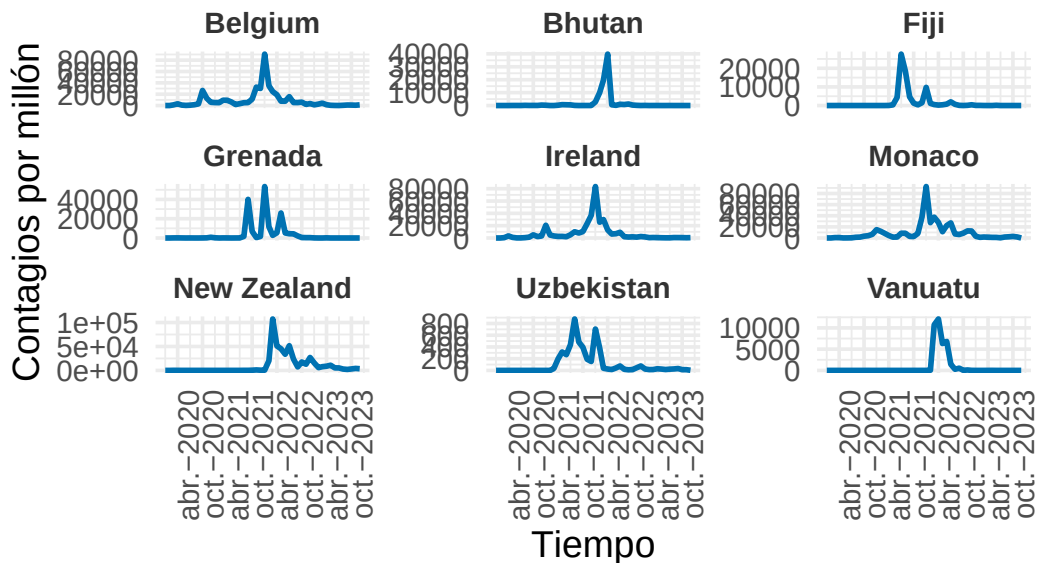
```
visualizar_ejemplos(serie_new_cases,cl_fuzzy_ndtw_3,2,9,c(3,3),42,ylab="Contagios
↳ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_cases,cl_fuzzy_ndtw_3,3,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 3



En los ejemplos se puede observar que pese a la homogeneidad del cluster 2, el cluster 1 parece solaparse con el 3, por ello posiblemente en las métricas importantes veremos que el cluster particional será mejor que este.

```
set.seed(42)
cvi(cl_ndtw_dba_3,type="internal")
```

Sil	SF	CH	DB	DBstar	D
0.04234107	0.46096160	101.12711703	1.35249337	1.40840462	0.13057901
COP					
0.62831404					

```
cvi(cl_fuzzy_ndtw_3,type="internal")
```

MPC	K	T	SC	PBMF
0.09579699	306.44322165	13.73668178	-1.07791785	0.13010002

Se han obtenido otras métricas, pero las métricas obtenidas para el cluster fuzzy son bastante bajas e indican bastante solapamiento. Hallemos manualmente de todas formas una métrica que sí nos sirva, el índice CH.

```
library(clusterCrit)
# Convertimos la lista de series en una matriz
data_matrix = do.call(rbind, serie_new_cases_z)

# Calculamos el índice Calinski-Harabasz
ch_fuzzy_ndtw = intCriteria(as.matrix(data_matrix), cl_fuzzy_ndtw_3@cluster,
  ↪ "Calinski_Harabasz")
ch_fuzzy_ndtw
```

```
$calinski_harabasz
[1] 7.930537
```

Efectivamente obtenemos un valor mucho más bajo lo que refuerza la idea de que el cluster particional que obtuvimos es bastante mejor. Usemos ahora la distancia Global Alignment Kernel.

Distancia Global Alignment Kernel

```
set.seed(42)
cl_fuzzy_gak_3 = tsclust(serie_new_cases_z, type = "fuzzy", k =3, distance =
  ↪ "gak", centroid = "fcddd", seed = 12345)
cl_fuzzy_gak_3
```

```
fuzzy clustering with 3 clusters
Using gak distance
Using fcddd centroids
```

```
Time required for analysis:
  user  system elapsed
45.49   0.20    3.04
```

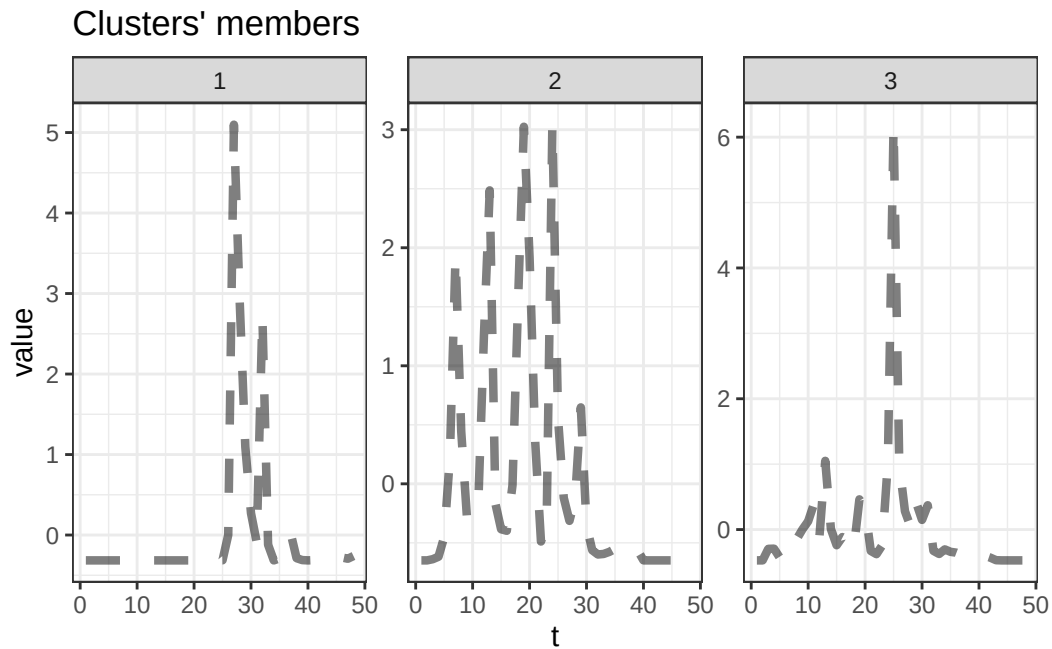
Head of fuzzy memberships:

	cluster_1	cluster_2	cluster_3
Afghanistan	0.0482546150	0.7957257191	0.156019666
Albania	0.0339115334	0.3455750965	0.620513370
Algeria	0.0004867427	0.9957554989	0.003757758
Andorra	0.0004133269	0.0002803841	0.999306289
Angola	0.0149506094	0.6723746339	0.312674757
Antigua and Barbuda	0.0290577860	0.0311337420	0.939808472

```
table(cl_fuzzy_gak_3@cluster)
```

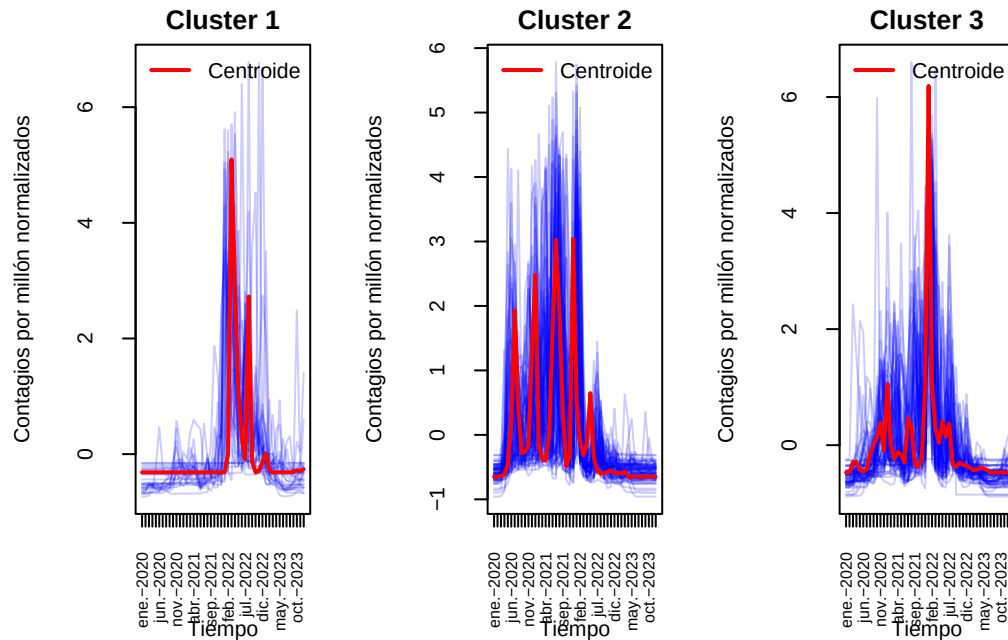
```
1 2 3  
29 85 78
```

```
plot(cl_fuzzy_gak_3,type="centroids")
```



Esta vez parece que hemos obtenido centroides más similares al de nuestro mejor cluster hasta el momento. representemos el resto de series también.

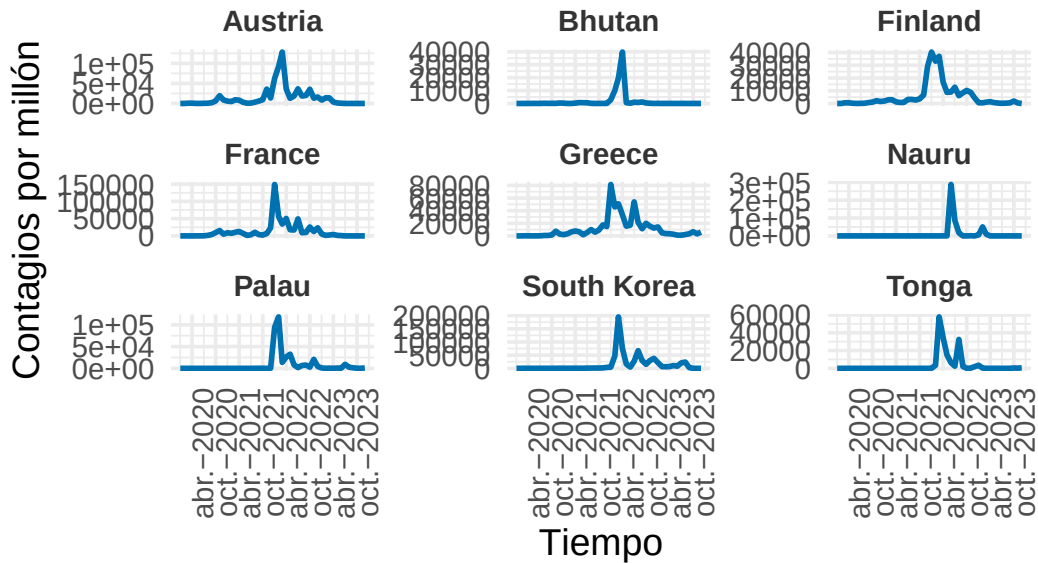
```
visualizar_series_centroides(serie_new_cases_z,cl_fuzzy_gak_3,3,ylab="Contagios  
↪ por millón normalizados")
```

En la gráfica también parece que los centroides son representativos. Veamos algunos ejemplos.

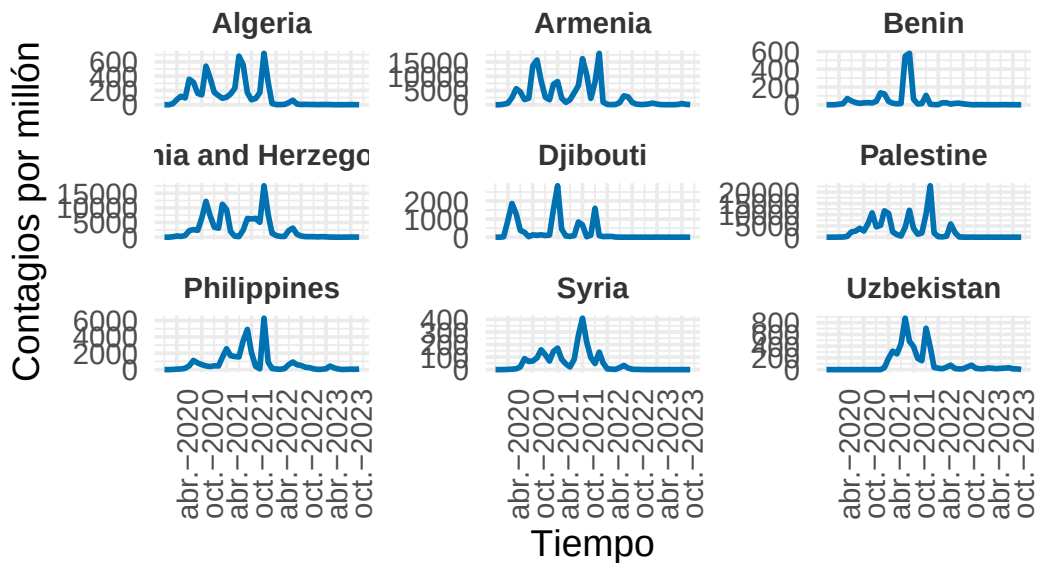
```
visualizar_ejemplos(serie_new_cases,cl_fuzzy_gak_3,1,9,c(3,3),42,ylab="Contagios
↳ por millón")
```

Ejemplos del Cluster 1



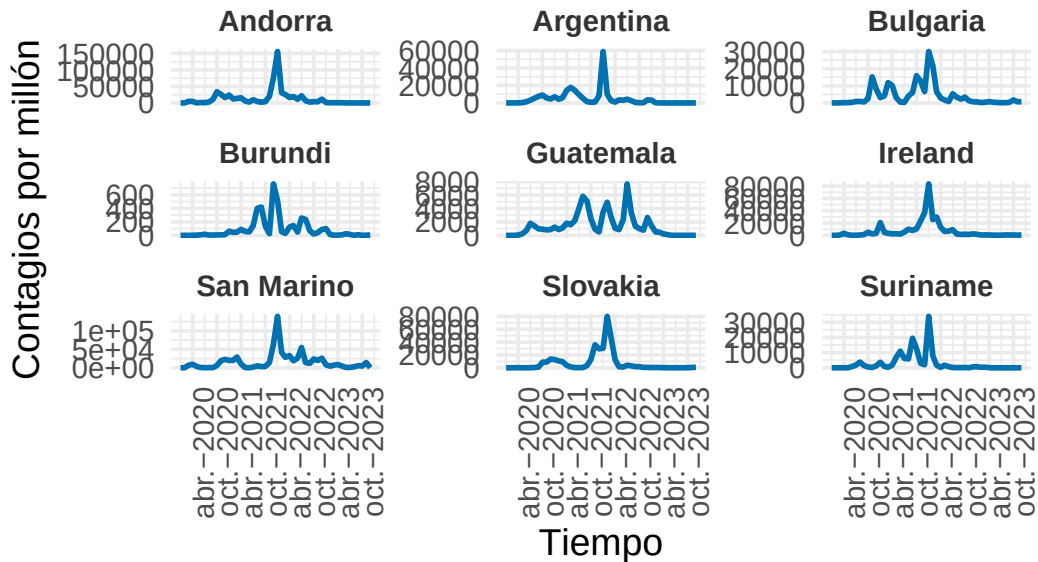
```
visualizar_ejemplos(serie_new_cases,cl_fuzzy_gak_3,2,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_cases,cl_fuzzy_gak_3,3,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 3



Se observa cierta heterogeneidad en el grupo 3 y una diferencia no muy significativa en observaciones entre el cluster 1 y 3. Aún así los clusters parecen tener sentido. Usemos criterios más objetivos como las métricas que miden la calidad del cluster.

```
set.seed(42)
cvi(cl_ndtw_dba_3,type="internal")
```

Si1	SF	CH	DB	DBstar	D
0.04234107	0.46096160	101.12711703	1.35249337	1.40840462	0.13057901
COP					
0.62831404					

```
cvi(cl_fuzzy_gak_3,type="internal")
```

MPC	K	T	SC	PBMF
5.996724e-01	5.918831e+01	4.867322e-03	2.315551e+00	3.129693e-04

```
# Calculamos el índice Calinski-Harabasz
ch_fuzzy_gak = intCriteria(as.matrix(data_matrix), cl_fuzzy_gak_3@cluster,
  ↪ "Calinski_Harabasz")
ch_fuzzy_gak
```

```
$calinski_harabasz
[1] 23.50355
```

En este caso las métricas obtenidas sobre el cluster Fuzzy son bastante mejores respecto al cluster anterior. Hemos conseguido un MPC de 0.6 más cercano a que 0.1, esto indica menor solapamiento que en el cluster anterior. Un valor de SC de 2.31 que ya es mayor que 1 indicando separación significativa (antes obtuvimos -1). Tiene sentido que obtengamos esto pues el cluster obtenido ahora presenta similitudes con el mejor cluster hasta el momento.

Sin embargo, comparando las métricas CH el cluster particional usando nDTW y DBA sigue presentando valores significativamente mayores por lo que seguiremos usando este cluster como el mejor. Aún así vemos como la distancia GAK nos ha ayudado a obtener un agrupamiento decente usando el método Fuzzy de clustering. Pasemos ahora a cluster jerárquico.

Hierarchical clustering

Debemos usar métricas simétricas en este caso.

Distancia DTW

```
library(cluster)
cl_hc_dtw_3 = tsclust(serie_new_cases_z, type = "hierarchical", k = 3,
  distance = "dtw_basic", control = hierarchical_control(method = diana),
  args = tsclust_args(dist = list(window.size = window)))
cl_hc_dtw_3
```

```
hierarchical clustering with 3 clusters
Using dtw_basic distance
Using PAM (Hierarchical) centroids
Using method diana
```

Time required for analysis:

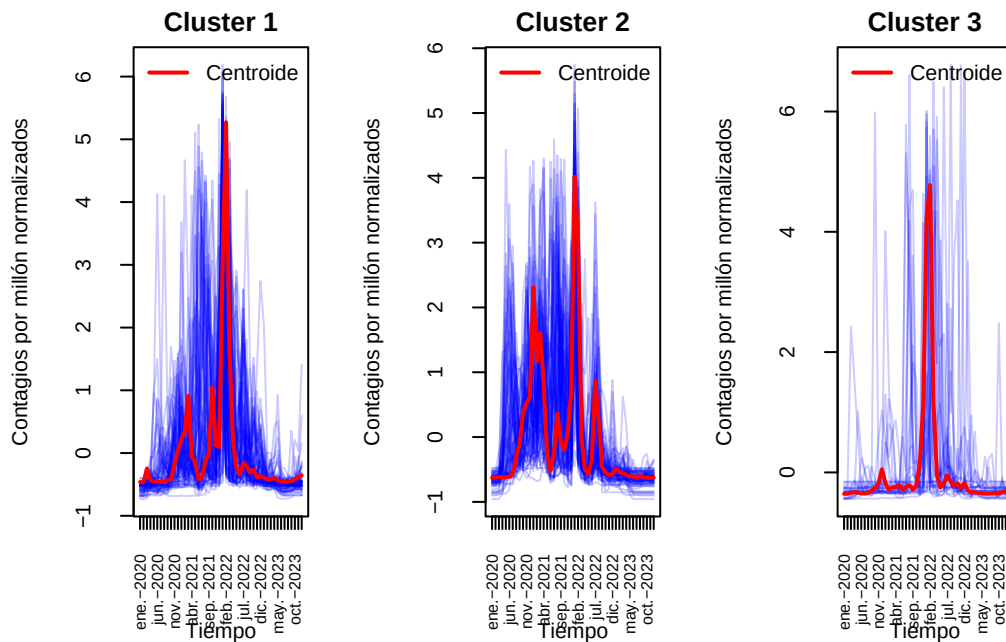
```
user  system elapsed
1.33   0.00   1.42
```

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	83	16.91218
2	80	19.39697
3	29	14.10043

Como siempre, veamos las series junto a los centroides correspondientes.

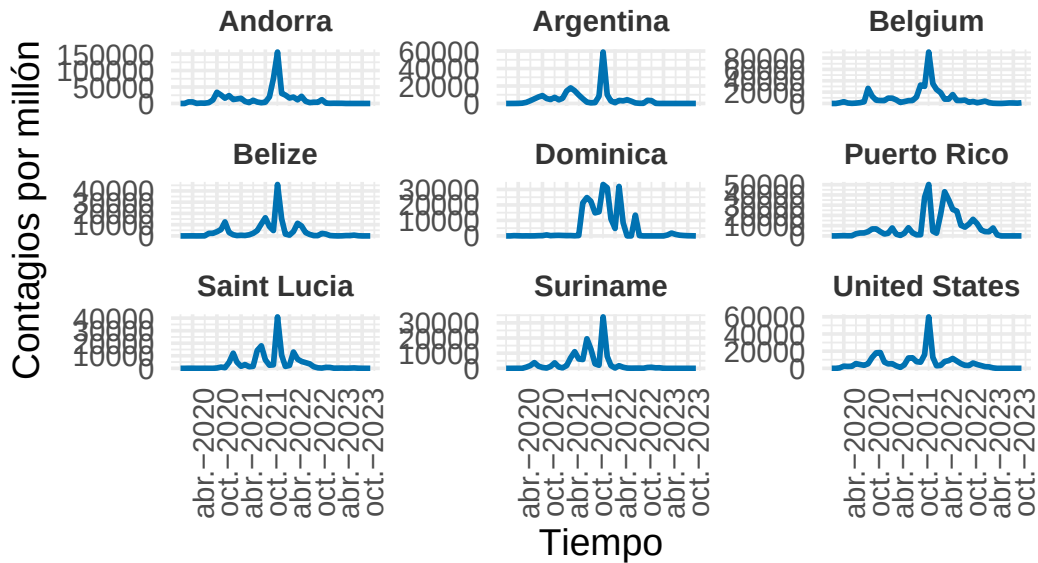
```
visualizar_series_centroides(serie_new_cases_z,cl_hc_dtw_3,3,ylab="Contagios
↳ por millón normalizados")
```



Parece haber variabilidad en los cluster 1 y 2, pero las agrupaciones tienen sentido y son en realidad similares a las de nuestro mejor cluster. Veamos algunos ejemplos.

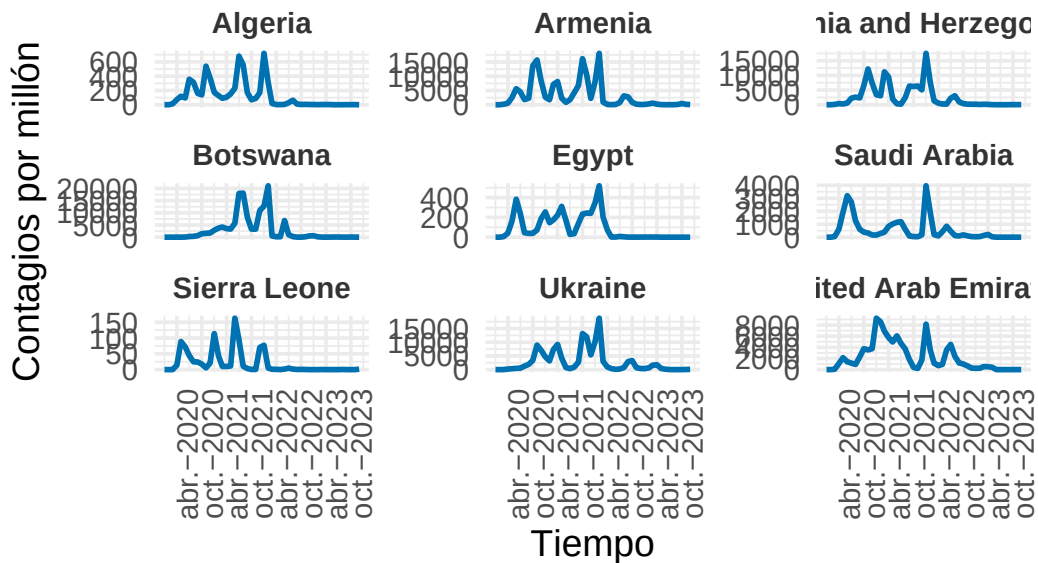
```
visualizar_ejemplos(serie_new_cases,cl_hc_dtw_3,1,9,c(3,3),42,ylab="Contagios
↳ por millón")
```

Ejemplos del Cluster 1



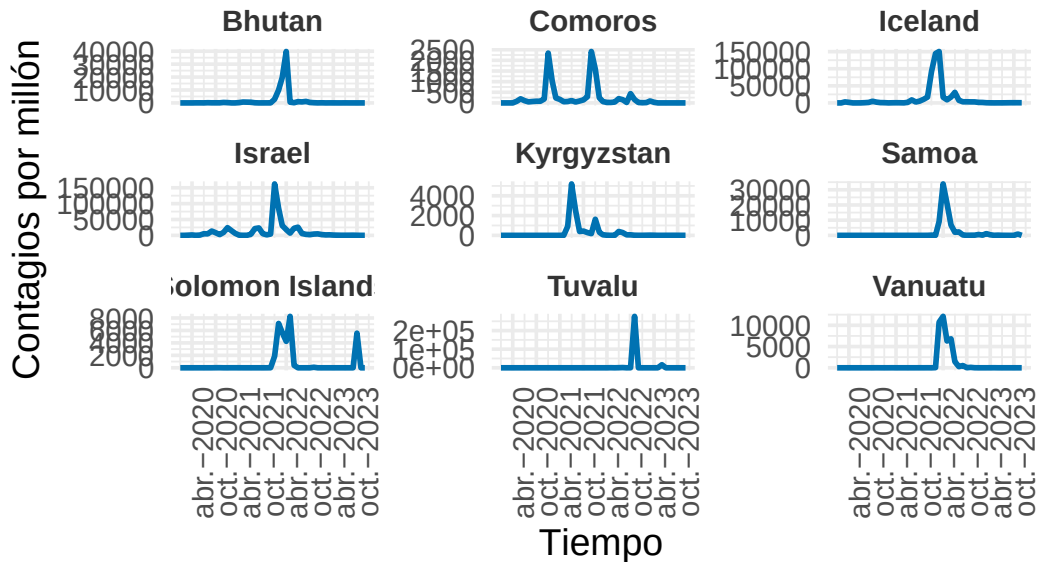
```
visualizar_ejemplos(serie_new_cases,cl_hc_dtw_3,2,9,c(3,3),42,ylab="Contagios
↳ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_cases,cl_hc_dtw_3,3,9,c(3,3),42,ylab="Contagios
↪ por millón")
```

Ejemplos del Cluster 3



Efectivamente parece confirmarse lo que decíamos anteriormente. Demos métricas objetivas de esto.

```
set.seed(42)
cvi(cl_ndtw_dba_3,type="internal")
```

Si1	SF	CH	DB	DBstar	D
0.04234107	0.46096160	101.12711703	1.35249337	1.40840462	0.13057901
COP					
0.62831404					

```
cvi(cl_hc_dtw_3,type="internal")
```

Si1	SF	CH	DB	DBstar	D	COP
0.1242725	0.0000000	75.3088840	2.0927718	2.2854443	0.2152783	0.4870100

Obtenemos algo que cabía esperar. Como los clusters tienen sentido obtenemos un valor de CH similar al que obtuvimos con la mejor agrupación. De igual forma el cluster considerado mejor hasta ahora sigue siendo mejor. Probemos otra distancia.

Distancia Shape-Based

```
cl_hc_sbd_3 = tsclust(serie_new_cases_z, type = "hierarchical", k = 3,  
distance = "sbd", centroid = shape_extraction, control =  
  ↪ hierarchical_control(method = diana),  
args = tsclust_args(dist = list(window.size = window)))  
cl_hc_sbd_3
```

hierarchical clustering with 3 clusters

Using sbd distance

Using shape_extraction centroids

Using method diana

Time required for analysis:

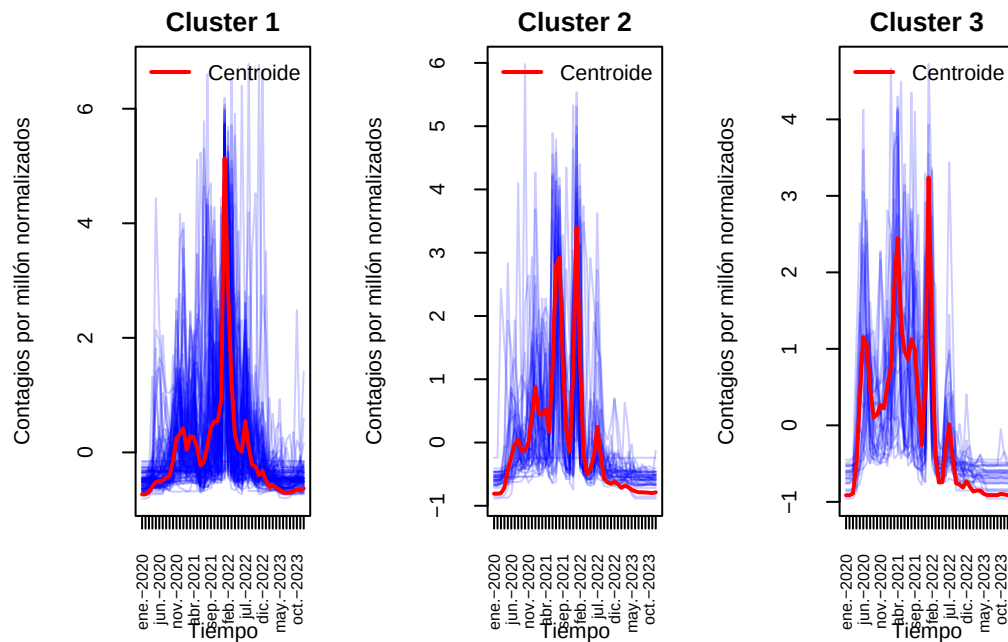
	user	system	elapsed
	0.21	0.02	0.05

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	121	0.2002054
2	42	0.2349907
3	29	0.2557372

Representemos la series junto a los centroides.

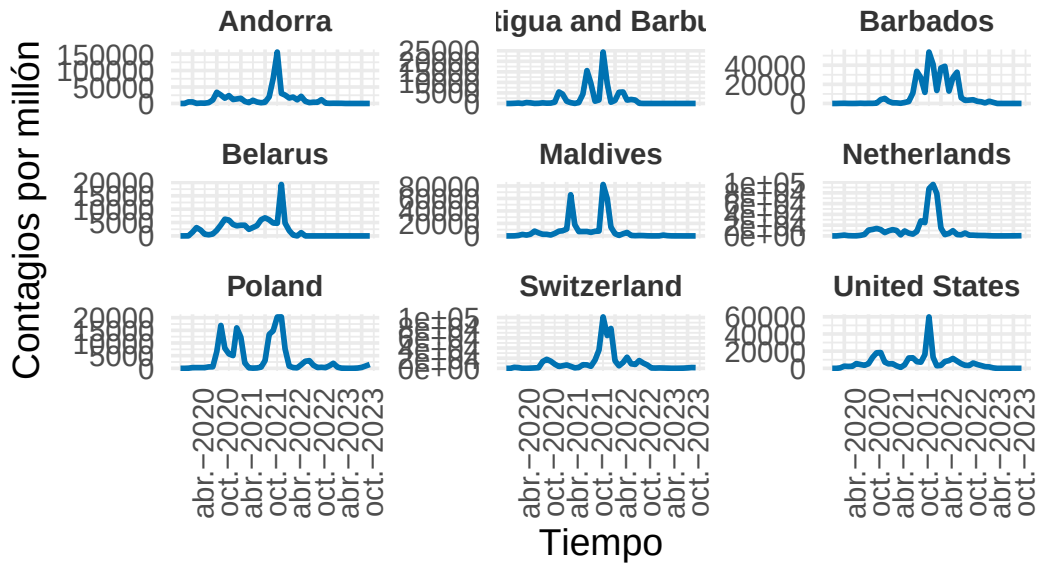
```
visualizar_series_centroides(serie_new_cases_z,cl_hc_sbd_3,3,ylab="Contagios  
  ↪ por millón normalizados")
```

Se observa una mayor variabilidad en general. Veamos algunos ejemplos.

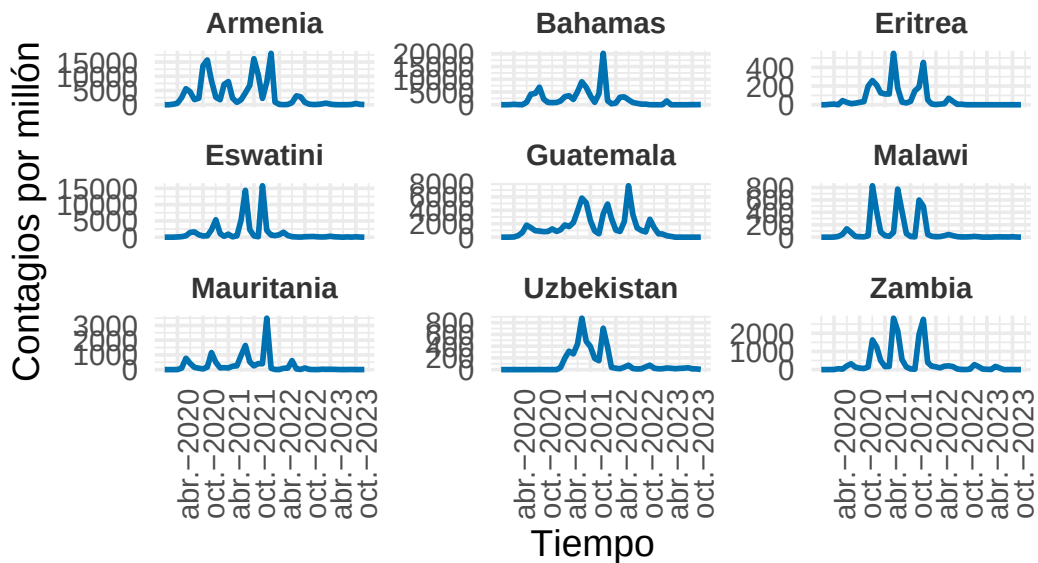
```
visualizar_ejemplos(serie_new_cases,cl_hc_sbd_3,1,9,c(3,3),42,ylab="Contagios
↳ por millón")
```

Ejemplos del Cluster 1



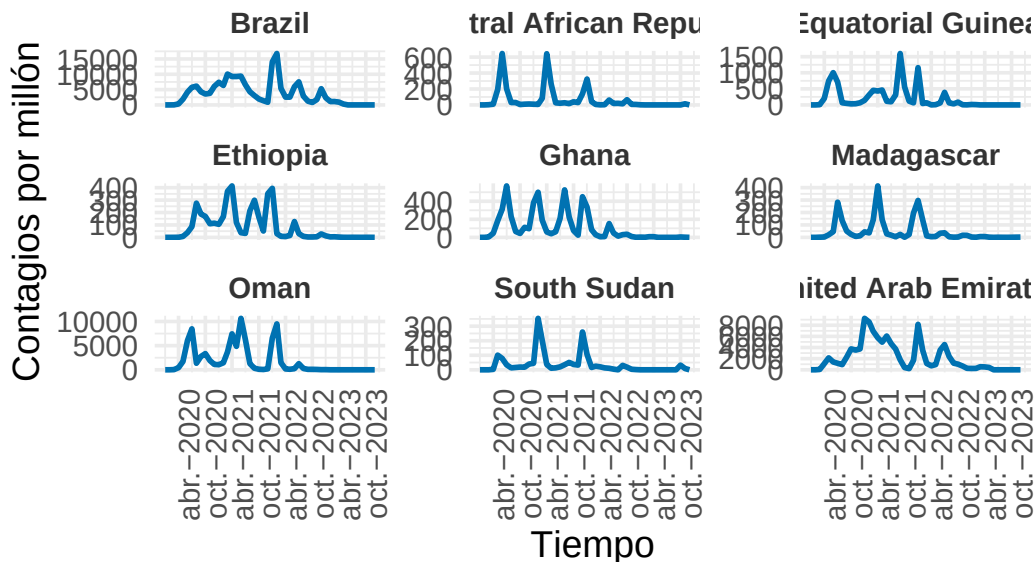
```
visualizar_ejemplos(serie_new_cases,cl_hc_sbd_3,2,9,c(3,3),42,ylab="Contagios
↳ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_cases,cl_hc_sbd_3,3,9,c(3,3),42,ylab="Contagios  
↪ por millón")
```

Ejemplos del Cluster 3



Se observa un agrupamiento peor que con nuestro mejor candidato. Veámoslo en las métricas.

```
set.seed(42)
cvi(cl_ndtw_dba_3,type="internal")
```

Sil	SF	CH	DB	DBstar	D
0.04234107	0.46096160	101.12711703	1.35249337	1.40840462	0.13057901
COP					
0.62831404					

```
cvi(cl_hc_sbd_3,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1631328	0.4033541	39.4963552	2.0770665	2.1010687	0.1067318	0.4176363

Como se preveía es mejor el agrupamiento particional con distancia nDTW y centroide usando DBA. Una vez estudiado y hallado un buen agrupamiento de países según contagios por millón, pasemos a estudiar una nueva variable muy importante, el número de muertes por millón de habitantes.

Países con trayectorias de muertes similares

Pasemos ahora a estudiar la variable de muertes. Por razones análogas al caso anterior usaremos la variable por millón de habitantes.

```
serie_new_deaths = preparar_series_temporales(df, "new_deaths_per_million")
```

Comprobemos que nuevamente las series son de longitud 48 y normalicemos.

```
unique(lengths(serie_new_deaths))
```

```
[1] 48
```

```
# Aplicar a la serie
serie_new_deaths_z = lapply(serie_new_deaths, z_score)
```

Comencemos a aplicar técnicas de clustering nuevamente.

Partitional Clustering

Probemos de nuevo con este tipo de cluster. Comencemos con la distancia asimétrica.

Distancia nDTW

Determinemos el k usando el coeficiente CH.

```
# Definimos el rango de k a evaluar
k_values = 2:6
ch_scores = numeric(length(k_values))

for (i in seq_along(k_values)) {
  k = k_values[i]
  cat("Evaluando k =", k, "...\\n")
  set.seed(42)
  cl = tsclust(serie_new_deaths_z, type = "partitional", k = k, distance =
    ↪ "ndtw", centroid = "dba", seed = 12345)

  ch = cvi(cl, type="internal")["CH"]
}
```

```
# Promedio de los coeficientes de silueta
ch_scores[i] = ch
}
```

Evaluando $k = 2 \dots$

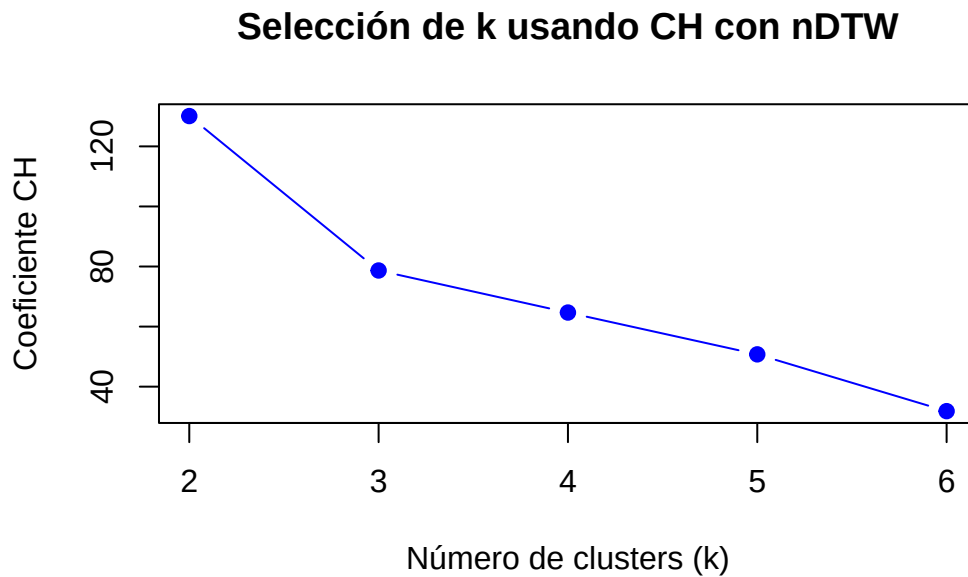
Evaluando $k = 3 \dots$

Evaluando $k = 4 \dots$

Evaluando $k = 5 \dots$

Evaluando $k = 6 \dots$

```
# Graficar los resultados
par(mfrow = c(1, 1))
plot(k_values, ch_scores, type = "b", pch = 19, col = "blue",
     xlab = "Número de clusters (k)", ylab = "Coeficiente CH",
     main = "Selección de k usando CH con nDTW")
```



Volvemos a obtener algo similar aunque ahora la diferencia entre $k = 3$ y $k = 4$ no es tan clara.

Caso $k = 2$

Al igual que antes, en este grupo esperamos obtener diferencias clara entre grupos pero una mayor variabilidad interna. Se buscan patrones generales.

```
set.seed(42)
cl_ndtw_dba_2_deaths = tsclust(serie_new_deaths_z, type = "partitional", k =
  ↪ 2, distance = "ndtw", centroid = "dba", seed = 12345)
cl_ndtw_dba_2_deaths
```

partitional clustering with 2 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

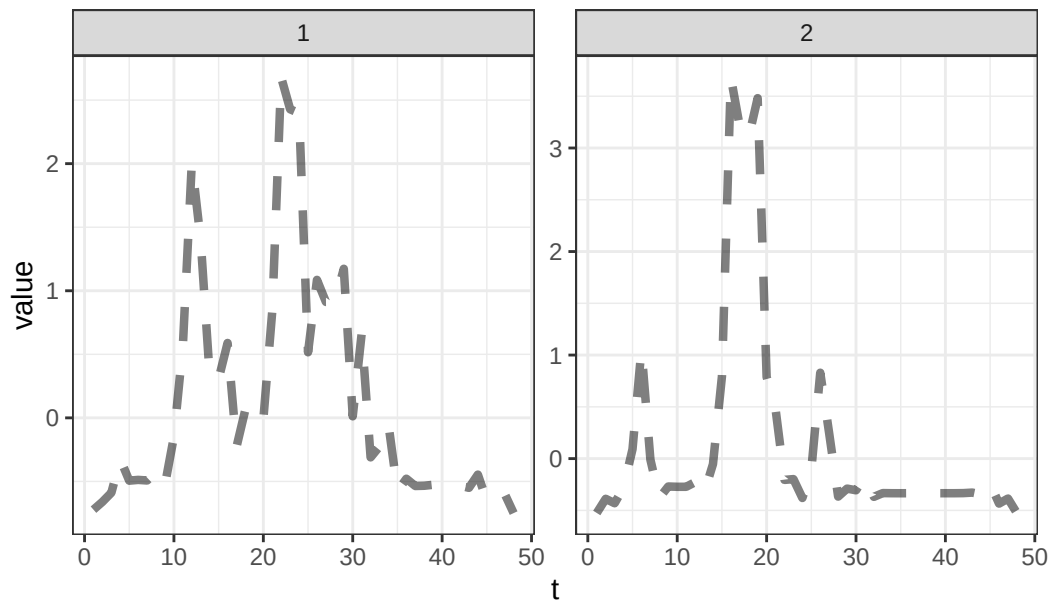
	user	system	elapsed
	2.06	0.46	1.27

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	80	0.2289586
2	105	0.1964298

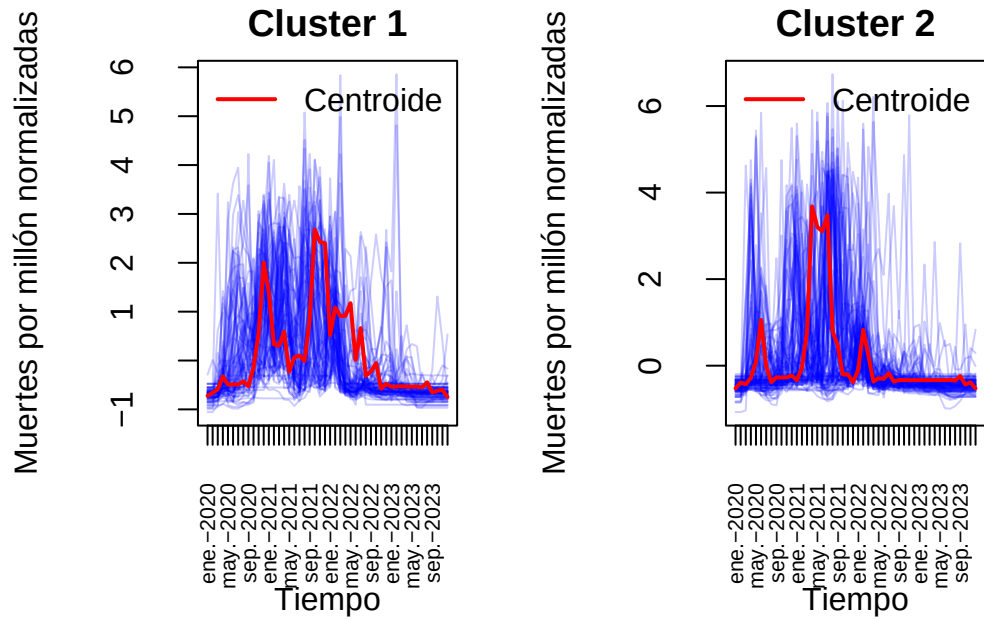
```
plot(cl_ndtw_dba_2_deaths,type="centroids")
```

Clusters' members



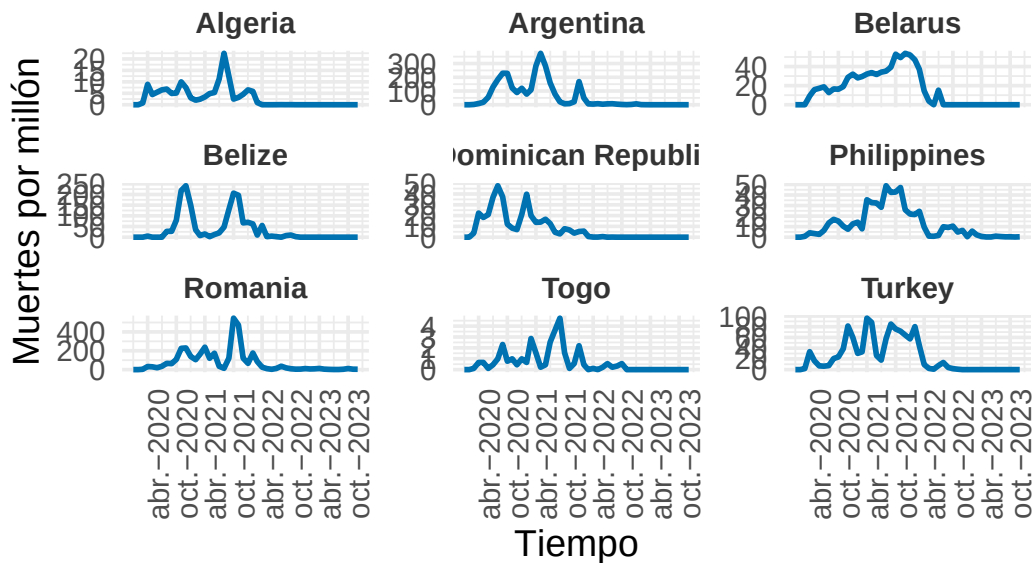
Dibujemos las series junto a los centroides y algunos ejemplos de cada cluster a ver si se ven diferencias claras.

```
visualizar_series_centroides(serie_new_deaths_z,cl_ndtw_dba_2_deaths,2,ylab="Muertes
↪ por millón normalizadas")
```



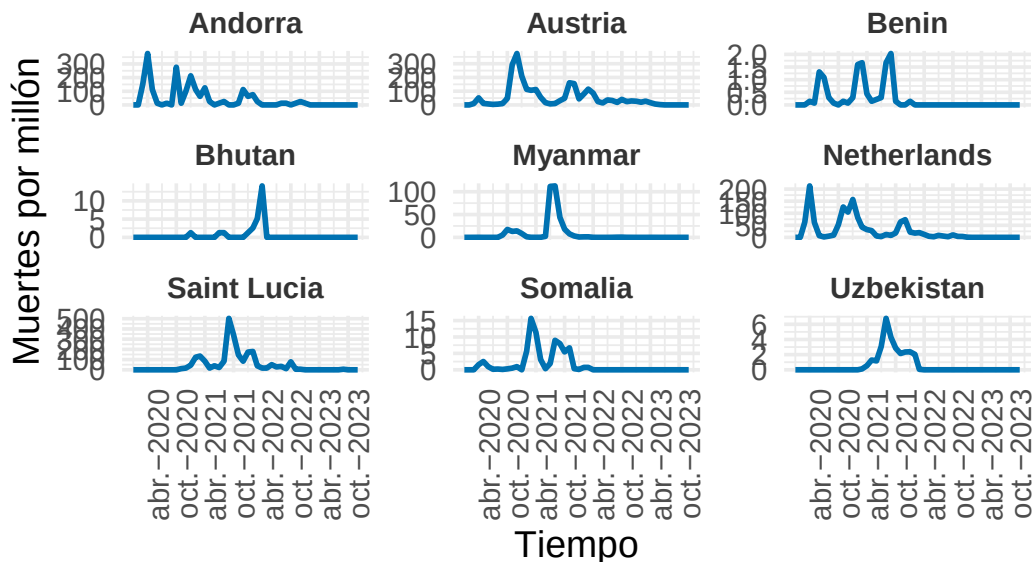
```
visualizar_ejemplos(serie_new_deaths,cl_ndtw_dba_2_deaths,1,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 1




```
visualizar_ejemplos(serie_new_deaths, cl_ndtw_dba_2_deaths, 2, 9, c(3, 3), 42, ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 2



Encontramos países en el primer grupo con brotes prolongados mientras que en el segundo grupo hay países que tienen brotes puntuales, subidas y bajadas rápidas. Estudiemos ahora el caso $K = 3$.

Caso $k = 3$

En este caso vamos a hacer multiarranque, una técnica muy conocida en el ámbito del cluster para aquellos métodos donde hay aleatoriedad. Consiste en realizar varias inicializaciones aleatorias y seleccionar la mejor basándonos en algún criterio. En este caso nos basaremos en el coeficiente CH.

```
set.seed(422)
clust_multiarranque_3 = tsclust(serie_new_deaths_z,
                                type = "partitionial",
                                k = 3,
                                distance = "ndtw",
                                centroid = "dba",
                                seed = 1234,
                                control = partitionial_control(nrep = 10L))
```

clust_multiarranque_3

```
[[1]]
partitional clustering with 3 clusters
Using ndtw distance
Using dba centroids
```

Time required for analysis:

	user	system	elapsed
	48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	58	0.2085946
2	44	0.1451539
3	83	0.2101712

```
[[2]]
partitional clustering with 3 clusters
Using ndtw distance
Using dba centroids
```

Time required for analysis:

	user	system	elapsed
	48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	72	0.1753290
2	95	0.2018697
3	18	0.2472483

```
[[3]]
partitional clustering with 3 clusters
Using ndtw distance
Using dba centroids
```

Time required for analysis:

	user	system	elapsed
	48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	85	0.2010033
2	67	0.2254296
3	33	0.1509464

[[4]]

partitional clustering with 3 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

	user	system	elapsed
	48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	62	0.1634217
2	58	0.2352202
3	65	0.2223954

[[5]]

partitional clustering with 3 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

	user	system	elapsed
	48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	11	0.1244045
2	79	0.2122241
3	95	0.1848039

[[6]]

partitional clustering with 3 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

	user	system	elapsed
	48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	112	0.2508124
2	52	0.1963281
3	21	0.1941633

[[7]]

partitional clustering with 3 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

	user	system	elapsed
	48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	73	0.2120245
2	46	0.1445347
3	66	0.2121160

[[8]]

partitional clustering with 3 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

	user	system	elapsed
	48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	75	0.2043859
2	42	0.1343401
3	68	0.2078956

```

[[9]]
partitional clustering with 3 clusters
Using ndtw distance
Using dba centroids

Time required for analysis:
      user  system elapsed
48.64    5.36   33.28

Cluster sizes with average intra-cluster distance:

      size  av_dist
1      87 0.2315545
2      15 0.1281861
3      83 0.1802648

[[10]]
partitional clustering with 3 clusters
Using ndtw distance
Using dba centroids

Time required for analysis:
      user  system elapsed
48.64    5.36   33.28

Cluster sizes with average intra-cluster distance:

      size  av_dist
1      15 0.2233591
2     128 0.2255913
3      42 0.1406123

attr(,"rng")
attr(,"rng")[[1]]
[1]      10407 -305777535 -1394370482 -1723143049 2071488076 1659356893
[7] -1081051142

attr(,"rng")[[2]]
[1]      10407 1325398954 -1161873690 -494229462 -159563550 560770250
[7] 333672325

attr(,"rng")[[3]]

```

```
[1]      10407   297685860   239054748  -160895377   224208364  -348437793
[7] -1537437949
```

```
attr(,"rng")[[4]]
[1]      10407 -1436965524  -782679284   2145224753   2079003111  -745179495
[7]  -788808701
```

```
attr(,"rng")[[5]]
[1]      10407  -925263360 -2140110818   1182240871  -289920184  -1342669798
[7]  -23349880
```

```
attr(,"rng")[[6]]
[1]      10407  1964307606  -660232513  -613467032   883799847  -2101658308
[7] -1005388727
```

```
attr(,"rng")[[7]]
[1]      10407  1949070442   161644254  -626705982   1904751025  -1371638087
[7]  1169819334
```

```
attr(,"rng")[[8]]
[1]      10407   916050734  -894480448   1159483818  -725429727  -1549288265
[7]   740358822
```

```
attr(,"rng")[[9]]
[1]      10407  -984664921 -1626530028 -1521787383   1115922615   2049520390
[7]   948743220
```

```
attr(,"rng")[[10]]
[1]      10407 -2106222585  -816860586    58210747 -1434067374   652939545
[7]  1205145458
```

```
set.seed(42)
# Calcular índice CH en cada repetición
scores = sapply(clust_multiarranque_3, function(model) {
  cvi(model,type="internal")["CH"]
})
scores
```

	CH	CH	CH	CH	CH	CH	CH
CH							
64.28709	68.85172	75.26604	101.96037	65.80325	68.84135	75.67180	
75.51539							
CH	CH						

87.48024 61.50093

```
# Elegir el mejor modelo
mejor_idx = which.max(scores)
cl_ndtw_dba_3_deaths = clust_multiarranque_3[[mejor_idx]]
cl_ndtw_dba_3_deaths
```

partitional clustering with 3 clusters
Using ndtw distance
Using dba centroids

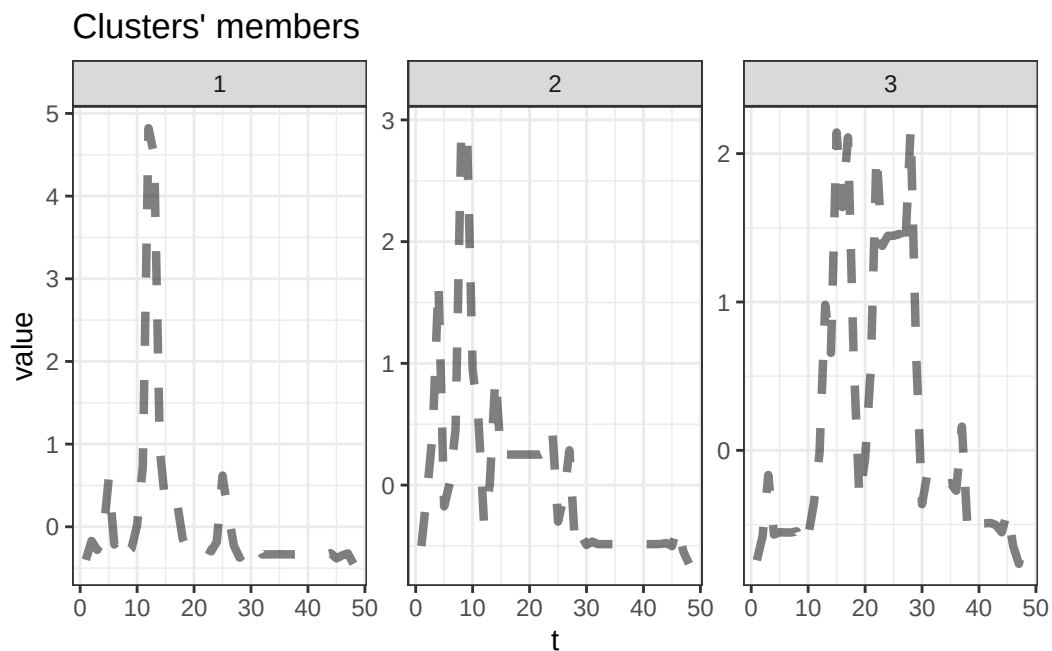
Time required for analysis:

user	system	elapsed
48.64	5.36	33.28

Cluster sizes with average intra-cluster distance:

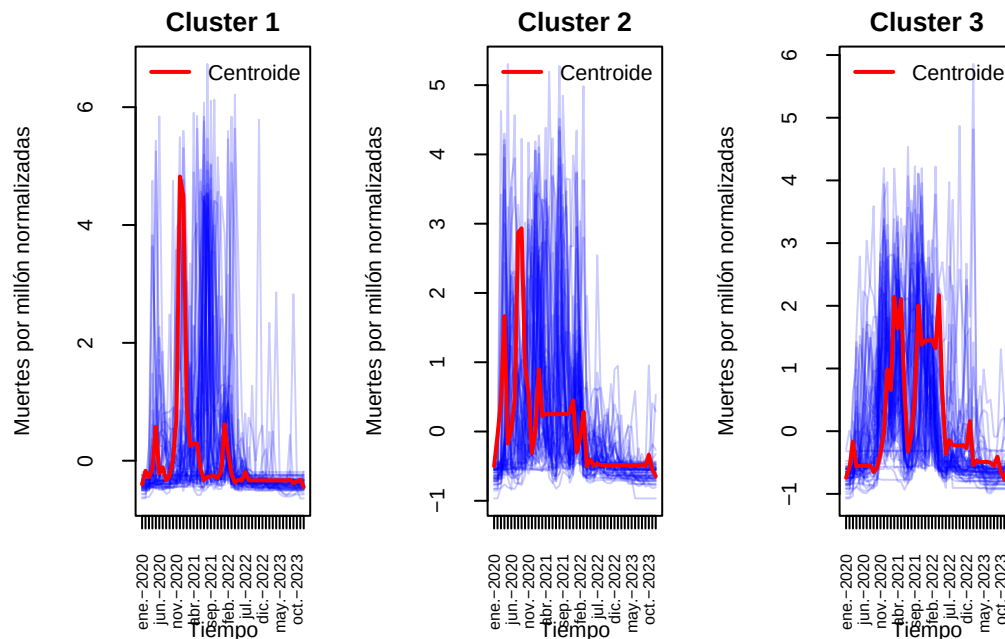
	size	av_dist
1	62	0.1634217
2	58	0.2352202
3	65	0.2223954

```
plot(cl_ndtw_dba_3_deaths,type="centroids")
```



Parece haber países con varios brotes (grupo 3), un pico muy marcado, pero con la subida y la bajada más suave (grupo 2) y un único brote marcado (grupo 1) con leves repuntes anteriores y posteriores, pero con la bajada y subida mucho más brusca que en el grupo 2. Parece que hemos obtenido una agrupación similar que en los contagios.

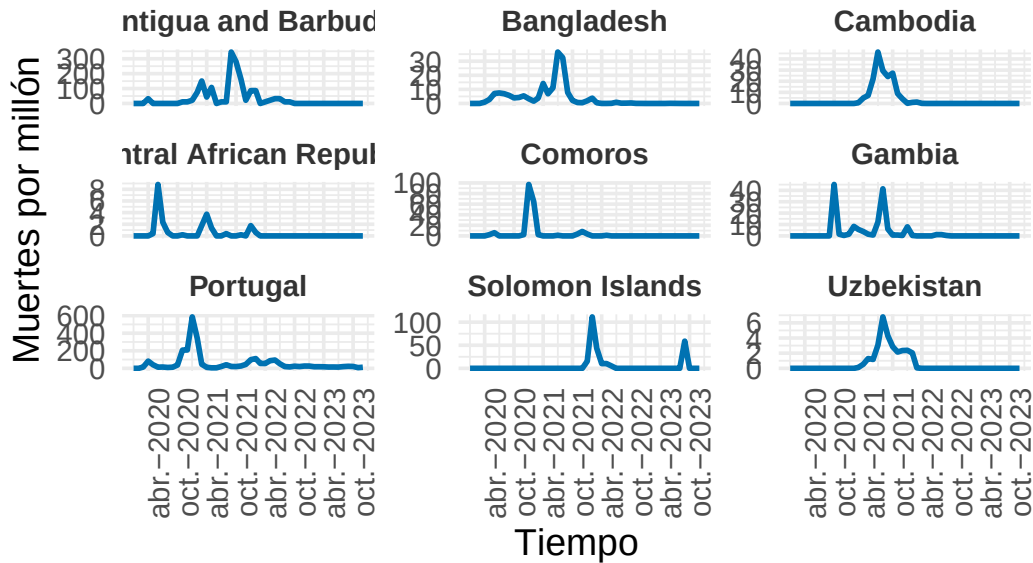
```
visualizar_series_centroides(serie_new_deaths_z,cl_ndtw_dba_3_deaths,3,ylab="Muertes
↳ por millón normalizadas")
```



Veamos algunos ejemplos a ver si se verifica lo que se dice y parece observarse en los gráficos.

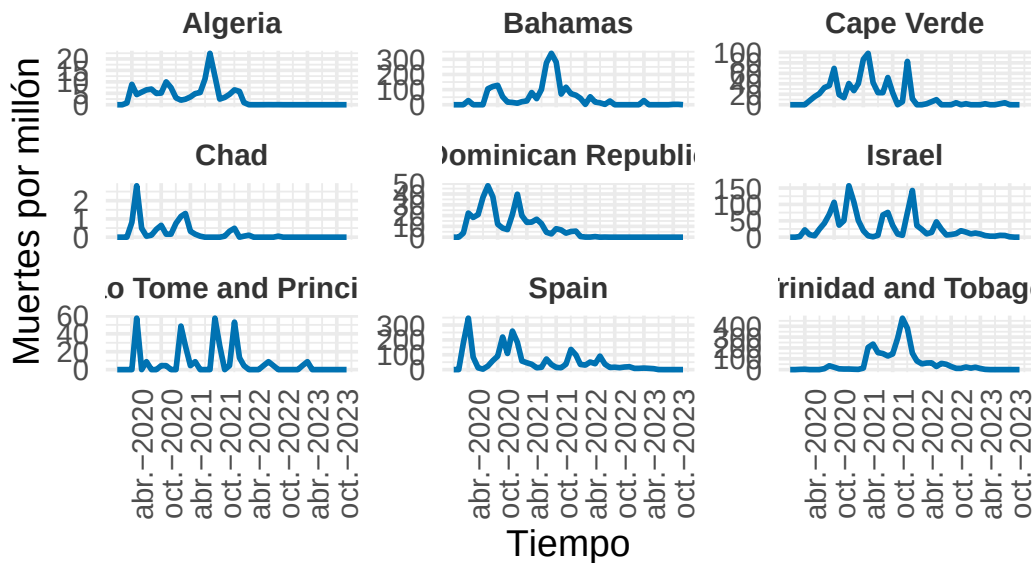
```
visualizar_ejemplos(serie_new_deaths,cl_ndtw_dba_3_deaths,1,9,c(3,3),42,ylab="Muertes
↳ por millón")
```


Ejemplos del Cluster 1



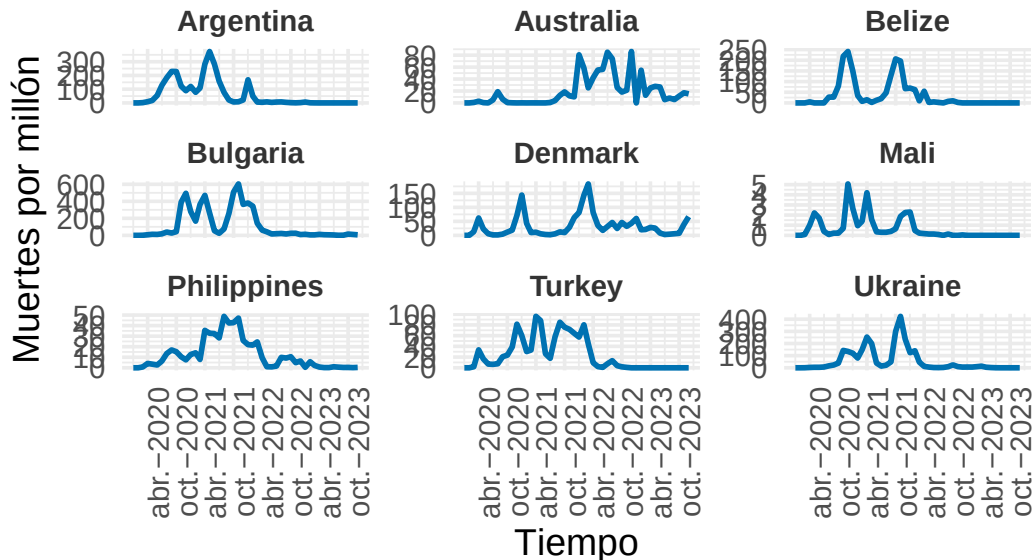
```
visualizar_ejemplos(serie_new_deaths,cl_ndtw_dba_3_deaths,2,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_deaths,cl_ndtw_dba_3_deaths,3,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 3



Se observan grupos que parecen homogéneos y diferenciados entre sí. El primer grupo presenta numerosos brotes de la epidemia en las muertes. Se diferencia del segundo en que dicho grupo tiene un brote muy marcado y algún brote mucho menos marcado antes o tras el mismo (con excepciones como Finlandia). En cambio el tercer grupo presenta países con un único pico muy marcado en la pandemia (salvo Burundi por ejemplo). Parece que se ha obtenido una buena agrupación veamos las métricas obtenidas.

```
set.seed(42)
cvi(cl_ndtw_dba_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1113985	0.4345835	78.2814446	1.8778449	1.9657758	0.1222198	0.5362779

Las métricas parecen bastante decentes.

Veamos qué ocurre para $k = 4$ y a partir de ahí estudiaremos diferentes distancias y métodos como hicimos anteriormente. ##### Caso $k = 4$ Aplicamos nuevamente multiarreglo.

```

set.seed(42)
clust_multiarranque_4 = tsclust(serie_new_deaths_z,
                                type = "partitional",
                                k = 4,
                                distance = "ndtw",
                                centroid = "dba",
                                seed = 12345,
                                control = partitional_control(nrep = 10L))
clust_multiarranque_4

```

```

[[1]]
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids

```

Time required for analysis:

	user	system	elapsed
	54.53	6.32	41.54

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	60	0.2123471
2	47	0.1882027
3	44	0.2122413
4	34	0.1466882

```

[[2]]
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids

```

Time required for analysis:

	user	system	elapsed
	54.53	6.32	41.54

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	75	0.2117268
2	46	0.2176323
3	10	0.1272559
4	54	0.1595570

```
[[3]]
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids
```

```
Time required for analysis:
      user  system elapsed
54.53    6.32   41.54
```

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	63	0.1509752
2	38	0.2178064
3	26	0.2167578
4	58	0.2239101

```
[[4]]
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids
```

```
Time required for analysis:
      user  system elapsed
54.53    6.32   41.54
```

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	33	0.1942415
2	17	0.1420149
3	45	0.2383168
4	90	0.1887192

```
[[5]]
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids
```

```
Time required for analysis:
      user  system elapsed
54.53    6.32   41.54
```

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	23	0.1476633
2	9	0.1480979
3	92	0.2132041
4	61	0.1904863

[[6]]

partitional clustering with 4 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

	user	system	elapsed
	54.53	6.32	41.54

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	87	0.2066708
2	18	0.2143640
3	42	0.1438822
4	38	0.2039998

[[7]]

partitional clustering with 4 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

	user	system	elapsed
	54.53	6.32	41.54

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	12	0.1145889
2	18	0.2195191
3	110	0.2179772
4	45	0.1446622

```

[[8]]
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids

Time required for analysis:
    user  system elapsed
54.53    6.32    41.54

Cluster sizes with average intra-cluster distance:

    size  av_dist
1    14 0.1075757
2    29 0.1585578
3    65 0.1986537
4    77 0.2005040

[[9]]
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids

Time required for analysis:
    user  system elapsed
54.53    6.32    41.54

Cluster sizes with average intra-cluster distance:

    size  av_dist
1    81 0.2293476
2    25 0.1561767
3    28 0.1924636
4    51 0.1666722

[[10]]
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids

Time required for analysis:
    user  system elapsed
54.53    6.32    41.54

```

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	50	0.2182924
2	55	0.1850746
3	32	0.2169021
4	48	0.1485129

attr(,"rng")

attr(,"rng")[[1]]

[1] 10407 -2132566924 1638542565 108172386 -1884566405 -1838154368
[7] -250773631

attr(,"rng")[[2]]

[1] 10407 -1645818963 548746318 440099794 143370804 -548492161
[7] 249546247

attr(,"rng")[[3]]

[1] 10407 -363950260 -864007039 1529726914 -409305868 -670976700
723026206

attr(,"rng")[[4]]

[1] 10407 -310576051 -443186231 1234569748 76923062 1387306546
-309616276

attr(,"rng")[[5]]

[1] 10407 -1515242020 -1576741206 -11449651 -783708047 1716218842
[7] 627172014

attr(,"rng")[[6]]

[1] 10407 938392635 -1982904907 -1517038136 -202780809 1451029597
[7] 1494431831

attr(,"rng")[[7]]

[1] 10407 -12081935 -1215743205 -1850867630 13015048 -175155923
[7] 978818417

attr(,"rng")[[8]]

[1] 10407 -547697792 879559495 -623273946 -2028524519 1825699665
[7] 1636958025

attr(,"rng")[[9]]

```
[1]      10407 -398910215 -724655204  141234094  212689848  912101053
-721622215
```

```
attr(,"rng")[[10]]
```

```
[1]      10407 -126602366   572111641   792103641   757116781 -1990669391
[7] -1530730288
```

```
set.seed(42)
# Calcular índice CH en cada repetición
scores = sapply(clust_multiarranque_4, function(model) {
  cvi(model,type="internal")["CH"]
})
scores
```

```
      CH      CH      CH      CH      CH      CH      CH      CH
44.88707 47.41290 62.22546 48.16015 54.10913 67.47059 62.18414 54.74829
      CH      CH
55.84650 60.76816
```

```
# Elegir el mejor modelo
mejor_idx = which.max(scores)
cl_ndtw_dba_4_deaths = clust_multiarranque_4[[mejor_idx]]
cl_ndtw_dba_4_deaths
```

```
partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids
```

```
Time required for analysis:
```

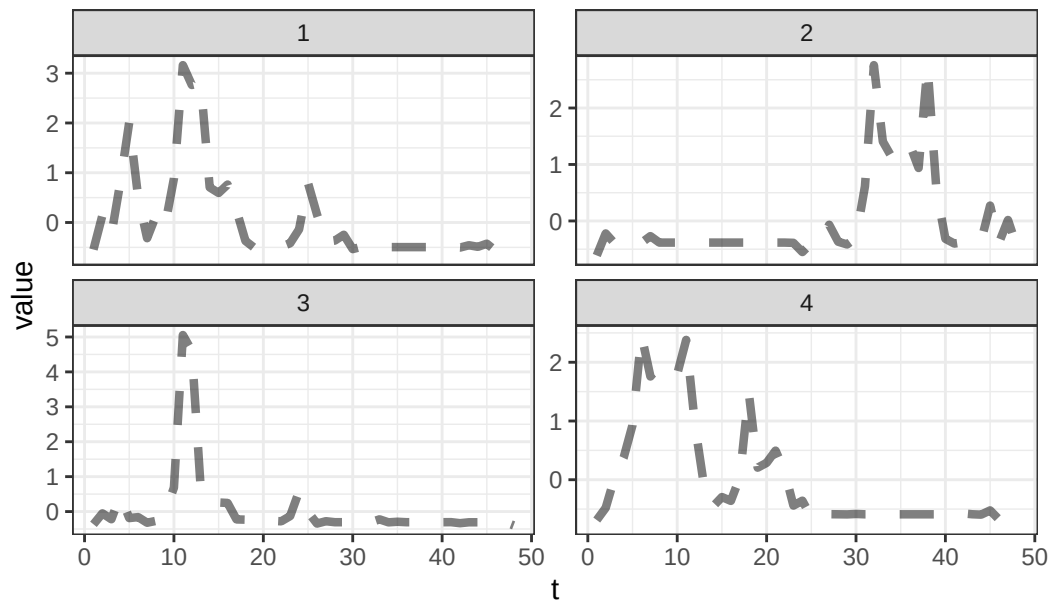
```
      user  system elapsed
54.53     6.32    41.54
```

```
Cluster sizes with average intra-cluster distance:
```

```
      size  av_dist
1      87 0.2066708
2      18 0.2143640
3      42 0.1438822
4      38 0.2039998
```

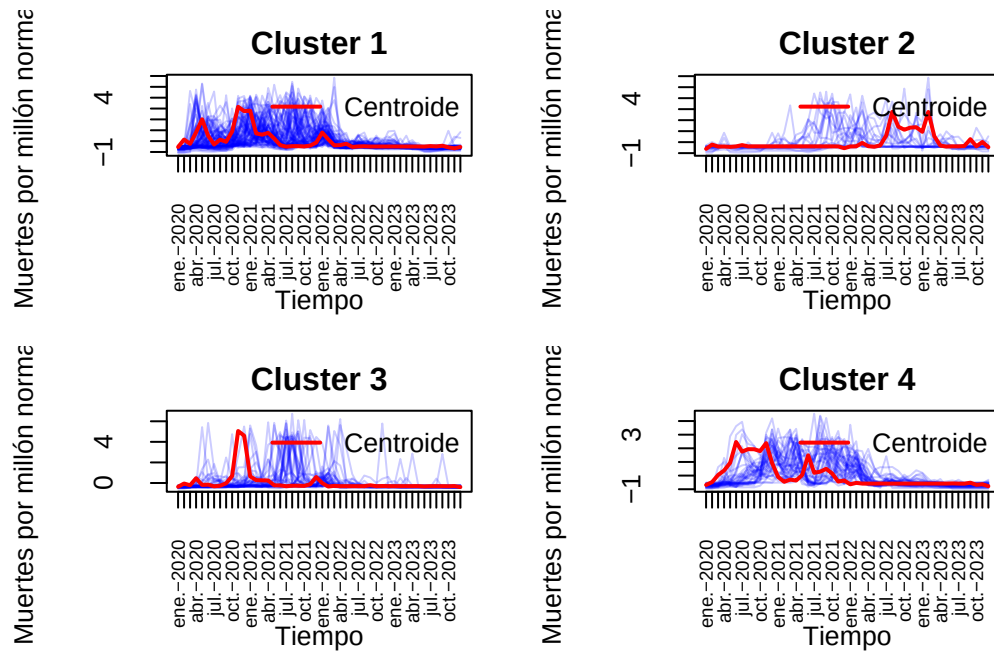
```
plot(cl_ndtw_dba_4_deaths,type="centroids")
```


Clusters' members



Ya se observa una mezcla entre grupos, esto posiblemente se debe a que segmentar en 4 grupos es demasiado. Veamos los centroides junto a las series.

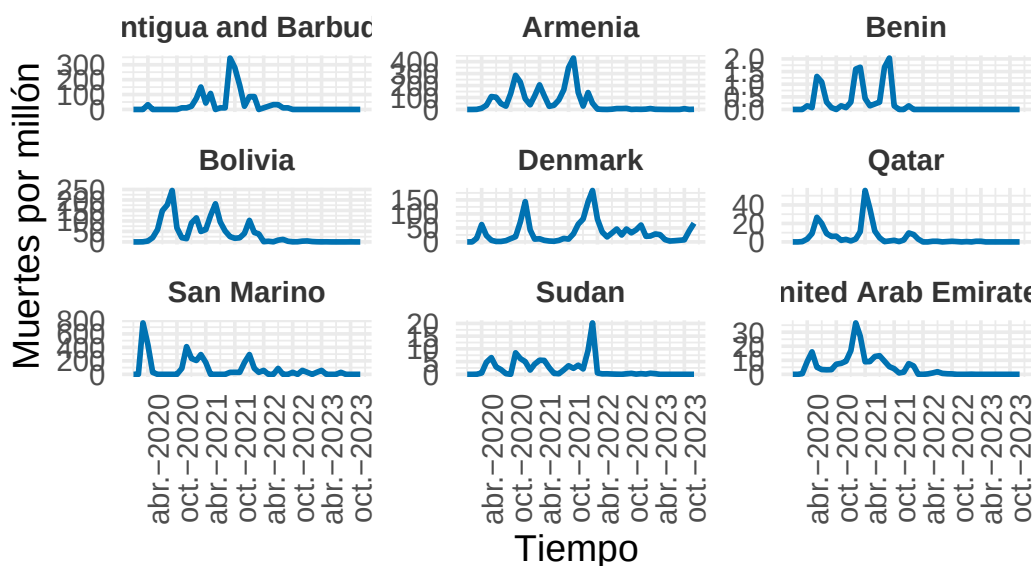
```
visualizar_series_centroides(serie_new_deaths_z,cl_ndtw_dba_4_deaths,4,ylab="Muertes
↳ por millón normalizadas",tamano_ventana = c(2,2))
```



Efectivamente la diferencia entre las series del grupo 1 y el 4 no es muy clara. Veamos los ejemplos.

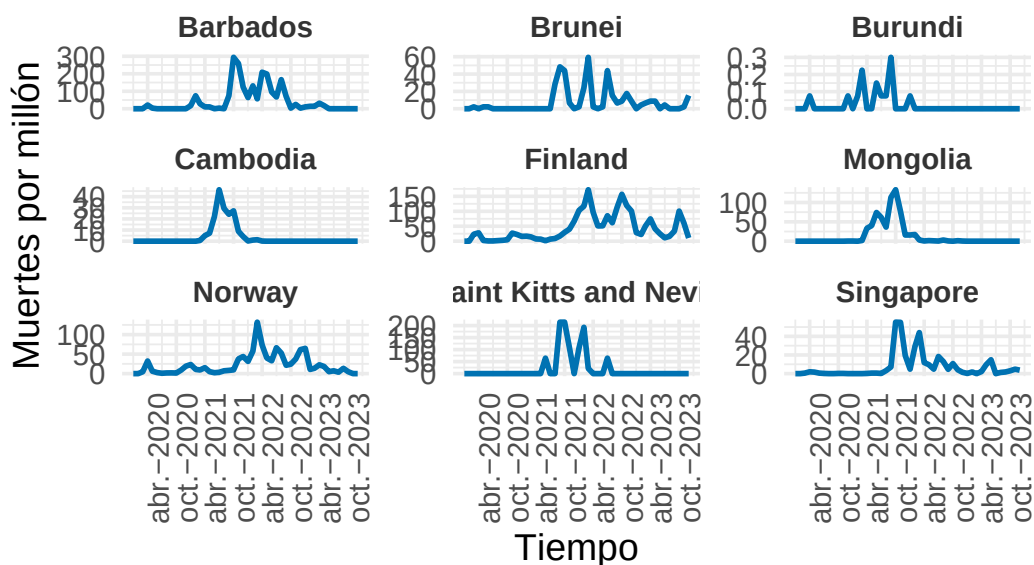
```
visualizar_ejemplos(serie_new_deaths,cl_ndtw_dba_4_deaths,1,9,c(3,3),42,ylab="Muertes
  ↪ por millón")
```

Ejemplos del Cluster 1



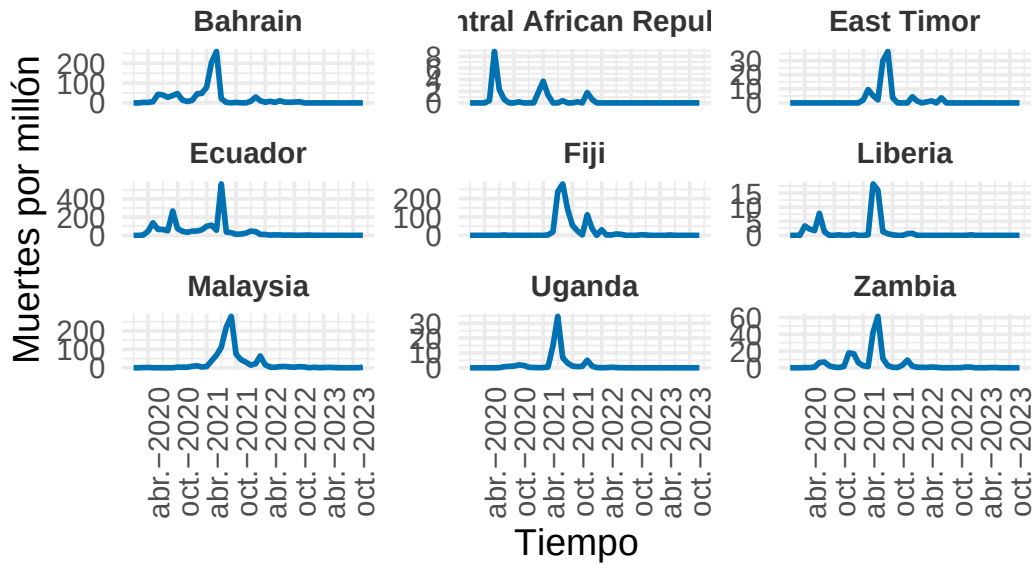
```
visualizar_ejemplos(serie_new_deaths,cl_ndtw_dba_4_deaths,2,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 2



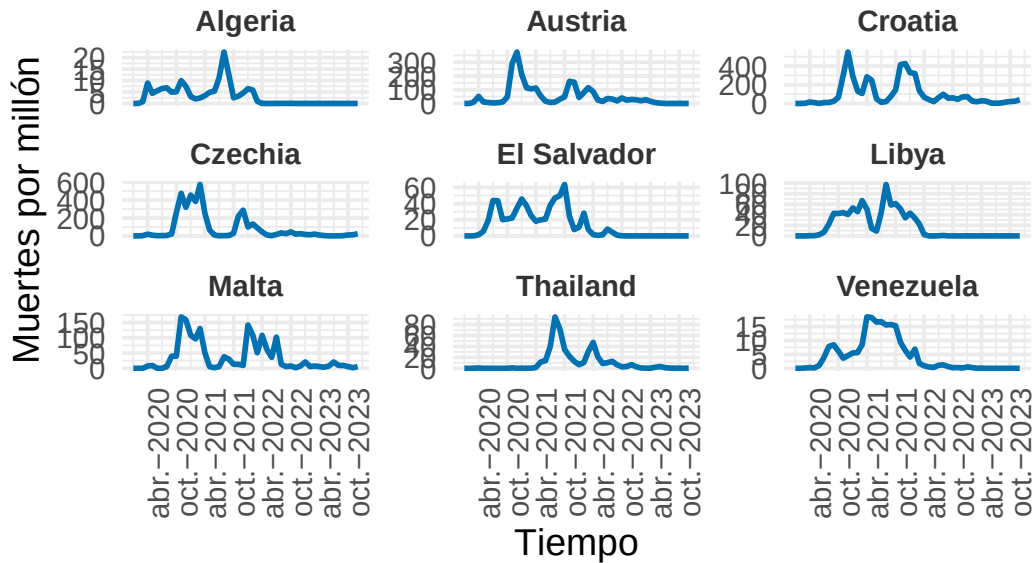
```
visualizar_ejemplos(serie_new_deaths,cl_ndtw_dba_4_deaths,3,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 3



```
visualizar_ejemplos(serie_new_deaths,cl_ndtw_dba_4_deaths,4,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 4



Se observa una menor homogeneidad. Además las puntuaciones obtenidas ahora son peores que cuando usamos $k = 3$. Veamos todas las métricas y comparemos.

```
set.seed(42)
cvi(cl_ndtw_dba_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1113985	0.4345756	78.2487584	1.8778449	1.9657758	0.1222198	0.5362779

```
cvi(cl_ndtw_dba_4_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D
0.08715625	0.38378074	54.57902807	1.68566293	1.72627975	0.11808810
COP					
0.47716170					

Efectivamente obtenemos métricas superiores para $k = 3$ como era de esperar. Vamos a estudiar ahora más distancias del caso $k = 3$ para elegir el mejor clúster posible.

Distancia Soft-DTW

```

set.seed(42)
clust_multiarranque_sdtw_3 = tsclust(serie_new_deaths_z,
                                     type = "partitional",
                                     k = 3,
                                     distance = "sdtw",
                                     centroid = "sdtw_cent",
                                     seed = 12345,
                                     control = partitional_control(nrep = 10L))

set.seed(42)
# Calcular índice CH en cada repetición
scores = sapply(clust_multiarranque_sdtw_3, function(model) {
  cvi(model,type="internal")["CH"]
})
scores

```

	CH	CH	CH	CH	CH	CH	CH
	CH						
45.39040	51.22023	50.74676	61.89751	61.38511	53.85044	74.89350	
58.34946							
	CH	CH					
102.81449	54.37540						

```

# Elegir el mejor modelo
mejor_idx = which.max(scores)
cl_sdtw_3_deaths = clust_multiarranque_sdtw_3[[mejor_idx]]
cl_sdtw_3_deaths

```

partitional clustering with 3 clusters
 Using sdtw distance
 Using sdtw_cent centroids

Time required for analysis:

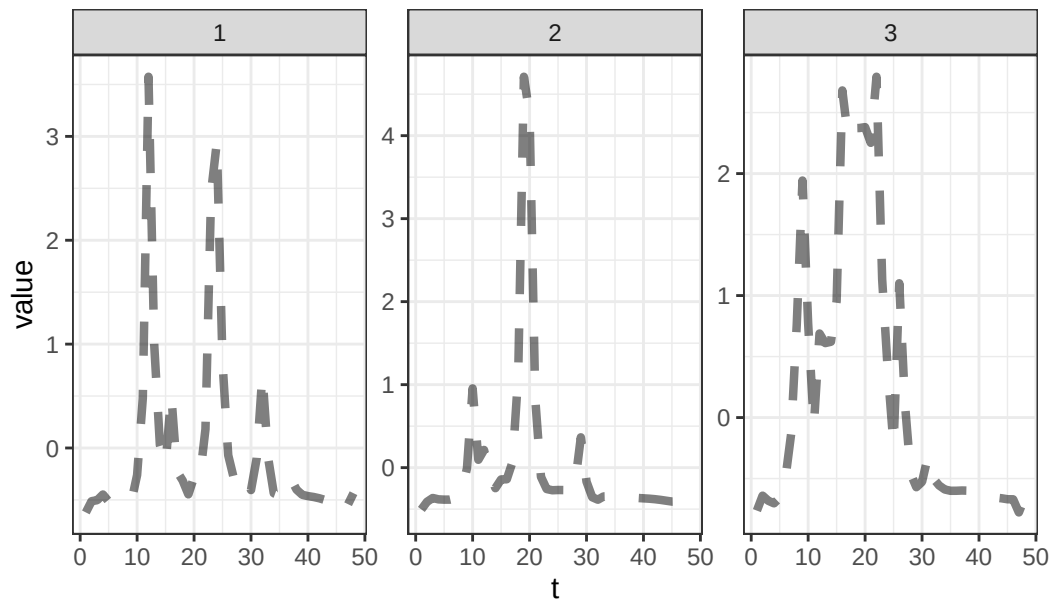
	user	system	elapsed
	419.14	3.28	31.06

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	53	7.321799
2	70	6.133176
3	62	5.599651

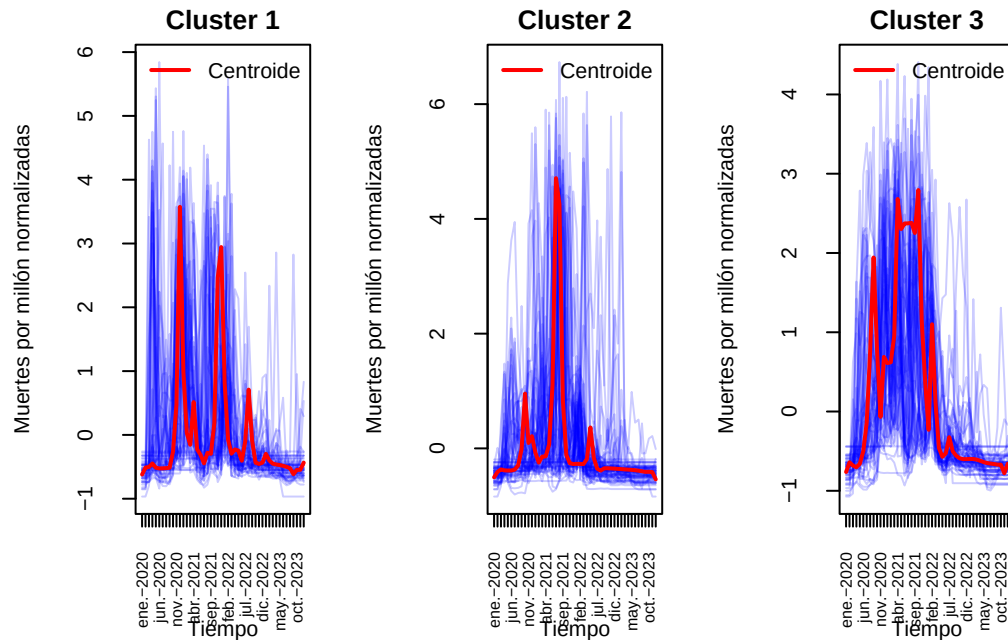
```
plot(cl_sdtw_3_deaths,type="centroids")
```

Clusters' members



Visualicemos los centroides junto a las series obtenidas.

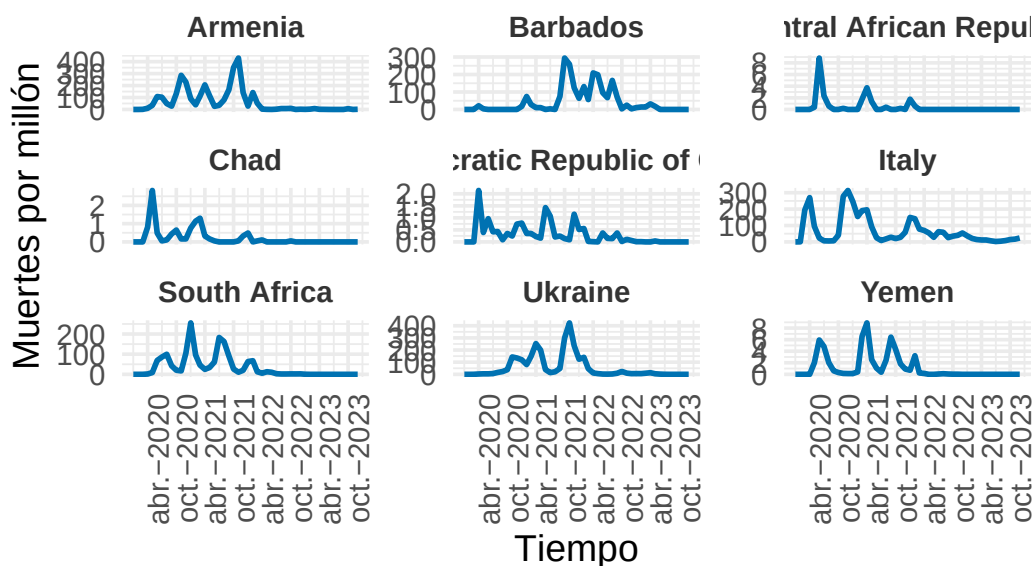
```
visualizar_series_centroides(serie_new_deaths_z,cl_sdtw_3_deaths,3,ylab="Muertes  
↪ por millón normalizadas")
```



Obtenemos clústeres algo similares al que obtuvimos anteriormente, veamos en los ejemplos como es la agrupación y analicemos las métricas.

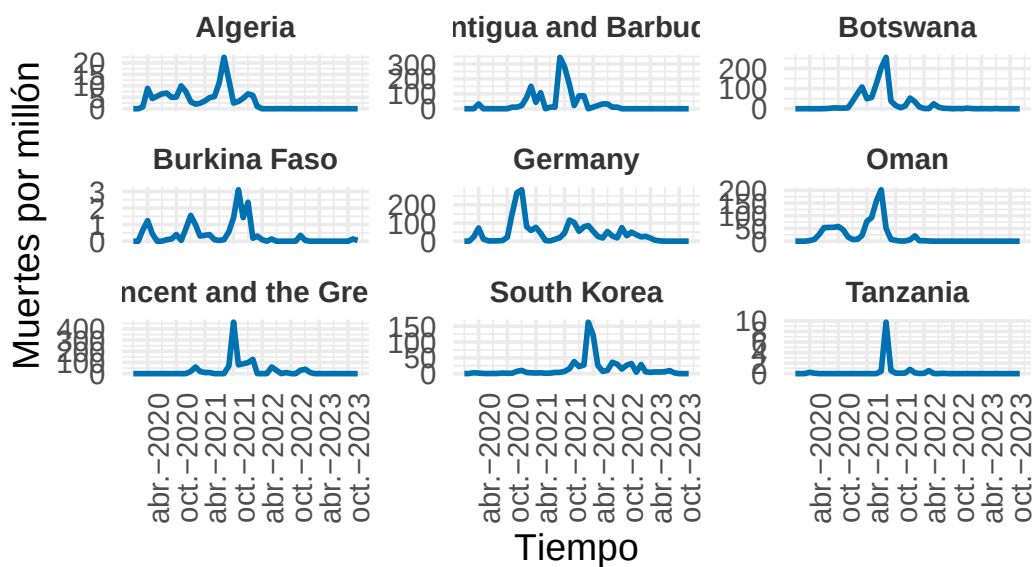
```
visualizar_ejemplos(serie_new_deaths,cl_sdtw_3_deaths,1,9,c(3,3),42,ylab="Muertes
↳ por millón")
```


Ejemplos del Cluster 1



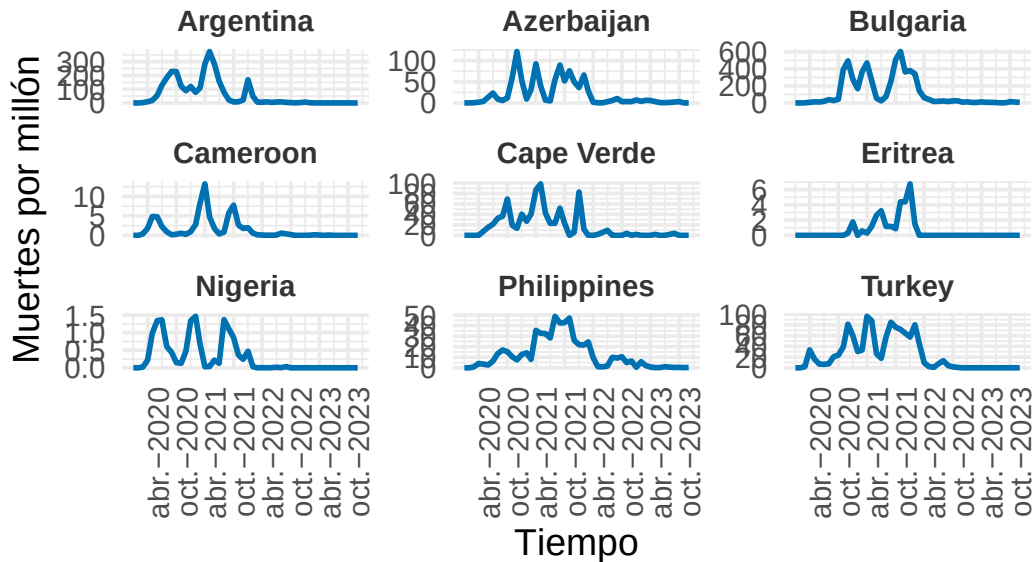
```
visualizar_ejemplos(serie_new_deaths,cl_sdtw_3_deaths,2,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_deaths,cl_sdtw_3_deaths,3,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 3



Vemos que los ejemplos del grupo 1 presentan poco homogeneidad. Esto significará que las métricas serán peores.

```
set.seed(42)
cvi(cl_ndtw_dba_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1113985	0.4345830	78.2794562	1.8778449	1.9657758	0.1222198	0.5362779

```
cvi(cl_sdtw_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D
2.215306e-01	2.582201e-08	6.863437e+01	1.877836e+00	1.910047e+00	3.434135e-02
COP					
2.534717e-01					

Vemos métricas superiores en el cluster usando nDTW y DBA.

k-Shape Clustering

Volvemos a hacer como hicimos para los nuevos casos.

```
set.seed(42)
cl_sbd_3_deaths = tsclust(serie_new_deaths_z, type = "partitional", k = 3,
  ↪ distance = "sbd", centroid = "shape", seed = 12345)
cl_sbd_3_deaths
```

partitional clustering with 3 clusters

Using sbd distance

Using shape centroids

Using zscore preprocessing

Time required for analysis:

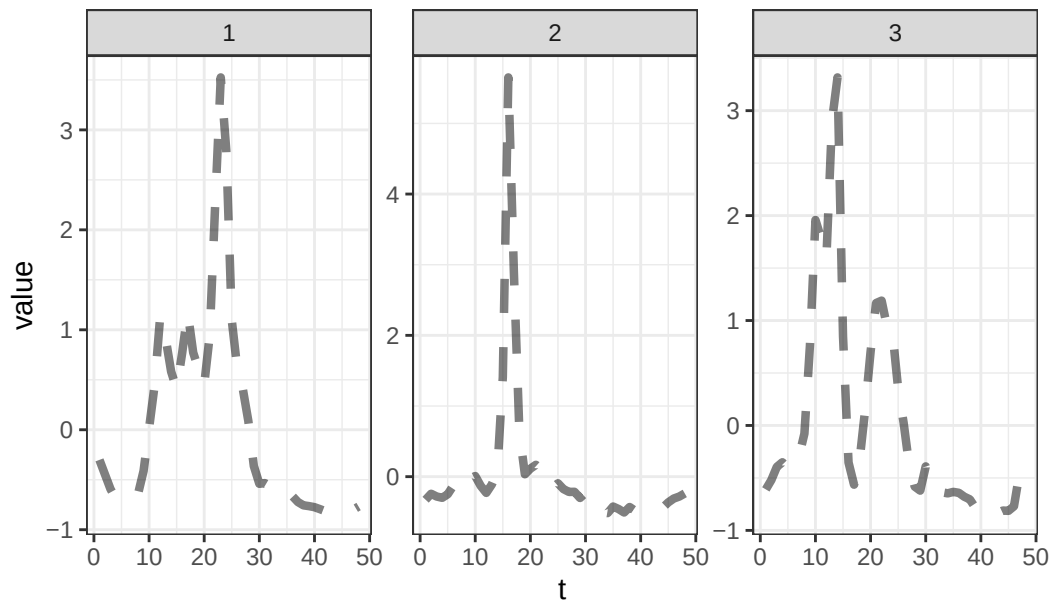
	user	system	elapsed
	0.13	0.02	0.10

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	49	0.1900942
2	105	0.2097787
3	31	0.2131835

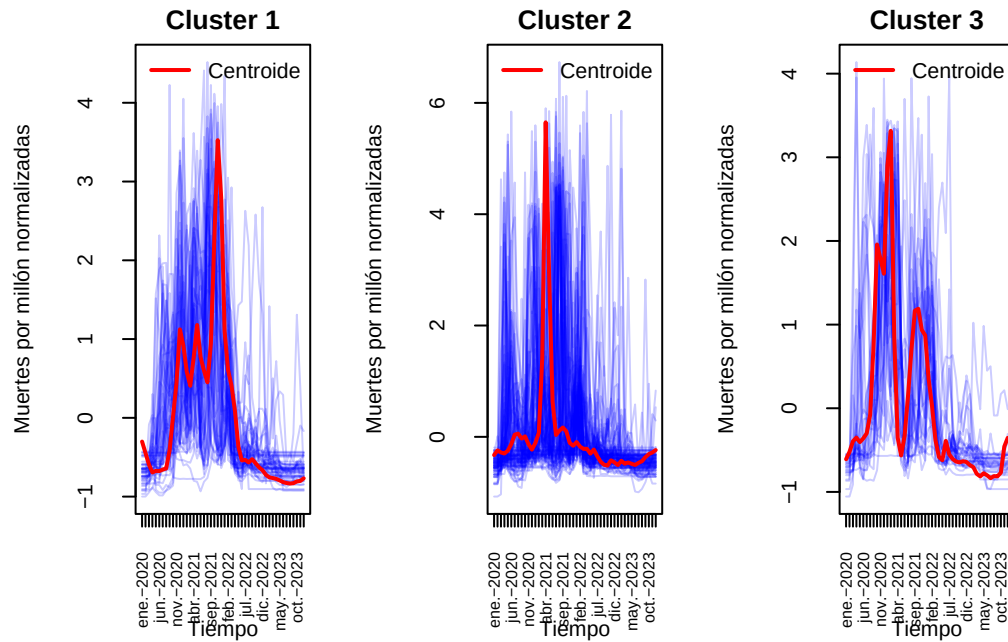
```
plot(cl_sbd_3_deaths, type = "centroids")
```

Clusters' members (z-normalized)



Realicemos las visualizaciones pertinentes.

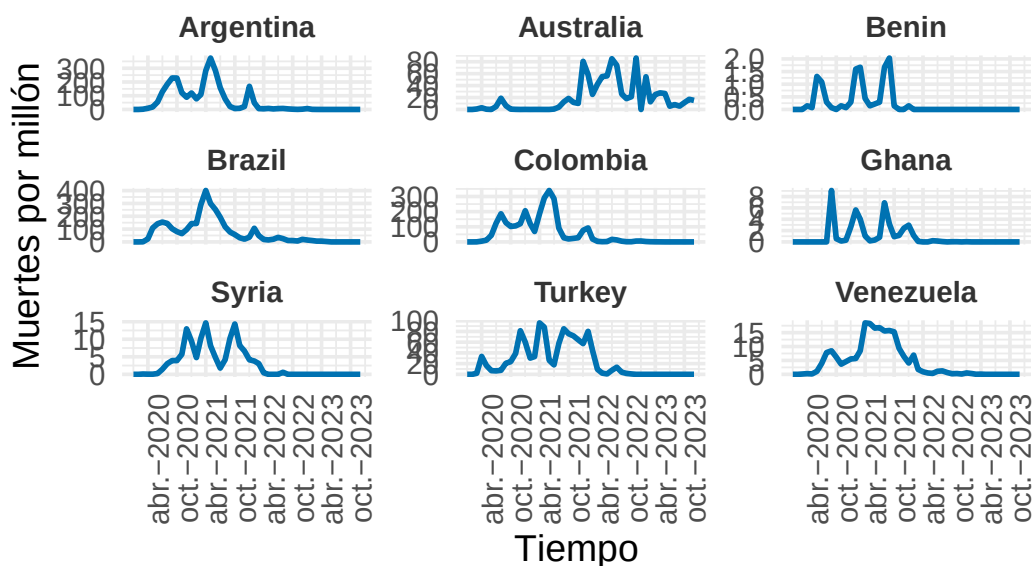
```
visualizar_series_centroides(serie_new_deaths_z, cl_sbd_3_deaths, 3, ylab="Muertes  
↪ por millón normalizadas")
```



Obtenemos centroides similares a los de la mejor agrupación hasta el momento, veamos algunos ejemplos.

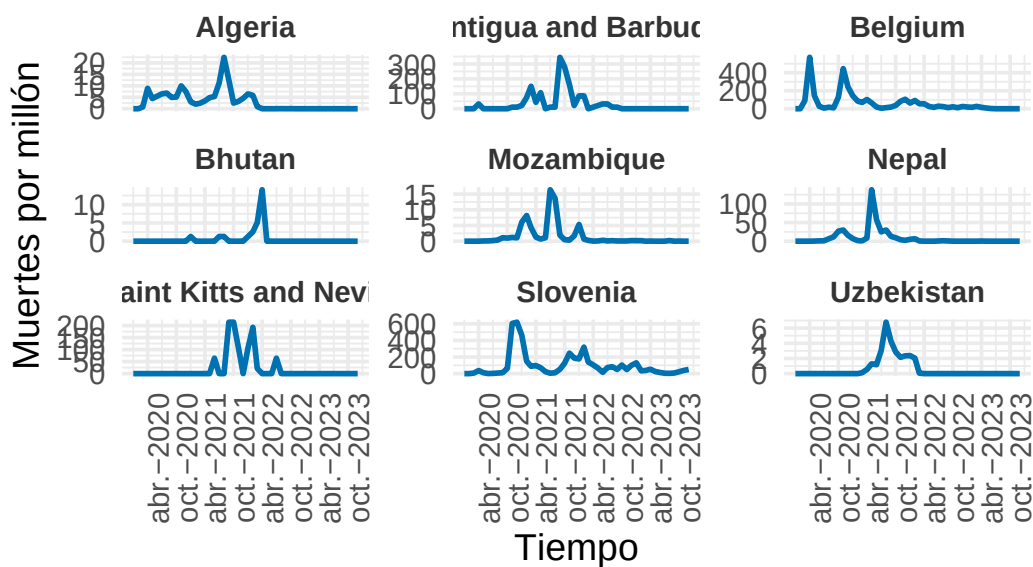
```
visualizar_ejemplos(serie_new_deaths,cl_sbd_3_deaths,1,9,c(3,3),42,ylab="Muertes
  ↪ por millón")
```

Ejemplos del Cluster 1



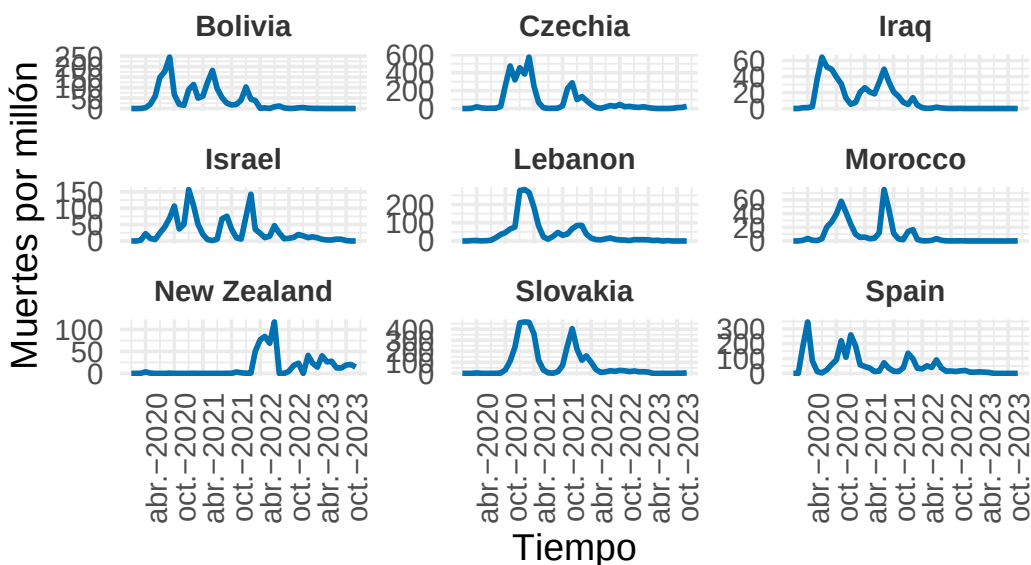
```
visualizar_ejemplos(serie_new_deaths,cl_sbd_3_deaths,2,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_deaths,cl_sbd_3_deaths,3,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 3



Vemos en los ejemplos que la agrupación es bastante decente. Veamos en las métricas cuál es la mejor.

```
set.seed(42)
cvi(cl_ndtw_dba_3_deaths,type="internal")
```

Si1	SF	CH	DB	DBstar	D	COP
0.1113985	0.4345852	78.2882912	1.8778449	1.9657758	0.1222198	0.5362779

```
cvi(cl_sbd_3_deaths,type="internal")
```

Si1	SF	CH	DB	DBstar	D
0.14814805	0.43555884	72.26231226	1.67559038	1.71515324	0.06857207
COP					
0.38215884					

Se obtienen métricas levemente inferiores que con nuestro mejor cluster por lo que seguiremos considerando como mejor nDTW con DBA. Nótese que la más fiable de ellas es el coeficiente CH.

Pasemos a otro tipo de cluster.

TADPole Clustering

Hagamos un procedimiento análogo al que hicimos con los contagios.

Para el valor de dc.

```
# Distancias DTW sobre nuestros datos normalizados
dist_matrix_dtw_deaths = proxy::dist(serie_new_deaths_z, method =
  ↪ "dtw_basic")
# Cálculo del percentil para dc
dc_deaths = quantile(dist_matrix_dtw_deaths, probs = 0.2)
```

El tamaño de ventana es el mismo que anteriormente debido a que las series presentan la misma longitud. Estamos en condiciones de aplicar este método de cluster.

```
set.seed(42)
cl_tadpole_3_deaths = tsclust(serie_new_deaths_z, k = 3, type = "t", trace =
  ↪ TRUE,
control = tadpole_control(dc = dc_deaths, window.size = window), seed = 12345)
```

Entering TADPole...

```
Computing lower and upper bound matrices
Pruning during local density calculation
Pruning during nearest-neighbor distance calculation (phase 1)
Pruning during nearest-neighbor distance calculation (phase 2)
Pruning percentage = 23.7%
Performing cluster assignment
```

TADPole completed for k = 3 & dc = 18.4

Elapsed time is 0.06 seconds.

```
cl_tadpole_3_deaths
```

```
tadpole clustering with 3 clusters
Using LB+DTW2 distance
Using PAM (TADPole) centroids
```

```
Time required for analysis:
  user  system elapsed
```

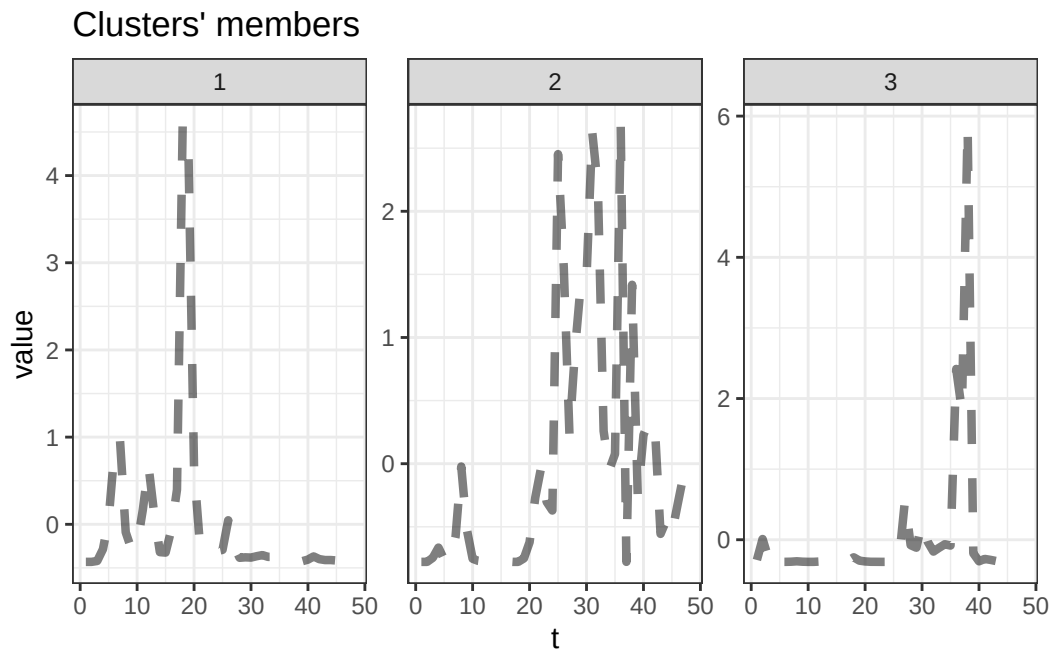


```
0.06    0.01    0.06
```

Cluster sizes with average intra-cluster distance:

```
size av_dist
1  160 4.277758
2   22 4.506924
3    3 2.447035
```

```
plot(cl_tadpole_3_deaths,type="centroids")
```



Se obtienen agrupaciones que casi no segmentan la población (hay un grupo con demasiadas observaciones).

```
set.seed(42)
cvi(cl_tadpole_3_deaths)
```

	Sil	SF	CH	DB	DBstar	D
COP	3.083175e-01	1.858355e-05	2.161440e+01	1.541264e+00	1.683022e+00	4.112949e-03
	5.978605e-01					

El coeficiente CH es muy bajo en comparación con el mejor candidato hasta el momento. Pasemos al siguiente tipo de cluster.

Fuzzy Clustering

Distancia nDTW

```
set.seed(42)
cl_fuzzy_ndtw_3_deaths = tsclust(serie_new_deaths_z, type = "fuzzy", k = 3,
  ↪ distance = "ndtw", centroid = "fcmd", seed = 12345)
cl_fuzzy_ndtw_3_deaths
```

fuzzy clustering with 3 clusters

Using ndtw distance

Using fcmd centroids

Time required for analysis:

	user	system	elapsed
	14.84	0.09	15.13

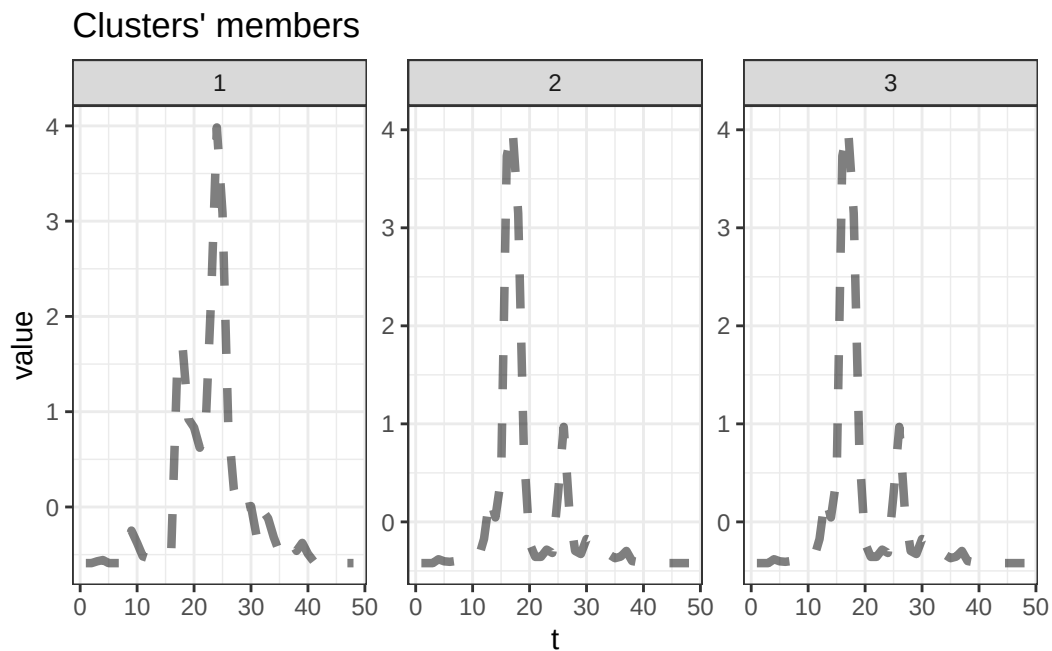
Head of fuzzy memberships:

	cluster_1	cluster_2	cluster_3
Afghanistan	0.1336332	0.4331834	0.4331834
Albania	0.3204436	0.3397782	0.3397782
Algeria	0.3596209	0.3201896	0.3201896
Andorra	0.3486568	0.3256716	0.3256716
Angola	0.4300060	0.2849970	0.2849970
Antigua and Barbuda	0.2808630	0.3595685	0.3595685

```
table(cl_fuzzy_ndtw_3_deaths@cluster)
```

```
1 2
90 95
```

```
plot(cl_fuzzy_ndtw_3_deaths, type = "centroids")
```



Los grupos parecen muy difusos. Seguramente la agrupación sea muy mala, veámoslo.

```
cvi(cl_fuzzy_ndtw_3_deaths,type="internal")
```

MPC	K	T	SC	PBMF
0.05304942	Inf	15.83866378	-1.87726888	0.05045710

```
# Calculamos el índice Calinski-Harabasz
data_matrix = do.call(rbind, serie_new_deaths_z)
ch_fuzzy_ndtw_deaths = intCriteria(as.matrix(data_matrix),
  ↪ cl_fuzzy_ndtw_3_deaths@cluster, "Calinski_Harabasz")
ch_fuzzy_ndtw_deaths
```

```
$calinski_harabasz
[1] 3.531694
```

Efectivamente la agrupación es mucho peor. Probemos con GAK.

Distancia Global Alignment Kernel

```
set.seed(42)
cl_fuzzy_gak_3_deaths = tsclust(serie_new_deaths_z, type = "fuzzy", k = 3,
  ↪ distance = "gak", centroid = "fcmd", seed = 12345)
cl_fuzzy_gak_3_deaths
```

fuzzy clustering with 3 clusters

Using gak distance

Using fcmd centroids

Time required for analysis:

user	system	elapsed
36.43	0.23	1.47

Head of fuzzy memberships:

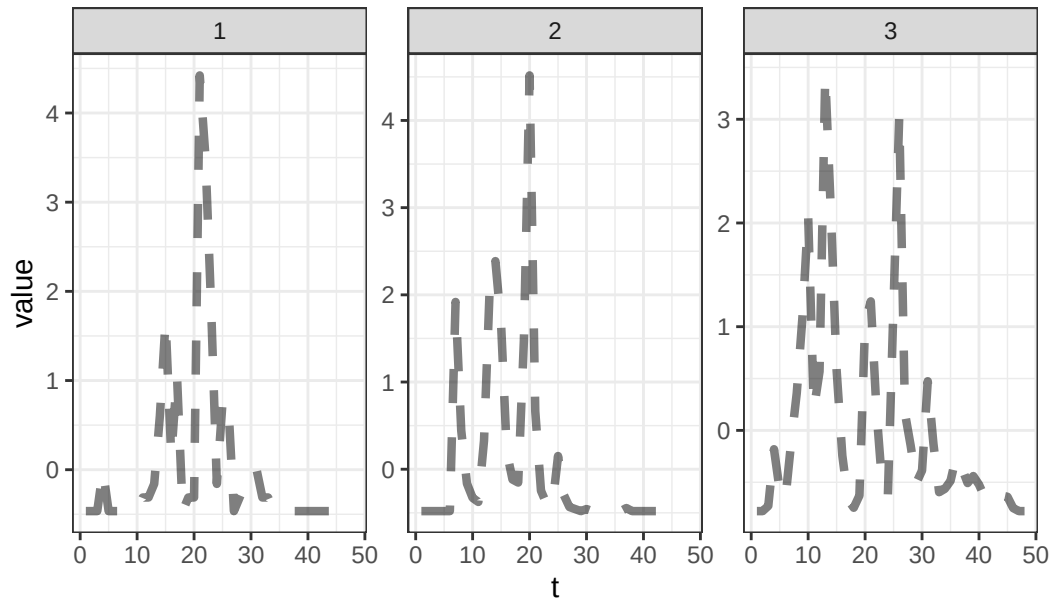
	cluster_1	cluster_2	cluster_3
Afghanistan	0.0752173	0.8960472	0.02873550
Albania	0.0238352	0.3327517	0.64341308
Algeria	0.1603004	0.7012263	0.13847334
Andorra	0.1418481	0.4223150	0.43583691
Angola	0.4416258	0.5027293	0.05564491
Antigua and Barbuda	1.0000000	0.0000000	0.00000000

```
table(cl_fuzzy_gak_3_deaths@cluster)
```

```
1 2 3
68 77 40
```

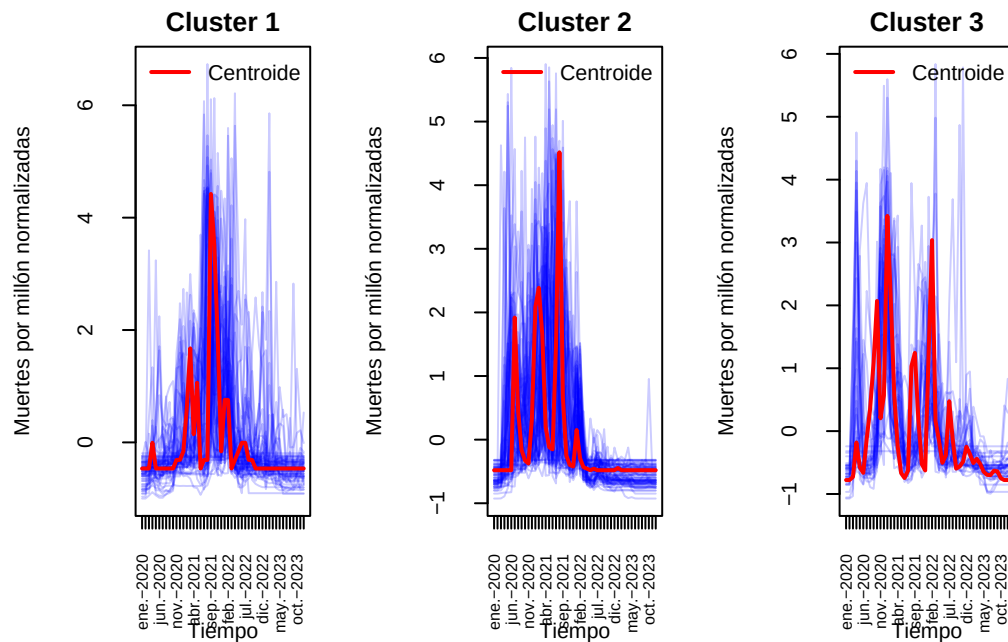
```
plot(cl_fuzzy_gak_3_deaths,type="centroids")
```

Clusters' members



Para este caso al menos vemos que son menos difusos y los grupos no son degenerados. Realicemos otras visualizaciones y comparemos métricas.

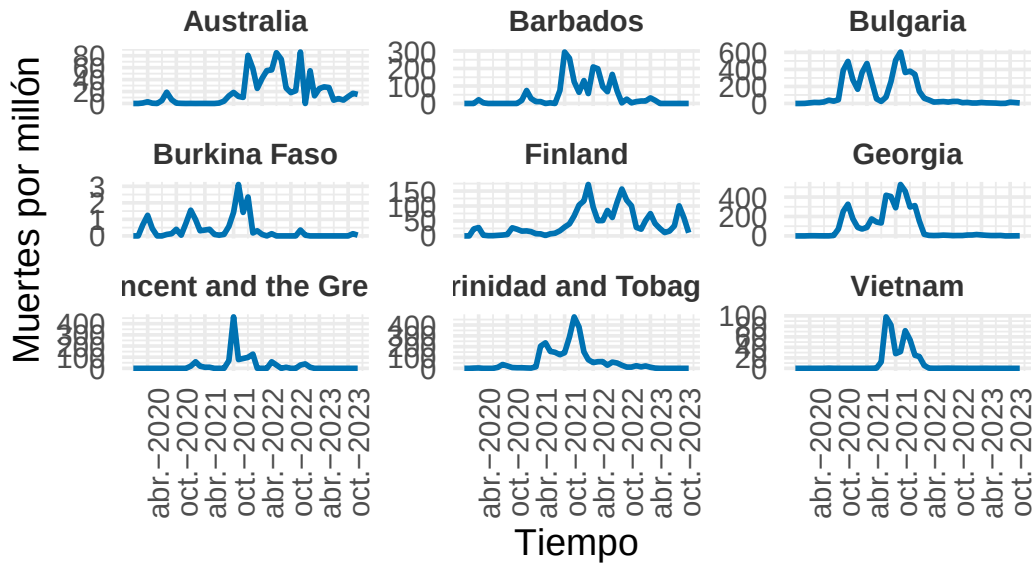
```
visualizar_series_centroides(serie_new_deaths_z,cl_fuzzy_gak_3_deaths,3,ylab="Muertes
↪ por millón normalizadas")
```



Parecen centroides razonables. Veamos algunos ejemplos.

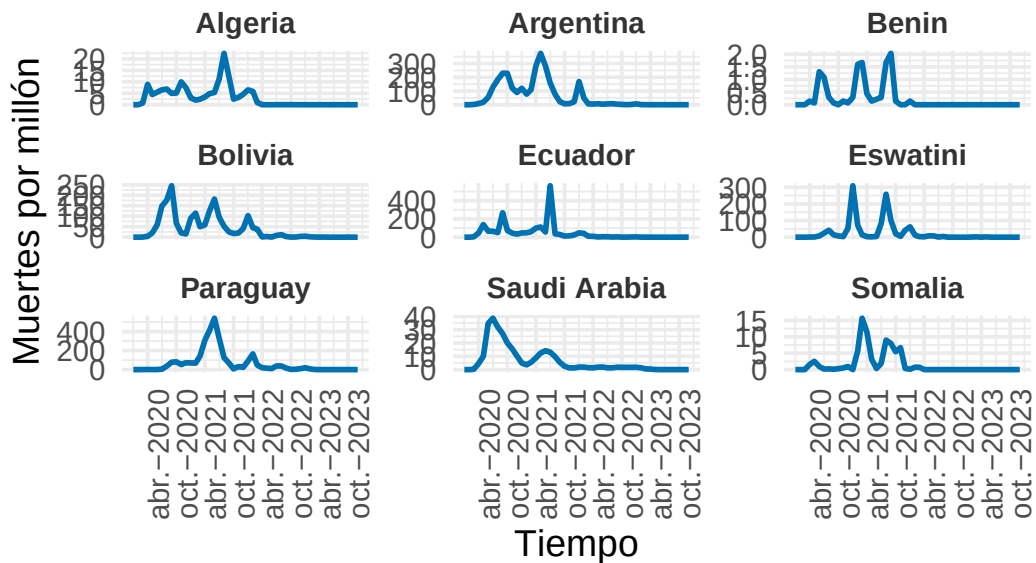
```
visualizar_ejemplos(serie_new_deaths,cl_fuzzy_gak_3_deaths,1,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 1



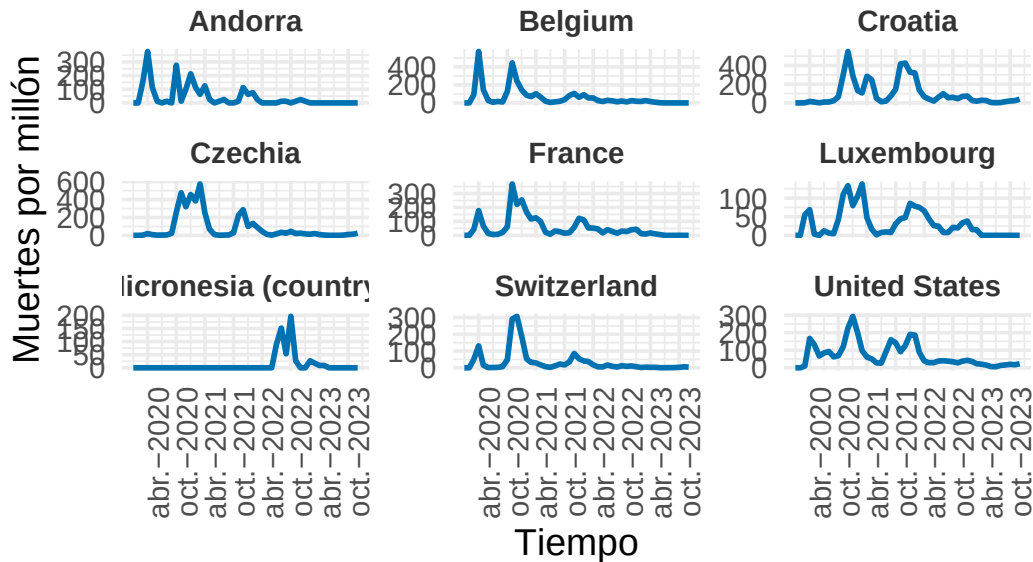
```
visualizar_ejemplos(serie_new_deaths,cl_fuzzy_gak_3_deaths,2,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_deaths,cl_fuzzy_gak_3_deaths,3,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 3



Vemos grupos no tan homogéneos y diferentes entre sí como con el mejor candidatos. Aún así la agrupación es decente. Veamos las métricas.

```
set.seed(42)
cvi(cl_ndtw_dba_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1113985	0.4345755	78.2484813	1.8778449	1.9657758	0.1222198	0.5362779

```
cvi(cl_fuzzy_gak_3_deaths,type="internal")
```

MPC	K	T	SC	PBMF
4.371909e-01	1.499412e+02	1.342824e-02	-6.564077e-01	9.992469e-05

```
# Calculamos el índice Calinski-Harabasz
ch_fuzzy_gak_deaths = intCriteria(as.matrix(data_matrix),
↪ cl_fuzzy_gak_3_deaths@cluster, "Calinski_Harabasz")
ch_fuzzy_gak_deaths
```



```
$scalinski_harabasz  
[1] 18.30487
```

Claramente las métricas del cluster particional usando nDTW con DBA son superiores (sobre todo ver el índice CH). Pasemos finalmente al cluster jerárquico.

Hierarchical clustering

Debemos usar métricas simétricas en este caso.

Distancia DTW

```
cl_hc_dtw_3_deaths = tsclust(serie_new_deaths_z, type = "hierarchical", k =  
  ↪ 3,  
  distance = "dtw_basic", control = hierarchical_control(method = diana),  
  args = tsclust_args(dist = list(window.size = window)))  
cl_hc_dtw_3_deaths
```

```
hierarchical clustering with 3 clusters  
Using dtw_basic distance  
Using PAM (Hierarchical) centroids  
Using method diana
```

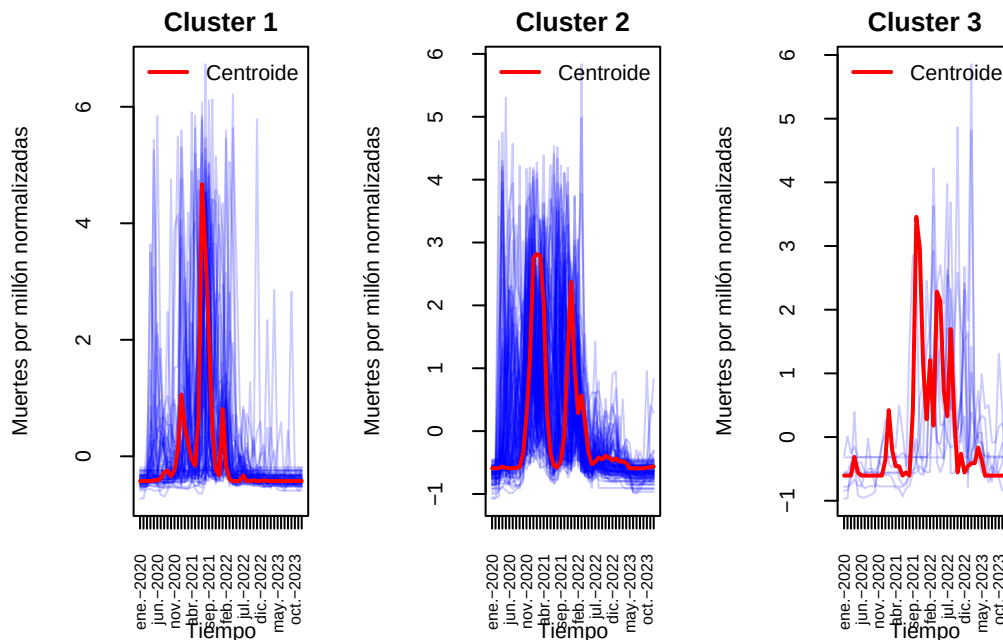
```
Time required for analysis:  
  user  system elapsed  
 1.27    0.02    0.80
```

Cluster sizes with average intra-cluster distance:

```
  size  av_dist  
1   69 15.62156  
2  107 18.84972  
3    9 22.09828
```

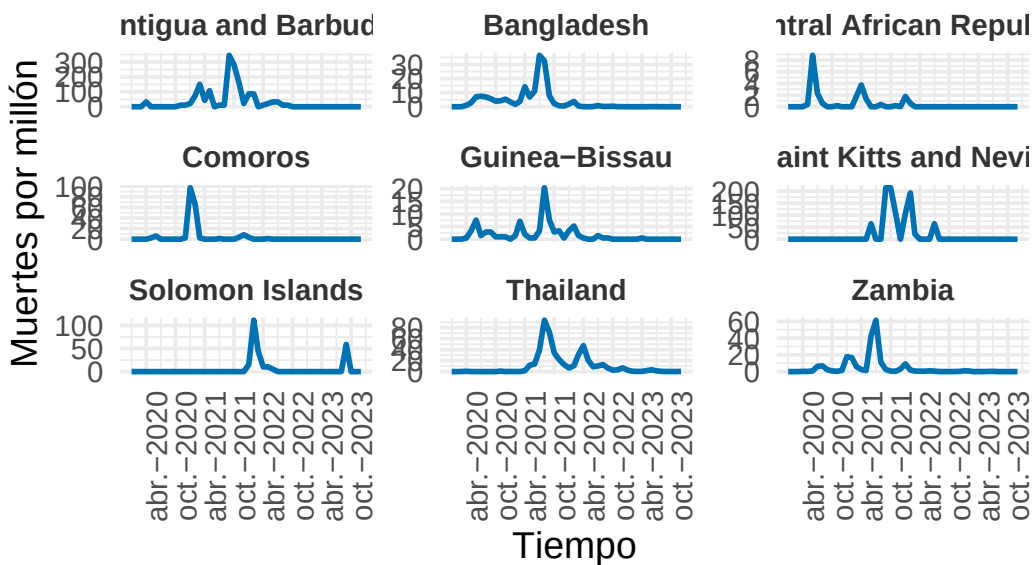
Volvemos a obtener un cluster en el que hay un grupo con muy pocas observaciones. Hagamos algunos gráficos.

```
visualizar_series_centroides(serie_new_deaths_z,cl_hc_dtw_3_deaths,3,ylab="Muertes  
  ↪ por millón normalizadas")
```



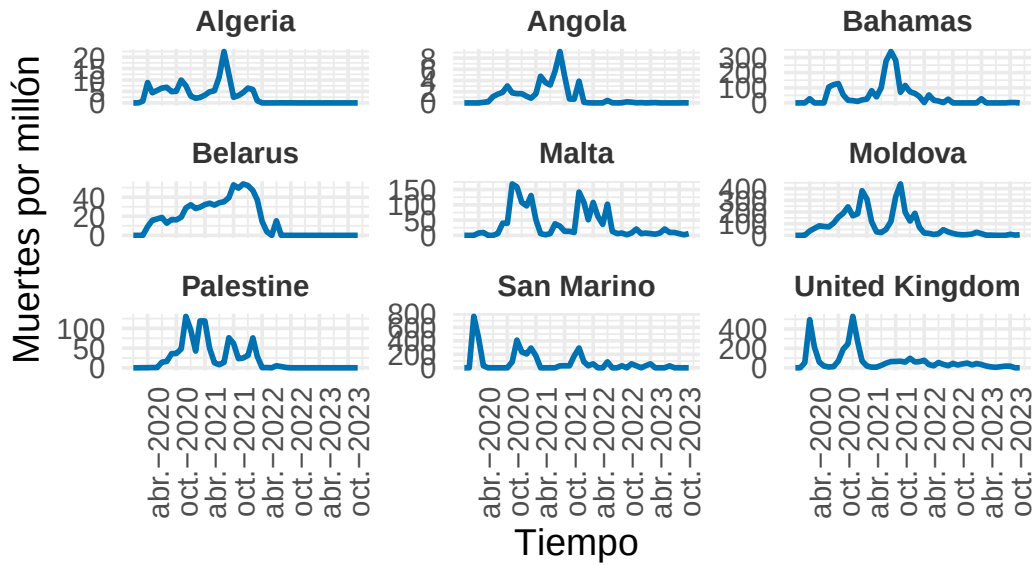
```
visualizar_ejemplos(serie_new_deaths,cl_hc_dtw_3_deaths,1,9,c(3,3),42,ylab="Muertes
↪ por millón")
```

Ejemplos del Cluster 1



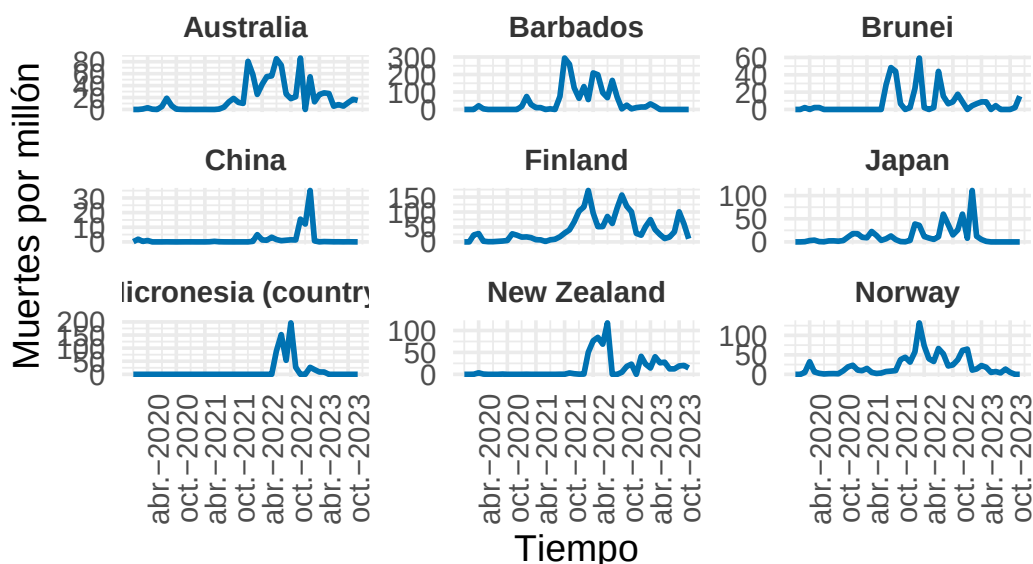
```
visualizar_ejemplos(serie_new_deaths,cl_hc_dtw_3_deaths,2,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_deaths,cl_hc_dtw_3_deaths,3,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 3



Pese a todo, parece que la agrupación es aceptable. Veamos las métricas.

```
set.seed(42)
cvi(cl_ndtw_dba_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1113985	0.4345843	78.2844764	1.8778449	1.9657758	0.1222198	0.5362779

```
cvi(cl_hc_dtw_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.2143724	0.0000000	75.2116170	2.0964065	2.1531657	0.2084851	0.4818585

Se obtiene mejor CH para el mejor agrupamiento hasta ahora, por tanto como es la métrica más fiable cuando usamos nDTW seguiremos eligiendo dicho modelo. Usemos otra distancia.

Distancia Shape-Based

```
cl_hc_sbd_3_deaths = tsclust(serie_new_deaths_z, type = "hierarchical", k =
  ↪ 3,
distance = "sbd", centroid = shape_extraction, control =
  ↪ hierarchical_control(method = diana),
```

```
args = tsclust_args(dist = list(window.size = window))
cl_hc_sbd_3_deaths
```

hierarchical clustering with 3 clusters

Using sbd distance

Using shape_extraction centroids

Using method diana

Time required for analysis:

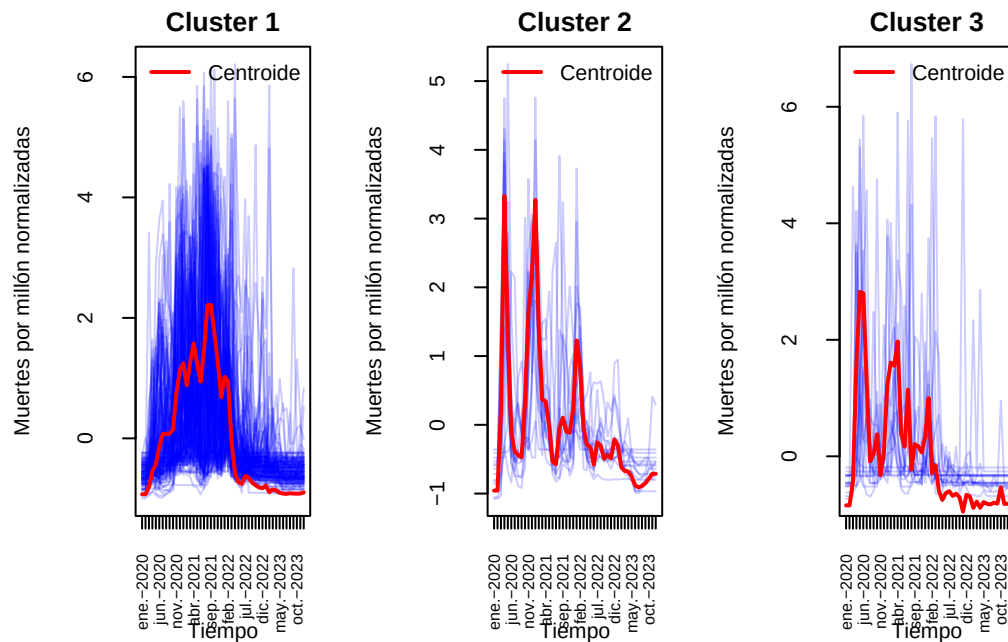
	user	system	elapsed
	0.27	0.00	0.03

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	154	0.3632790
2	14	0.2353487
3	17	0.4167338

Volvemos a tener el mismo problema, un grupo tiene demasiadas observaciones. Representemos la series junto a los centroides.

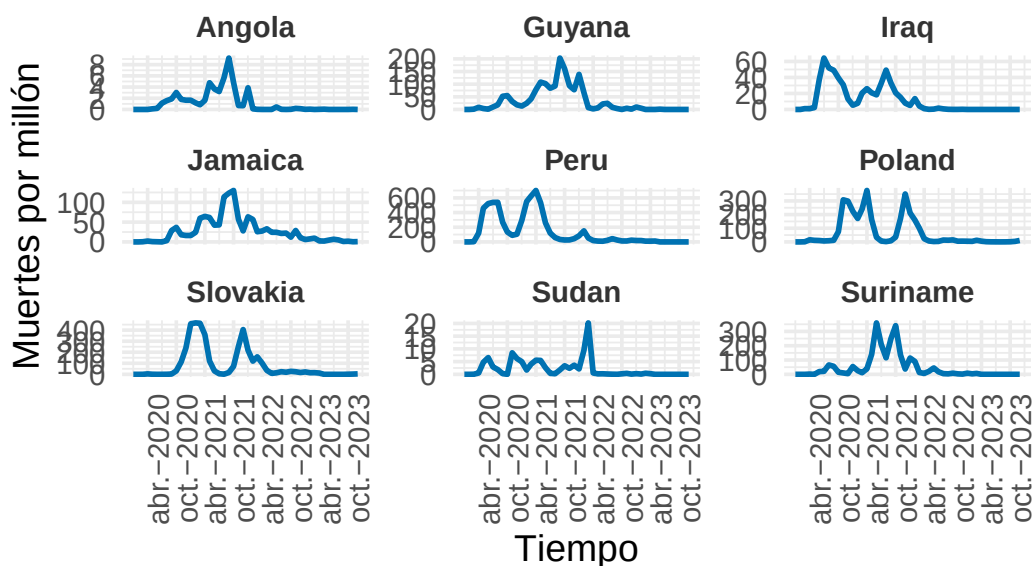
```
visualizar_series_centroides(serie_new_deaths_z,cl_hc_sbd_3_deaths,3,ylab="Muertes  
↔ por millón normalizadas")
```



Esta agrupación parece pobre, pues además casi todas las observaciones se concentran en el grupo 1.

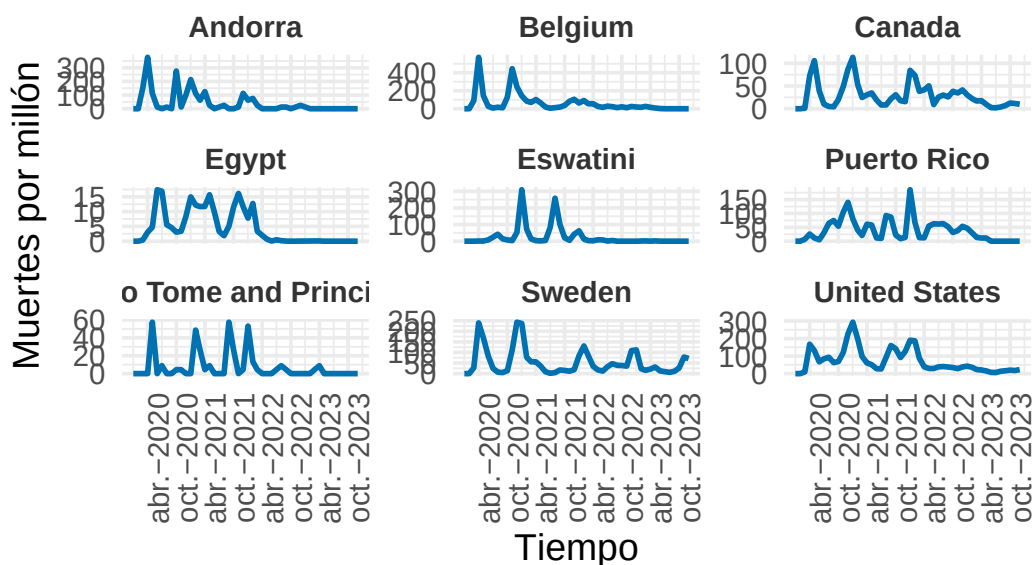
```
visualizar_ejemplos(serie_new_deaths,cl_hc_sbd_3_deaths,1,9,c(3,3),42,ylab="Muertes
  ↪ por millón")
```

Ejemplos del Cluster 1



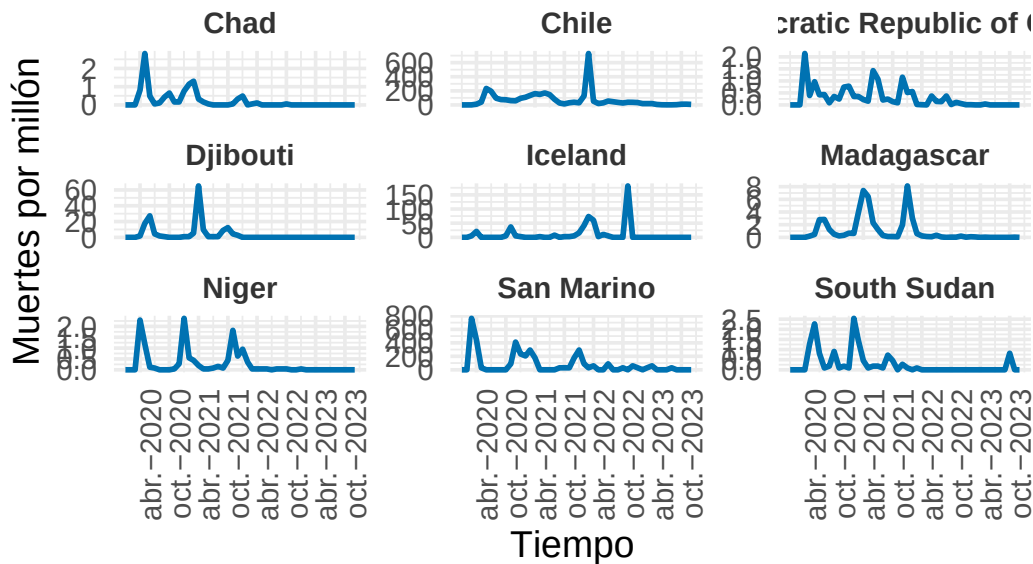
```
visualizar_ejemplos(serie_new_deaths,cl_hc_sbd_3_deaths,2,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_new_deaths,cl_hc_sbd_3_deaths,3,9,c(3,3),42,ylab="Muertes  
↪ por millón")
```

Ejemplos del Cluster 3



El agrupamiento parece peor que en el mejor cluster encontrado.

```
set.seed(42)
cvi(cl_ndtw_dba_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1113985	0.4345758	78.2496594	1.8778449	1.9657758	0.1222198	0.5362779

```
cvi(cl_hc_sbd_3_deaths,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1761248	0.3099045	17.9764527	2.2416082	2.3894459	0.1069128	0.6701176

Efectivamente se obtiene un valor de CH mucho peor.

En conclusión para las muertes volvemos a considerar cluster particional con distancia nDTW y centroides hallados usando DBA. La agrupación obtenida parece bastante decente en términos de separación entre cluster y variabilidad interna. Pasemos ahora a hacer un análisis multivariante considerando tanto contagios como muertes.

Países con trayectorias de contagios y muertes similares

Ahora pasemos a un enfoque multivariante. El paquete dtwclust soporta también series multivariantes, sin embargo pocas distancias pueden soportar series de esta forma. De hecho dtwclust solo soporta DTW y GAK.

Pongamos los datos como los espera dtwclust.

```
# Nos quedamos con los paises que estan en ambas
# Intersección de nombres comunes
países_comunes = intersect(names(serie_new_cases), names(serie_new_deaths))

# Filtrar ambas listas
serie_new_cases_f = serie_new_cases[países_comunes]
serie_new_deaths_f = serie_new_deaths[países_comunes]
serie_new_cases_z_f = serie_new_cases_z[países_comunes]
serie_new_deaths_z_f = serie_new_deaths_z[países_comunes]

# Formato adecuado
series_multivar = mapply(function(casos, muertes) {
  rbind(casos, muertes)
}, serie_new_cases_f, serie_new_deaths_f, SIMPLIFY = FALSE)

# Para la variable normalizada
series_multivar_z = mapply(function(casos, muertes) {
  rbind(casos, muertes)
}, serie_new_cases_z_f, serie_new_deaths_z_f, SIMPLIFY = FALSE)
```

Empecemos con el clustering particional.

Partitional Clustering

Distancia DTW

```
# Definimos el rango de k a evaluar
k_values = 2:10
sil_scores = numeric(length(k_values))
# Matriz de distancias entre todas las series (usando dtw para multivariante)
dist_matrix = proxy::dist(series_multivar_z, method = "dtw_basic",
  ↪ window.size = window) # Lo que cogimos anteriormente

for (i in seq_along(k_values)) {
```

```

k = k_values[i]
cat("Evaluando k =", k, "...\\n")
set.seed(42)
cl = tsclust(series_multivar_z, type = "partitional", k = k, distance =
↪  "dtw_basic", centroid = "dba", seed = 12345)

sil = silhouette(cl@cluster, dist_matrix)

# Promedio de los coeficientes de silueta
sil_scores[i] = mean(sil[, 3])
}

```

```

Evaluando k = 2 ...
Evaluando k = 3 ...
Evaluando k = 4 ...
Evaluando k = 5 ...
Evaluando k = 6 ...
Evaluando k = 7 ...
Evaluando k = 8 ...
Evaluando k = 9 ...
Evaluando k = 10 ...

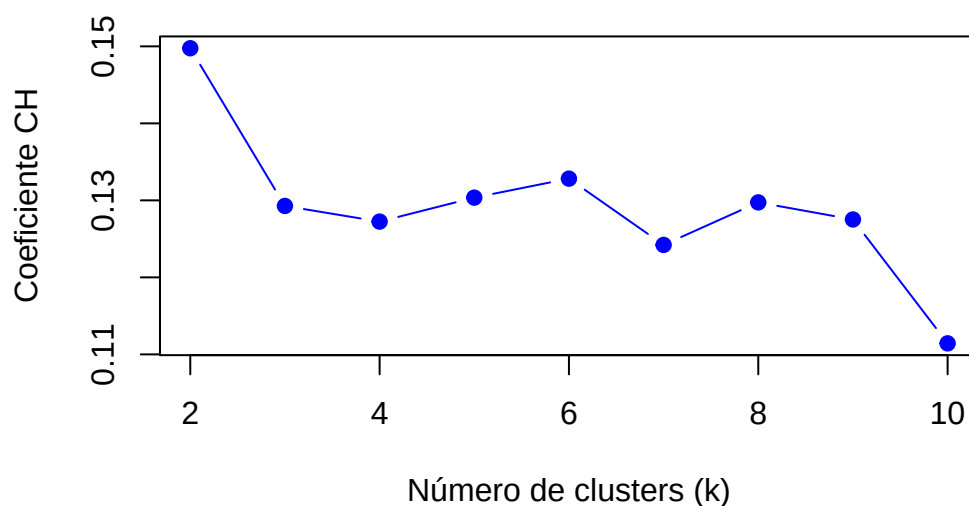
```

```

# Graficar los resultados
par(mfrow = c(1, 1))
plot(k_values, sil_scores, type = "b", pch = 19, col = "blue",
     xlab = "Número de clusters (k)", ylab = "Coeficiente CH",
     main = "Selección de k usando Silhouette con DTW")

```

Selección de k usando Silhouette con DTW



Obtenemos el máximo de la métrica para $k = 2$ nuevamente. Haremos como anteriormente, estudiemos el caso trivial $k = 2$ y después el segundo k más alto, en este caso $k = 6$.

Caso $k = 2$

```
set.seed(42)
cl_dtw_dba_mv_2 = tsclust(series_multivar_z, type = "partitional", k = 2,
  ↪ distance = "dtw_basic", centroid = "dba", seed = 12345)
cl_dtw_dba_mv_2
```

```
partitional clustering with 2 clusters
Using dtw_basic distance
Using dba centroids
```

Time required for analysis:

	user	system	elapsed
	10.21	0.80	0.47

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	89	65.29280
2	96	63.29271

Vamos a representar los centroides junto a las series.

```
visualizar_series_centroides_mv <- function(series, cl, k, variable = NULL,
                                             ylab = "Valor",
                                             fecha_inicio = "2020-01-01",
                                             frecuencia = "month",
                                             tamano_ventana = c(1,k)) {
  par(mfrow = tamano_ventana, mar = c(5, 6, 2, 1), mgp = c(4, 1.3, 0))
  max_len <- max(sapply(series, length))
  fechas <- seq(as.Date(fecha_inicio), by = frecuencia, length.out = max_len)

  for (cluster_id in 1:k) {
    series_cluster <- series[cl@cluster == cluster_id]
    series_cluster <- lapply(series_cluster, as.numeric)

    matplot(fechas[seq_len(max_len)],
             do.call(cbind, lapply(series_cluster, function(s) {
               length(s) <- max_len
               return(s)
             })),
             type = "l", lty = 1, col = rgb(0, 0, 1, 0.2),
             main = paste("Cluster", cluster_id),
             ylab = ylab, xlab = "Tiempo", xaxt = "n")

    ticks <- seq(min(fechas), max(fechas), by = "6 months")
    axis(1, at = ticks, labels = format(ticks, "%b-%Y"),
         las = 2, cex.axis = 0.8)

    centroide <- cl@centroids[[cluster_id]]

    if (is.null(variable)) {
      num_vars <- nrow(centroide)
      cols <- c("red", "green", "orange", "purple", "brown")
      for (v in 1:num_vars) {
        lines(fechas[seq_len(ncol(centroide))],
              as.numeric(centroide[v, ]), col = cols[v], lwd = 2)
      }
      legend("topright", legend = paste("Centroide var", 1:num_vars),
            col = cols[1:num_vars], lwd = 2, bty = "n")
    } else {
      lines(fechas[seq_len(ncol(centroide))],
            as.numeric(centroide[variable, ]), col = "red", lwd = 2)
      legend("topright", legend = paste("Centroide var", variable),
```

```

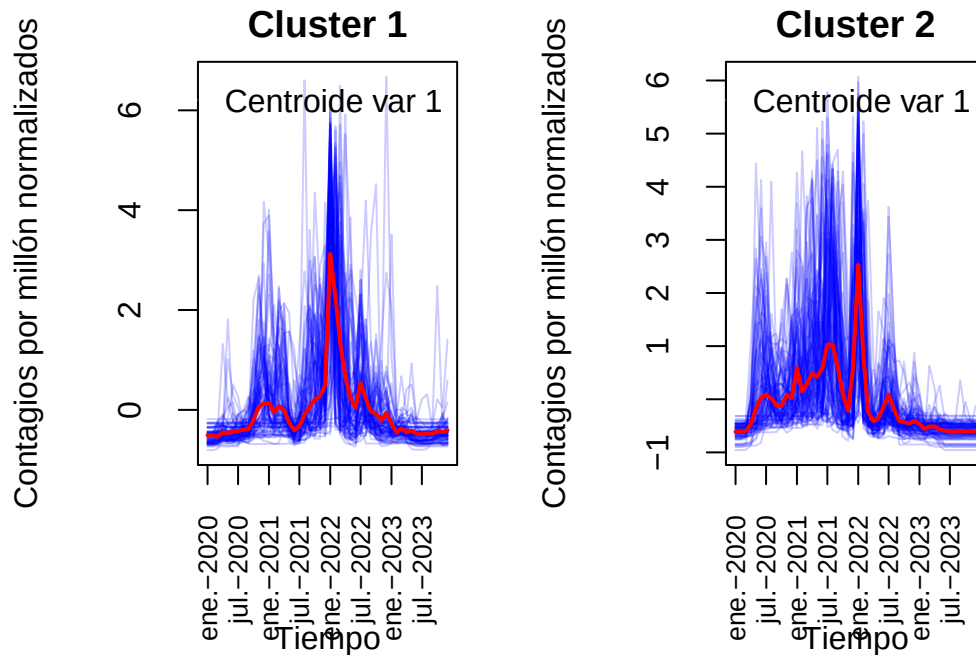
        col = "red", lwd = 2, bty = "n")
    }
}
}

```

```

visualizar_series_centroides_mv(serie_new_cases_z_f, cl_dtw_dba_mv_2, k =
  ↪ 2, variable=1, ylab="Contagios por millón normalizados")

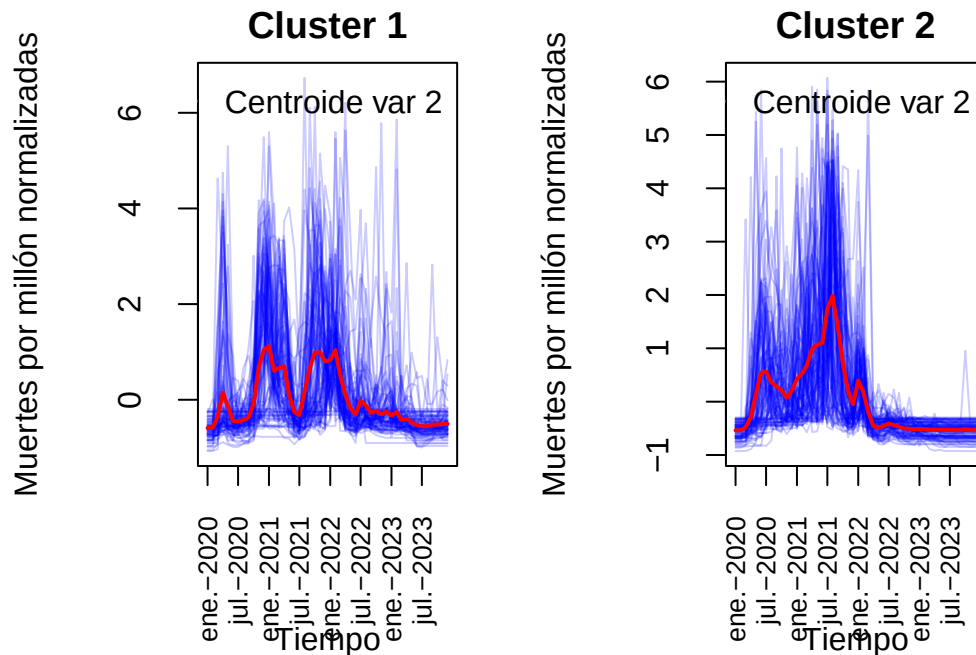
```



```

visualizar_series_centroides_mv(serie_new_deaths_z_f, cl_dtw_dba_mv_2, k = 2,
  ↪ variable = 2, ylab = "Muertes por millón normalizadas")

```



Se puede ver que el primer cluster corresponde con series con más de un pico de muertes y un solo pico de contagios, mientras que en el cluster 2 suele haber más brotes de contagios y un solo pico de muertes que destaque del resto. Veamos algunos ejemplos.

```
visualizar_ejemplos_multivariado = function(series, cl, n_cluster,
  ↪ n_ejemplos,

                                tamaño_ventana = c(1,1),
                                ↪ semilla = NULL,
                                variables = c("casos",
                                ↪ "muertes"),
                                nombres_vars =
                                ↪ c("new_cases_per_million",
                                ↪ "new_deaths_per_million"))
                                ↪ {

  if (!is.null(semilla)) set.seed(semilla)

  # Índices de las series del cluster especificado
  ind_cluster = sample(which(cl@cluster == n_cluster), min(n_ejemplos,
  ↪ sum(cl@cluster == n_cluster)))

  # Nombres de series
  nombres = names(series)
  if (is.null(nombres)) nombres <- paste("Serie", seq_along(series))
```

```

# Extraer y transformar series
datos = purrr::map_df(ind_cluster, function(i) {
  s = series[[i]]

  if (!is.matrix(s)) stop("Cada serie debe ser una matriz con dimensiones:
    ↪ variables x tiempo.")

  s_sub = s[variables, , drop = FALSE]
  df = as.data.frame(t(s_sub))
  df$Tiempo = seq_len(ncol(s_sub))

  df_long = tidyr::pivot_longer(df, cols = -Tiempo, names_to = "Variable",
    ↪ values_to = "Valor")
  df_long$Serie = nombres[i]

  df_long$Variable = factor(df_long$Variable,
                           levels = variables,
                           labels = nombres_vars)

  df_long
})

# Separar por variable
for (var in levels(datos$Variable)) {
  datos_var = datos %>% filter(Variable == var)

  print(
    ggplot2::ggplot(datos_var, aes(x = Tiempo, y = Valor, color = Serie,
    ↪ group = Serie)) +
    geom_line(alpha = 0.7, linewidth = 1) +
    facet_wrap(~ Serie, nrow = tamaño_ventana[1], ncol =
    ↪ tamaño_ventana[2], scales = "free_y") +
    labs(title = paste("Ejemplos del Clúster", n_cluster, "-", var),
         x = "Tiempo", y = var) +
    theme_minimal(base_size = 14) +
    theme(strip.text = element_text(face = "bold"),
          plot.title = element_text(size = 18, face = "bold", hjust =
    ↪ 0.5))
  )
}
}

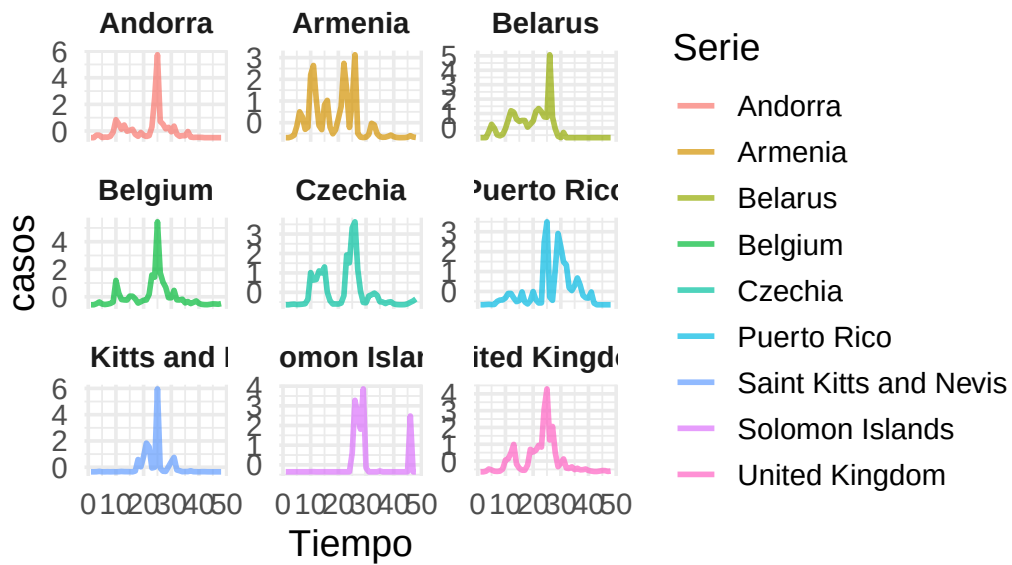
```

```

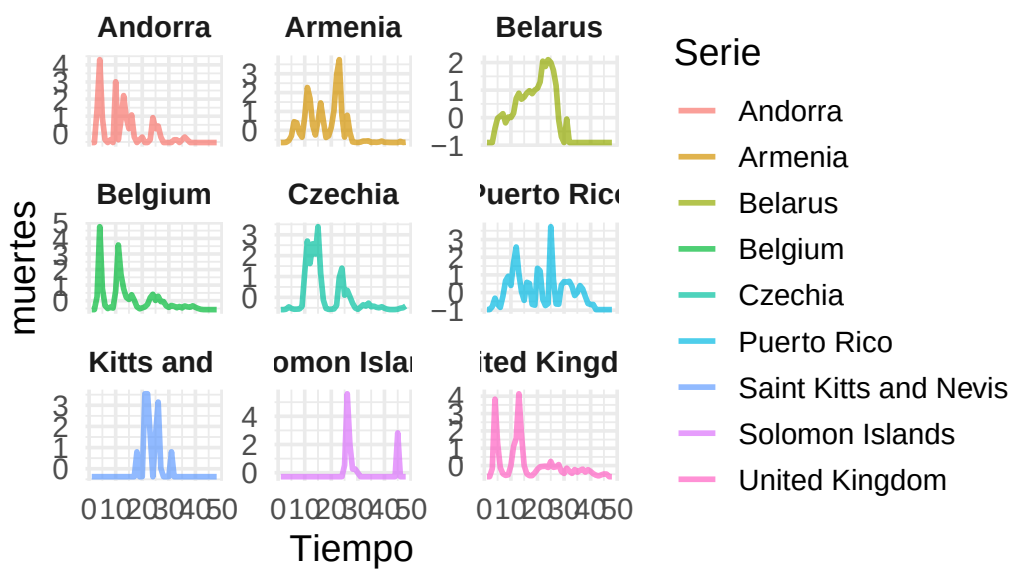
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_dtw_dba_mv_2,
  n_cluster = 1, n_ejemplos = 9,
  tamano_ventana = c(3, 3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)

```

Ejemplos del Clúster 1 – casos

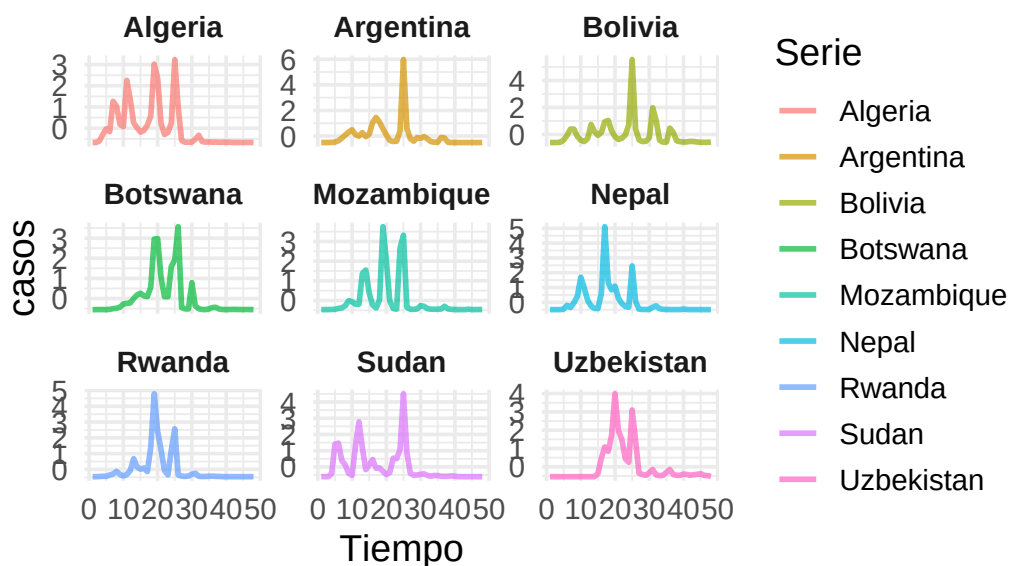


Ejemplos del Clúster 1 – muertes

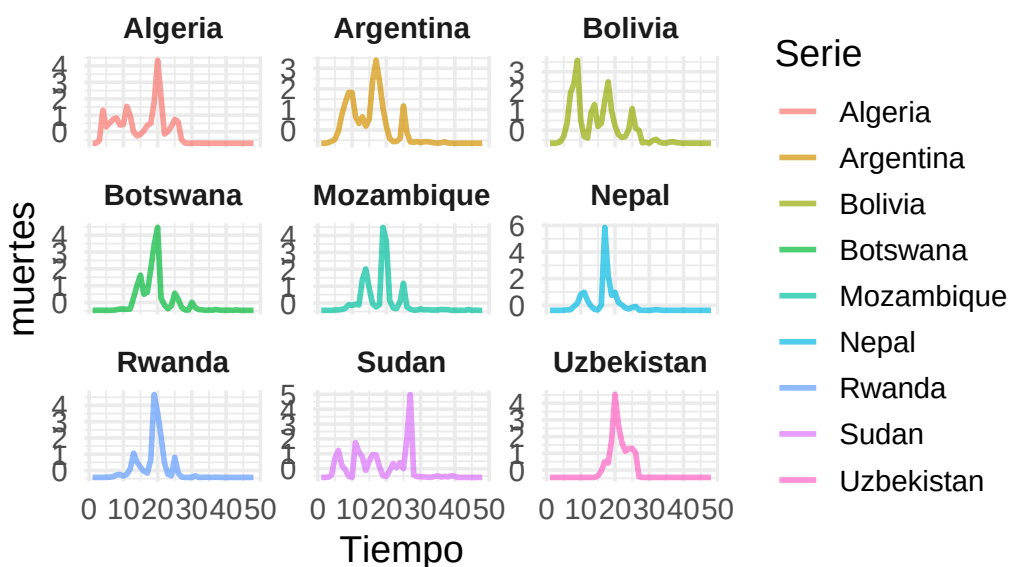


```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_dtw_dba_mv_2,
  n_cluster = 2, n_ejemplos = 9,
  tamaño_ventana = c(3, 3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

Ejemplos del Clúster 2 – casos



Ejemplos del Clúster 2 – muertes



Parece reflejarse lo que dijimos anteriormente.

```
cvi(cl_dtw_dba_mv_2,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1497552	0.0000000	88.0163829	2.2459458	2.2459458	0.2117054	0.5639719

Hemos conseguido segmentar en dos tipos de países, países con un único pico de contagios marcado y varios picos de muertes y países con varios picos de contagios además del importnate y un único pico de muertes que sobresale del resto. Estudiemos ahora el caso $k = 6$.

Caso $k = 6$

```
set.seed(42)
cl_dtw_dba_mv_6 = tsclust(series_multivar_z, type = "partitional", k = 6,
  ↪ distance = "dtw_basic", centroid = "dba", seed = 12345)
cl_dtw_dba_mv_6
```

```
partitional clustering with 6 clusters
Using dtw_basic distance
Using dba centroids
```

Time required for analysis:

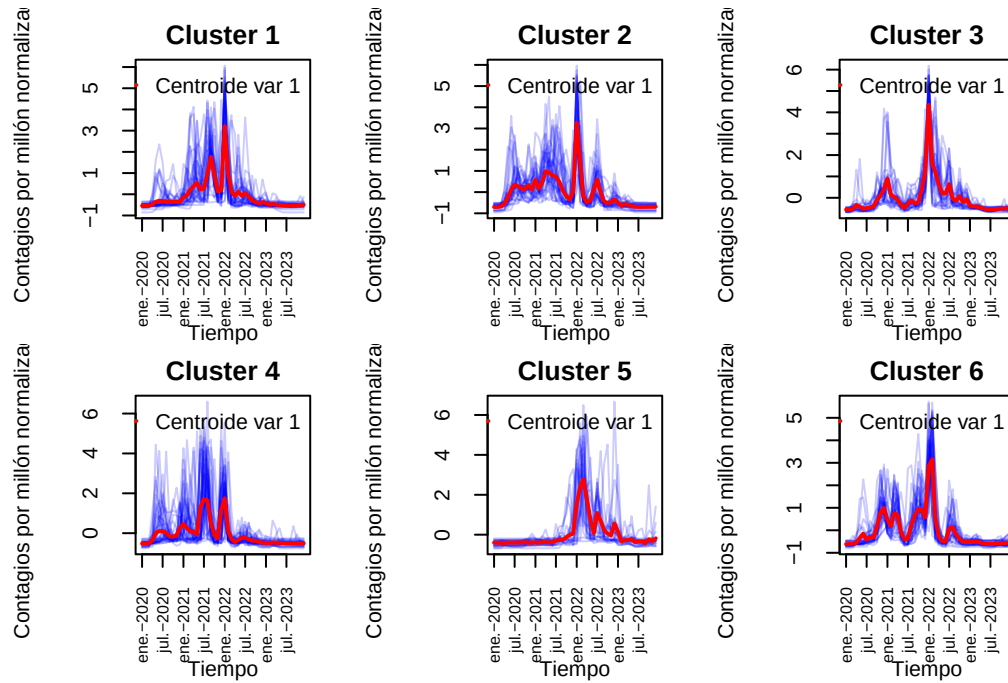
user	system	elapsed
2.81	0.39	0.37

Cluster sizes with average intra-cluster distance:

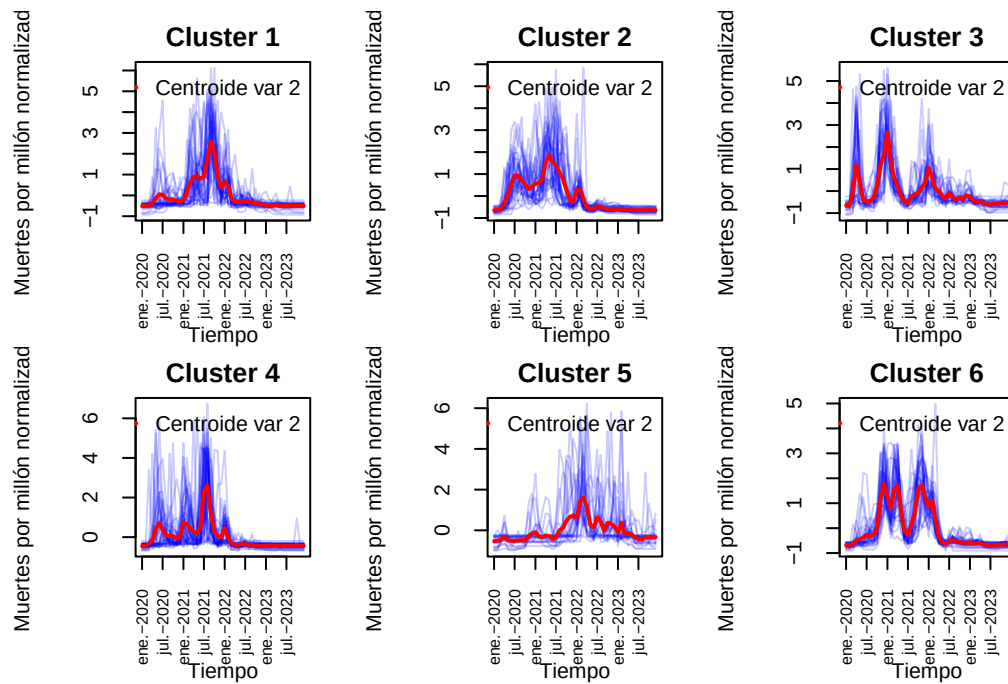
	size	av_dist
1	33	54.82798
2	32	55.85513
3	28	51.36290
4	39	56.43263
5	22	61.14819
6	31	46.77912

Visualicemos los centroides de cada cluster junto a las series temporales

```
visualizar_series_centroides_mv(serie_new_cases_z_f, cl_dtw_dba_mv_6, k =
  ↪ 6, variable=1, "Contagios por millón normalizados", tamaño_ventana = c(2,3))
```



```
visualizar_series_centroides_mv(serie_new_deaths_z_f, cl_dtw_dba_mv_6, k =6,
↵ variable = 2,"Muertes por millón normalizadas",tamano_ventana = c(2,3))
```

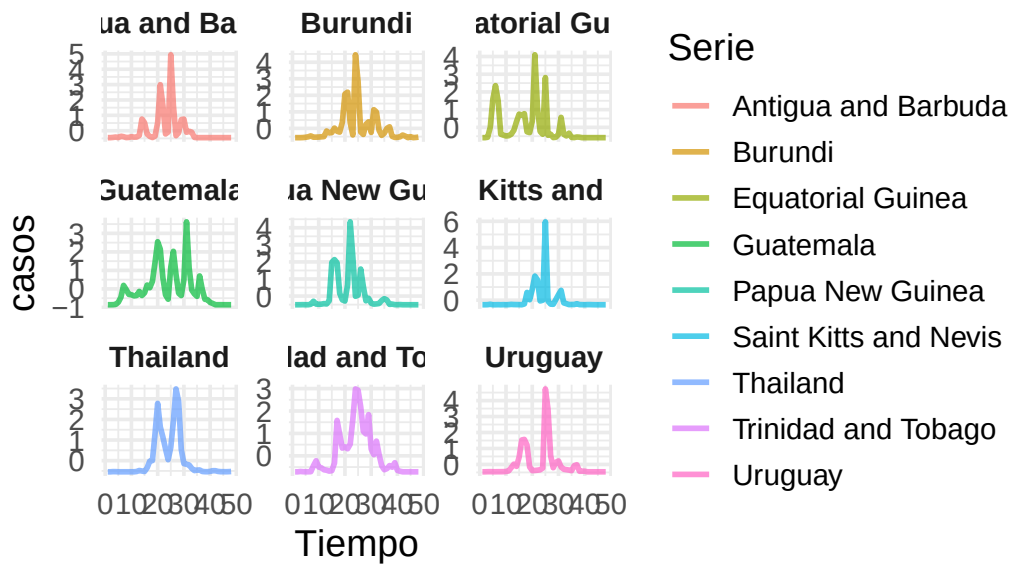


```

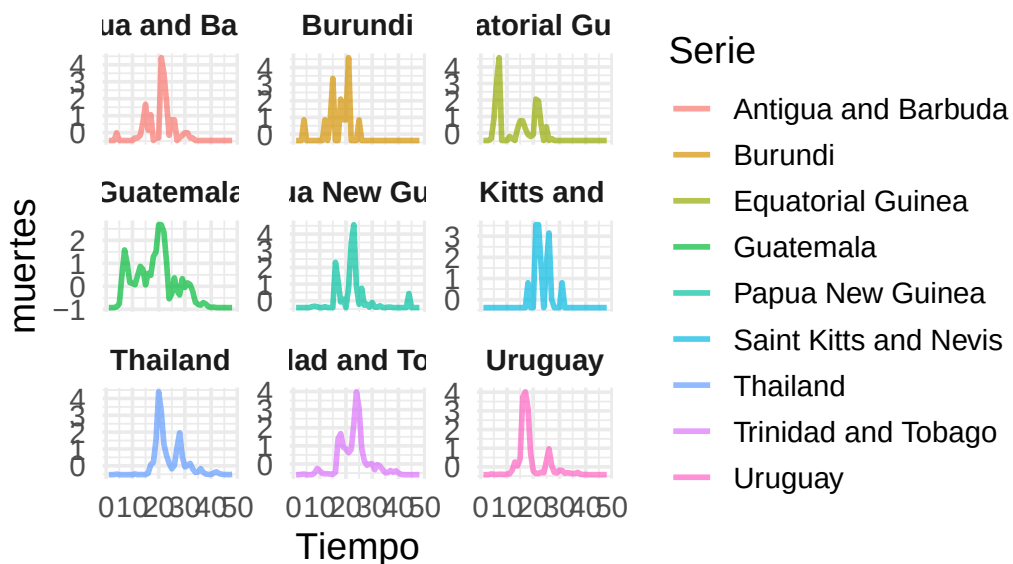
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_dtw_dba_mv_6,
  n_cluster = 1, n_ejemplos = 9,
  tamaño_ventana = c(3, 3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)

```

Ejemplos del Clúster 1 – casos

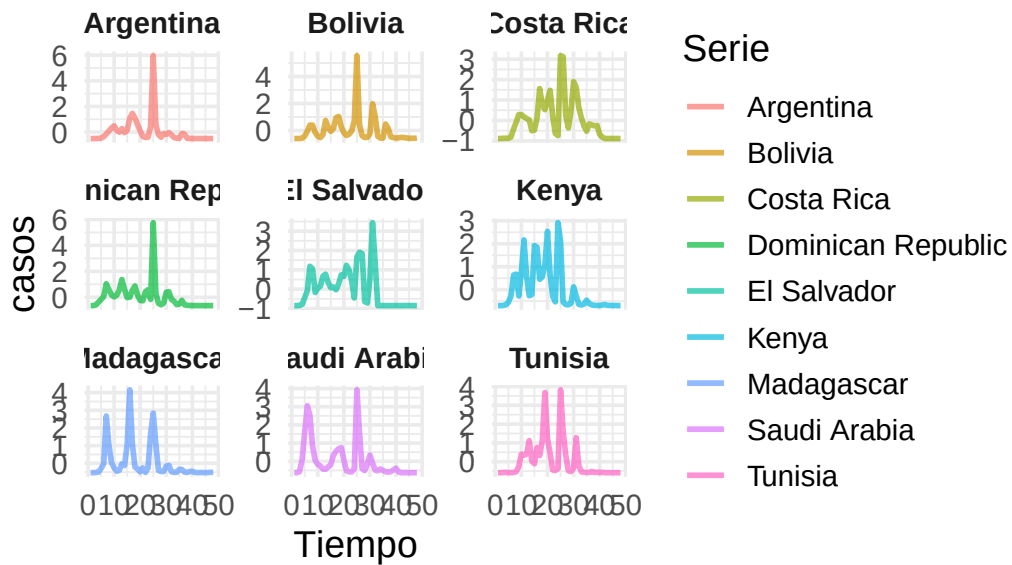


Ejemplos del Clúster 1 – muertes

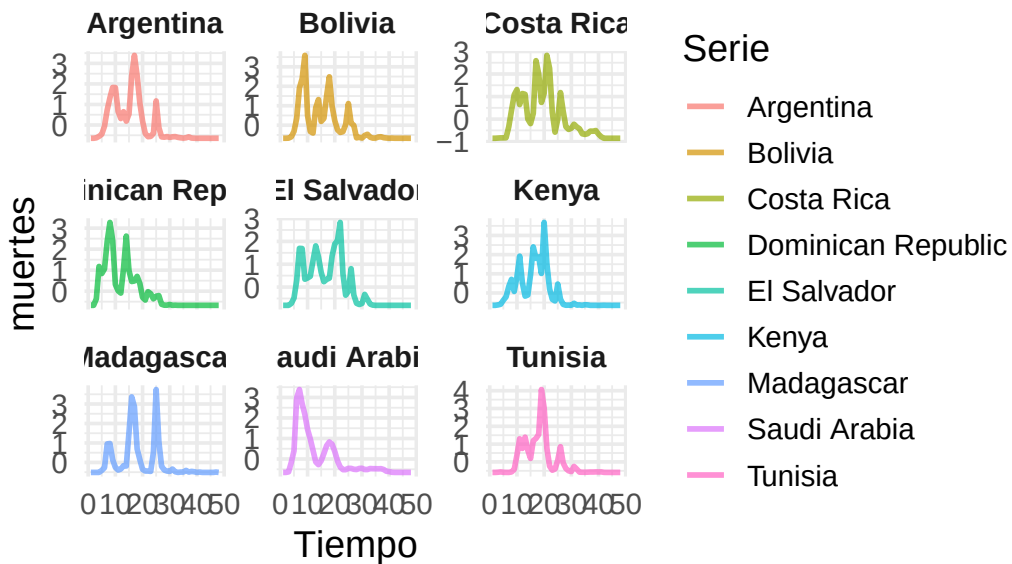


```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_dtw_dba_mv_6,
  n_cluster = 2, n_ejemplos = 9,
  tamaño_ventana = c(3, 3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

Ejemplos del Clúster 2 – casos



Ejemplos del Clúster 2 – muertes



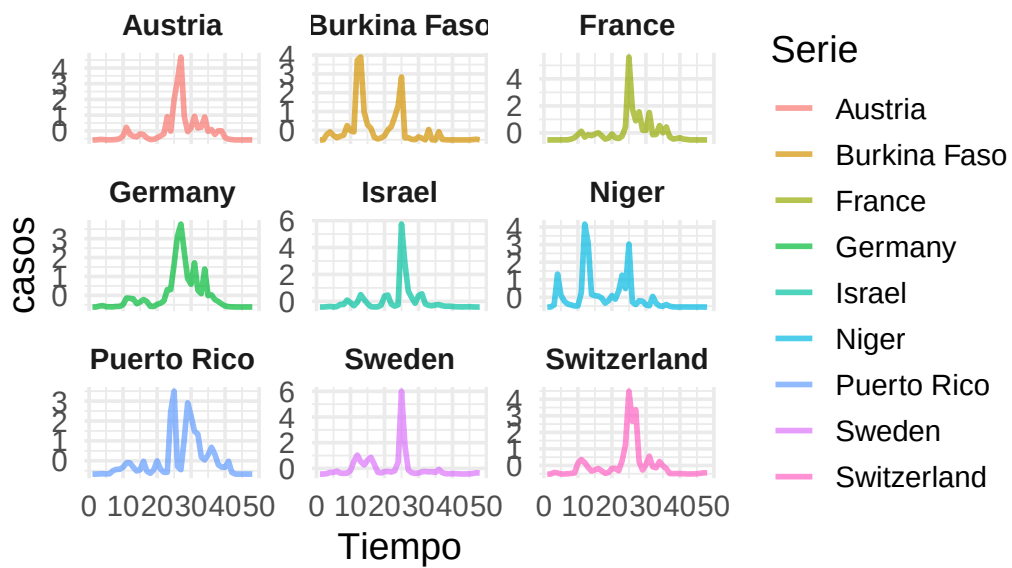
```
visualizar_ejemplos_multivariado(  
  series_multivar_z, cl_dtw_dba_mv_6,  
  n_cluster = 3, n_ejemplos = 9,
```

```

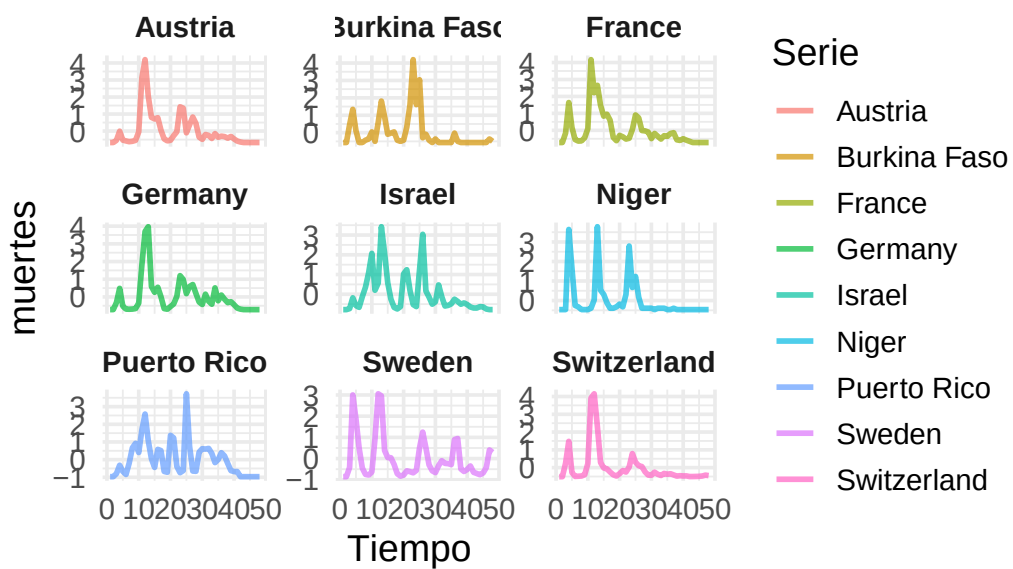
tamano_ventana = c(3, 3), semilla = 42,
variables = c("casos", "muertes"),
nombres_vars = c("casos", "muertes")
)

```

Ejemplos del Clúster 3 – casos

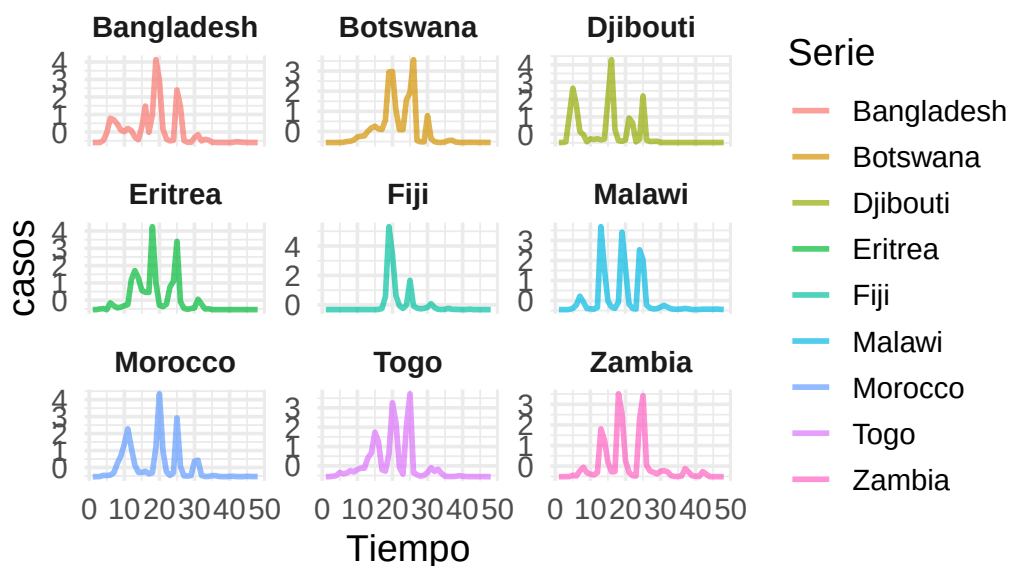


Ejemplos del Clúster 3 – muertes

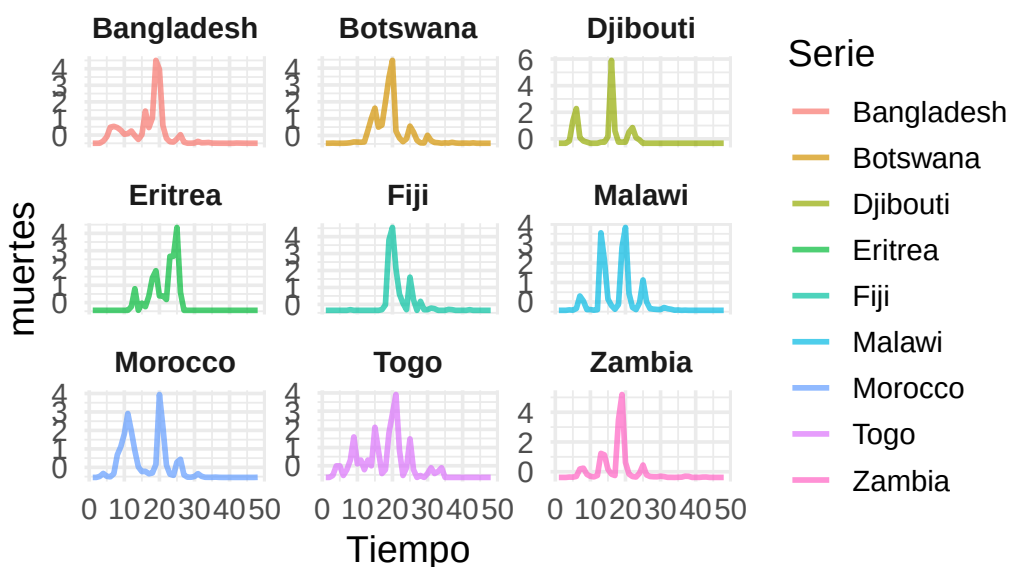


```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_dtw_dba_mv_6,
  n_cluster = 4, n_ejemplos = 9,
  tamano_ventana = c(3,3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

Ejemplos del Clúster 4 – casos



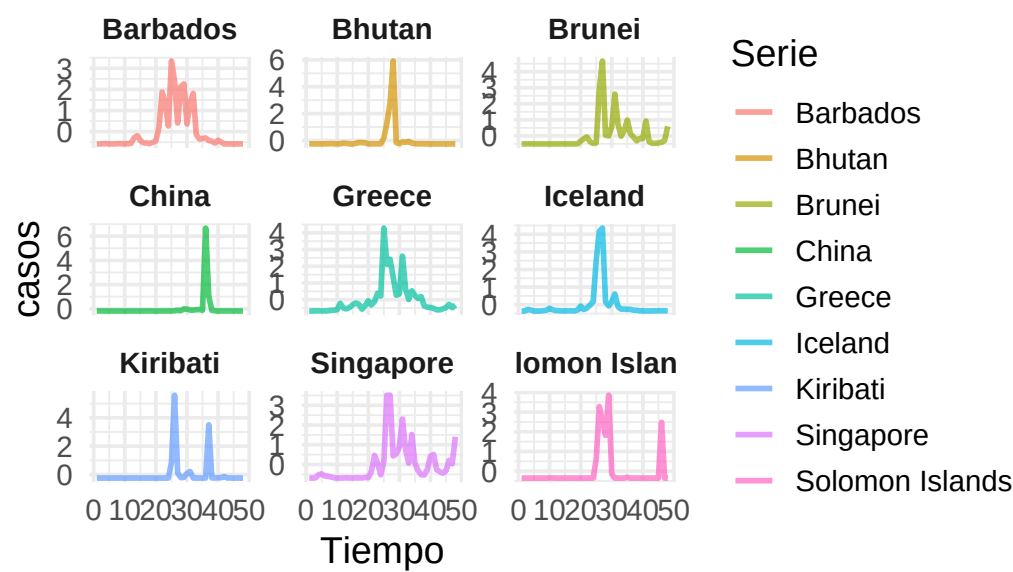
Ejemplos del Clúster 4 – muertes



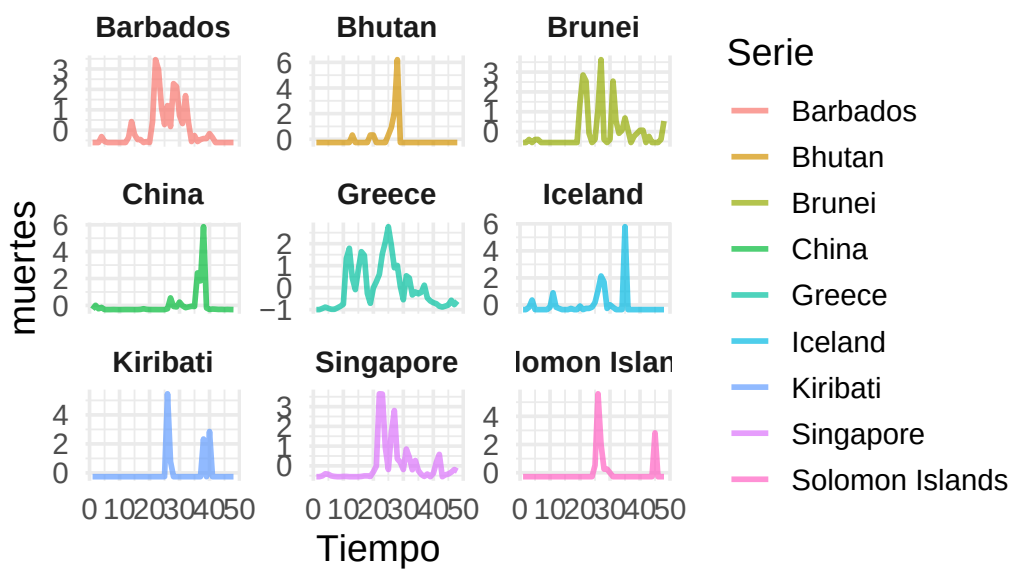
```
visualizar_ejemplos_multivariado(  
  series_multivar_z, cl_dtw_dba_mv_6,  
  n_cluster = 5, n_ejemplos = 9,
```

```
tamano_ventana = c(3,3), semilla = 42,  
variables = c("casos", "muertes"),  
nombres_vars = c("casos", "muertes")  
)
```

Ejemplos del Clúster 5 – casos

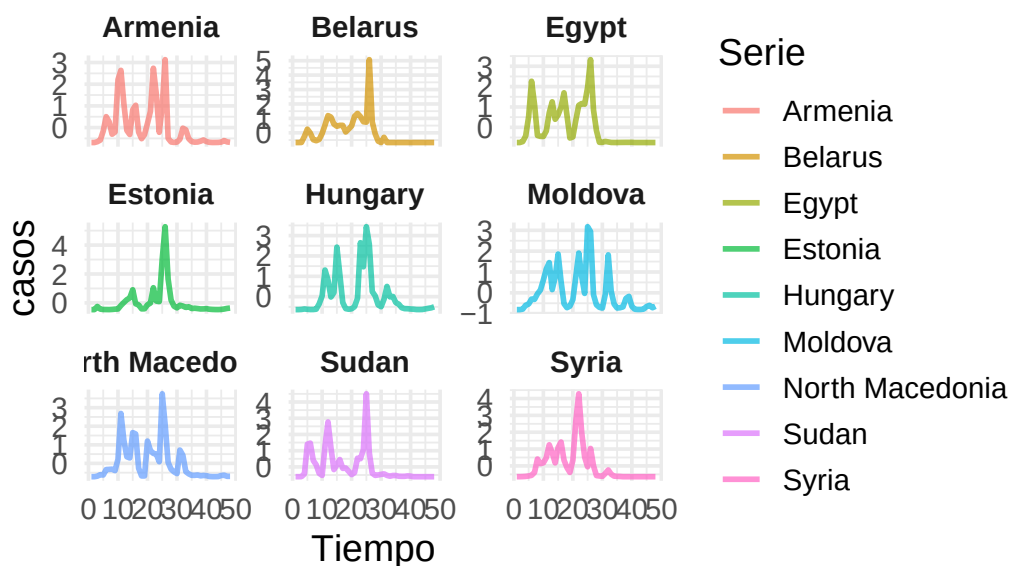


Ejemplos del Clúster 5 – muertes

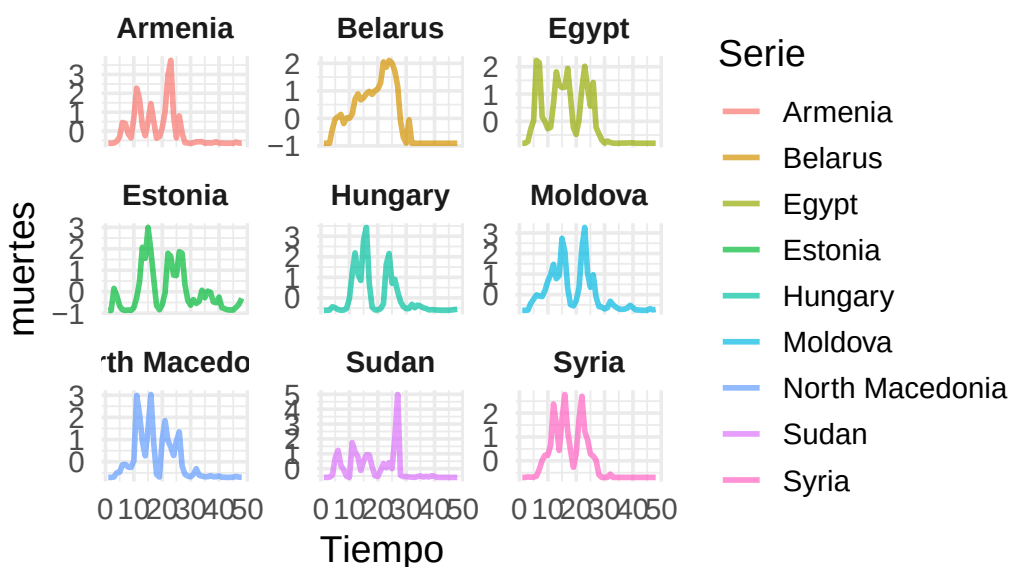


```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_dtw_dba_mv_6,
  n_cluster = 6, n_ejemplos = 9,
  tamaño_ventana = c(3,3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

Ejemplos del Clúster 6 – casos



Ejemplos del Clúster 6 – muertes



Parece que hemos conseguido clústers con mucho sentido. Se pueden observar dos cosas importantes.

- Respecto a la similitud intra-clúster, parece que hemos conseguido clusters bastante

homogéneos, esto es signo de buena agrupación.

- Además los clusters son bastante distintos entre sí, aunque haya agrupaciones con contagios similares (por ejemplo cluster 3 y 5), si observamos las muertes, estas siguen patrones completamente diferentes. Esto también es buena señal de que nuestra agrupación parece coherente.

```
cvi(cl_dtw_dba_mv_6,type="internal")
```

	Sil	SF	CH	DB	DBstar	D	COP
	0.1328205	0.0000000	29.1601125	2.0727817	2.1910412	0.2714423	0.4441670

Usemos ahora otra distancia.

Distancia Global Alignment Kernel

```
set.seed(42)
cl_dtw_gak_mv_6 = tsclust(series_multivar_z, type = "partitional", k = 6,
  ↪ distance = "gak", centroid = "pam", seed = 12345)
cl_dtw_gak_mv_6
```

partitional clustering with 6 clusters

Using gak distance

Using pam centroids

Time required for analysis:

	user	system	elapsed
	5.29	0.01	5.48

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	174	0.9942528
2	1	0.0000000
3	1	0.0000000
4	1	0.0000000
5	1	0.0000000
6	7	0.8571429

No hemos conseguido segmentar prácticamente.

```
cvi(cl_dtw_gak_mv_6,type="internal")
```

	Sil	SF	CH	DB	DBstar	D
	4.409589e-10	1.466467e-01	2.200000e+00	1.279967e+00	1.279967e+00	1.000000e+00
COP						
	9.675675e-01					

Como era de esperar obtenemos métricas horribles. Pasemos a hacer ahora fuzzy.

Fuzzy Clustering

Distancia DTW

Veamos el k óptimo a parte de 2 para este caso.

```
# Definimos el rango de k a evaluar
k_values = 2:10
sil_scores = numeric(length(k_values))
# Matriz de distancias entre todas las series (usando dtw para multivariante)
dist_matrix = proxy::dist(series_multivar_z, method = "dtw_basic",
  ↪ window.size = window) # Lo que cogimos anteriormente

for (i in seq_along(k_values)) {
  k = k_values[i]
  cat("Evaluando k =", k, "...\\n")
  set.seed(42)
  cl = tsclust(series_multivar_z, type = "fuzzy", k = k, distance =
  ↪ "dtw_basic", centroid = "fcmdd", seed = 12345)

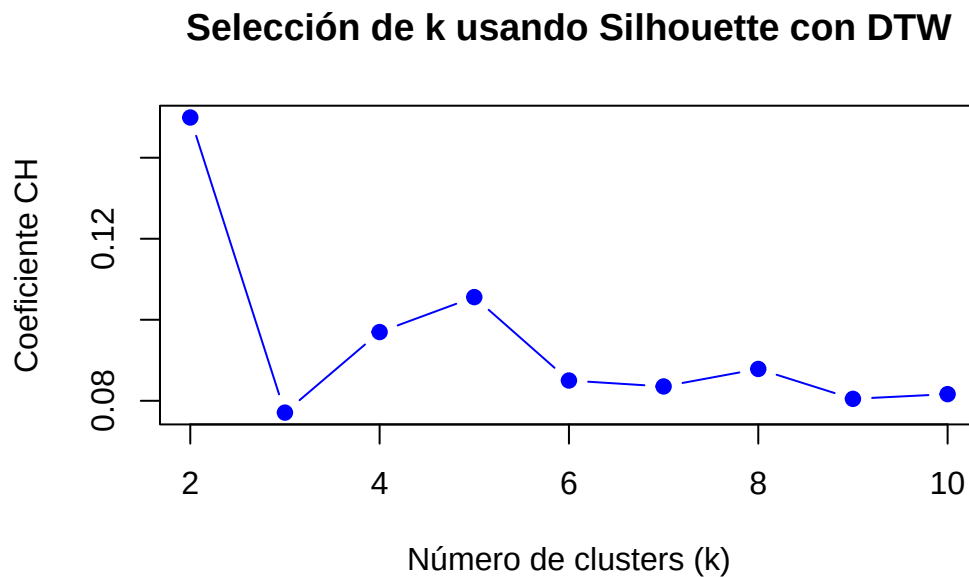
  sil = silhouette(cl@cluster, dist_matrix)

  # Promedio de los coeficientes de silueta
  sil_scores[i] = mean(sil[, 3])
}
```

```
Evaluando k = 2 ...
Evaluando k = 3 ...
Evaluando k = 4 ...
Evaluando k = 5 ...
Evaluando k = 6 ...
Evaluando k = 7 ...
```

```
Evaluando k = 8 ...  
Evaluando k = 9 ...  
Evaluando k = 10 ...
```

```
# Graficar los resultados  
par(mfrow = c(1, 1))  
plot(k_values, sil_scores, type = "b", pch = 19, col = "blue",  
      xlab = "Número de clusters (k)", ylab = "Coeficiente CH",  
      main = "Selección de k usando Silhouette con DTW")
```



Hemos obtenido $k = 5$ como el que elegiremos.

```
set.seed(42)  
cl_fuzzy_dtw_mv_5 = tsclust(series_multivar_z, type = "fuzzy", k = 5, distance  
  ↪ = "dtw_basic", centroid = "fcmd", save.data = TRUE, seed = 12345)  
cl_fuzzy_dtw_mv_5
```

fuzzy clustering with 5 clusters
Using dtw_basic distance
Using fcmd centroids

Time required for analysis:
user system elapsed

0.03 0.00 0.04

Head of fuzzy memberships:

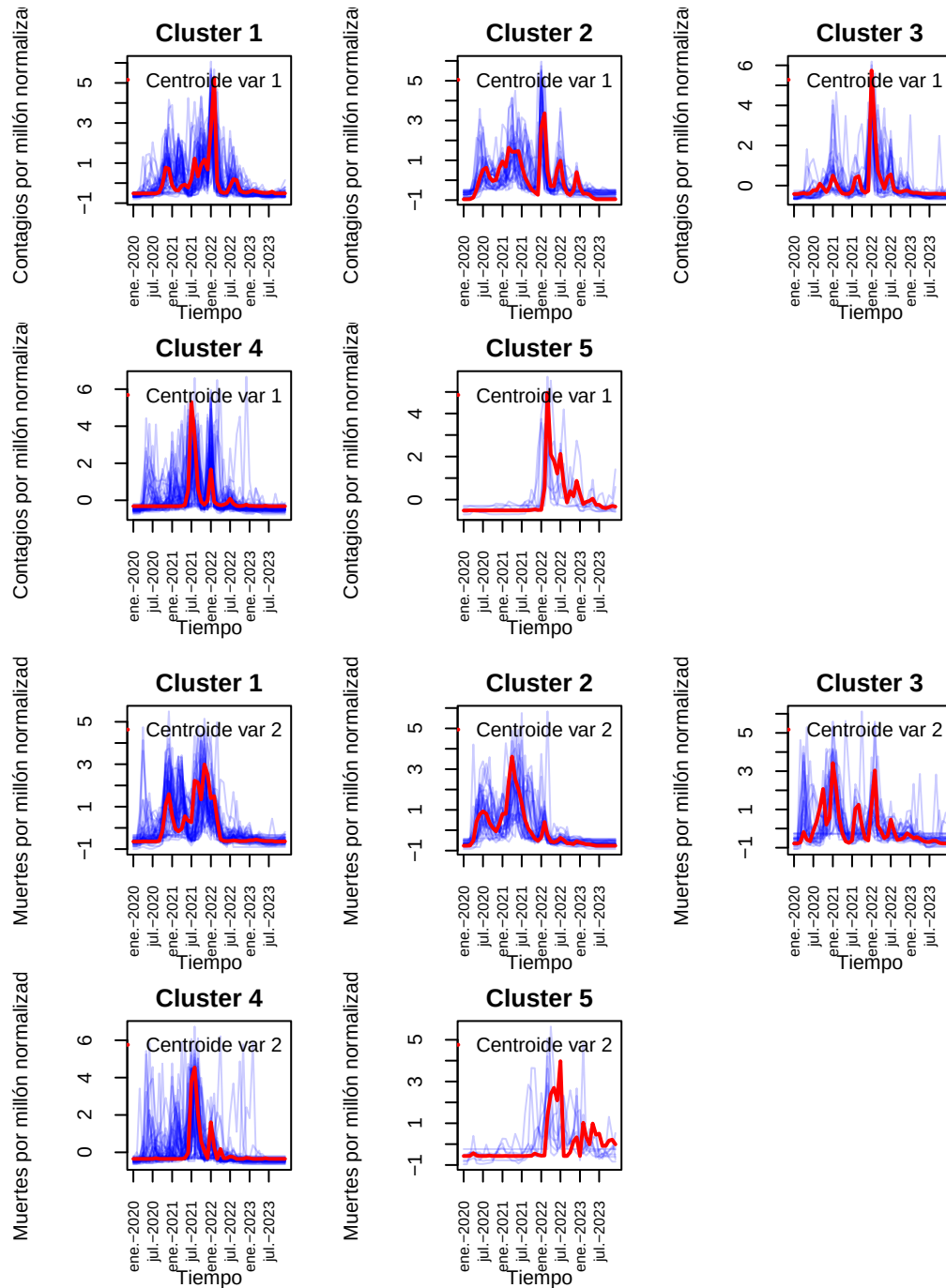
	cluster_1	cluster_2	cluster_3	cluster_4	cluster_5
Afghanistan	0.1735052	0.2079470	0.1607551	0.3493371	0.10845564
Albania	0.2908721	0.2074420	0.2769372	0.1460927	0.07865595
Algeria	0.2763969	0.2248471	0.1687015	0.2695485	0.06050613
Andorra	0.2264924	0.1597209	0.3115973	0.1895350	0.11265445
Angola	0.3210049	0.1839020	0.1984124	0.2306638	0.06601686
Antigua and Barbuda	0.2540080	0.1272524	0.2484519	0.2650146	0.10527305

```
table(cl_fuzzy_dtw_mv_5@cluster)
```

```
 1  2  3  4  5  
54 33 31 58  9
```

Representemos los centroides y las series de cada grupo.

```
visualizar_series_centroides_mv(serie_new_cases_z_f, cl_fuzzy_dtw_mv_5, k =  
  ↪ 5, variable=1, "Contagios por millón normalizados", tamano_ventana = c(2,3))  
visualizar_series_centroides_mv(serie_new_deaths_z_f, cl_fuzzy_dtw_mv_5, k  
  ↪ =5, variable = 2, "Muertes por millón normalizadas", tamano_ventana =  
  ↪ c(2,3))
```



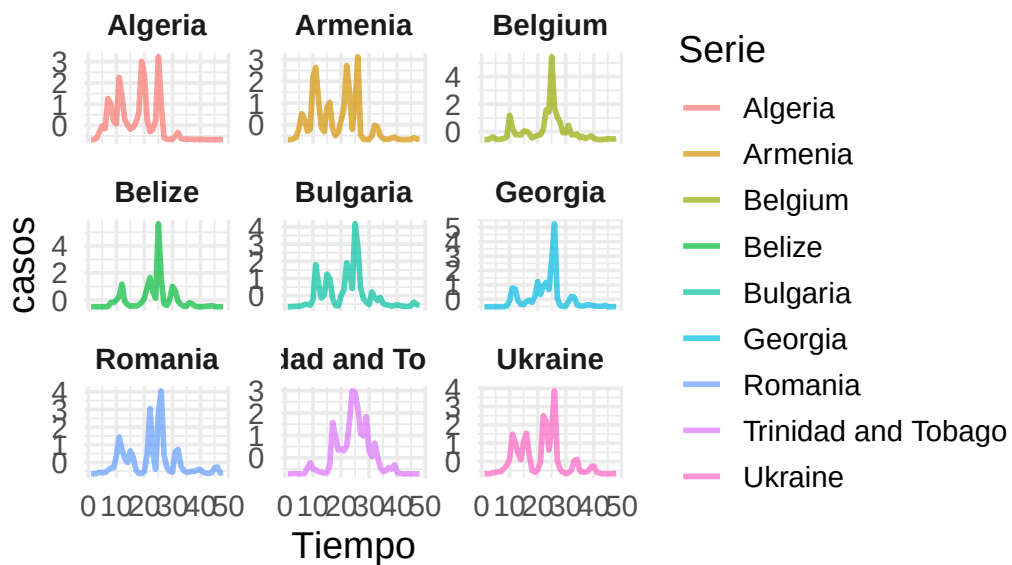
En este caso parece que vemos menos homogeneidad que antes. Visualicemos algunos ejemplos.

```

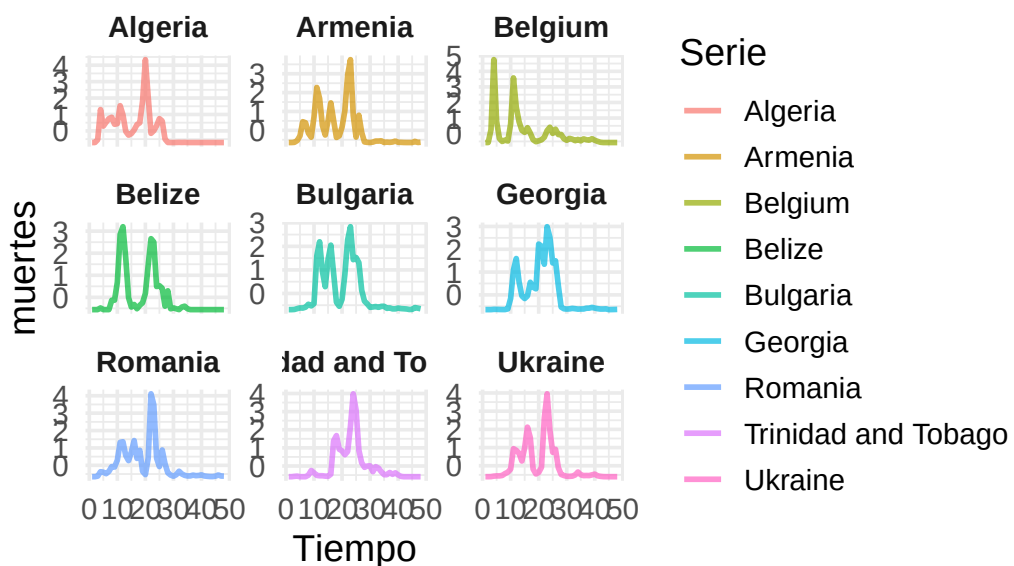
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_fuzzy_dtw_mv_5,
  n_cluster = 1, n_ejemplos = 9,
  tamaño_ventana = c(3, 3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)

```

Ejemplos del Clúster 1 – casos

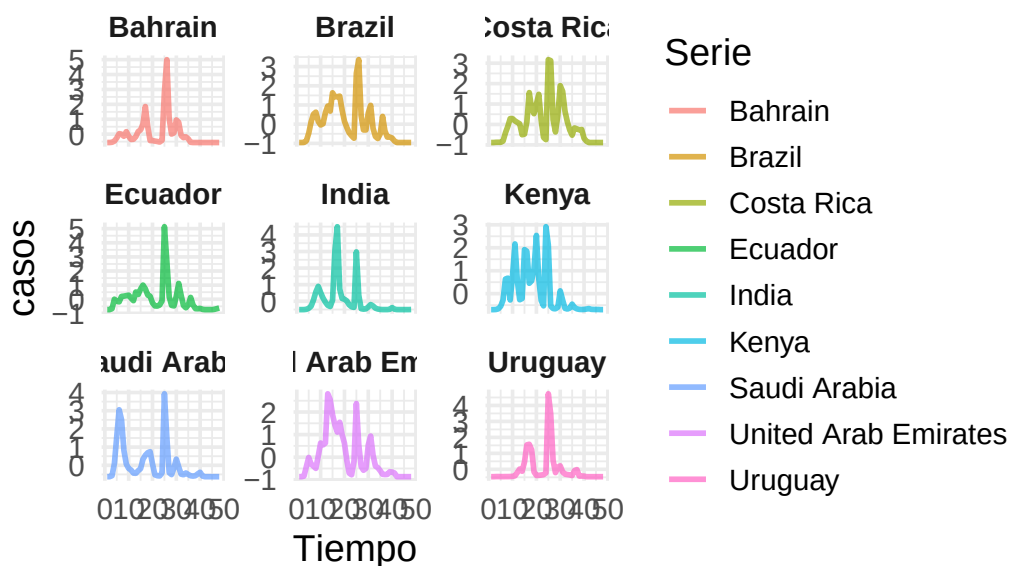


Ejemplos del Clúster 1 – muertes

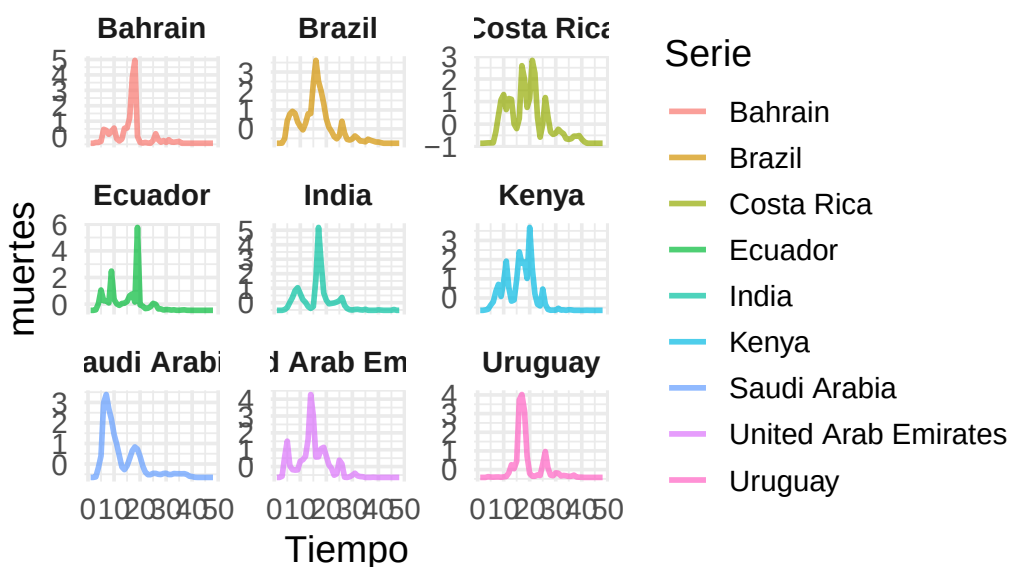


```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_fuzzy_dtw_mv_5,
  n_cluster = 2, n_ejemplos = 9,
  tamaño_ventana = c(3, 3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

Ejemplos del Clúster 2 – casos



Ejemplos del Clúster 2 – muertes



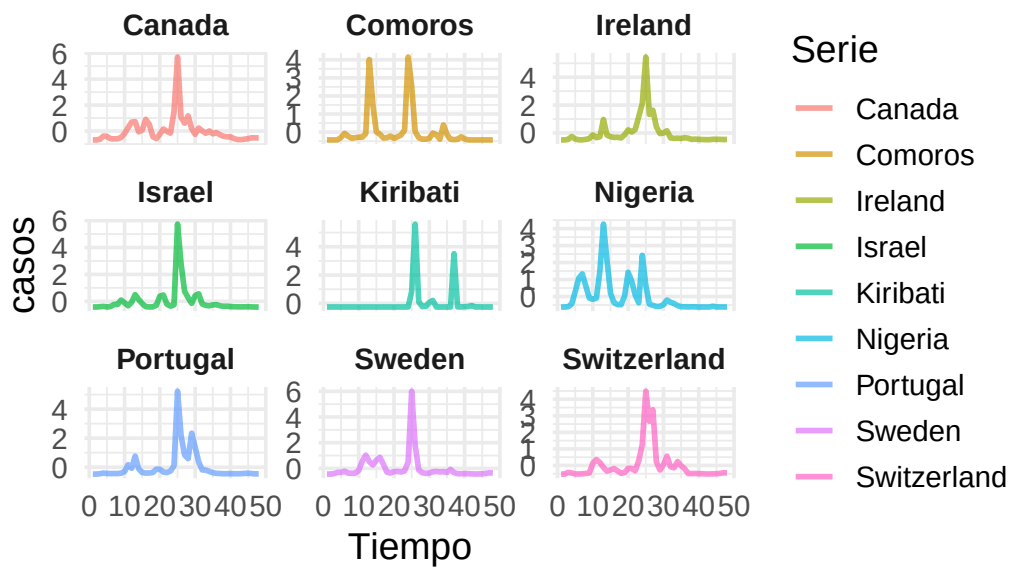
```
visualizar_ejemplos_multivariado(
    series_multivar_z, cl_fuzzy_dtw_mv_5,
    n_cluster = 3, n_ejemplos = 9,
```

```

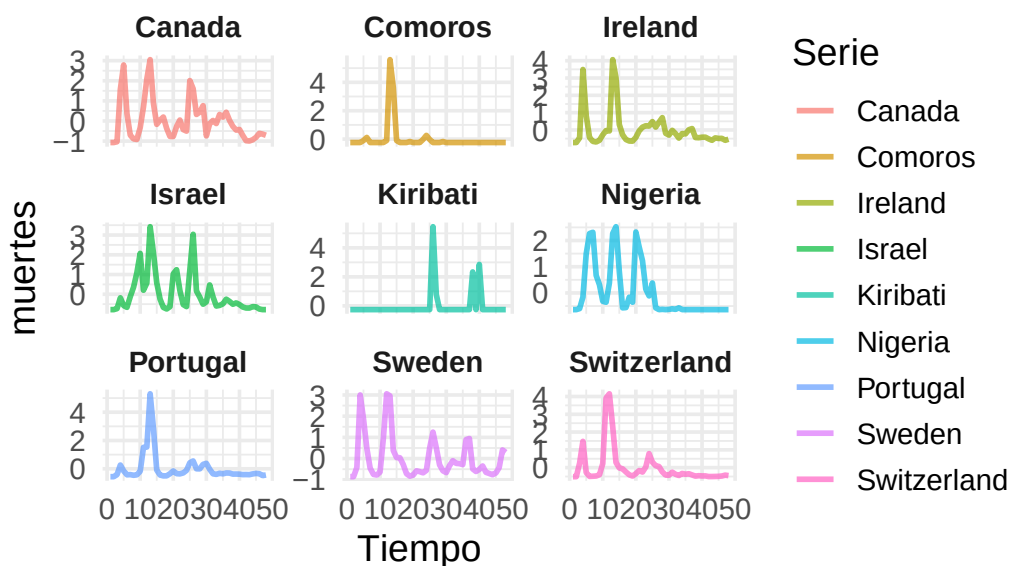
tamano_ventana = c(3, 3), semilla = 42,
variables = c("casos", "muertes"),
nombres_vars = c("casos", "muertes")
)

```

Ejemplos del Clúster 3 – casos

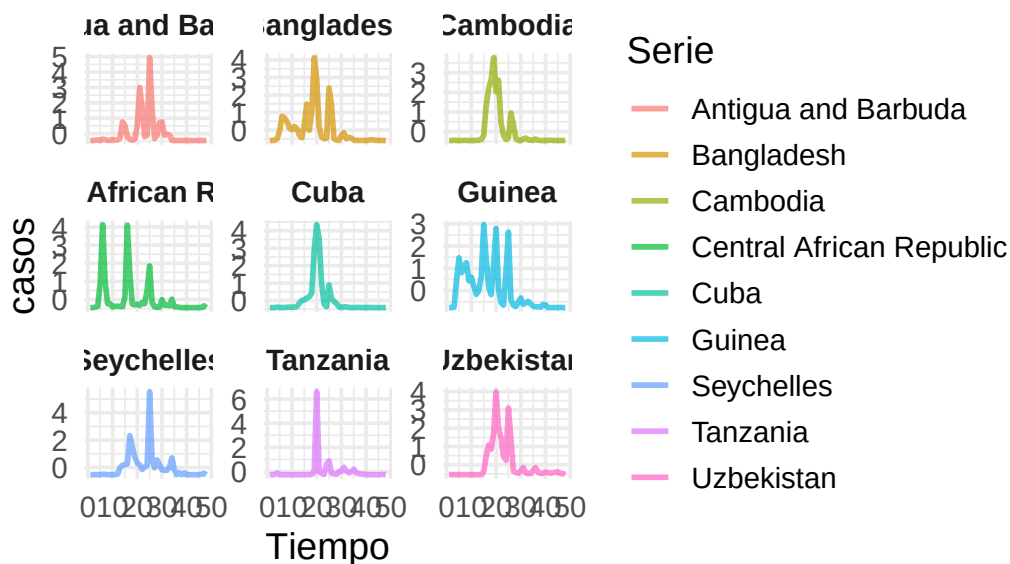


Ejemplos del Clúster 3 – muertes

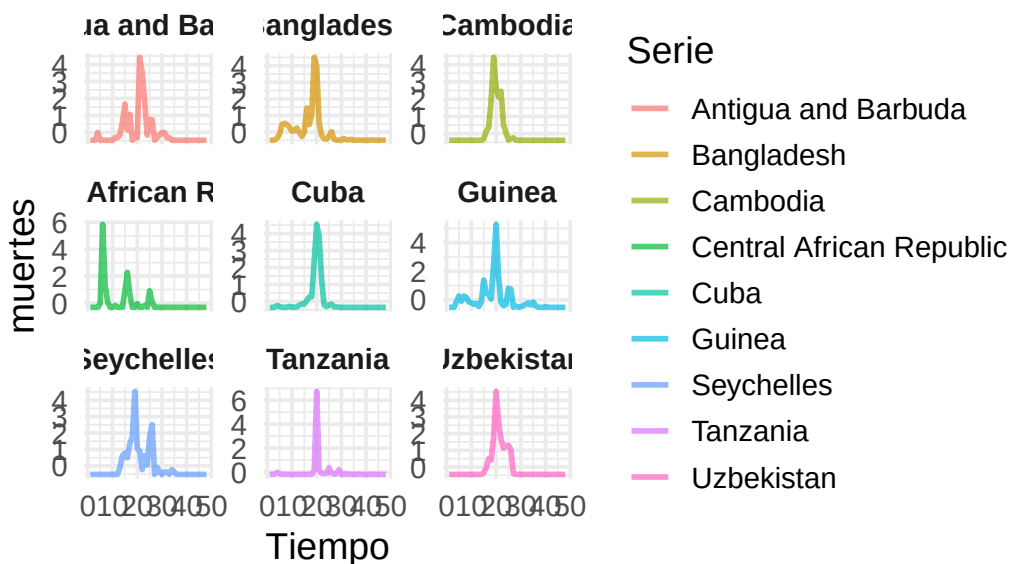


```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_fuzzy_dtw_mv_5,
  n_cluster = 4, n_ejemplos = 9,
  tamano_ventana = c(3,3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

Ejemplos del Clúster 4 – casos



Ejemplos del Clúster 4 – muertes



```
visualizar_ejemplos_multivariado(
    series_multivar_z, cl_fuzzy_dtw_mv_5,
    n_cluster = 5, n_ejemplos = 9,
```

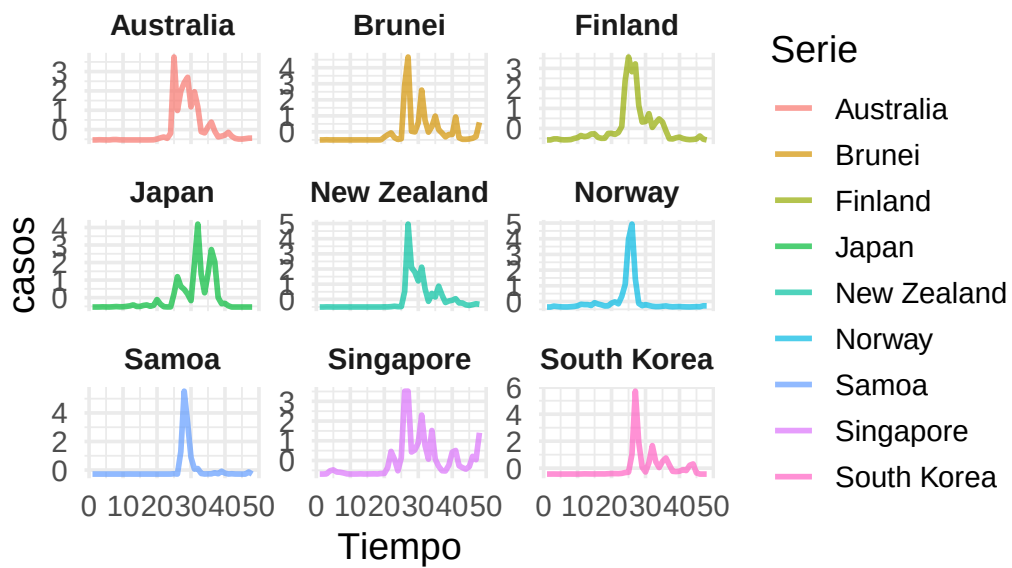


```

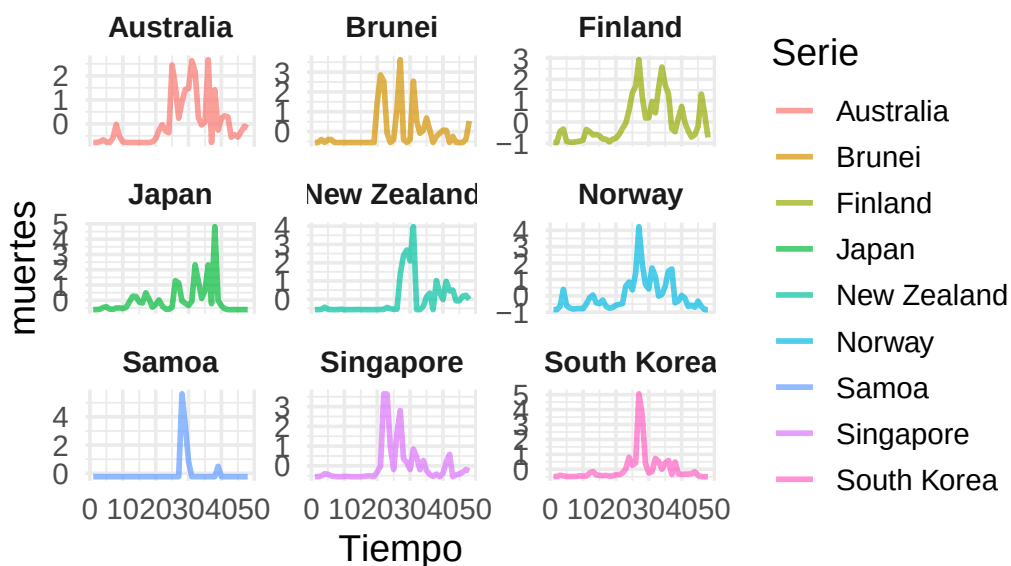
tamano_ventana = c(3,3), semilla = 42,
variables = c("casos", "muertes"),
nombres_vars = c("casos", "muertes")
)

```

Ejemplos del Clúster 5 – casos



Ejemplos del Clúster 5 – muertes



No se observa mal agrupamiento, pero parece peor. Veamos las métricas.

```
set.seed(42)
cvi(cl_dtw_dba_mv_6,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1328205	0.0000000	28.9366054	2.0727817	2.1910412	0.2714423	0.4441670

```
# Calculamos la matriz de distancias DTW
dist_matrix = proxy::dist(serie_new_cases_z_f, method = "dtw_basic")

# Calculamos coeficiente silhouette
sil = silhouette(cl_fuzzy_dtw_mv_5@cluster, dist_matrix)
summary(sil)$avg.width
```

```
[1] -0.02180047
```

Efectivamente el agrupamiento no es tan bueno, de hecho el índice silhouette es prácticamente 0. Probemos con la distancia GAK.

Distancia Global Alignment Kernel

```
set.seed(42)
cl_fuzzy_dtw_mv_5 = tsclust(series_multivar_z, type = "fuzzy", k = 5, distance
  ↪ = "gak", centroid = "fcmd", seed = 12345)
cl_fuzzy_dtw_mv_5
```

fuzzy clustering with 5 clusters

Using gak distance

Using fcmd centroids

Time required for analysis:

user	system	elapsed
0.04	0.00	0.03

Head of fuzzy memberships:

	cluster_1	cluster_2	cluster_3	cluster_4	cluster_5
Afghanistan	0.2	0.2	0.2	0.2	0.2
Albania	0.2	0.2	0.2	0.2	0.2
Algeria	0.2	0.2	0.2	0.2	0.2
Andorra	0.2	0.2	0.2	0.2	0.2
Angola	0.2	0.2	0.2	0.2	0.2
Antigua and Barbuda	0.2	0.2	0.2	0.2	0.2

```
table(cl_fuzzy_dtw_mv_5@cluster)
```

1	2	3	4	5
181	1	1	1	1

Volvemos a obtener clusters que no nos aportan información. Finalmente pasemos al cluster jerárquico.

Hierarchical clustering

Distancia DTW

Volvamos a determinar el k ideal además de $k = 2$.

```

# Definimos el rango de k a evaluar
k_values = 2:10
sil_scores = numeric(length(k_values))
# Matriz de distancias entre todas las series (usando dtw para multivariante)
dist_matrix = proxy::dist(series_multivar_z, method = "dtw_basic",
  ↪ window.size = window) # Lo que cogimos anteriormente

for (i in seq_along(k_values)) {
  k = k_values[i]
  cat("Evaluando k =", k, "...\\n")
  set.seed(42)
  cl = tsclust(series_multivar_z, type = "hierarchical", k = k,
    distance = "dtw_basic",
    control = hierarchical_control(method = diana),
    args = tsclust_args(dist = list(window.size = window)))

  sil = silhouette(cl@cluster, dist_matrix)

  # Promedio de los coeficientes de silueta
  sil_scores[i] = mean(sil[, 3])
}

```

```

Evaluando k = 2 ...
Evaluando k = 3 ...
Evaluando k = 4 ...
Evaluando k = 5 ...
Evaluando k = 6 ...
Evaluando k = 7 ...
Evaluando k = 8 ...
Evaluando k = 9 ...
Evaluando k = 10 ...

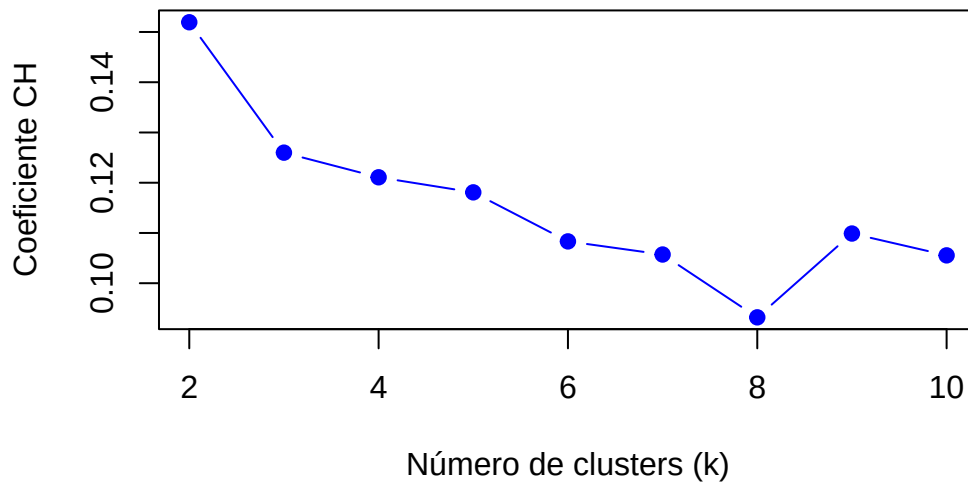
```

```

# Graficar los resultados
par(mfrow = c(1, 1))
plot(k_values, sil_scores, type = "b", pch = 19, col = "blue",
  xlab = "Número de clusters (k)", ylab = "Coeficiente CH",
  main = "Selección de k usando Silhouette con DTW")

```

Selección de k usando Silhouette con DTW



probaremos con $k = 5$ aunque no sea el segundo mayor valor, pues está muy cercano al mismo y así compararemos clusters de 5 o 6 grupos.

```
cl_hc_dtw_5_mv = tsclust(series_multivar_z, type = "hierarchical", k = 5,  
                        distance = "dtw_basic",  
                        control = hierarchical_control(method = "complete"),  
                        args = tsclust_args(dist = list(window.size = window)))  
cl_hc_dtw_5_mv
```

hierarchical clustering with 5 clusters
Using dtw_basic distance
Using PAM (Hierarchical) centroids
Using method complete

Time required for analysis:

user	system	elapsed
0.75	0.02	0.80

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	21	67.34987
2	60	73.82163

```

3 27 58.39775
4 56 64.15573
5 21 71.00582

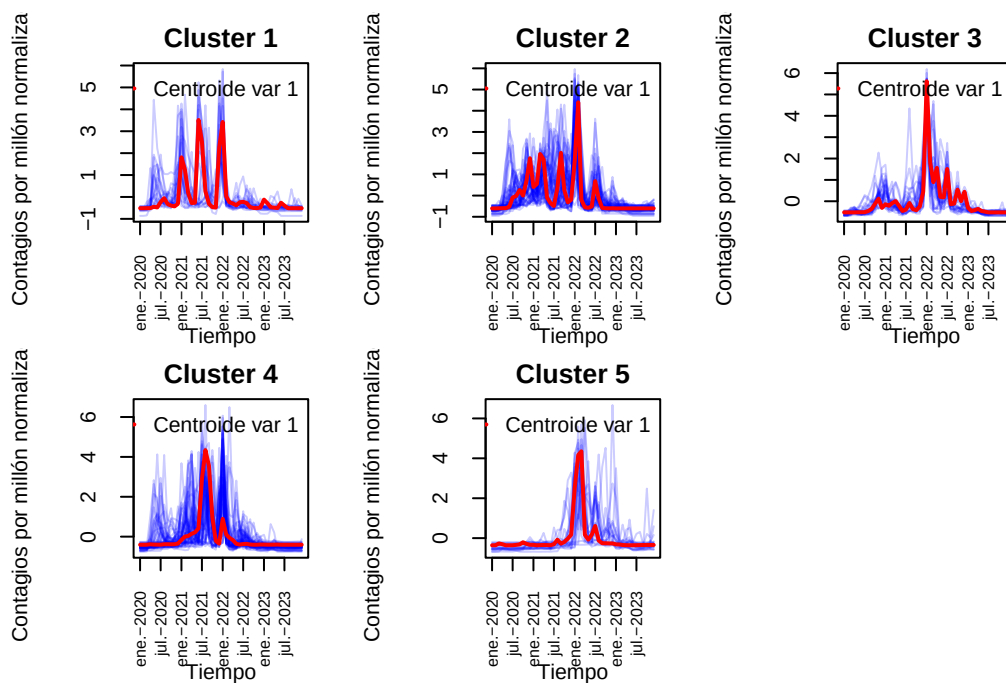
```

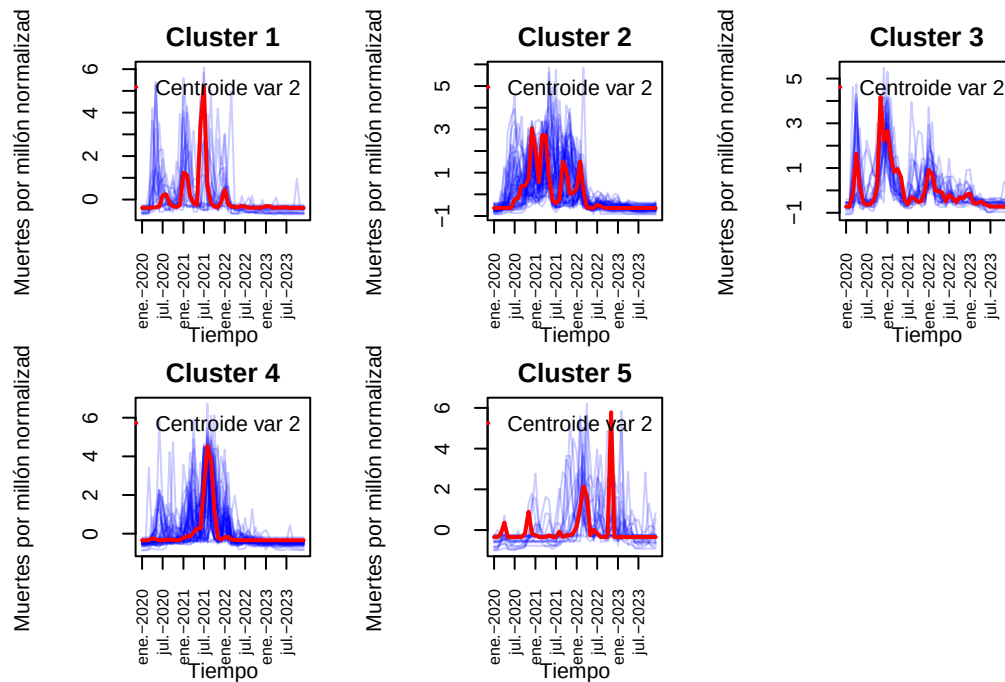
Se observan primeramente grupos no muy desequilibrados.

```

visualizar_series_centroides_mv(serie_new_cases_z_f, cl_hc_dtw_5_mv, k =
  ↪ 5,variable=1,ylab="Contagios por millón normalizados",tamano_ventana =
  ↪ c(2,3))
visualizar_series_centroides_mv(serie_new_deaths_z_f, cl_hc_dtw_5_mv, k =5,
  ↪ variable = 2,ylab="Muertes por millón normalizadas",tamano_ventana =
  ↪ c(2,3))

```

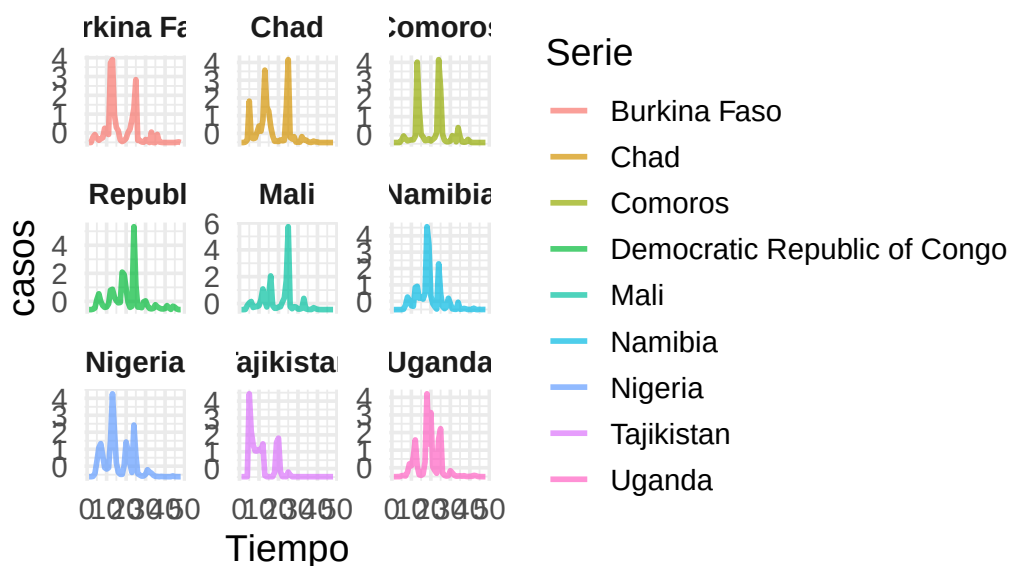




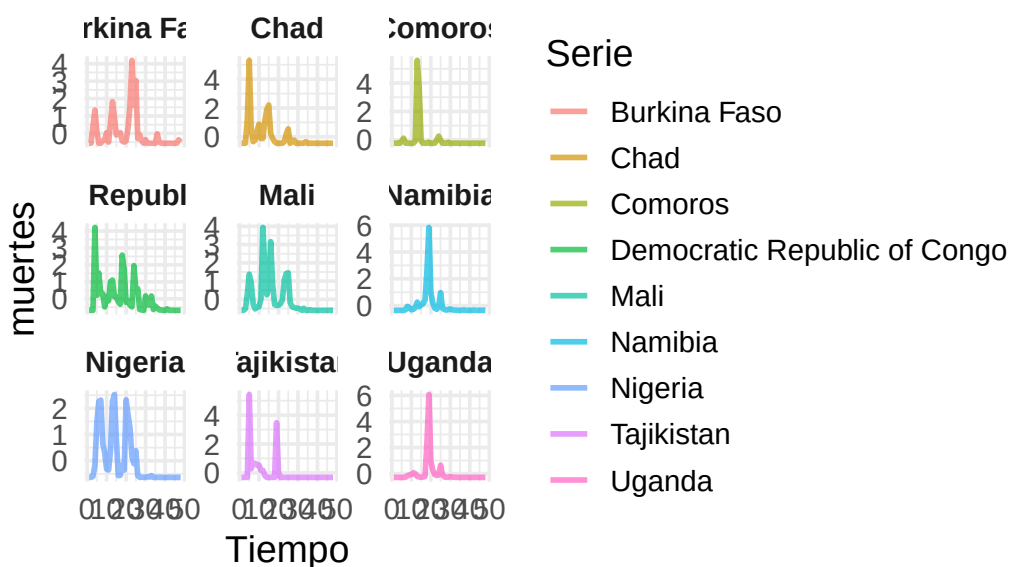
Parece que hemos obtenido una agrupación decente. Veamos algunos ejemplos.

```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_hc_dtw_5_mv,
  n_cluster = 1, n_ejemplos = 9,
  tamano_ventana = c(3, 3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

ejemplos del Clúster 1 – casos



ejemplos del Clúster 1 – muertes



```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_hc_dtw_5_mv,
  n_cluster = 2, n_ejemplos = 9,
```

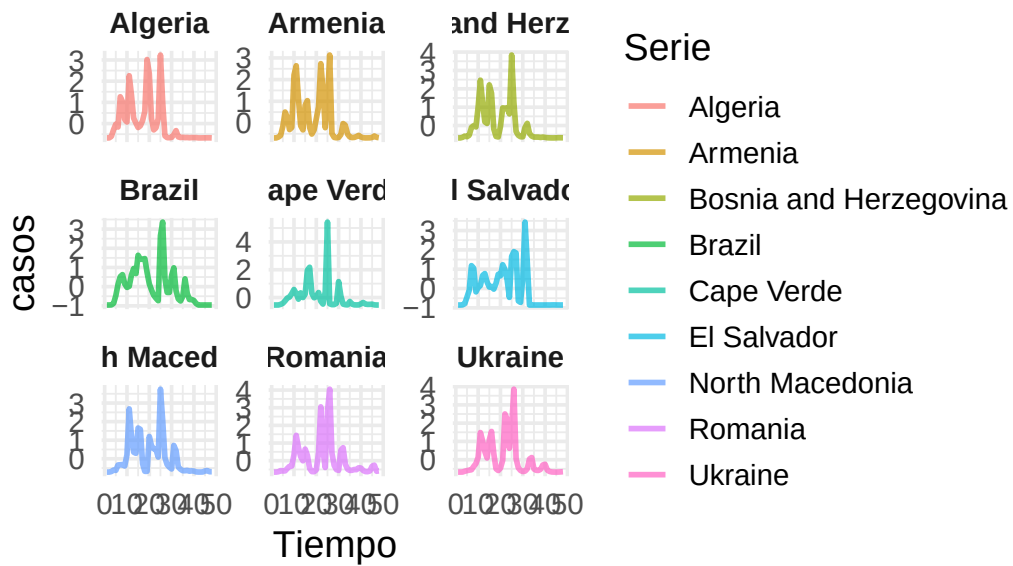


```

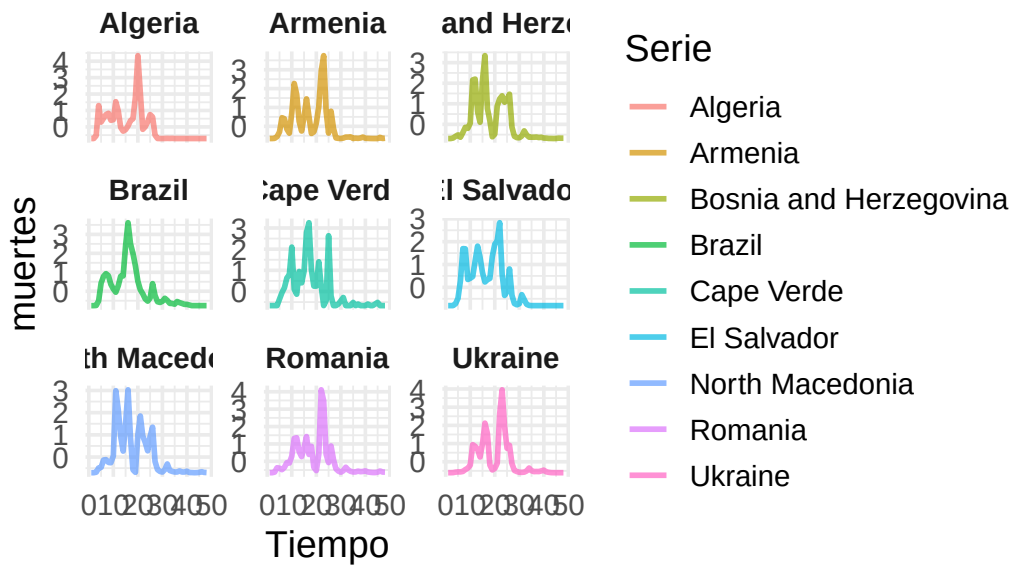
tamano_ventana = c(3,3), semilla = 42,
variables = c("casos", "muertes"),
nombres_vars = c("casos", "muertes")
)

```

Ejemplos del Clúster 2 – casos

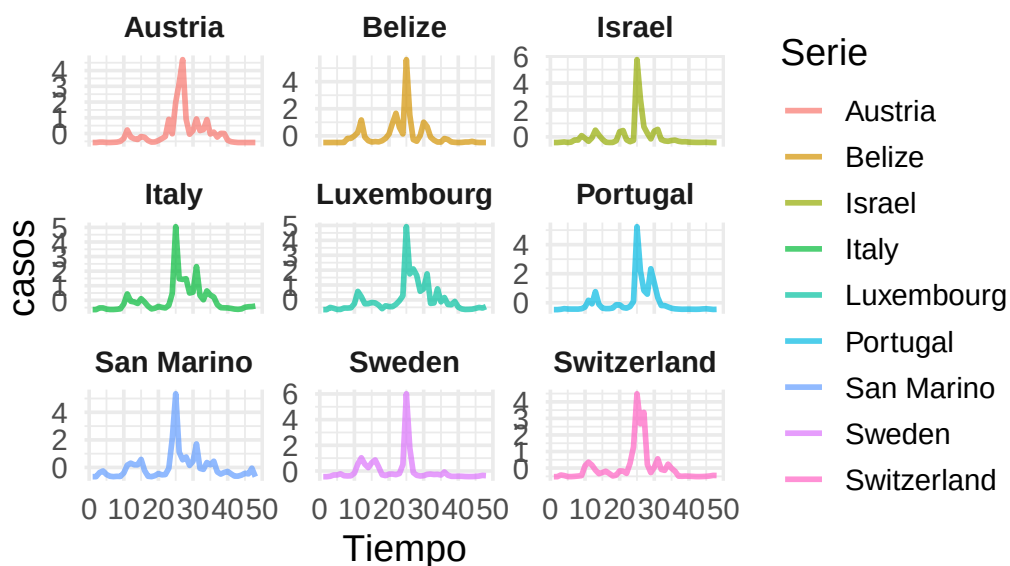


Ejemplos del Clúster 2 – muertes

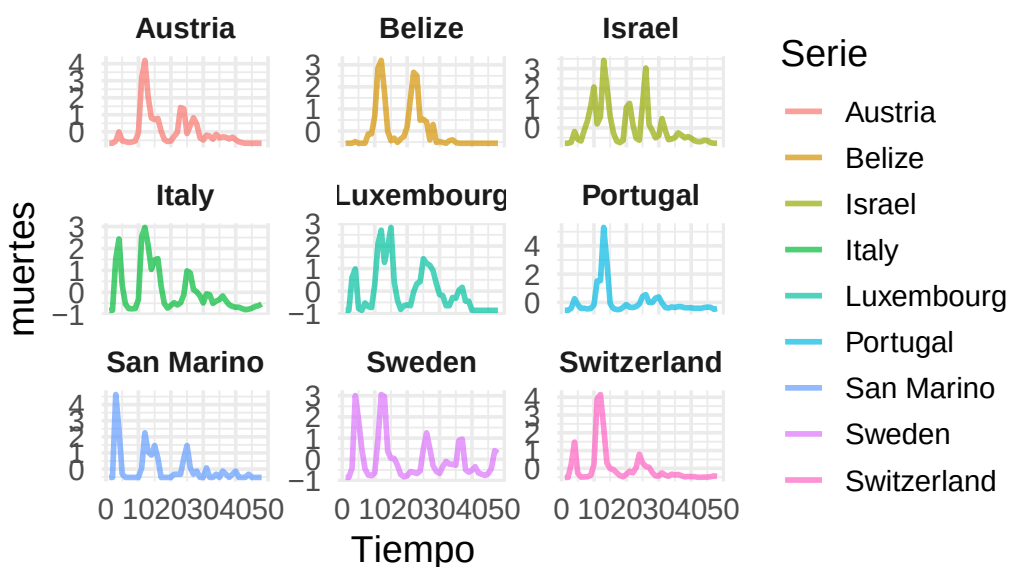


```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_hc_dtw_5_mv,
  n_cluster = 3, n_ejemplos = 9,
  tamaño_ventana = c(3,3), semilla = 42,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

Ejemplos del Clúster 3 – casos



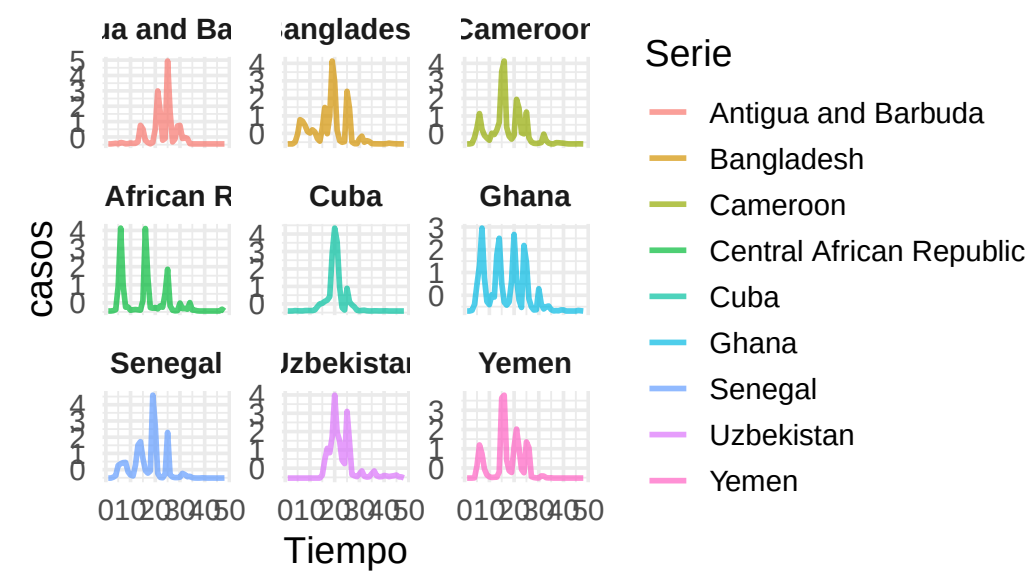
Ejemplos del Clúster 3 – muertes



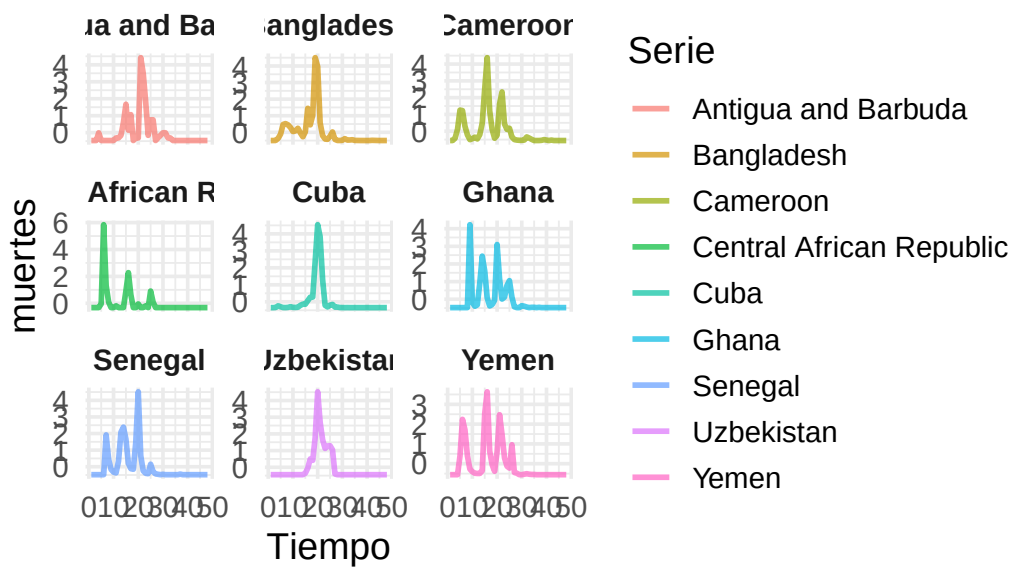
```
visualizar_ejemplos_multivariado(  
  series_multivar_z, cl_hc_dtw_5_mv,  
  n_cluster = 4, n_ejemplos = 9,
```

```
tamano_ventana = c(3,3), semilla = 42,  
variables = c("casos", "muertes"),  
nombres_vars = c("casos", "muertes")  
)
```

Ejemplos del Clúster 4 – casos

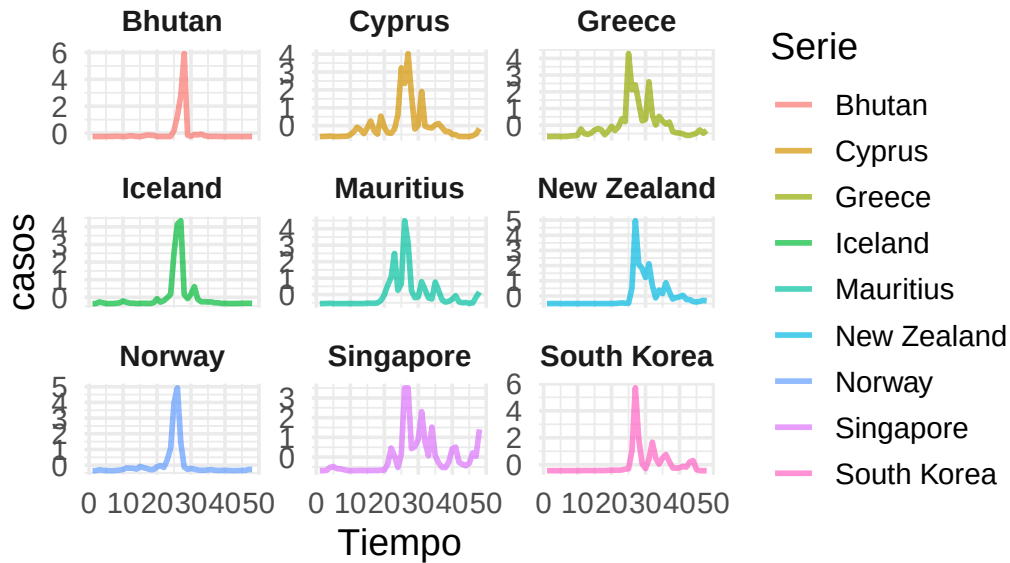


Ejemplos del Clúster 4 – muertes

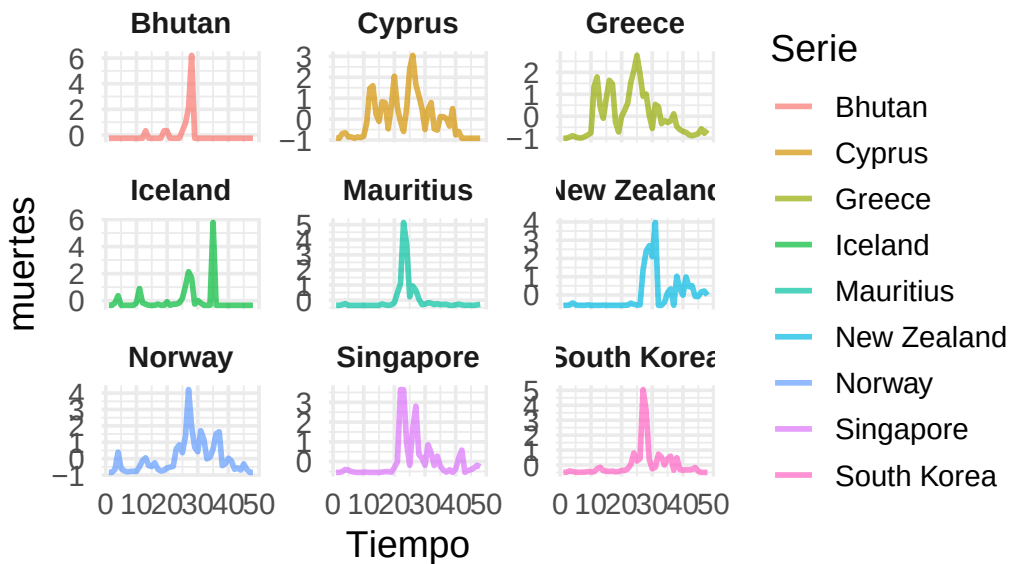


```
visualizar_ejemplos_multivariado(
  series_multivar_z, cl_hc_dtw_5_mv,
  n_cluster = 5, n_ejemplos = 9,
  tamaño_ventana = c(3, 3), semilla = 142,
  variables = c("casos", "muertes"),
  nombres_vars = c("casos", "muertes")
)
```

Ejemplos del Clúster 5 – casos



Ejemplos del Clúster 5 – muertes



Vemos clusters con bastante sentido, veamos las métricas para ver con cuál nos quedamos.

```
cvi(cl_dtw_dba_mv_6,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1328205	0.0000000	28.8860591	2.0727817	2.1910412	0.2714423	0.4441670

```
cvi(cl_hc_dtw_5_mv,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.0882470	0.0000000	51.0487796	1.7306644	1.8538197	0.2883227	0.5632804

En las métricas vemos que el cluster jerárquico tiene en casi todos los casos mejores métricas, por ello elegiremos este cluster como el mejor hasta el momento. Pasemos a usar la distancia GAK.

Distancia Global Alignment Kernel

```
set.seed(42)
cl_hc_gak_5_mv = tsclust(series_multivar_z, type = "hierarchical", k = 5,
  distance = "gak",
  control = hierarchical_control(method = diana),
  args = tsclust_args(dist = list(window.size = window)),
  seed = 12345)
cl_hc_gak_5_mv
```

hierarchical clustering with 5 clusters
 Using gak distance
 Using PAM (Hierarchical) centroids
 Using method diana

Time required for analysis:

	user	system	elapsed
	2.77	0.04	2.94

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	2	0.5000000
2	177	0.9943503
3	2	0.5000000
4	2	0.5000000
5	2	0.5000000

Hemos vuelto a obtener grupos con muy pocas observaciones así que no seguiremos.

Por tanto, consideramos el cluster jerárquico con 5 grupos y distancia DTW el mejor cluster.

Estudiemos ahora la pandemia desde el punto de vista operativo en relación con los hospitales. Realicemos un clustering del número de pacientes UCI, teniendo en cuenta que, debido a la naturaleza de estos datos (sensibilidad o dificultad en su recogida), no se dispone de información para muchos países.

Esto permite evaluar la presión real sobre el sistema sanitario allí donde están disponibles los datos pues permiten comparar la gravedad y la gestión de la pandemia, minimizando el sesgo de diferencias en diagnóstico de casos leves, pues ahora nos estamos preocupando de estudiar aquellos casos de COVID realmente preocupantes..

Países con trayectoria de hospitalizados similar

Construyamos primero las series como hicimos anteriormente.

```
serie_icu = preparar_series_temporales(df,"icu_patients_per_million")
```

Veamos si son diferentes las longitudes de cada serie.

```
unique(lengths(serie_icu))
```

```
[1] 48
```

Por tanto tenemos series de idéntica longitud.

Realicemos un escalado para que las técnicas de cluster funcionen mejor.

```
serie_icu_z = lapply(serie_icu, z_score)
```

Antes de pasar a realizar clúster veamos que efectivamente el número de países es mucho menor.

```
length(serie_icu)
```

```
[1] 38
```

Ahora solo tenemos 38 países.

Usaremos algunas de las técnicas para no alargar excesivamente el estudio.

Partitional Clustering

Distancia nDTW

Prototipo DBA

En vez de la distancia DTW usaremos la versión asimétrica y normalizada que llevamos usando durante la práctica. Como se describe en la memoria, el prototipo ideal cuando usamos DTW es DBA (DTW Barycenter Averaging). Para determinar el k que usaremos nos basaremos en el índice CH. Como nuestra matriz de distancias es asimétrica no podemos fiarnos de índices como el Silhouette o el índice de Dunn. Además al usar DBA, el índice de Davies Bouldin también puede inducirnos error, por ello se ha elegido el índice CH para ayudarnos a determinar el k .

```
# Definimos el rango de k a evaluar
k_values = 2:6
ch_scores = numeric(length(k_values))

for (i in seq_along(k_values)) {
  k = k_values[i]
  cat("Evaluando k =", k, "...\\n")
  set.seed(42)
  cl = tsclust(serie_icu_z, type = "partitional", k = k, distance = "ndtw",
  ↪ centroid = "dba", seed = 12345)

  resumen_metricas = cvi(cl,type="internal")
  ch = resumen_metricas["CH"]

  # Promedio de los coeficientes de silueta
  ch_scores[i] = ch
}
```

Evaluando k = 2 ...

Evaluando k = 3 ...

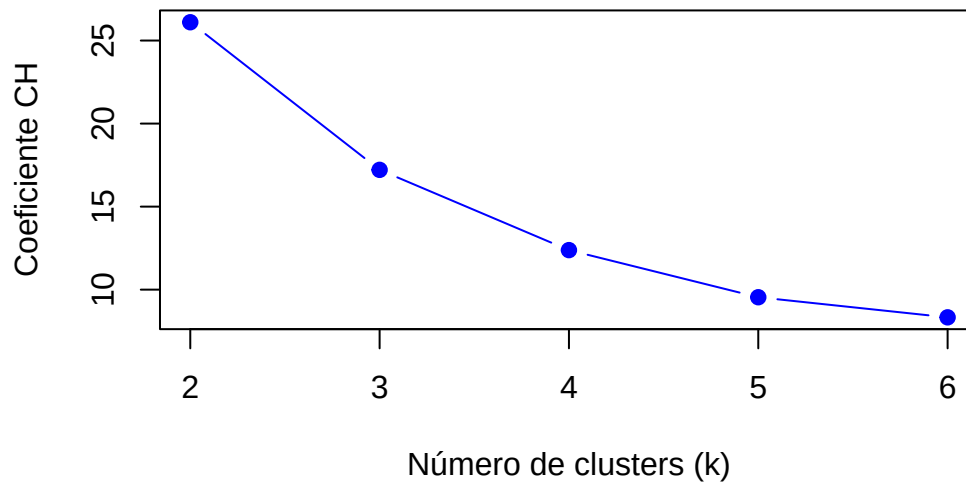
Evaluando k = 4 ...

Evaluando k = 5 ...

Evaluando k = 6 ...

```
# Graficar los resultados
par(mfrow = c(1, 1))
plot(k_values, ch_scores, type = "b", pch = 19, col = "blue",
     xlab = "Número de clusters (k)", ylab = "Coeficiente CH",
     main = "Selección de k usando CH con DTW")
```

Selección de k usando CH con DTW



```
ch_scores
```

```
[1] 26.102681 17.213748 12.379767  9.542373  8.331654
```

Empecemos con $k = 2$ y vemos que obtenemos.

Caso $k = 2$

Vamos a construir el cluster correspondiente. Recordemos que tomaremos la distancia ndtw definida y centroides por DBA.

```
set.seed(42)
cl_ndtw_dba_2_icu = tsclust(serie_icu_z, type = "partitional", k = 2, distance
↪ = "ndtw", centroid = "dba", seed = 12345)
cl_ndtw_dba_2_icu
```

```
partitional clustering with 2 clusters
Using ndtw distance
Using dba centroids
```

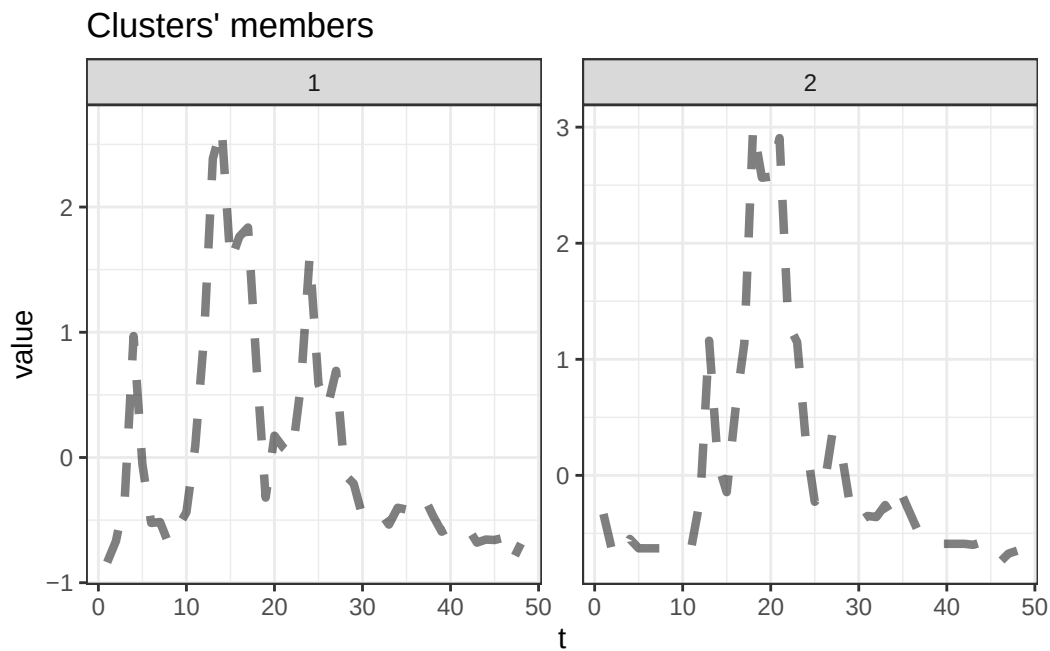
Time required for analysis:

user	system	elapsed
1.07	0.11	0.52

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	26	0.1883481
2	12	0.1717215

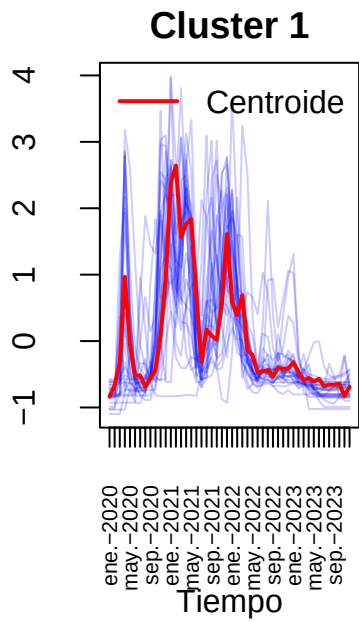
```
plot(cl_ndtw_dba_2_icu,type="centroids")
```



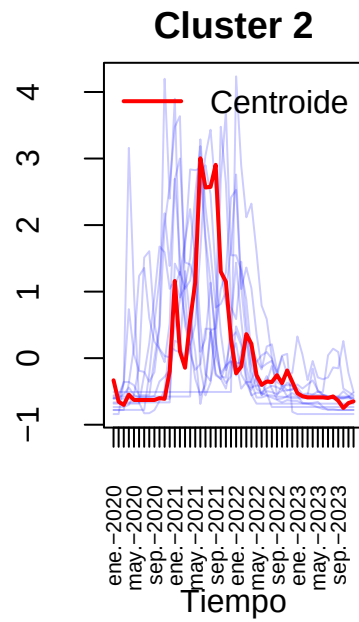
En los centroides se ven dos tipos de países, países con un pico inicial y uno intermedio más marcado y países con un gran pico intermedio marcado. Realicemos algunas visualizaciones.

```
visualizar_series_centroides(serie_icu_z,cl_ndtw_dba_2_icu,2,ylab="Pacientes  
↪ UCI por millón normalizados")
```

Pacientes UCI por millón normalizado



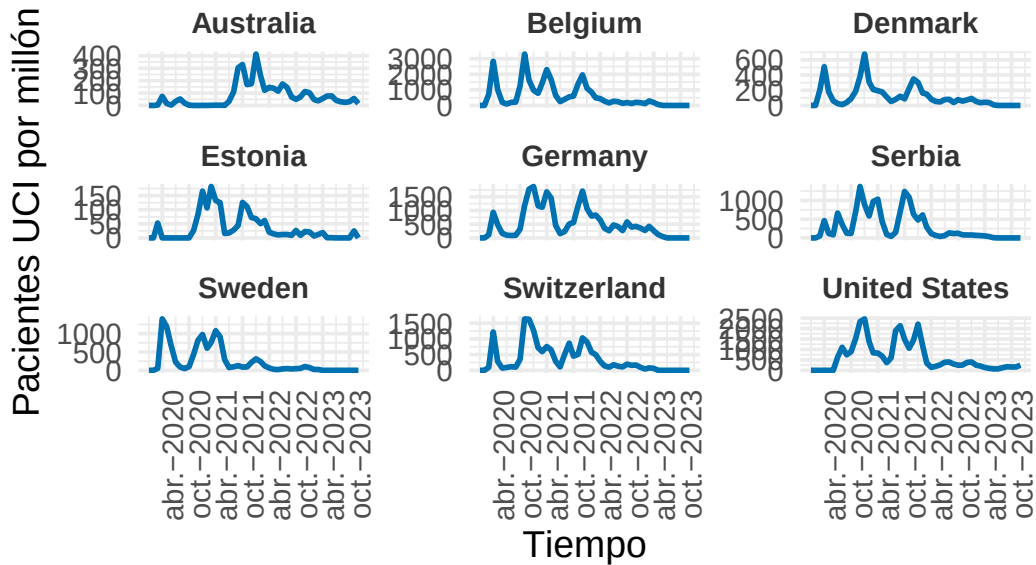
Pacientes UCI por millón normalizado



Veamos algunos ejemplos para hacernos a la idea de cómo son los grupos.

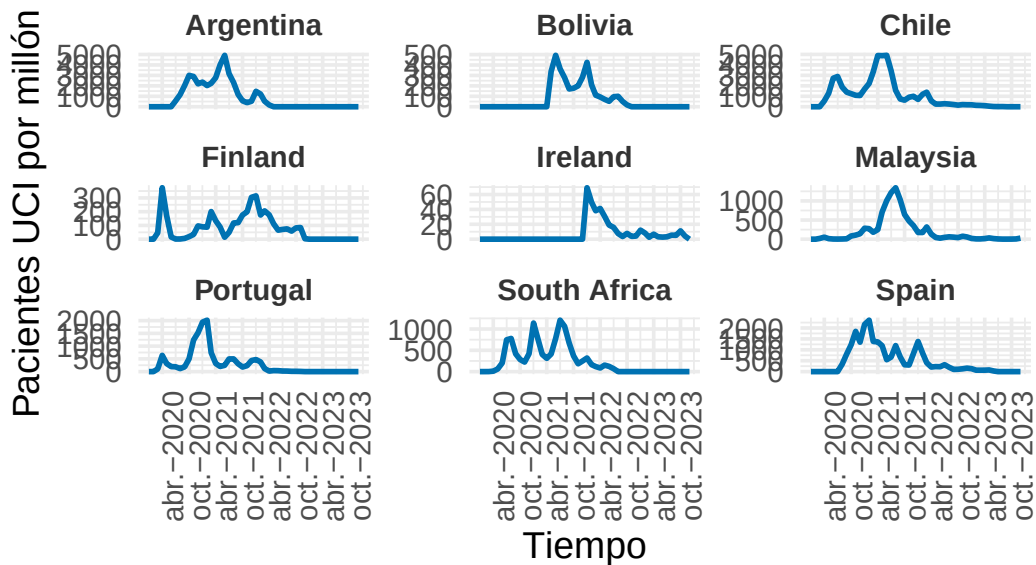
```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_2_icu,1,9,c(3,3),42,ylab="Pacientes
↳ UCI por millón")
```

Ejemplos del Cluster 1



```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_2_icu,2,9,c(3,3),42,ylab="Pacientes
↪ UCI por millón")
```

Ejemplos del Cluster 2



Vemos similitudes en las curvas aceptables, cumpliendo con lo dicho anteriormente, veamos las métricas y probemos $k = 3$.

```
cvi(cl_ndtw_dba_2_icu,type="internal")
```

Sil	SF	CH	DB	DBstar	D
0.06561179	0.52174742	21.91803241	2.00206197	2.00206197	0.19721908
COP					
0.61543493					

Estudiemos el caso $k = 3$ a ver si conseguimos segmentar de mejor manera.

Caso $k = 3$

Vamos a construir el cluster correspondiente. Recordemos que tomaremos la distancia ndtw definida y centroides por DBA.

```
set.seed(42)
cl_ndtw_dba_3_icu = tsclust(serie_icu_z, type = "partitional", k = 3, distance
  ↪ = "ndtw", centroid = "dba", seed = 12345)
cl_ndtw_dba_3_icu
```

partitional clustering with 3 clusters

Using ndtw distance

Using dba centroids

Time required for analysis:

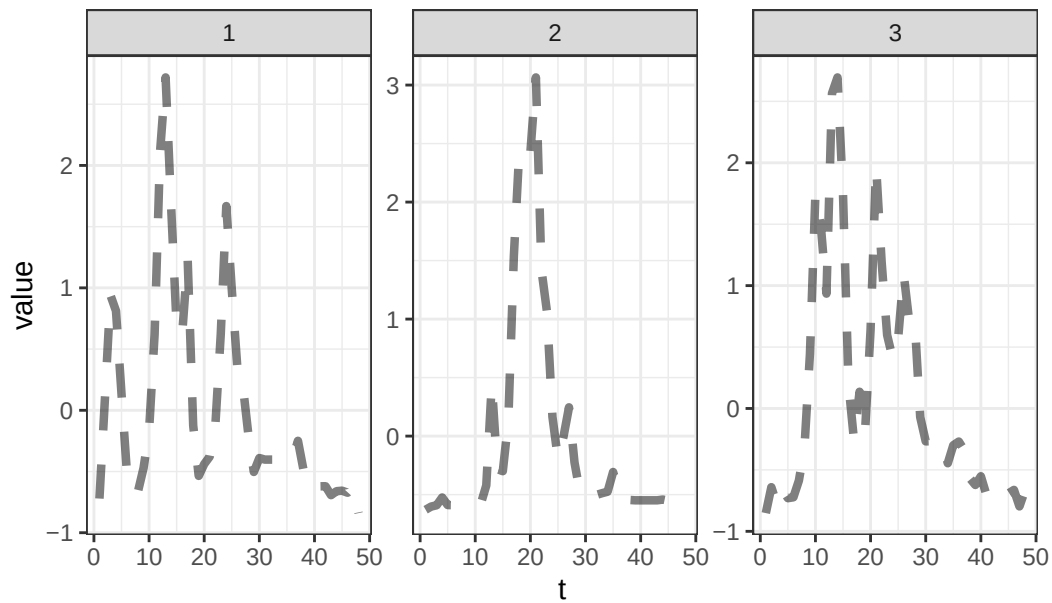
user	system	elapsed
1.04	0.08	0.53

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	15	0.1833087
2	7	0.1515757
3	16	0.1729469

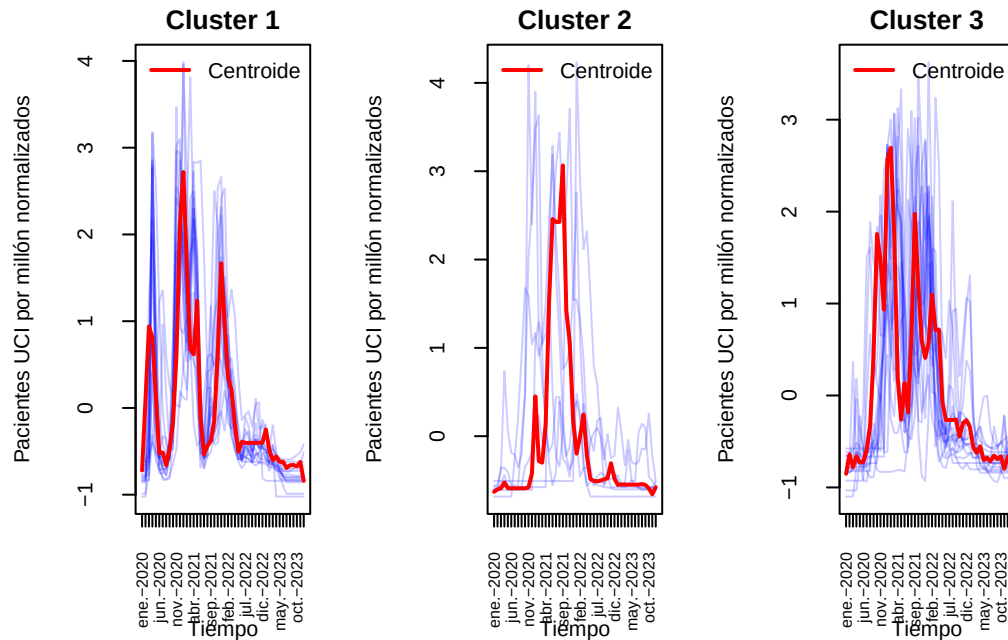
```
plot(cl_ndtw_dba_3_icu,type="centroids")
```

Clusters' members



Ahora el grupo que ha aparecido tiene como centroide una curva con varios picos marcados alrededor del gran pico central (se diferencia con el grupo 1 en que los picos están más juntos y el primer pico del grupo 1 ocurre al principio).

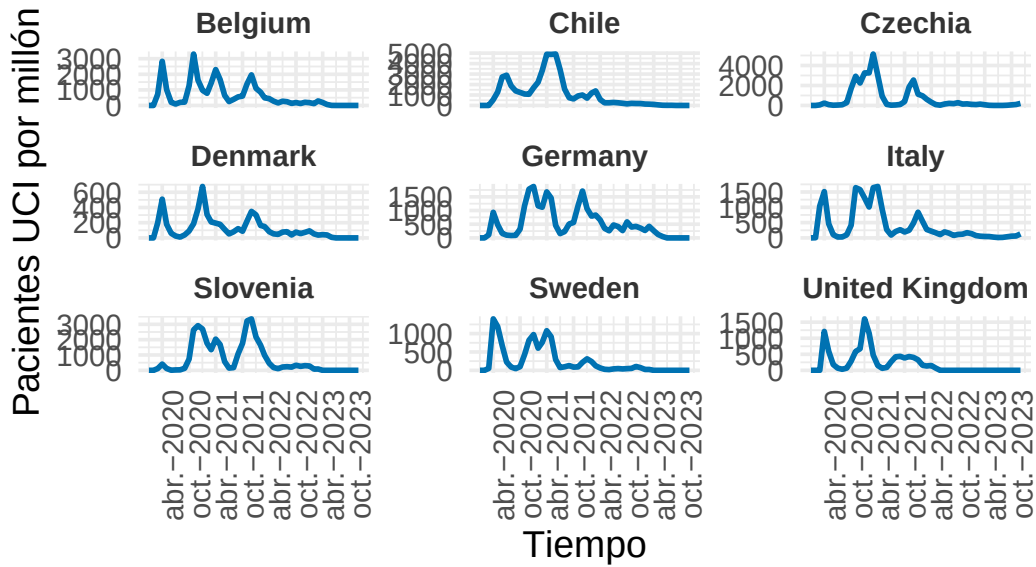
```
visualizar_series_centroides(serie_icu_z,cl_ndtw_dba_3_icu,3,ylab="Pacientes
↪ UCI por millón normalizados")
```



Como dijimos, se observa un primer grupo con 3 brotes y entre ellos, un brote en el medio más marcado, otro grupo con un brote muy marcado y un último grupo con varios brotes en torno al centro y mas juntos entre sí.

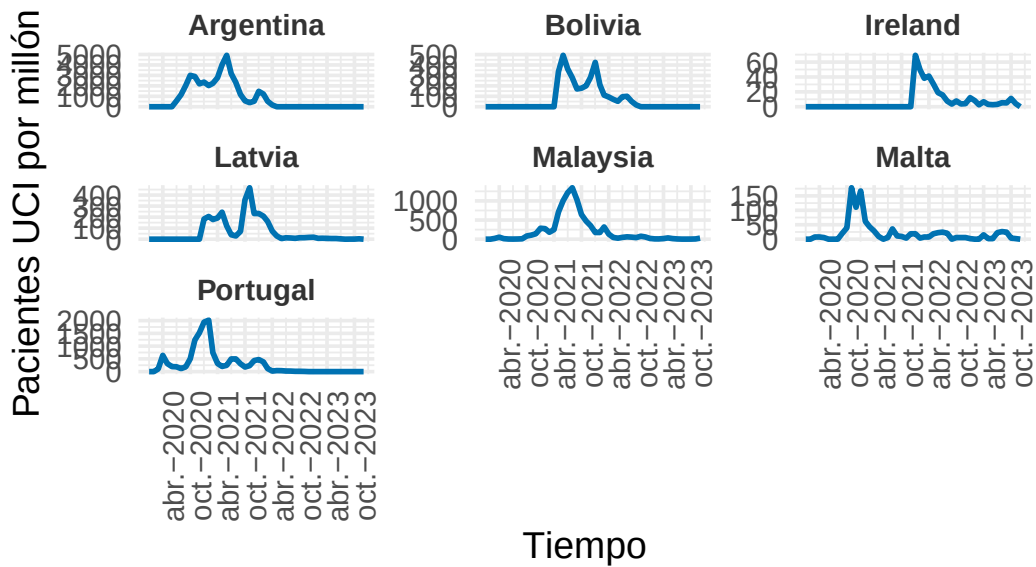
```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_3_icu,1,9,c(3,3),42,ylab="Pacientes
↪ UCI por millón")
```


Ejemplos del Cluster 1



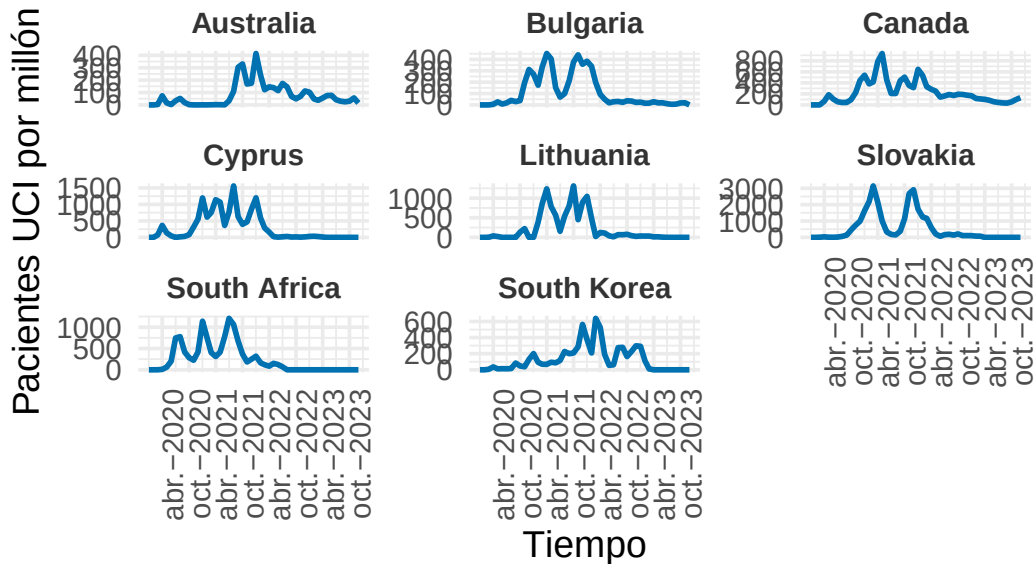
```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_3_icu,2,7,c(3,3),42,ylab="Pacientes
↪ UCI por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_3_icu,3,8,c(3,3),42,ylab="Pacientes
↪ UCI por millón")
```

Ejemplos del Cluster 3



Vemos grupos con bastante sentido, parece una agrupación muy coherente. Comparemos métricas

```
set.seed(42)
cvi(cl_ndtw_dba_3_icu,type="internal")
```

	Si1	SF	CH	DB	DBstar	D
	0.08456112	0.47554394	21.21118659	1.60635435	1.60764626	0.27509421
COP						
	0.52867232					

```
cvi(cl_ndtw_dba_2_icu,type="internal")
```

	Si1	SF	CH	DB	DBstar	D
	0.06561179	0.52645374	27.15516520	2.00206197	2.00206197	0.19721908
COP						
	0.61543493					

Basándonos en la métrica más fiable (CH) la diferencia es baja aunque es mayor en el clúster con $k = 2$, sin embargo en métricas como Silhouette y DB, el clúster con $k = 3$ es algo superior. Seguiremos estudiando qué k elegir en métodos posteriores, pues aún no está claro.

Caso $k = 4$

Veamos si ahora los clústeres empeoran.

```
set.seed(42)
cl_ndtw_dba_4_icu = tsclust(serie_icu_z, type = "partitional", k = 4, distance
↪ = "ndtw", centroid = "dba", seed = 12345)
cl_ndtw_dba_4_icu
```

partitional clustering with 4 clusters
Using ndtw distance
Using dba centroids

Time required for analysis:

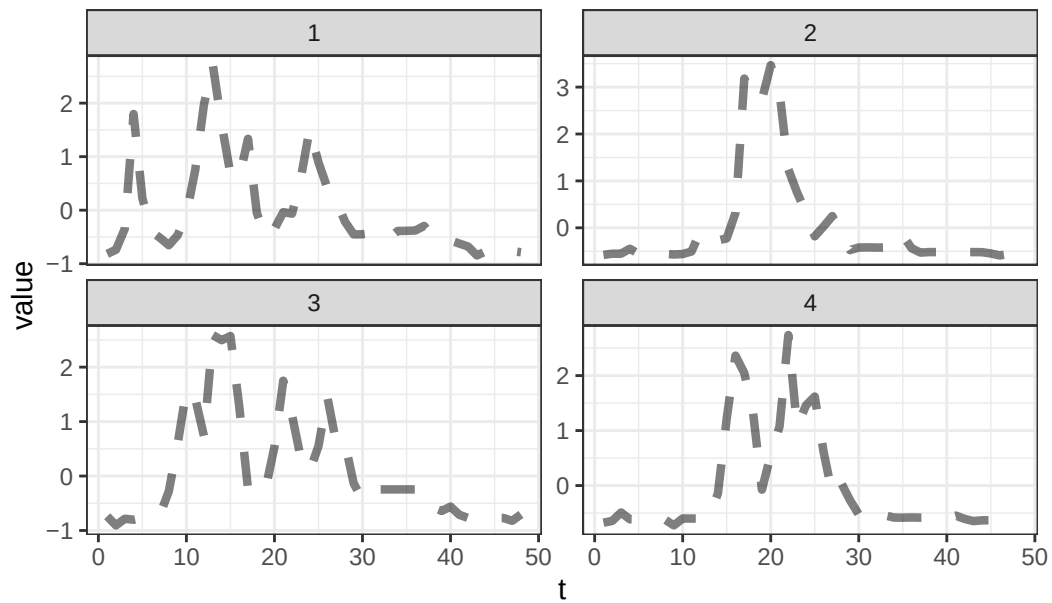
	user	system	elapsed
	0.75	0.11	0.59

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	14	0.1702186
2	4	0.1429766
3	9	0.1688265
4	11	0.1354195

```
plot(cl_ndtw_dba_4_icu, type = "centroids")
```

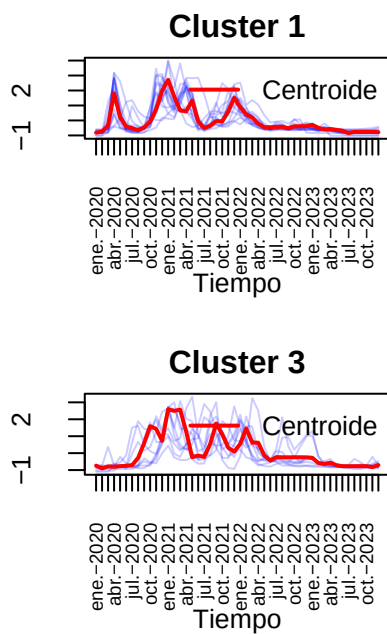
Clusters' members



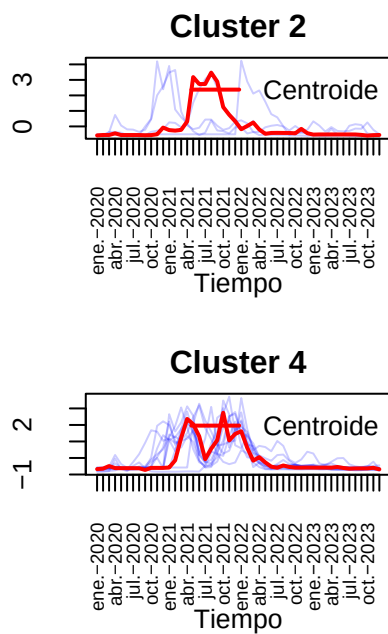
<se observa un grupo con tan solo 4 observaciones. Esto ya es mala señal.

```
visualizar_series_centroides(serie_icu_z,cl_ndtw_dba_4_icu,4,ylab="Pacientes
↪ UCI por millón normalizados",tamano_ventana = c(2,2))
```

Pacientes UCI por millón no



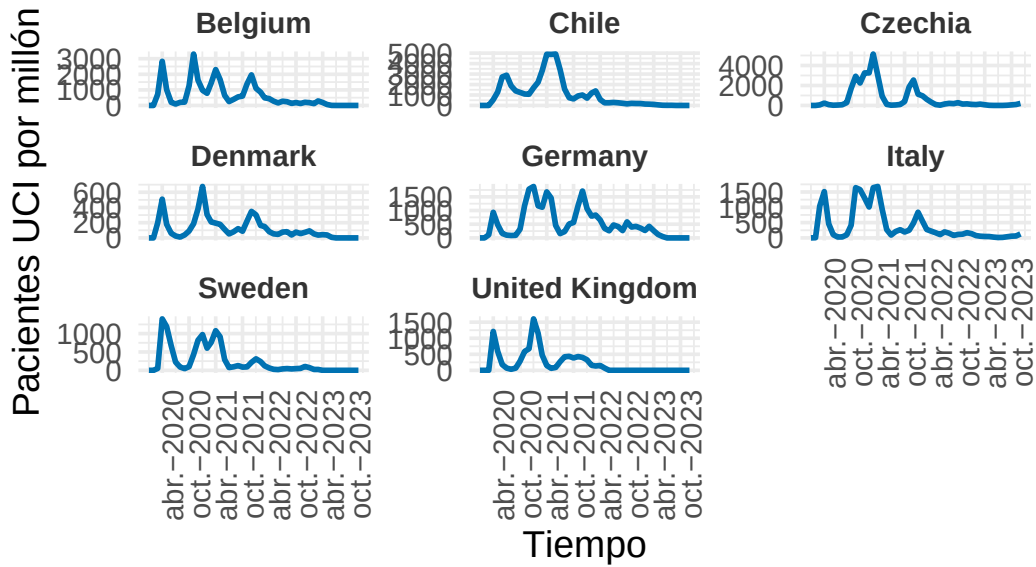
Pacientes UCI por millón no



No parece haber muchísima diferencia entre el clúster 3 y el 4. Veamos algunos ejemplos.

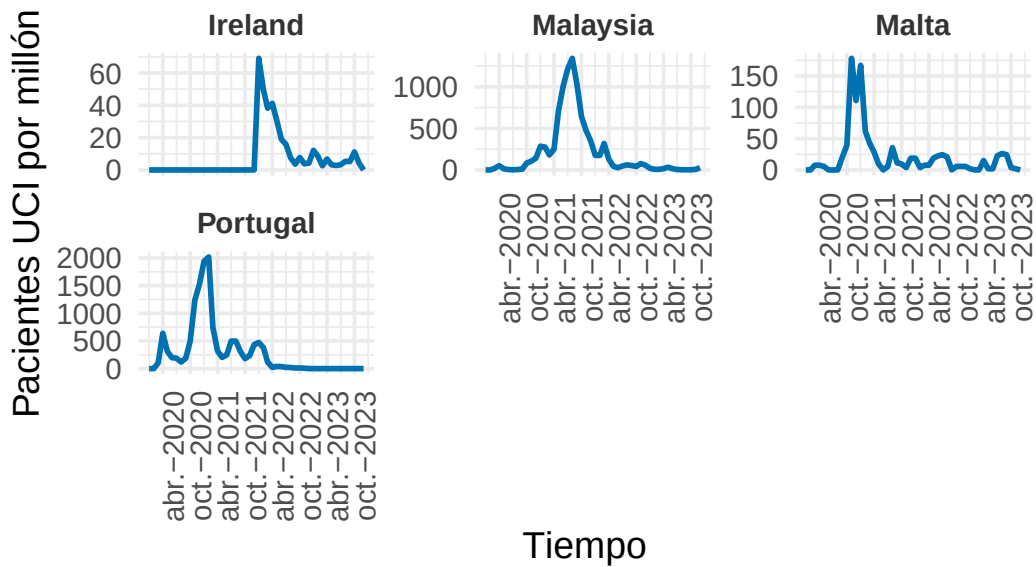
```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_4_icu,1,8,c(3,3),42,ylab="Pacientes
↳ UCI por millón")
```

Ejemplos del Cluster 1



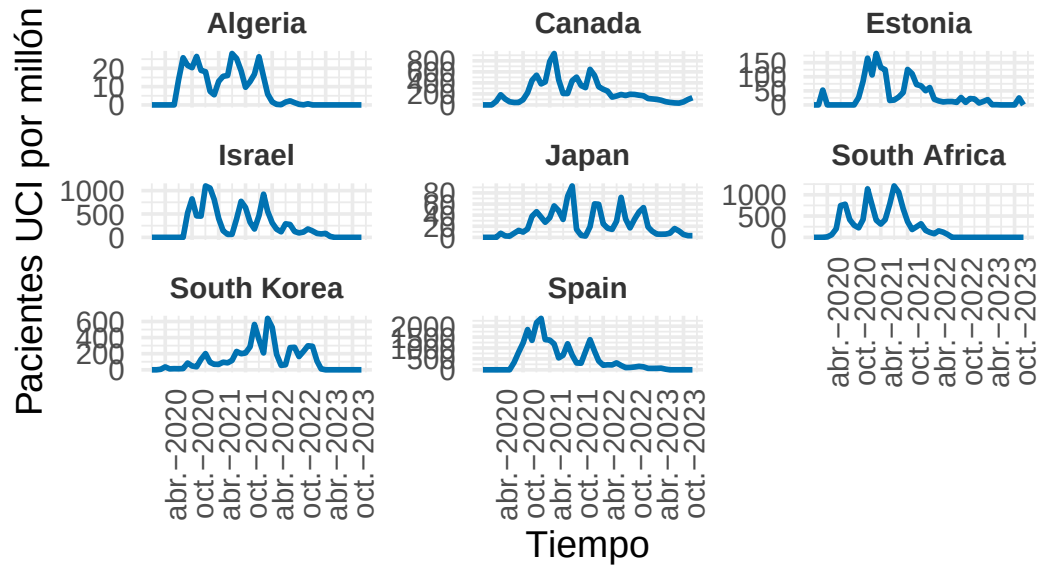
```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_4_icu,2,4,c(3,3),42,ylab="Pacientes
↪ UCI por millón")
```

Ejemplos del Cluster 2



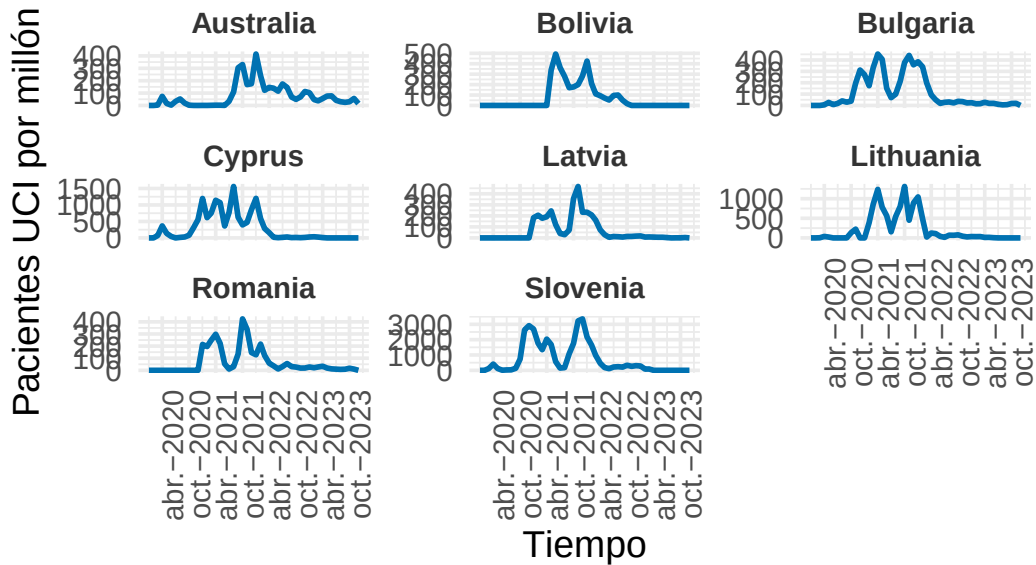
```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_4_icu,3,8,c(3,3),42,ylab="Pacientes
↳ UCI por millón")
```

Ejemplos del Cluster 3



```
visualizar_ejemplos(serie_icu,cl_ndtw_dba_4_icu,4,8,c(3,3),42,ylab="Pacientes
↳ UCI por millón")
```

Ejemplos del Cluster 4



Se ve que la agrupación es peor pues hay mayor solapamiento.

```
set.seed(42)
cvi(cl_ndtw_dba_3_icu,type="internal")
```

Si1	SF	CH	DB	DBstar	D
0.08456112	0.47554394	21.21118659	1.60635435	1.60764626	0.27509421
COP					
0.52867232					

```
cvi(cl_ndtw_dba_4_icu,type="internal")
```

Si1	SF	CH	DB	DBstar	D
0.08995069	0.42910246	11.14830487	1.72439717	1.79355719	0.27884862
COP					
0.48040204					

Parece no haber ninguna mejora respecto $k = 3$ (de hecho obtenemos métricas mejores para $k = 3$). Cambiemos de distancia.

Distancia Soft-DTW

Veamos que k conviene basándonos en el coeficiente Silhouette.

```
# Definimos el rango de k a evaluar
k_values = 2:6
sil_scores = numeric(length(k_values))

for (i in seq_along(k_values)) {
  k = k_values[i]
  cat("Evaluando k =", k, "...\\n")
  set.seed(42)
  cl = tsclust(serie_icu_z, type = "partitional", k = k, distance = "sdtw",
  ↪ centroid = "sdtw_cent", seed = 12345)

  resumen_metricas = cvi(cl,type="internal")
  sil = resumen_metricas["Sil"]

  # Promedio de los coeficientes de silueta
  sil_scores[i] = sil
}
```

Evaluando k = 2 ...

Evaluando k = 3 ...

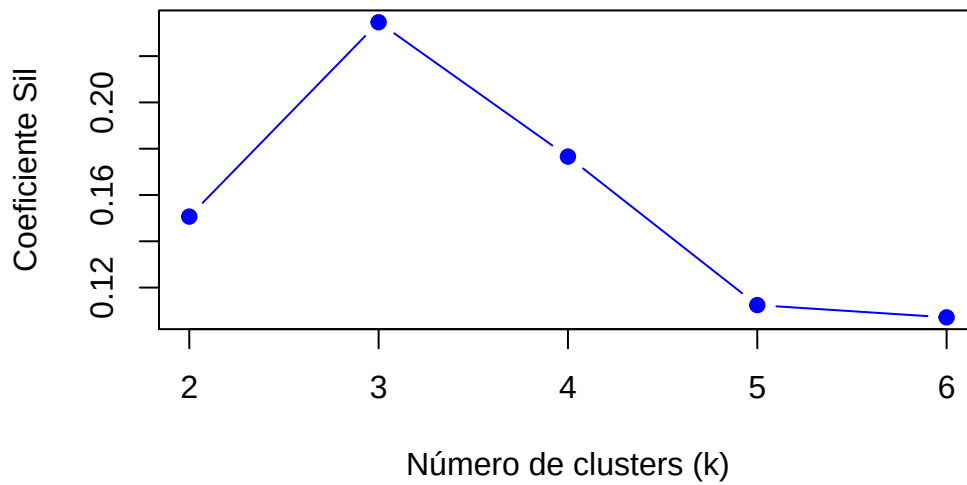
Evaluando k = 4 ...

Evaluando k = 5 ...

Evaluando k = 6 ...

```
# Graficar los resultados
par(mfrow = c(1, 1))
plot(k_values, sil_scores, type = "b", pch = 19, col = "blue",
     xlab = "Número de clusters (k)", ylab = "Coeficiente Sil",
     main = "Selección de k usando Silhouette con Soft-DTW")
```

Selección de k usando Silhouette con Soft-DTW



Se obtiene el óptimo de forma clara para $k = 3$. Igualmente hagamos $k = 2$ y $k = 3$ y elegiremos la que sea mejor.

Caso $k = 2$

```
set.seed(42)
cl_sdtw_2_icu = tsclust(serie_icu_z, type = "partitional", k = 2, distance =
  ↪ "sdtw", centroid = "sdtw_cent", seed = 12345)
cl_sdtw_2_icu
```

partitional clustering with 2 clusters

Using sdtw distance

Using sdtw_cent centroids

Time required for analysis:

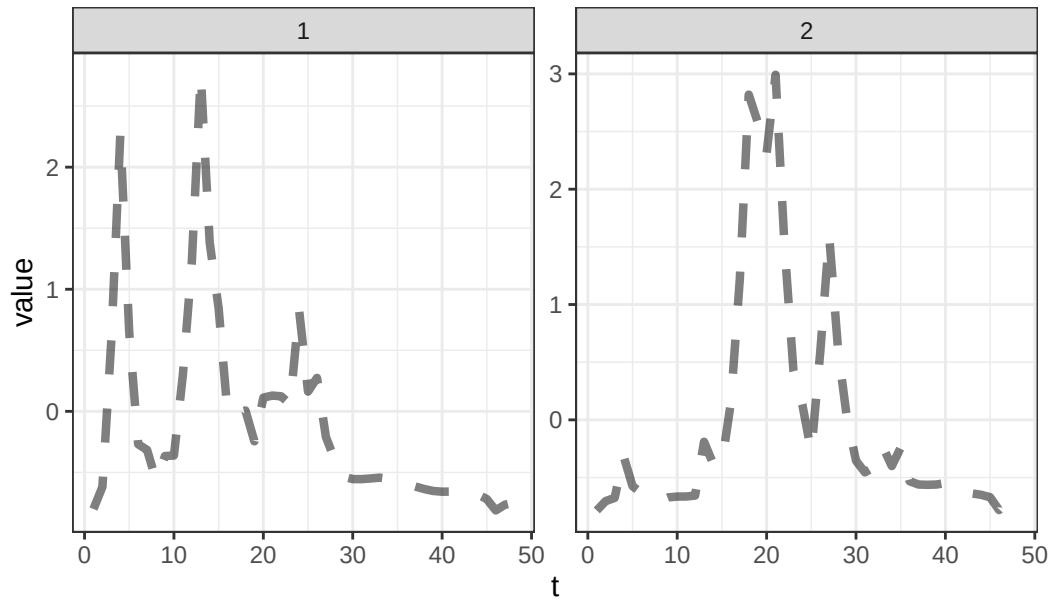
	user	system	elapsed
	9.33	0.34	0.94

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	19	5.174135
2	19	4.875432

```
plot(cl_sdtw_2_icu,type="centroids")
```

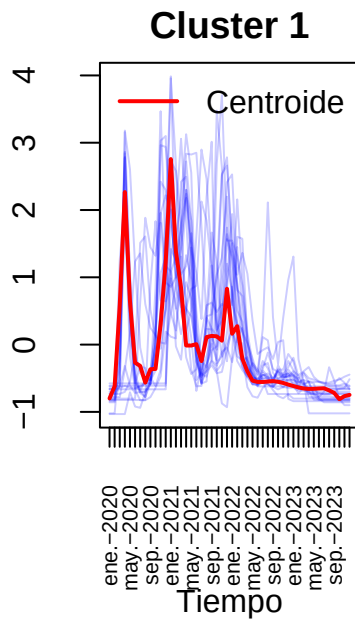
Clusters' members



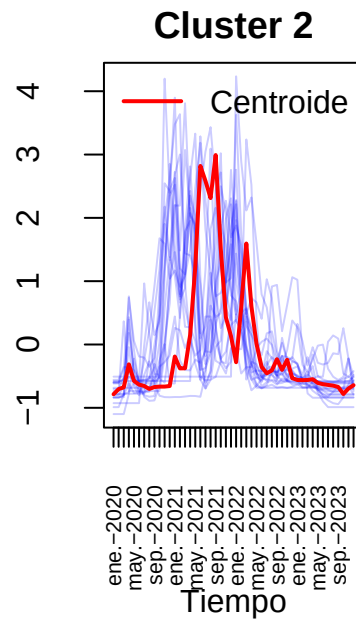
Visualicemos los centroides junto a las series.

```
visualizar_series_centroides(serie_icu_z,cl_sdtw_2_icu,2,ylab="Pacientes UCI  
↪ por millón normalizado")
```

Pacientes UCI por millón normalizad



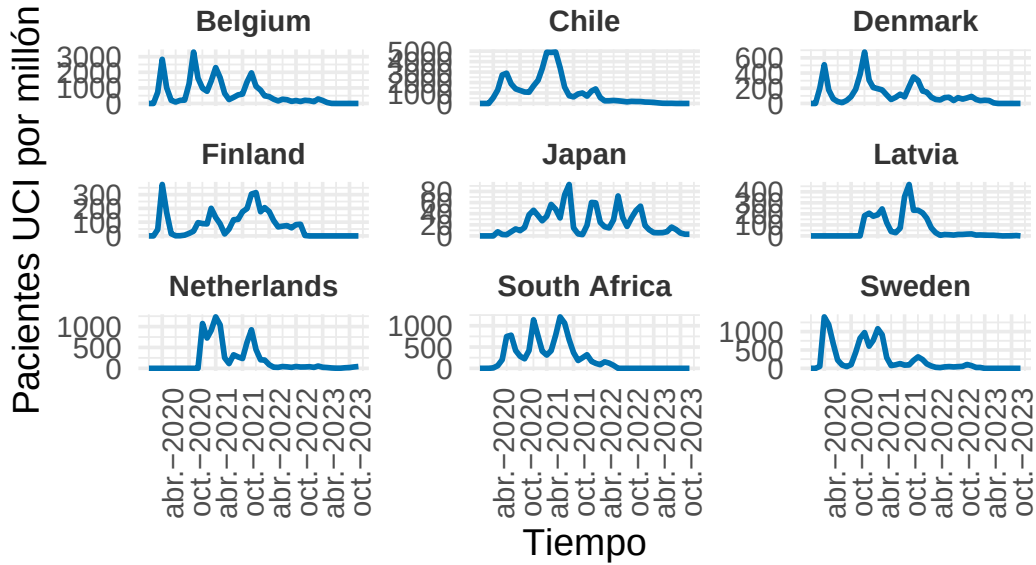
Pacientes UCI por millón normalizad



Algo similar a lo anterior, un primer grupo con brotes tempranos (y en general varios brotes) y un segundo grupo con un gran brote central. Veamos ejemplos.

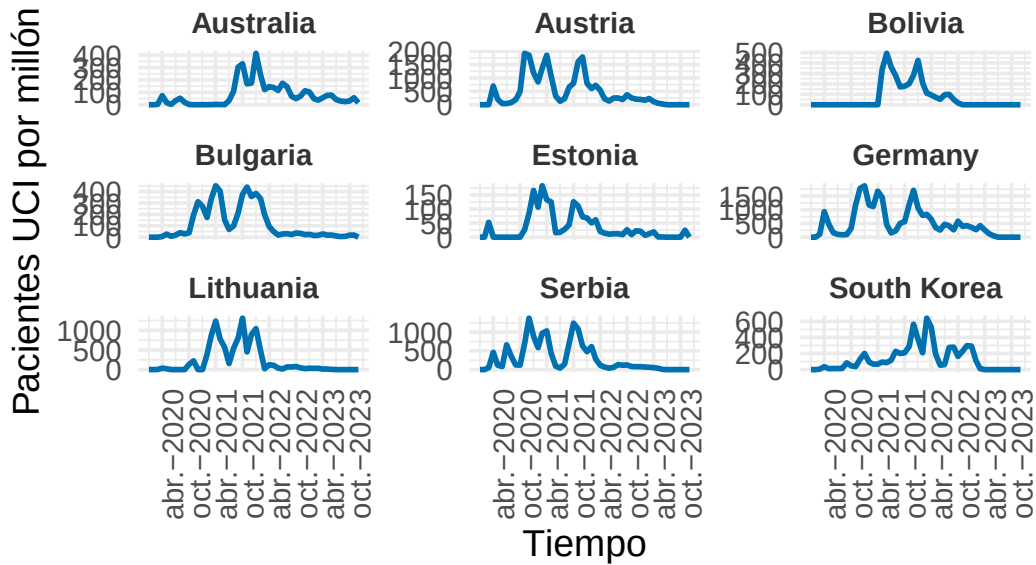
```
visualizar_ejemplos(serie_icu,cl_sdtw_2_icu,1,9,c(3,3),42,ylab="Pacientes UCI  
↪ por millón")
```

Ejemplos del Cluster 1



```
visualizar_ejemplos(serie_icu,cl_sdtw_2_icu,2,9,c(3,3),42,ylab="Pacientes UCI  
↪ por millón")
```

Ejemplos del Cluster 2



Vemos quizá grupos no tan homogéneos.

```
set.seed(42)
cvi(cl_sdtw_2_icu,type="internal")
```

	Sil	SF	CH	DB	DBstar	D
	1.506607e-01	2.200572e-04	2.332835e+01	1.917511e+00	1.917511e+00	1.183554e-01
COP						
	3.442294e-01					

```
cvi(cl_ndtw_dba_2_icu,tipe="internal")
```

	Sil	SF	CH	DB	DBstar	D
	0.06561179	0.52645374	27.15516520	2.00206197	2.00206197	0.19721908
COP						
	0.61543493					

Obtenemos métricas similares en general, veamos si con $k = 3$ conseguimos mejor agrupación.

Caso $k = 3$

```
set.seed(42)
cl_sdtw_3_icu = tsclust(serie_icu_z, type = "partitional", k = 3, distance =
  ↪ "sdtw", centroid = "sdtw_cent", seed = 12345)
cl_sdtw_3_icu
```

partitional clustering with 3 clusters

Using sdtw distance

Using sdtw_cent centroids

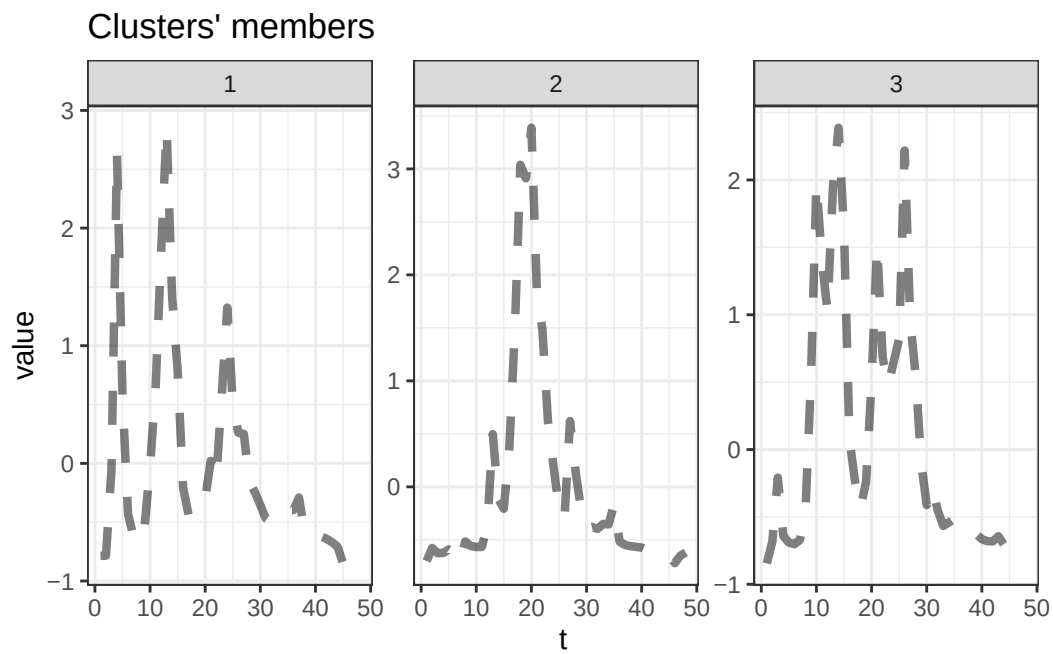
Time required for analysis:

	user	system	elapsed
	2.62	0.01	0.34

Cluster sizes with average intra-cluster distance:

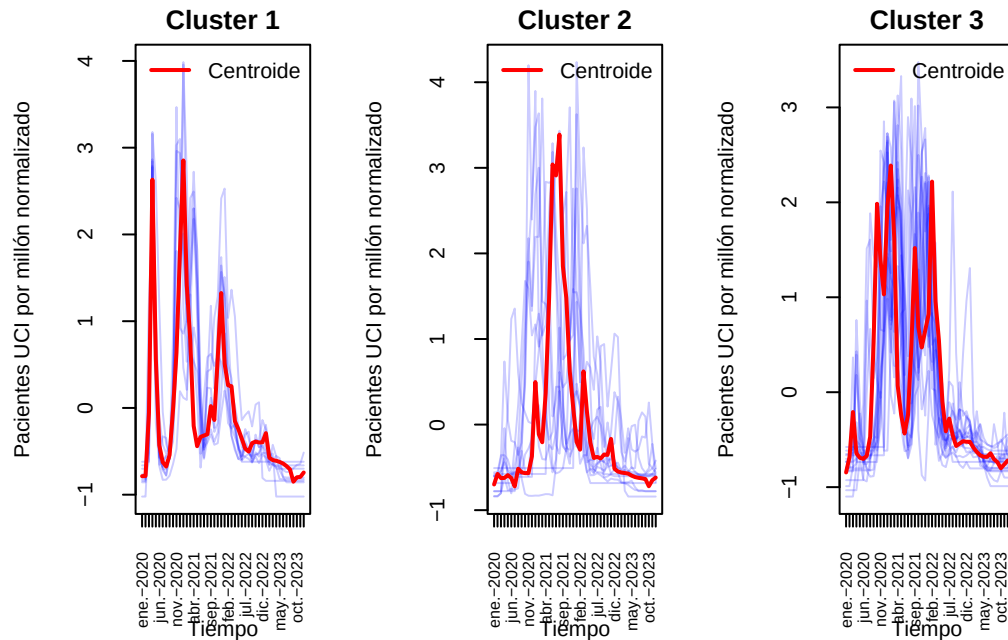
	size	av_dist
1	9	4.465702
2	12	4.511265
3	17	4.478392

```
plot(cl_sdtw_3_icu,type="centroids")
```



Obtenemos centroides similares en forma a los anteriores.

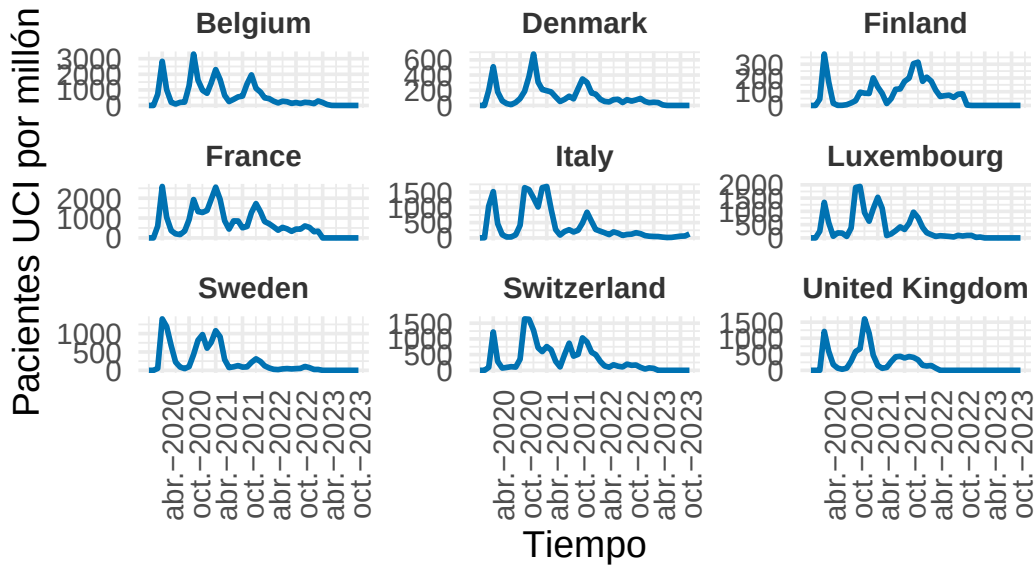
```
visualizar_series_centroides(serie_icu_z,cl_sdtw_3_icu,3,ylab="Pacientes UCI  
↪ por millón normalizado")
```



Parece que los centroides representan adecuadamente a las series. Hemos obtenido, al igual que antes un primer grupo con brotes más separados, con un brote inicial muy temprano, un grupo con un gran brote central y un tercer grupo con varios brotes centrados. Veamos los ejemplos.

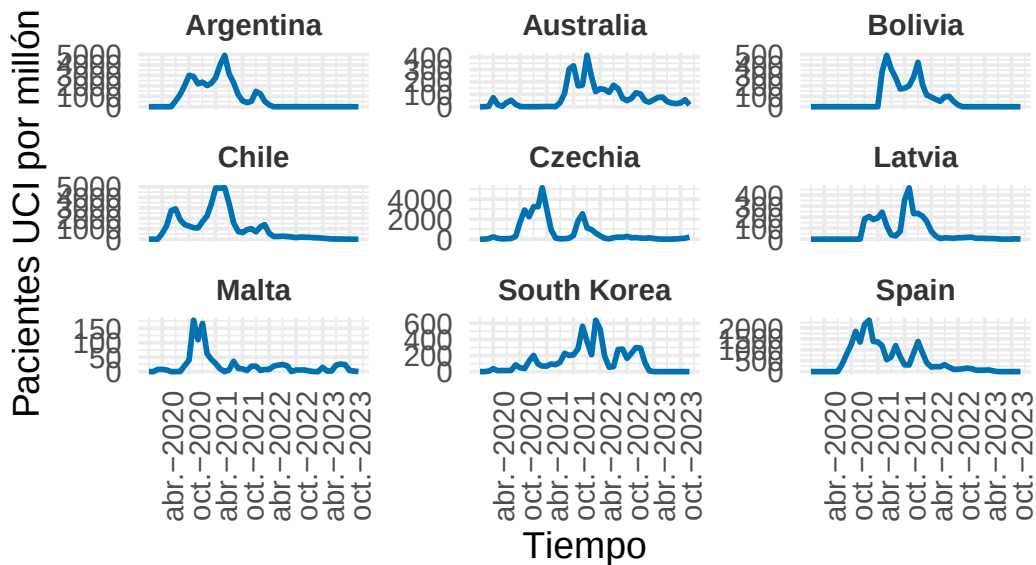
```
visualizar_ejemplos(serie_icu,cl_sdtw_3_icu,1,9,c(3,3),42,ylab="Pacientes UCI
↪ por millón")
```


Ejemplos del Cluster 1



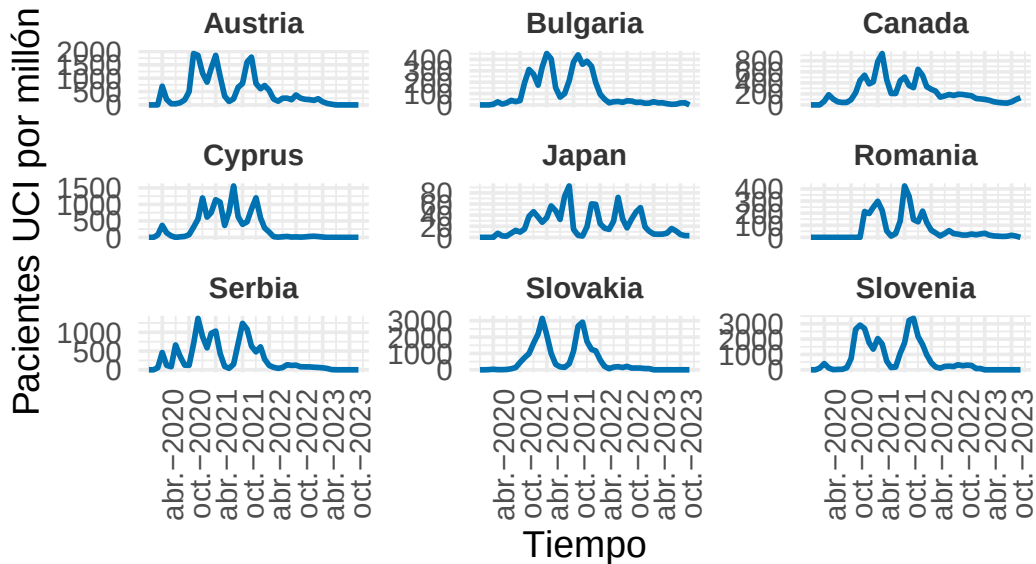
```
visualizar_ejemplos(serie_icu, cl_sdtw_3_icu, 2, 9, c(3, 3), 42, ylab="Pacientes UCI  
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_icu,cl_sdtw_3_icu,3,9,c(3,3),42,ylab="Pacientes UCI
↪ por millón")
```

Ejemplos del Cluster 3



Hemos obtenido grupos con mucho sentido y coherencia respecto a los centroides. Esta agrupación parece superior (como sugería el gráfico del k).

```
set.seed(42)
cvi(cl_ndtw_dba_3_icu,tipe="internal")
```

Si1	SF	CH	DB	DBstar	D
0.08456112	0.47554413	21.21135369	1.60635435	1.60764626	0.27509421
COP					
0.52867232					

```
cvi(cl_ndtw_dba_2_icu,tipe="internal")
```

Si1	SF	CH	DB	DBstar	D
0.06561179	0.52645374	27.15516520	2.00206197	2.00206197	0.19721908
COP					
0.61543493					

```
cvi(cl_sdtw_3_icu,type="internal")
```

	Sil	SF	CH	DB	DBstar	D
	2.346329e-01	6.146763e-06	1.703795e+01	1.242582e+00	1.243175e+00	1.778096e-01
COP						
	2.856340e-01					

Aunque la métrica CH presente un valor más alto para $k = 2$ con la distancia nDTW, la diferencia respecto al clúster `cl_sdtw_3_icu` no es sustancial. Este último, además, muestra valores muy aceptables en otras métricas, incluso bastante superiores en muchos casos. Si bien sabemos que algunas de ellas no son completamente fiables en este contexto, como hemos observado en los ejemplos, los clústeres que se forman muy homogéneos y coherentes con los centroides. Además se verifica que hay una separación clara entre los grupos. Es decir, se ha observado y comprobado con las métricas, que hay una buena separación inter-clúster y una baja separación intra-clúster.

***k*-Shape Clustering**

Este método se obtiene en `dtwclust` al usar la distancia Shape-Based (SBD) y el centroide shape extraction como bien sabemos. Volveremos a ver que k sugiere.

```
# Definimos el rango de k a evaluar
k_values = 2:6
sil_scores = numeric(length(k_values))

for (i in seq_along(k_values)) {
  k = k_values[i]
  cat("Evaluando k =", k, "...\\n")
  set.seed(42)
  cl = tsclust(serie_icu_z, type = "partitional", k = k, distance = "sbd",
    ↪ centroid = "shape", seed = 12345)

  resumen_metricas = cvi(cl,type="internal")
  sil = resumen_metricas["Sil"]

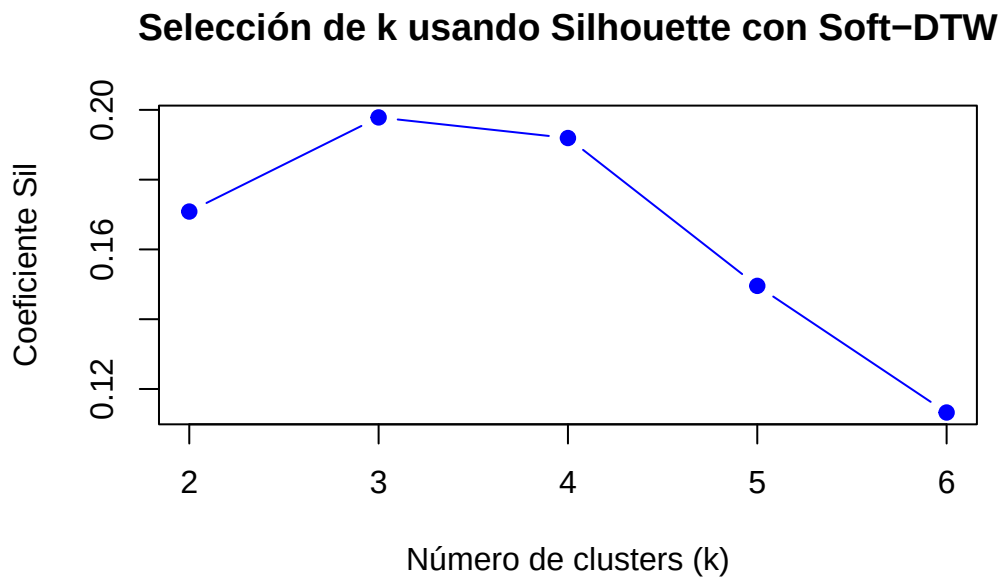
  # Promedio de los coeficientes de silueta
  sil_scores[i] = sil
}
```

```
Evaluando k = 2 ...
```

```
Evaluando k = 3 ...
```

```
Evaluando k = 4 ...  
Evaluando k = 5 ...  
Evaluando k = 6 ...
```

```
# Graficar los resultados  
par(mfrow = c(1, 1))  
plot(k_values, sil_scores, type = "b", pch = 19, col = "blue",  
     xlab = "Número de clusters (k)", ylab = "Coeficiente Sil",  
     main = "Selección de k usando Silhouette con Soft-DTW")
```



Sugiere $k = 3$, aunque $k = 4$ presenta un valor similar. Probemos solo con estos dos valores.

Caso $k = 3$

```
set.seed(42)  
cl_sbd_3_icu = tsclust(serie_icu_z, type = "partitional", k = 3, distance =  
  ↪ "sbd", centroid = "shape", seed = 12345)  
cl_sbd_3_icu
```

```
partitional clustering with 3 clusters  
Using sbd distance  
Using shape centroids  
Using zscore preprocessing
```

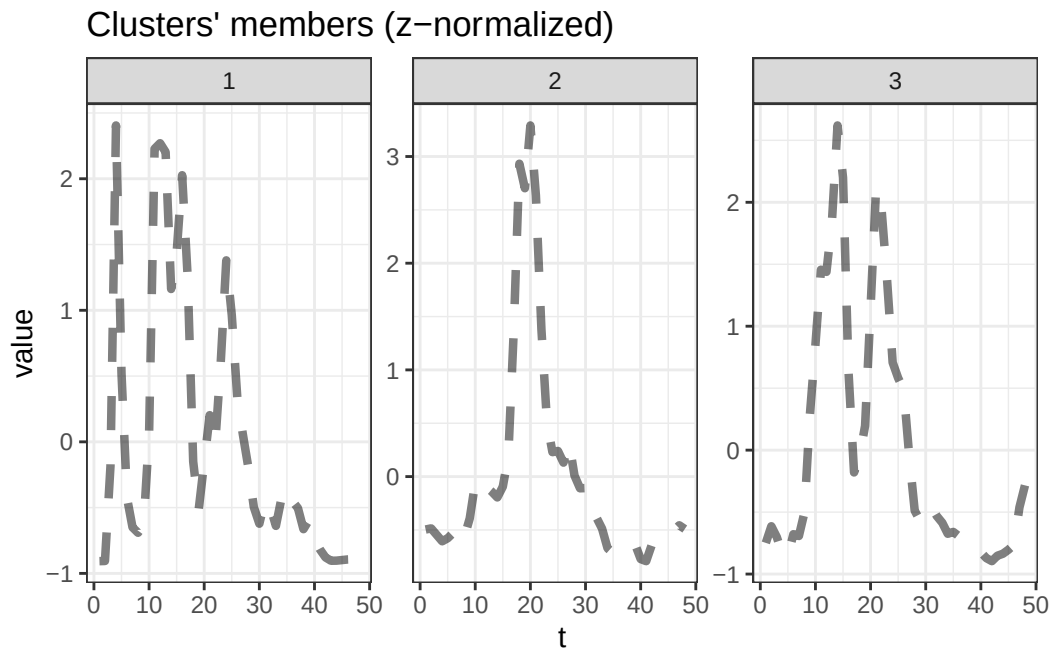
Time required for analysis:

user	system	elapsed
0.08	0.00	0.08

Cluster sizes with average intra-cluster distance:

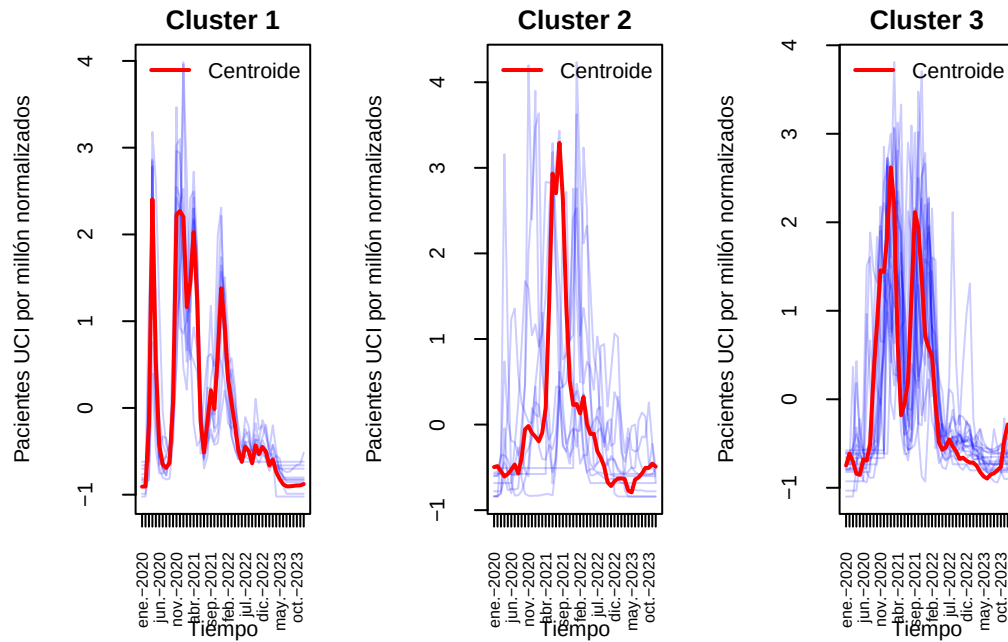
	size	av_dist
1	10	0.1084545
2	10	0.1736793
3	18	0.1678114

```
plot(cl_sbd_3_icu,type="centroids")
```



Volvemos a obtener centroides similares. Esto es buena señal pues los patrones captados por los algoritmos son parecidos.

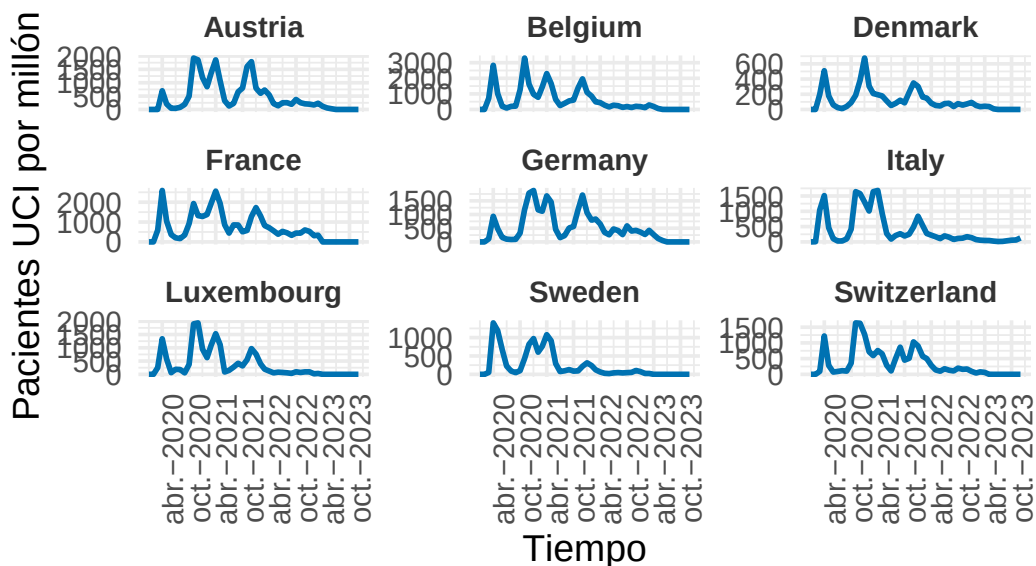
```
visualizar_series_centroides(serie_icu_z,cl_sbd_3_icu,3,ylab="Pacientes UCI  
↪ por millón normalizados")
```



Similar como acabamos de decir.

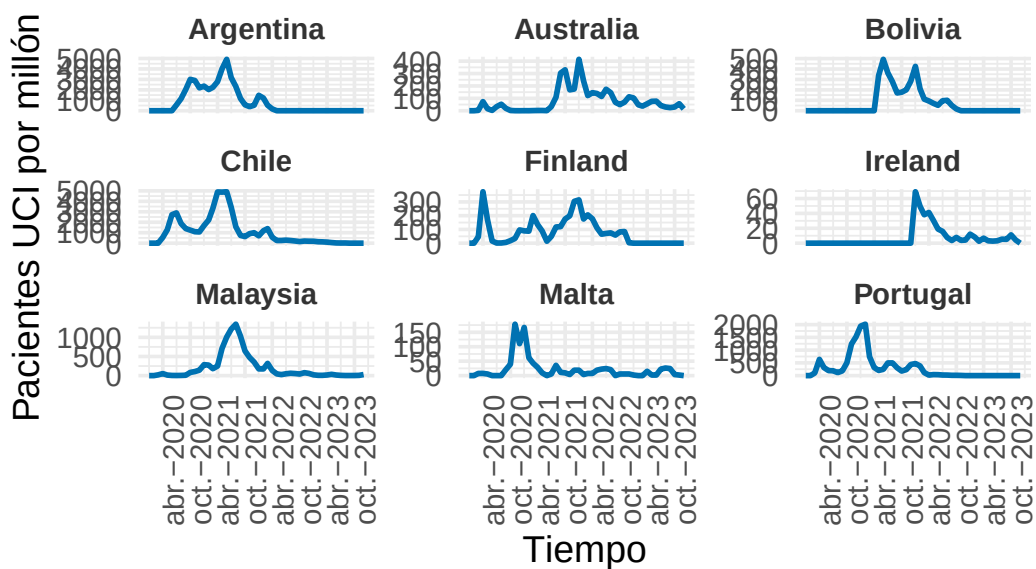
```
visualizar_ejemplos(serie_icu,cl_sbd_3_icu,1,9,c(3,3),42,ylab="Pacientes UCI
↪ por millón")
```

Ejemplos del Cluster 1



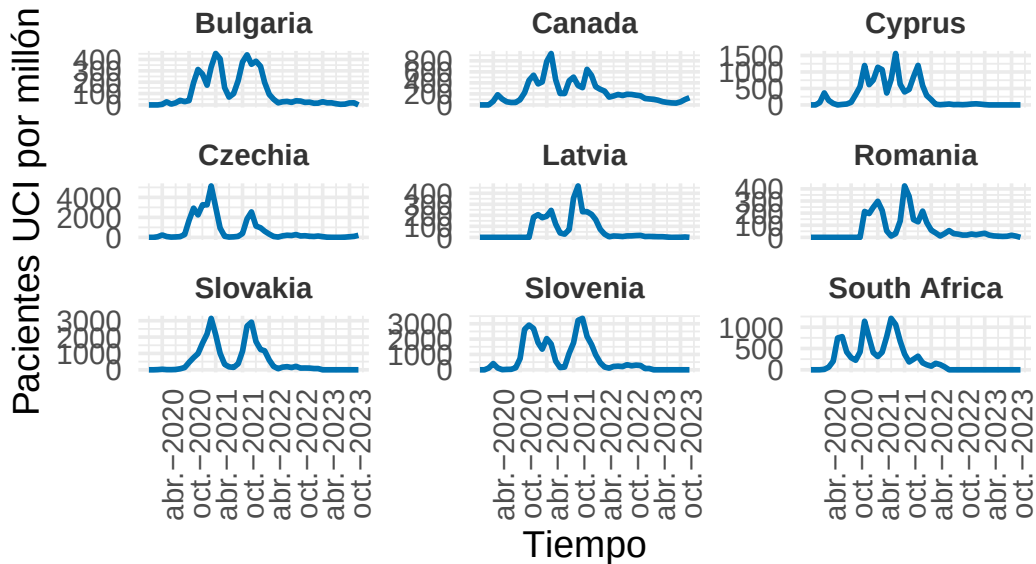
```
visualizar_ejemplos(serie_icu,cl_sbd_3_icu,2,9,c(3,3),42,ylab="Pacientes UCI
↪ por millón")
```

Ejemplos del Cluster 2



```
visualizar_ejemplos(serie_icu,cl_sbd_3_icu,3,9,c(3,3),42,ylab="Pacientes UCI
↪ por millón")
```

Ejemplos del Cluster 3



En los ejemplos se ven nuevamente agrupaciones muy decentes (aunque quizá parece mejor agrupamiento el anterior). Veamos las métricas que sugieren.

```
set.seed(42)
cvi(cl_sdtw_3_icu,type="internal")
```

Si1	SF	CH	DB	DBstar	D
2.346329e-01	6.284317e-06	1.729697e+01	1.242582e+00	1.243175e+00	1.778096e-01
COP					
2.856340e-01					

```
cvi(cl_sbd_3_icu,type="internal")
```

Si1	SF	CH	DB	DBstar	D	COP
0.1978387	0.4855752	14.1838915	1.1660908	1.1730181	0.1238631	0.3375975

En general se obtienen mejores métricas para el agrupamiento usando la distancia Soft-DTW. El coeficiente DB es mejor en esta última agrupación aunque la diferencia no es muy grande.

Por ello seguiremos eligiendo la agrupación Soft-DTW como la mejor. Pasemos por último al caso $k = 4$, pues en realidad ya hemos conseguido una agrupación muy buena.

Caso $k = 4$

```
set.seed(42)
cl_sbd_4_icu = tsclust(serie_icu_z, type = "partitional", k = 4, distance =
  ↪ "sbd", centroid = "shape", seed = 12345)
cl_sbd_4_icu
```

partitional clustering with 4 clusters

Using sbd distance

Using shape centroids

Using zscore preprocessing

Time required for analysis:

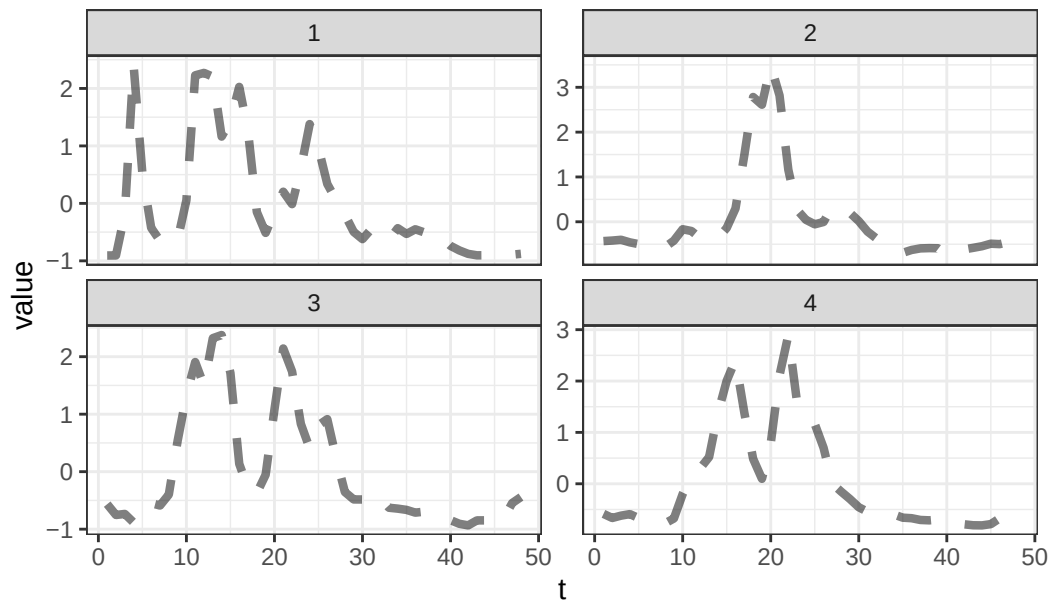
	user	system	elapsed
	0.05	0.00	0.05

Cluster sizes with average intra-cluster distance:

	size	av_dist
1	10	0.1084545
2	10	0.1647165
3	9	0.1503553
4	9	0.1267891

```
plot(cl_sbd_4_icu, type = "centroids")
```

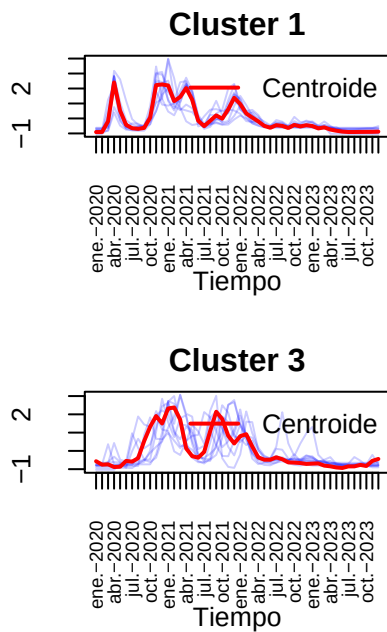
Clusters' members (z-normalized)



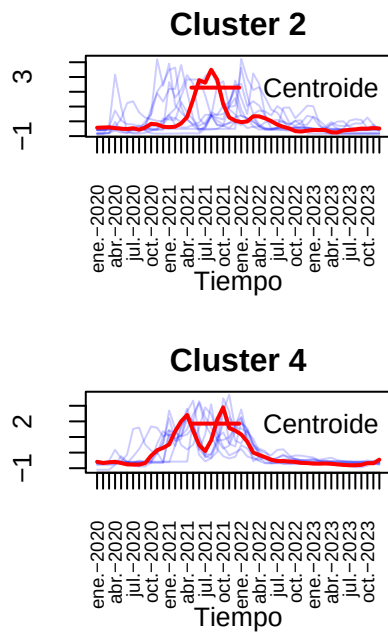
Parece haber cierto solapamiento entre el grupo 3 y 4.

```
visualizar_series_centroides(serie_icu_z,cl_sbd_4_icu,4,ylab="Pacientes UCI
↪ por millón normalizados",tamano_ventana = c(2,2))
```

Pacientes UCI por millón no



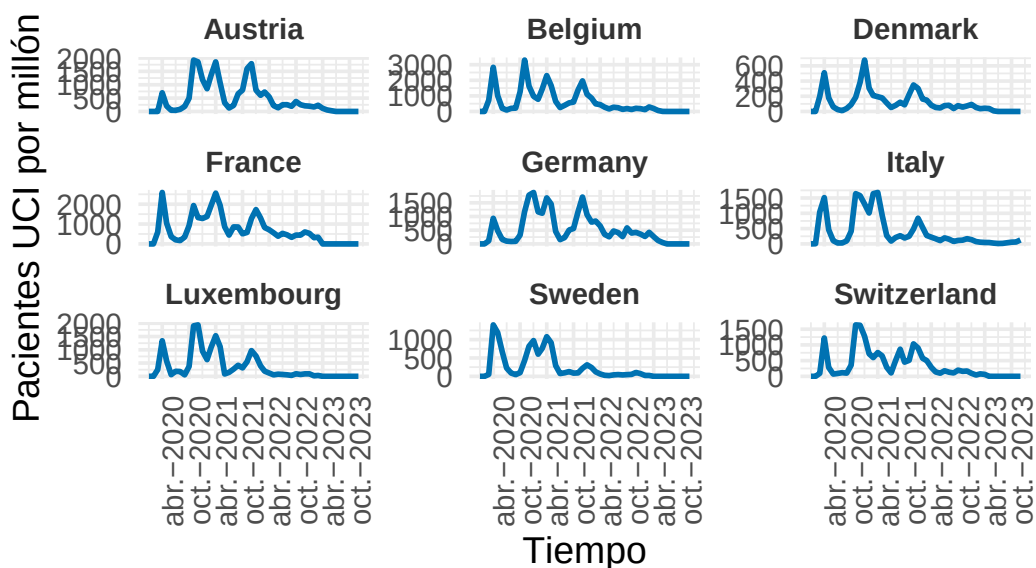
Pacientes UCI por millón no



Veamos algunos ejemplos.

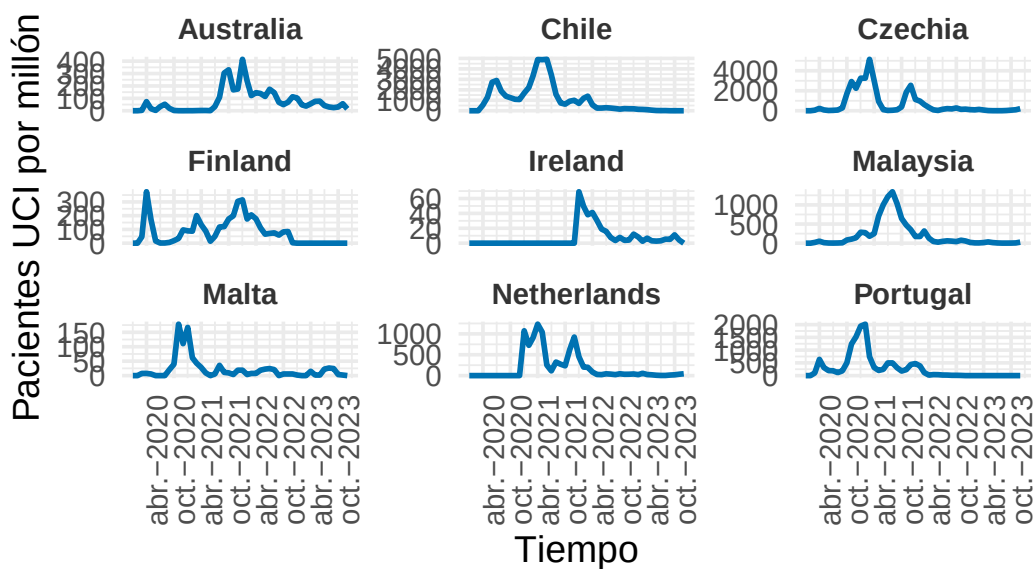
```
visualizar_ejemplos(serie_icu,cl_sbd_4_icu,1,9,c(3,3),42,ylab="Pacientes UCI
↳ por millón")
```

Ejemplos del Cluster 1



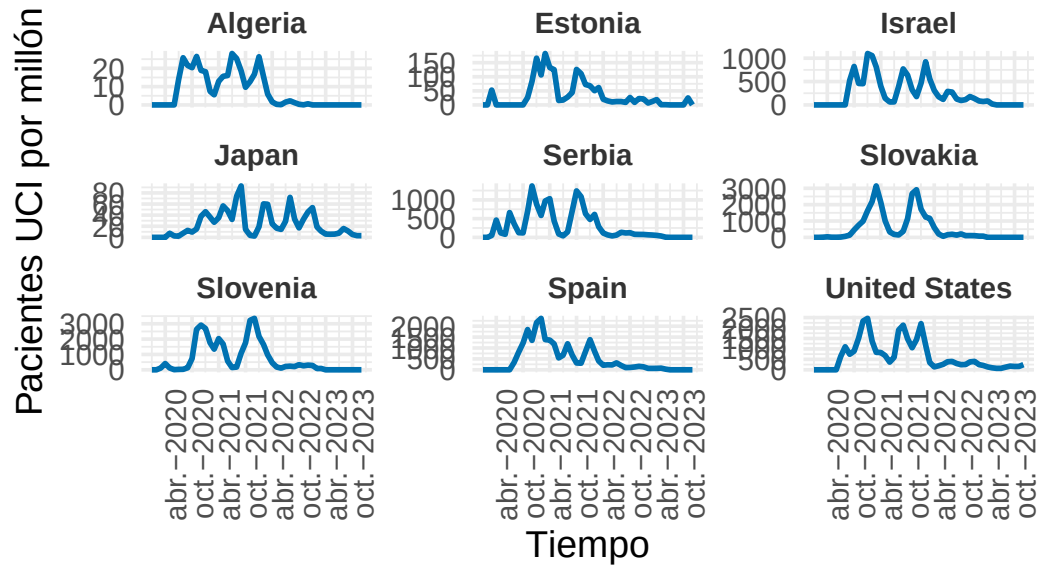
```
visualizar_ejemplos(serie_icu,cl_sbd_4_icu,2,9,c(3,3),42,ylab="Pacientes UCI  
↪ por millón")
```

Ejemplos del Cluster 2



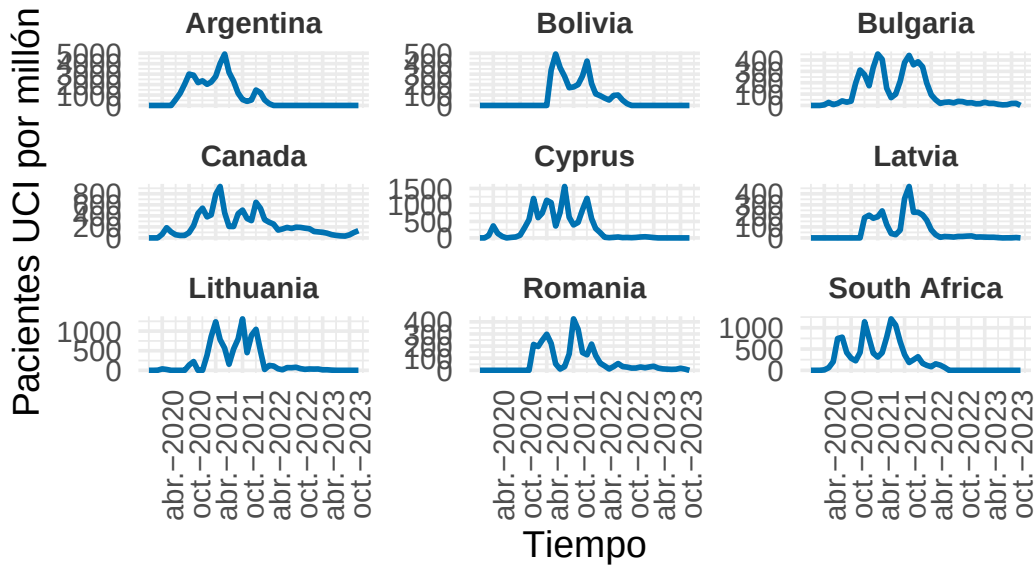
```
visualizar_ejemplos(serie_icu,cl_sbd_4_icu,3,9,c(3,3),42,ylab="Pacientes UCI  
↪ por millón")
```

Ejemplos del Cluster 3



```
visualizar_ejemplos(serie_icu,cl_sbd_4_icu,4,9,c(3,3),42,ylab="Pacientes UCI  
↪ por millón")
```

Ejemplos del Cluster 4



Hemos obtenido agrupaciones similares, aunque no parece haber mucha diferencia entre los grupos 3 y 4. Posiblemente esto se refleje en métricas como el índice DB.

```
set.seed(42)
cvi(cl_sdtw_3_icu,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
2.346329e-01	6.284317e-06	1.729697e+01	1.242582e+00	1.243175e+00	1.778096e-01	
2.856340e-01						

```
cvi(cl_sbd_4_icu,type="internal")
```

Sil	SF	CH	DB	DBstar	D	COP
0.1919276	0.4500062	11.8328196	1.4482601	1.5456371	0.1238631	0.3008139

Efectivamente ahora las métricas sugieren claramente que la agrupación con distancia Soft-DTW es la mejor. Vamos a elegir definitivamente esta agrupación elegida como la elegida, pues ya de por sí es muy buena. Concluimos la parte práctica en la que construimos los modelos.

Para terminar, guardaremos las series univariantes junto al mejor cluster encontrado y lo mismo para la multivariante.

Primero para los casos

```

# Obtener nombres de países
nombres_paises_casos = names(serie_new_cases)

# Extraer los clusters como vector
clusters_casos = cl_ndtw_dba_3@cluster

# Crear una nueva lista con el cluster incluido
serie_new_cases_cluster = mapply(function(pais, serie, cluster) {
  list(
    casos = serie,
    cluster = cluster
  )
}, pais = nombres_paises_casos, serie = serie_new_cases, cluster =
  ↪ clusters_casos, SIMPLIFY = FALSE)

# Asignar nombres de países a la nueva lista
names(serie_new_cases_cluster) = nombres_paises_casos
serie_new_cases_cluster$Afghanistan$cluster

```

[1] 3

```

# Guardamos como objeto Rdata
save(serie_new_cases_cluster, file = "serie_new_cases_cluster.RData")

```

Ahora para las muertes.

```

# Obtener nombres de países
nombres_paises_muertes = names(serie_new_deaths)

# Extraer los clusters como vector
clusters_muertes = cl_ndtw_dba_3_deaths@cluster

# Crear una nueva lista con el cluster incluido
serie_new_deaths_cluster = mapply(function(pais, serie, cluster) {
  list(
    muertes = serie,
    cluster = cluster
  )
}, pais = nombres_paises_muertes, serie = serie_new_deaths, cluster =
  ↪ clusters_muertes, SIMPLIFY = FALSE)

```

```
# Asignar nombres de países a la nueva lista
names(serie_new_deaths_cluster) = nombres_paises_muertes
serie_new_deaths_cluster$Afghanistan$cluster
```

```
[1] 1
```

```
# Guardamos como objeto Rdata
save(serie_new_deaths_cluster, file = "serie_new_deaths_cluster.RData")
```

Ahora para contagios y muertes.

```
# Obtener nombres de países
nombres_paises_casos_muertes = names(series_multivar)

# Extraer los clusters como vector
clusters_casos_mv = cl_hc_dtw_5_mv@cluster

# Crear una nueva lista con el cluster incluido
serie_new_cases_muertes_cluster = mapply(function(pais, serie, cluster) {
  list(
    casos = serie,
    cluster = cluster
  )
}, pais = nombres_paises_casos_muertes, serie = series_multivar, cluster =
  ↪ clusters_casos_mv, SIMPLIFY = FALSE)

# Asignar nombres de países a la nueva lista
names(serie_new_cases_muertes_cluster) = nombres_paises_casos_muertes
serie_new_cases_muertes_cluster$Afghanistan$cluster
```

```
[1] 1
```

```
# Guardamos como objeto Rdata
save(serie_new_cases_muertes_cluster, file =
  ↪ "serie_new_cases_muertes_cluster.RData")
```

Finalmente para los pacientes UCI.


```

# Obtener nombres de países
nombres_paises_icu = names(serie_icu)

# Extraer los clusters como vector
clusters_icu = cl_sdtw_3_icu@cluster

# Crear una nueva lista con el cluster incluido
serie_icu_cluster = mapply(function(pais, serie, cluster) {
  list(
    icu = serie,
    cluster = cluster
  )
}, pais = nombres_paises_icu, serie = serie_icu, cluster = clusters_icu,
  ↪ SIMPLIFY = FALSE)

# Asignar nombres de países a la nueva lista
names(serie_icu_cluster) = nombres_paises_icu
serie_icu_cluster$Algeria

```

```

$icu
 [1] 0.000 0.000 0.000 0.000 0.000 0.000 13.719 25.767 21.557 20.354
[11] 26.478 18.883 18.129 7.439 5.544 12.892 15.631 16.078 28.084 25.366
[21] 18.573 9.645 12.737 16.812 26.303 15.411 5.972 1.624 0.330 0.132
[31] 1.537 2.253 1.227 0.377 0.044 0.625 0.000 0.000 0.000 0.000
[41] 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000

```

```

$cluster
[1] 3

```

```

# Guardamos como objeto Rdata
save(serie_icu_cluster, file = "serie_icu_cluster.RData")

```